MindVideo AI

TensorFlow

من الأساس إلى الإنتاج

الأساسيات › بناء النماذج › التدريب والتقييم › النشر والإنتاج



م/ أحمد صالح المغز

2026

مرجع عربي شامل

TensorFlow الاحترافي

من الأساس إلى الإنتاج

مرجع متكامل للتعلّم العميق والشبكات العصبية وتطبيقاتها الواقعية، من أول تنسور إلى النشر والمراقبة في الإنتاج.

م/أحمد صالح المغز

TensorFlow 2.21 · Keras 3.15.1 · Python 3.13 (الرسمي TensorFlow توافق)

الإصدار الأول · 2026

بيانات الكتاب

العنوان	TensorFlow الاحترافي – من الأساس إلى الإنتاج
المؤلف	م/أحمد صالح المغز
الإصدار	الإصدار الأول · 2026
المراجع التقنية	Python 3.13 · Keras 3.15.1 · TensorFlow 2.21 (توافق TensorFlow الرسمي)
اللغة	العربية الفصحى المعاصرة
التصنيف	تعلّم الآلة · الشبكات العصبية · هندسة البرمجيات

© 2026 م/أحمد صالح المغز. جميع الحقوق محفوظة. لا يُسمح بإعادة إنتاج هذا الكتاب أو توزيعه أو تخزينه في نظام استرجاع، كليًّا أو جزئيًّا، بأي وسيلة إلكترونية أو ميكانيكية، دون إذن كتابي مسبق من المؤلف، باستثناء الاقتباسات القصيرة لأغراض المراجعة والتدريس مع الإشارة إلى المصدر.

أسماء TensorFlow Kerasg Pythong علامات تجارية مسجّلة لأصحابها. استُخدمت في هذا الكتاب لأغراض تعليمية وصفية فقط، ولا يعني ذلك أي رعاية أو ارتباط رسمي.

أُعدّت الأمثلة البرمجية لتعمل على الإصدارات المذكورة. قد تتغيّر الواجهات في الإصدارات اللاحقة؛ راجع التوثيق الرسمي عند الترقية. المعلومات الواردة في هذا الكتاب تُقدَّم لأغراض تعليمية، ولا تُعدّ استشارة مهنية أو قانونية.

> «الفكرة الجيدة لا تكفي؛ يجب أن تصل.» – من مقدمة الكتاب

للتواصل والملاحظات: عبر صفحة الكتاب الرسمية · الإصدار الأول 2026

بسم الله الرحمن الرحيم

إهداء

إلى زوجتي **أم عماد**، رفيقة دربي وشريكة أيامي؛ إلى من منحت البيت سكينته، والحلم صبره، والخطوات المتعثرة سببًا جديدًا للمضي. ما كان لهذا الجهد أن يكتمل لولا دعاؤك، ومساندتك، وإيمانك بأن وراء كل فكرة صادقة عملًا يستحق أن يُبذل.

إلى ابنيَّ **عماد** و**م/ عمار**، وهما ما يزالان على مقاعد الجامعة؛ أرى في أسئلتكما فضول العلم، وفي سعيكما وعدًا بمستقبل تصنعانه بأيديكما. أسأل الله أن يفتح لكما أبواب المعرفة، وأن يجعل هذا الكتاب رفيقًا في طريقكما، لا نهايةً له.

إلى ابنتي **الملكة**، آخر العنقود وبهجة البيت؛ لكِ من القلب محبة لا يحدّها وصف، وأمنية بأن تظلي مقبلة على الحياة، واثقةً بأن لكل حلم جميل بداية تستحق المحاولة.

إلى **عمرو**، الذي يرى في التفاصيل عوالم تستحق الاكتشاف؛ وإلى **عصماء**، صاحبة الصبر والعزيمة؛ وإلى **علا**، التي تمنح الحكايات دفأها؛ وإلى **عزام**، الذي يليق باسمه الإصرار: أنتم فرح العمر وسنده.

وإلى كل طالب وطالبة وباحث وباحثة، بدأوا من سؤال صغير، ثم واصلوا التعلم كلما استعصى عليهم الطريق: لعل هذه الصفحات تقرّب فكرة، أو تفتح باب تجربة، أو تذكّركم بأن المحاولة التالية قد تكون بداية الفهم.

لكم جميعًا أهدي هذا الكتاب، محبةً وامتنانًا وأملًا.

— م/أحمد صالح المغز

مقدمة المؤلف

في صيفٍ بعيد، جلستُ أمام شاشةٍ تُضيءُ غرفةً صغيرة، أراقبُ رقمًا واحدًا يتغيّر ببطء: قيمة الخسارة. كان النموذجُ الأولُ الذي كتبتُه لا يعرف شيئًا عن الصور ولا عن اللغة؛ كان يحاول أن يرسم خطًا مستقيمًا يمرُّ بين نقاطٍ مبعثرة. وحين نجح، بعد ساعاتٍ من المحاولة، شعرتُ بأنّي لم أكتبْ أسطرًا من بايثون، بل فتحتُ نافذةً على طريقةٍ جديدةٍ في التفكير: أن تُعطي الآلةَ أمثلةً، فتبحثَ هي عن القانون.

من ذلك اليوم إلى اليوم، تغيّر الكثير. كبرتْ المكتبةُ، وتغيّرتْ واجهاتُها، وتبدّلتْ أسماؤها، ودخلتْ الذكاءاتُ الاصطناعيةُ كلَّ بيتٍ وكلَّ هاتف. لكنّ السؤالَ الذي دفعني إلى الكتابة ظلَّ كما هو: **كيف نفهمُ ما نفعله، بدلًا من أن نُشغّلَ ما لا نفهمه؟**

هذا الكتابُ محاولةٌ صادقةٌ للإجابة. ليس قاموسًا للأوامر، ولا مجموعةً من الوصفات الجاهزة، بل رحلةٌ متدرّجةٌ تبدأ من السؤال «ما هو التنسور؟»، وتمرّ بالمعادلة والكود والتجربة والخطأ، وتنتهي عند نموذجٍ يعملُ في العالم الحقيقي، على هاتفٍ أو خادمٍ أو جهازٍ طبي. بين البداية والنهاية، ستجدُ قصصًا، وأمثلةً محلولة، وتمارينَ تُشبهُ ما ستواجهه فعلًا، وأخطاءً شائعةً سقطتُ فيها أنا نفسي، وأخرى رأيتُها تتكرّر أمامي في قاعات الدرس وفي غرفِ الخوادم.

لماذا TensorFlow في عام 2026؟

قد يسألُ سائل: «أليس هناك ما هو أسهل؟». نعم، هناك أدواتٌ أسهل، وأخرى أسرع في البحث، وثالثةٌ يحبّها مجتمعُ الأبحاث. لكنّ TensorFlow، بعد أكثر من عقدٍ من النضوج، صار **نظامًا بيئيًا كاملًا** لا مكتبةً واحدة: لغةٌ للتنسورات، وواجهةٌ عاليةٌ لبناء النماذج، وخطوطُ بياناتٍ عالية الأداء، وأدواتُ تدريبٍ موزّع، ومنصّةُ نشرٍ على الأجهزة الطرفية، وخوادمُ استدلال، وأدواتُ مراقبةٍ وحوكمة. من يتعلّمُ TensorFlow يتعلّمُ، في الحقيقة، **كيف تنتقل الفكرة من دفترٍ إلى منتج**.

ولا يعني ذلك أنّه الخيارُ الوحيد. الفصلُ الأول يقارنُ بين TensorFlow وPyTorch وJAX بلا تحيّز، ويشرحُ متى يكون كلٌّ منها الخيارَ الصحيح. فالهدفُ ليس أن تُحبّ أداة، بل أن تعرفَ **لماذا** تختارها.

عن الإصدارات: كلمة صريحة

أُكتبُ هذه الأسطرَ في عام 2026، والنسخةُ المرجعيةُ للكتاب هي **TensorFlow 2.21** و**Keras 3.15.1**، وأمثلةُ اللغة مكتوبةٌ بمعايير **Python 3.14**. لكنّ الأمانةَ العلميةَ تفرضُ عليّ قولَ الحقيقة: عجلاتُ TensorFlow الرسميةَ عند لحظة الكتابة تدعمُ Python حتى 3.13، ولم تصدرْ بعد عجلةٌ رسميةٌ مستقرّةٌ لـ 3.14. لذلك خصّصتُ الفصلَ الثاني لثلاثة مساراتٍ عملية: تشغيلُ TensorFlow داخل بيئة Python 3.13، أو داخل الحاوية الرسمية، أو تجربةُ `tf-nightly` عند توفّر عجلةٍ متوافقة. أمّا كودُ الكتاب نفسُه فقد كُتبَ بأسلوبٍ حديثٍ يعملُ على 3.13 و3.14 معًا، مع الإشارة إلى ما يعتمدُ على ميزاتٍ جديدة.

لن أُخفيَ عليك أنّك ستواجهُ أخطاءً. سأُخفّفُ عنك، لكن لن أُخدعك: من يُعلّمُك أنّ الطريقَ مفروشٌ بالورود يبيعُك وهمًا. الطريقُ مليءٌ بـNaN، وبأشكالٍ لا تتوافق، وبذاكرةٍ تنفد، وبمقاييسَ ترفعُ يدها كأنّها تقول: «لستُ واثقًا». ولهذا خُصّصتْ صناديقُ «أخطاء شائعة» في كل فصل، لأنّ الخطأَ ليس عيبًا في الرحلة؛ إنّه جزءٌ من المنهج.

لمن هذا الكتاب؟

- لطالبِ الهندسةِ أو علومِ الحاسبِ الذي يريدُ أن يبني أساسًا لا يُنسى.
- للمهندسِ الذي يعرفُ البرمجةَ ويقفُ على بابِ التعلّمِ العميقِ يبحثُ عن خريطة.
- للباحثِ التطبيقيِّ الذي يريدُ تحويلَ ورقةٍ علميةٍ إلى كودٍ يعمل.
- للمدرّبِ الذي يبحثُ عن مادةٍ عربيةٍ متسلسلةٍ، واضحةٍ، وقابلةٍ للتدريس.
- ولكلّ من قرأَ كثيرًا، وطبّقَ قليلًا، ثم قرّرَ أن يُكملَ الطريق.

لا يُشترطُ أن تكونَ خبيرًا في الرياضيات. الملحقُ (أ) يُعيدُ ترتيبَ ما تحتاجه من جبرٍ خطيٍّ وتفاضلٍ واحتمالات. ولا يُشترطُ أن تملكَ GPU؛ كلُّ أمثلةِ الكتاب تعملُ على CPU، وإن كانتْ بعضُ التجاربِ ستكونُ أسرعَ كثيرًا على كرتٍ رسومي.

كيف تقرأ هذا الكتاب؟

ثلاثُ طرقٍ، اخترْ منها ما يناسبُك:

1. **الطريقُ المتدرّج:** اقرأْ بالترتيب، ونفّذِ الكودَ في كل فصل قبل الانتقال. لا تقرأْ ثلاثةَ فصولٍ ثم تقول: «فهمتُ». الفهمُ الحقيقيُّ يظهرُ عند أول خطأٍ تُصلحُه بيدك.

2. **الطريقُ العملي:** ابدأْ من الفصل الخامس، ثم عُدْ إلى الأساسيات كلّما احتجت. هذا مسارٌ للمهندسِ الذي يريدُ نتيجةً سريعة، لكنّه لا يُغني عن فهمِ التنسورات والاشتقاق.
3. **الطريقُ المرجعي:** استخدمْ فهرسَ الموضوعات والملاحقَ عند الحاجة. الكتابُ صُمّمَ ليعودَ إليه المحترفُ بعد أن يُنهيَه، لا ليُوضعَ على الرف.

شكر وتقدير

إلى زوجتي «الملكة»، التي تعلّمتُ منها أنّ الإنجازَ ليس لحظةَ وصول، بل عادةُ سيرٍ يوميّة. إلى أبنائي: د/ عماد، د/ عمار، عمرو، عصماء، علا، عزام. إلى والديّ، الذين زرعوا فيّ فضولَ السؤال. إلى أساتذتي، وإلى كلِّ من كتبَ سطرًا تعلّمتُ منه. وإلى مجتمعِ TensorFlow مفتوحِ المصدر، الذي جعلَ المعرفةَ مشاعًا بين الناس.

وأخيرًا إليك، أيّها القارئ. إنّك تحملُ بين يديك دعوةً لا كتابًا: افتحْ محرّرَ الأكواد، وشغّلْ أول سطر، ودعِ الفضولَ يفعلُ الباقي.

م/أحمد صالح المغز

2026

كيف تستخدم هذا الكتاب؟

خريطة الرموز

الرمز	المعنى
💡	**نصيحة عملية** جرّبتُها ونجحت
⚠️	**تحذير:** خطأ شائع قد يُضيّع وقتك
🚫	**خطر:** قد يُفسد النموذج أو البيانات
🧪	**تجربة:** كود قابل للتنفيذ والتعديل
📌	**ملاحظة نظرية** تربط الفكرة بجذورها
🏭	**حالة واقعية** من الصناعة
🎯	**هدف الفصل**
✍️	**تمرين** مع الناتج المتوقع

عادات ثلاث تُغيّر نتيجتك

1. اكتبْ بيدك، لا تنسخْ فقط

قراءةُ الكودِ تُشبهُ مشاهدةَ السباحة من المدرّجات. لن تتعلّمَ السباحةَ حتى تنزلَ الماء. اكتبْ المثالَ سطرًا سطرًا، ثم أغلقِ المصدرَ وأعدْ كتابتَه من الذاكرة. حين تخطئ، ستعرفُ بالضبط أين يسكنُ سوءُ الفهم.

2. شغّلِ التجربةَ الصغيرة أولًا

قبل أن تُدرّبَ على 60 ألف صورة، درّبْ على 600. قبل أن تبني Transformer، ابنِ طبقةَ انتباهٍ واحدة. التجربةُ الصغيرةُ تُعطي إشارةً سريعة، والتجربةُ الكبيرةُ تُعطي إنذارًا متأخرًا.

3. اقرأِ الأخطاءَ كما تقرأُ الدرس

رسالةُ الخطأ ليست عدوًّا؛ إنّها أوّلُ من يُخبرك بالحقيقة. تعلّمْ أن تقرأَ: - نوعُ الخطأ (Shape/Dtype/ResourceExhausted/InvalidArgument). - الشكلُ المتوقّع مقابل الشكلِ الفعلي. - آخرُ طبقةٍ نجحتْ قبل الفشل.

بنية الفصل

كلُّ فصلٍ يتبعُ إيقاعًا ثابتًا:

1. **قصة افتتاحية:** مشكلةٌ واقعية تُشعلُ السؤال.
2. **أهداف الفصل:** ما ستُتقنهُ بنهايته.
3. **الشرح النظري:** الفكرة، ثم المعادلة، ثم الرموز.
4. **التنفيذ:** كودٌ كاملٌ مع تعليقاتٍ عربية.
5. **المقارنة والحالة الواقعية:** متى تختار؟ وماذا يحدث في الصناعة؟
6. **أخطاء شائعة:** العرض، السبب، الحل.
7. **تمارين:** 6-8 تمارين متنوعة.
8. **ملخص:** خلاصةٌ في نقاط.

البيئة والملفات

- كلُّ الأكواد في الكتاب مكتوبةٌ لتعملَ في **دفتر Jupyter** أو سكربت بايثون.
- المسارُ الموصى به: `book/` للمصادر، و`notebooks/` لتجاربك.
- لا تُشغّلْ كلَّ الأمثلةِ في ملفٍ واحد. افصلْ كلَّ تجربةٍ في دفترٍ مستقل، حتى تعرفَ مصدرَ الخطأ.
- احفظْ نسخةً من كلِّ تجربةٍ ناجحة، مع رقم الإصدار والنتيجة. ستحتاجُ إليها لاحقًا.

اصطلاحات الكود

```python
# تعليق عربي يشرح الفكرة، لا يكرّر السطر
import tensorflow as tf
import keras

keras.utils.set_random_seed(42)   # تكرار النتائج قدر الإمكان

# النتيجة المتوقعة تُكتب في تعليق مستقل
print(tf.__version__)             # 2.21.0
```

- **الأسماءُ الإنجليزية** للمتغيّرات والدوال، لأنّها لغةُ المكتبة والمجتمع.
- **التعليقاتُ عربية**، لأنّها مساحةُ الفهم.
- **الأرقامُ صغيرة** في الأمثلة، لتُشغَّلَ في ثوانٍ.
- **لا تُخفِ الخطأ:** إن كان المثالُ يحتاجُ GPU، سيُقالُ ذلك صراحةً.

ماذا تفعل إذا لم يعمل الكود؟

1. تحقّقْ من الإصدارات: `tf.__version__` و`keras.__version__`.
2. أعدْ تشغيلَ الجلسة من الصفر؛ كثيرٌ من الأخطاءِ سببُها حالةٌ قديمة.
3. ابحثْ عن رسالة الخطأ كما هي، ثم اقرأْ أوّلَ إجابةٍ رسمية، لا أوّلَ إجابةٍ عشوائية.
4. بسّطِ المثال: احذفْ كلَّ شيءٍ حتى يبقى الخطأ، ثم أعدْ بناءَه خطوةً خطوة.
5. إن استمرّ الخطأ، فتحقّقْ من الفصل الثاني (دليل البيئة) والملحق (ب).

ما لا يفعله هذا الكتاب

- لن يُعطيكَ «مشروعًا جاهزًا» تنسخُه وتنسى. المشاريعُ في الفصل الأخير هياكلُ كاملة، لكنّك أنت من يملأُ بياناتها ويواجهُ أخطاءها.
- لن يُحوّلكَ إلى باحثٍ في ثلاثة أسابيع. البحثُ يحتاجُ رياضياتٍ وأوراقًا وتجربة. لكنّه سيُعطيكَ أرضيةً صلبةً تنطلقُ منها.
- لن يُخفيَ عنك التعقيد. التعقيدُ جزءٌ من الحقيقة، ومن يُبسّطُ أكثر من اللازم يُنتجُ ثقةً زائفة.

خطتك الأولى: 30 يومًا

الأسبوع	الهدف	المخرَج
1	الفصول 1-5	بيئة تعمل + أول نموذج Keras
2	الفصول 6-8	شبكة من الصفر + تقييم صحيح
3	الفصول 9-10	خط بيانات + CNN على صور حقيقية
4	الفصل 11 + مشروع صغير	نقل تعلّم + تقرير تجربة

بعد الشهر الأول، ستكونُ قد كسرتَ الحاجزَ النفسي. الباقي رحلةُ عمقٍ وتخصص.

خريطة الطريق: من المبتدئ إلى المحترف

المستويات الخمسة

المستوى 0 — الاستكشاف (أسبوع)

- ما الذكاء الاصطناعي؟ ما الفرق بينه وبين تعلّم الآلة والتعلّم العميق؟
- تثبيت البيئة، وتشغيل أول نموذج.
- قراءة: الفصلان 1-2.

المستوى 1 — الأساسيات (3–4 أسابيع)

- التنسورات والأشكال والبث.
- الاشتقاق التلقائي والرسوم البيانية.
- Keras ودورة التدريب.
- قراءة: الفصول 3-5، مع تنفيذ كل تمرين.

المستوى 2 — بناء النماذج (6–8 أسابيع)

- الشبكات العصبية والانتشار الخلفي.
- المحسّنات والتنظيم والتقييم.
- خطوط البيانات.
- الرؤية الحاسوبية.
- قراءة: الفصول 6-10.

المستوى 3 — التخصص (8–10 أسابيع)

اختَرْ مسارًا واحدًا أولًا، ثم عُدْ للثاني: - **الرؤية:** 10-11، 19، 23. - **اللغة والتسلسلات:** 12-14، 19. - **التوليد:** 15-16، 19. - **الإنتاج:** 17-20، 24.

المستوى 4 — الاحتراف (مستمر)

- الأداء والتدريب الموزّع (18).
- MLOps والحوكمة (20، 23).
- التعلم المعزّز (21).
- البيانات الجدولية والرسوم (22).
- مشاريع كاملة ومعرض أعمال (24).

مسارات جاهزة

الهدف	الفصول بالترتيب
مهندس رؤية حاسوبية	1-11، 17-19، 23-24
مهندس NLP	1-9، 12-14، 17-19، 24
مهندس نماذج توليدية	1-9، 13-16، 18-19، 24
مهندس MLOps	1-9، 17-20، 23-24
باحث تطبيقي	1-18، 21-24
مدرّس/مدرب	الكتاب كاملًا + الملاحق + التمارين

قواعد التقدّم

1. **لا تنتقلْ من فصلٍ قبل أن تُنجزَ 70% من تمارينه.** التمرينُ هو المكانُ الذي يتحوّلُ فيه الشرحُ إلى مهارة.
2. **مشروعٌ صغيرٌ بعد كلِّ جزء.** بعد الجزء الأول: نموذجُ تصنيفٍ بسيط. بعد الثاني: CNN على بياناتك. بعد الثالث: تطبيقٌ لغويٌّ أو توليدي. بعد الرابع: خدمةٌ منشورة.
3. **اكتبْ ما تعلّمتَه.** مدوّنةٌ أو دفترُ ملاحظاتٍ أو شرحٌ لصديق. التعليمُ أفضلُ مراجعة.
4. **قِسْ، لا تُخمّنْ.** احتفظْ بجدولٍ للتجارب: التاريخ، الفكرة، التغيير، النتيجة، الوقت.
5. **اعتنِ بصحتك.** الجلوسُ الطويلُ أمام الشاشة ليس بطولة. النومُ والحركةُ جزءٌ من جودة التفكير.

ماذا بعد الكتاب؟

- اقرأْ ورقةً بحثيةً واحدةً أسبوعيًا، وأعدْ تنفيذَ فكرتها الأساسية.
- شاركْ في مسابقةٍ أو مشروعٍ مفتوح المصدر.
- اكتبْ نموذجًا صغيرًا بلغةٍ أخرى (PyTorch/JAX) لفهمِ الحدودِ بين الأدوات.
- تعلّمِ الرياضياتَ التي تحتاجها عند الحاجة، لا كلَّ الرياضيات قبل البدء.
- ابنِ معرضًا من 3 مشاريع عميقة، لا 20 مشروعًا سطحيًا.

اصطلاحات رياضية

الرمز	المعنى	ملاحظة
x	مُدخل (عدد أو متجه أو تنسور)	قد يكون صورة أو كلمة مُرمَّزة
y	الهدف/التسمية الحقيقية	في التصنيف: فئة
$\hat{y}$	التنبؤ	ناتج النموذج
W	مصفوفة الأوزان	قابلة للتعلّم
b	التحييز (bias)	قابل للتعلّم
z	المُدخل الخطّي للطبقة	$z = Wx + b$
a	التنشيط	$a = f(z)$
f	دالة التنشيط	ReLU، sigmoid، …
L أو $\mathcal{L}$	دالة الخسارة	كلّما صغرتْ كان أفضل
θ	وسائط النموذج	الأوزان والتحييزات
η	معدّل التعلّم	خطوة التحديث
$\nabla_\theta L$	تدرّج الخسارة	اتجاه أسرع زيادة
$\odot$	ضرب عنصري	Hadamard product
$\lVert W \rVert_2^2$	مربع معيار L2	مجموع مربعات العناصر
$\mathbb{E}$	التوقّع الرياضي	المتوسط النظري
σ	دالة sigmoid	$\frac{1}{1+e^{-z}}$
$\mathrm{softmax}$	توزيع احتمالي	مجموع المخرجات = 1
N	حجم الدفعة/العيّنة	batch size
D	بُعد الميزات	features
T	طول التسلسل	خطوات زمنية

الرمز	المعنى	ملاحظة
α	معدّل/وزن	حسب السياق
λ	قوة التنظيم	في L2
ϵ	قيمة صغيرة	لتفادي القسمة على صفر
μ, σ^2	المتوسط والتباين	في التطبيع

اصطلاحات الأشكال

الترميز	المعنى
`(N,)`	متجه بطول N
`(N, D)`	مصفوفة: N عيّنة، D ميزة
`(N, H, W, C)`	دفعة صور: ارتفاع، عرض، قنوات
`(N, T, D)`	دفعة تسلسلات: طول، بُعد
`None`	بُعد حرّ (غالبًا حجم الدفعة)
`...`	أبعاد محذوفة للاختصار

ملاحظة على الترقيم

- الفصول مرقّمة 1-24، والأقسام $N.1, N.2, \ldots$
- الأشكال مرقّمة $N.M$ حيث N رقم الفصل.
- المعادلات مرقّمة عند الحاجة فقط، حفاظًا على سلاسة القراءة.
- التمارين مرقّمة داخل كل فصل، والحلول المختارة في الملحق (هـ).

خريطة الكتاب

فهرس المحتويات

أربعة أجزاء · أربعة وعشرون فصلًا · سبعة ملاحق

مقدّمات الكتاب

تمهيد

إهداء iv
مقدمة المؤلف iv
كيف تستخدم هذا الكتاب؟ vii
خريطة الطريق xi
اصطلاحات رياضية xiv

الجزء الأول · الأساسات: كيف تفكّر في TensorFlow؟

1 TensorFlow: الحكاية والمنظومة والمكانة 2
1.1 أهداف الفصل 2
1.2 ما الذكاء الاصطناعي؟ ثلاث دوائر متداخلة 2
1.3 من DistBelief إلى TensorFlow 2.21: لماذا كل هذه الأسماء؟ 3
1.4 المنظومة: سبعة أجزاء تعمل معًا 4
1.5 لماذا TensorFlow؟ خمسة أسباب بلا مبالغة 5
1.6 TensorFlow أم PyTorch أم JAX؟ 6
1.7 أول نموذج: خطٌّ مستقيم يتعلّم 7
1.8 متى لا تختار TensorFlow؟ 9
1.9 حالة واقعية: عندما أنقذت LiteRT المشروع 9
1.10 أخطاء شائعة 10
1.11 تمارين 10
1.12 ملخص 11
2 بيئة 2026: Python 3.14 وTensorFlow 2.21 12
2.1 أهداف الفصل 12

2.2 التوافق يسبق أمر التثبيت 12
2.3 أنشئ بيئة واعرف ما ثبّتّ 13
2.4 CPU وGPU: افصل بين التعريف والمكتبة 14
2.5 تشخيص الأعطال 15
2.6 حالة واقعية: تجهيز فريق قبل بدء التدريب 15
2.7 مصفوفة التثبيت العملية 15
2.8 ملف اعتماد قابل لإعادة الإنتاج 16
2.9 فحص GPU بطريقة منصفة 17
2.10 التخطيط بين مبتدئ وباحث وفريق إنتاج 17
2.11 إدارة البيئة عبر عدة مشاريع 17
2.12 التثبيت ليس نهاية التحقق 18
2.13 أمثلة تنفيذية إضافية 18
2.14 أخطاء شائعة 19
2.15 تمارين 19
2.16 ملخص 20
3 التنسورات: التفكير بالأبعاد 21
3.1 أهداف الفصل 21
3.2 أربعة أوصاف للتنسور 21
3.3 اقرأ شكل الصورة قبل أن تدخل النموذج 23
3.4 إعادة تشكيل دون تغيير الترتيب 25
3.5 البث: محاذاة المحاور من اليمين 25
3.6 قيم ثابتة وأوزان قابلة للتعلم 26
3.7 حالة واقعية: دفعة صور لا تدخل النموذج 26
3.8 فحص الأشكال كعقد بين مكونات النظام 26
3.9 النوع العددي والدقة والذاكرة 27
3.10 قناع الحشو في التسلسلات 27
3.11 أدوات تشخيص سريعة 28
3.12 البث بالتفصيل: محور مفقود لا يساوي قيمة مفقودة 28
3.13 المتغيرات وسجل الحالة 28
3.14 الجهاز والنقل 29

3.15 اختبار خصائص الأشكال 29
3.16 مثال حساب يدوي للبث 29
3.17 أمثلة تنفيذية إضافية 30
3.18 توثيق عقد البيانات 30
3.19 فهرسة آمنة للأمثلة 30
3.20 سجلات المتغيرات 31
3.21 توحيد أنواع البيانات 31
3.22 أخطاء شائعة 31
3.23 تمارين 31
3.24 ملخص 32
4 الاشتقاق التلقائي والرسوم البيانية 33
4.1 أهداف الفصل 33
4.2 المشتقة: لغة التغيير 33
4.3 الشريط: كيف يتذكّر TensorFlow؟ 35
4.4 من الاشتقاق إلى التدريب 37
4.5 tf.function: الرسم البياني الذي يُبنى مرة واحدة 40
4.6 XLA: حين يُعاد تشكيل الرسم 42
4.7 حالة واقعية: ثماني دقائق بدل دقيقة 43
4.8 أخطاء شائعة 44
4.9 تمارين 44
4.10 ملخص 45
5 Keras: من الفكرة إلى النموذج 46
5.1 أهداف الفصل 46
5.2 أي بنية تناسب الحساب؟ 46
5.3 الخسارة والمقياس يجيبان عن سؤالين مختلفين 48
5.4 دورة الحفظ والنشر 49
5.5 حالة واقعية: الطلبات بفرعين 49
5.6 خط أساس كامل من البيانات إلى القياس 49
5.7 مدخلان ومخرجان في Functional API 50
5.8 الحفظ الآمن ودورة النشر 50

5.9 عقود التدريب: خسارة، مقياس، ووزن عينة 51
5.10 بناء input pipeline مع Keras 51
5.11 اكشف انهيار التعلم بالتشخيص 51
5.12 قابلية إعادة الإنتاج 52
5.13 Model subclassing ومتى تحتاجه 52
5.14 فصل البحث عن التقييم النهائي 52
5.15 Keras callbacks تعاقد وليست سحرًا 52
5.16 فحص قبل الإنتاج 53
5.17 دورة تعلم بحسب المستوى 53
5.18 أمثلة تنفيذية إضافية 53
5.19 مواصفات التشغيل والتسليم 53
5.20 تحقق من التسمية والتوزيع 54
5.21 استدلال بدفعات صغيرة 54
5.22 اختيار metric مرتبط بالتكلفة 54
5.23 احتفظ بعقد الخدمة مع النموذج 54
5.24 أخطاء شائعة 54
5.25 تمارين 55
5.26 ملخص 55

الجزء الثاني · بناء الشبكات وتدريبها

6 الشبكات العصبية من الصفر 57
6.1 أهداف الفصل 57
6.2 من جمع موزون إلى قرار 57
6.3 الانتشار الخلفي قاعدة سلسلة منظمة 59
6.4 التهيئة والاستقرار 60
6.5 حالة واقعية: مصنف القطع 60
6.6 اشتقاق رجوع الخطأ من طبقة واحدة 60
6.7 التنشيط مشتقةً وسلوكًا 61
6.8 التهيئة وحجم الطبقة 61
6.9 نموذج رقمي كامل للمبتدئ 62

6.10 المقارنة بين التنشيطات على رسم واحد 62
6.11 من المشتقة إلى تحديث الدفعة 63
6.12 كيف تقرأ model.summary؟ 63
6.13 حدود نظرية التقريب الشامل 63
6.14 تشخيص كل طبقة على حدة 63
6.15 منطق اختيار عدد الطبقات 64
6.16 مثال انحدار متعدد الميزات 64
6.17 ماذا تعرض في التقرير؟ 64
6.18 أمثلة تنفيذية إضافية 64
6.19 أخطاء شائعة 65
6.20 تمارين 65
6.21 ملخص 66
7 المحسّنات والتنظيم: فنّ التدريب 67
7.1 أهداف الفصل 67
7.2 المحسّن يحدد طريقة الحركة 67
7.3 التنظيم يحدّ من اعتماد النموذج على تفاصيل العينة 69
7.4 معدل التعلم والقص والدقة المختلطة 69
7.5 حالة واقعية: تعميم نموذج المستشفى 70
7.6 مسارات ضبط منهجية 70
7.7 تطبيع داخل الشبكة لا يطبع البيانات الأصلية 70
7.8 وصفة تشخيص عملية لفرط التعلّم 71
7.9 الدقة المختلطة والاستقرار العددي 71
7.10 جداول معدل التعلم 71
7.11 BatchNorm وDropout في inference 72
7.12 تنظيم يعكس فرضية 72
7.13 خطة اختبار لعدم الاستقرار 72
7.14 سجل المقارنة 72
7.15 فرط التعلّم تحت المجهر 73
7.16 وزن الفئات والـ focal loss 73
7.17 Early stopping ومشكلة epoch المختارة 73

7.18 تنظيم البيانات أهم من تنظيم الوزن 73
7.19 جدول قرار للتقنيات 74
7.20 أمثلة تنفيذية إضافية 74
7.21 أخطاء شائعة 74
7.22 تمارين 75
7.23 ملخص 75
8 التقييم والضبط: كيف تعرف أن نموذجك جيد؟ 76
8.1 أهداف الفصل 76
8.2 ثلاثة صناديق: لماذا لا تكفي مجموعة واحدة؟ 76
8.3 التحقق المتقاطع: أن تختبر أكثر من مرة 77
8.4 مصفوفة الالتباس: تشريح الخطأ 77
8.5 المعايرة: هل الثقة تعني الاحتمال؟ 79
8.6 عندما تكون الأقلية هي المهمة 80
8.7 الضبط والتتبع وتحليل الأخطاء 81
8.8 حالة واقعية: حين صار قرار المراجعة أهم من AUC 82
8.9 أمثلة تنفيذية إضافية 82
8.10 أخطاء شائعة 83
8.11 تمارين 84
8.12 ملخص 84
9 tf.data: هندسة خطوط البيانات 85
9.1 أهداف الفصل 85
9.2 Dataset سلسلة تحويلات 85
9.3 ترتيب التحويلات قرار ذاكرة وزمن 87
9.4 الملفات الكبيرة وTFRecord 88
9.5 حالة واقعية: صور خط إنتاج 88
9.6 من ملف صورة إلى دفعة مدربة 88
9.7 متى تختار TFRecord؟ 89
9.8 الحتمية والتوزيع 89
9.9 تشخيص اختناق البيانات 89
9.10 الثبات وإعادة التشغيل 90

9.11 schemag TFRecord 90
9.12 زيادة البيانات على مستوى batch 90
9.13 مراجعة قبل التدريب 90
9.14 نمط Dataset J reproducible 91
9.15 مسألة cache عمليًا 91
9.16 معايير قبول خط البيانات 91
9.17 هندسة CSV والنصوص 91
9.18 البيانات التي تتغير عند القراءة 92
9.19 أمثلة تنفيذية إضافية 92
9.20 فحوص قبول قبل حساب مكلف 92
9.21 حجم الملفات والمشاركة 93
9.22 فحص label mapping 93
9.23 أخطاء شائعة 93
9.24 تمارين 93
9.25 ملخص 94
10 الرؤية الحاسوبية: CNN من الالتفاف إلى الإنتاج 95
10.1 أهداف الفصل 95
10.2 مرشح محلي يعاد استخدامه 95
10.3 من LeNet إلى ConvNeXt 97
10.4 تفسير بصري لا يثبت السببية 98
10.5 حالة واقعية: كشف خدش 98
10.6 مجال الرؤية وحساب الكلفة 98
10.7 زيادة البيانات يحافظ على معنى الملصق 98
10.8 النقل من نموذج مدرب مسبقًا 98
10.9 تفسير Grad-CAM ومسؤولية التحقق 99
10.10 من التدريب إلى القياس التشغيلي 99
10.11 وصفة تدريب قابلة للتكرار 99
10.12 لماذا يختلف أداء CNN بين المجالين؟ 99
10.13 ضوابط BatchNorm ودفعة صغيرة 100
10.14 تهيئة صور الخدمة 100

10.15 Grad-CAM: تطبيق وتفسير حذر 100
10.16 تصميم مشروع تصنيف صور كامل 100
10.17 الموازنة بين الدقة والكمون 101
10.18 أخطاء من الواقع 101
10.19 المراجعة العلمية 101
10.20 استكشاف feature maps 101
10.21 التهيئة المسبقة والملصقات الصاخبة 101
10.22 متطلبات اكتساب البيانات 102
10.23 أمثلة تنفيذية إضافية 102
10.24 أخطاء شائعة 102
10.25 تمارين 103
10.26 ملخص 103

الجزء الثالث · الرؤية والتسلسلات واللغة والتوليد

11 الرؤية المتقدمة: النقل والكشف والتجزئة 105
11.1 أهداف الفصل 105
11.2 التعلم بالنقل مرحلتان 105
11.3 الكشف يحدد أين وماذا 107
11.4 U-Net للتجزئة 108
11.5 حالة واقعية: صور من جهاز جديد 108
11.6 الكشف: من الصندوق إلى قرار التشغيل 108
11.7 التجزئة: قناع صحيح مكانيًا 108
11.8 التنبؤ بوجود هدف نادر 109
11.9 TTA وموازنة الكمون 109
11.10 تقرير مشروع قابل للمراجعة 109
11.11 Transfer learning بميزانية بيانات صغيرة 109
11.12 مكونات كاشف الأجسام 110
11.13 أقنعة segmentation ومعالجة الحدود 110
11.14 تقييم مقاوم لتغير المجال 110
11.15 annotator agreement وإدارة الأقنعة 110

11.16 كشف الأجسام الصغيرة وتعدد المقاييس 111
11.17 تكلفة البيانات وتحديد الأولوية 111
11.18 عتبات تشغيل حسب المخاطر 111
11.19 وثائق المشروع والقيود 111
11.20 حين لا توجد صناديق لكل البيانات 111
11.21 القياس الميداني وإدارة الإصدارات 112
11.22 إمكانية إعادة إنتاج المشروع 112
11.23 أمثلة تنفيذية إضافية 112
11.24 أخطاء شائعة 113
11.25 تمارين 113
11.26 ملخص 113
12 التسلسلات: RNN/LSTM/GRU والسلاسل الزمنية 114
12.1 أهداف الفصل 114
12.2 ترتيب الزمن جزء من البيانات 114
12.3 لماذا LSTM وGRU؟ 116
12.4 حالة واقعية: تخطيط مخزون 117
12.5 النافذة والهدف وخط الأساس 117
12.6 LSTM بواباتٌ وحالةٌ 117
12.7 تقييم زمني واقعي 117
12.8 بيانات متغيرة وحالات مفقودة 118
12.9 Stationarity والاتجاه والموسمية 118
12.10 التسرب الزمني وfeature availability 118
12.11 اختيار نافذة الإدخال 118
12.12 teacher forcing والتنبؤ المتكرر 119
12.13 تفسير نموذج زمني 119
12.14 تدريب LSTM عمليًا 119
12.15 سلاسل متعددة ومتغيرات خارجية 119
12.16 مقاييس الانحدار الزمني 120
12.17 اختبار انزياح الزمن 120
12.18 عدم اليقين والتخطيط 120

12.19 اختيار RNN أو Conv1D أو Transformer 120
12.20 تقرير متانة زمني 121
12.21 أمثلة تنفيذية إضافية 121
12.22 توقع احتمالي واحتياط المخزون 121
12.23 أخطاء شائعة 121
12.24 تمارين 122
12.25 ملخص 122
13 الانتباه والمحوّلات 123
13.1 أهداف الفصل 123
13.2 من السياق المضغوط إلى أوزان الانتباه 123
13.3 لماذا رؤوس متعددة ومعلومة موضع؟ 125
13.4 الكلفة والبدائل 125
13.5 حالة واقعية: تصنيف تذاكر الدعم 126
13.6 بناء الكتلة خطوةً خطوة 126
13.7 موضعية مطلقة ونسبية 126
13.8 طول السياق والذاكرة 127
13.9 اختيار بنية المحول 127
13.10 أساليب تدريب ومعايرة 127
13.11 ضبط شكل الاستعلام والمفتاح والقيمة 127
13.12 قناع padding في دفعة لغوية 128
13.13 لماذا انتشر Flash Attention؟ 128
13.14 التحليل الحسابي والذاكرة 128
13.15 فحوص قبل fine-tuning 128
13.16 تصميم pretraining وfine-tuning 129
13.17 التعامل مع طول السياق 129
13.18 التحسين والتدرج 129
13.19 المعايرة والتقييم 129
13.20 إدارة checkpoints 130
13.21 تجربة استئصال (ablation) سليمة 130
13.22 Transformer للتصنيف على بيانات صغيرة 130

13.23 حدود الانتباه كشرح 130
13.24 أمثلة تنفيذية إضافية 130
13.25 أخطاء شائعة 131
13.26 تمارين 131
13.27 ملخص 132
14 معالجة اللغة الطبيعية والعربية 133
14.1 أهداف الفصل 133
14.2 من الحرف إلى الرمز: لماذا لا يرى النموذج نصك؟ 133
14.3 التضمين: حين تصبح الكلمة متجهًا 136
14.4 BERT: قراءة الجملة في اتجاهين 138
14.5 العربية: لغةٌ تتحدى المعجم 140
14.6 من النص إلى نموذج: تصنيف المشاعر خطوة بخطوة 141
14.7 التقييم: ما الذي نقيسه فعلًا؟ 144
14.8 أخطاء شائعة 145
14.9 تمارين 146
14.10 ملخص 147
15 التوليد: Autoencoder وVAE وGAN 148
15.1 أهداف الفصل 148
15.2 Autoencoder يتعلم إعادة البناء 148
15.3 VAE يعلّم فضاءً احتماليًا 148
15.4 GAN لعبة خصومية لا مقياس جودة وحيد 149
15.5 تقييم العينات 150
15.6 حالة واقعية: عيوب صناعية نادرة 151
15.7 أمثلة تنفيذية إضافية 151
15.8 ورشة عملية: ما الذي نراقبه أثناء التدريب؟ 151
15.9 بناء تجربة توليد يمكن للآخرين مراجعتها 152
15.10 بروتوكول عملي على صور صغيرة 153
15.11 اختيار latent dimension وتفسيره 153
15.12 الاستخدام البحثي والأخلاقي 153
15.13 دراسات ablation ومقارنة عادلة 153

15.14 التوليد المشروط بالمعلومات 154
15.15 اختيار مقياس الصورة 154
15.16 صلاحية استخدام البيانات 154
15.17 تصميم دراسة ومقارنة عادلة 154
15.18 سياسة الاستخدام 155
15.19 نشر النتائج 155
15.20 أخطاء شائعة 155
15.21 تمارين 156
15.22 ملخص 156
16 نماذج الانتشار: التوليد الحديث 157
16.1 أهداف الفصل 157
16.2 إضافة ضوضاء وفق جدول 157
16.3 إزالة الضوضاء مشروطة بالزمن 157
16.4 التوجيه والتكلفة 158
16.5 حالة واقعية: صور قطع تحت إضاءة مختلفة 159
16.6 أمثلة تنفيذية إضافية 159
16.7 جدول الضوضاء والتدريب 159
16.8 U-Net مشروطة بالزمن 160
16.9 أخذ عينات DDPM وDDIM 160
16.10 تقييم ومخاطر بيانات التوليد 160
16.11 اختبار forward process قبل التدريب 160
16.12 أخذ عينات مرئي ومقاييس هندسية 161
16.13 اختيار دقة التدريب 161
16.14 تلخيص دراسة نموذج 161
16.15 اشتقاق معاملات الجدول 161
16.16 تشخيص artifacts 162
16.17 التقطير والتسريع 162
16.18 تسجيل نموذج بحثي 162
16.19 حدود التطبيق 162
16.20 مقارنة sampler وتقليل الكلفة 163

16.21 الأثر البحثي والبيئي 163
16.22 اختيار تطبيق مسؤول 163
16.23 التحقق من شرط التوليد 163
16.24 استقرار التحديث 164
16.25 أخطاء شائعة 164
16.26 تمارين 164
16.27 ملخص 165

الجزء الرابع · الاحتراف والإنتاج

17 الطبقات والنماذج المخصصة والتدريب المتقدم 167
17.1 أهداف الفصل 167
17.2 وزن ينشأ مرة واحدة 167
17.3 خسائر ومقاييس وسلوك تدريب خاص 168
17.4 حالة واقعية: خسارة موزونة لعدة مخرجات 169
17.5 أمثلة تنفيذية إضافية 169
17.6 تعديل trainstep دون كسر Keras 170
17.7 الأقنعة والحالة في call 171
17.8 التهيئة والتسلسل والتحميل 171
17.9 عزل أخطاء التدرج 172
17.10 مقاييس مخصصة ذات حالة 172
17.11 addloss وقيود الطبقة 172
17.12 دعم أكثر من مخرج وsample weights 172
17.13 tf.function والتوافق مع التصدير 172
17.14 خطة اختبار طبقة احترافية 173
17.15 طبقة Attention مصغرة 173
17.16 حساب الخسارة لعدة أهداف 173
17.17 توزيع الطبقة المخصصة 173
17.18 التوافق بين backends 174
17.19 فشل مختبر التدريب المخصص 174
17.20 دورة تدريب مخصصة عند الضرورة 174

17.21 اختبار الطبقة عبر الأشكال والأنواع 174
17.22 دعم backend ومراجعة الشفرة 174
17.23 واجهة عامة وصيانة طويلة الأجل 175
17.24 قراءة سجل التدرجات 175
17.25 دورة مراجعة على دفعة واحدة 175
17.26 أخطاء شائعة 175
17.27 تمارين 175
17.28 ملخص 176
18 الأداء والتدريب الموزّع 177
18.1 أهداف الفصل 177
18.2 قِس خط أساس 177
18.3 XLA والدقة المختلطة 179
18.4 التوزيع وتراكم التدرجات 179
18.5 حالة واقعية: قياس خدمة تدريب 180
18.6 أمثلة تنفيذية إضافية 180
18.7 استراتيجيات التوزيع والتكلفة 181
18.8 تراكم التدرج 181
18.9 اقرأ trace لا رقم واحد 181
18.10 الدقة المختلطة وحدود الأعداد 181
18.11 تقرير أداء قابل للتكرار 182
18.12 مختبر profiler خطوة بخطوة 182
18.13 تقدير الذاكرة 182
18.14 استراتيجية اختيار الدقة 182
18.15 الاتصالات في التدريب الموزع 183
18.16 حوكمة القياس 183
18.17 Mixed precision حسب backend 183
18.18 توازي البيانات والنموذج 183
18.19 مراجعة الأشكال الديناميكية 183
18.20 جمع الدرجات والتحقق من scale 184
18.21 لوحة أداء للفريق 184

18.22 XLA وكلفة التجميع 184
18.23 الذاكرة وحجم الدفعة 184
18.24 اختبار تراكم التدرج 184
18.25 استراتيجية نمو 185
18.26 أخطاء شائعة 185
18.27 تمارين 185
18.28 ملخص 186
19 النشر: TF Servingg TF.jsg LiteRT 187
19.1 أهداف الفصل 187
19.2 صيغة الاستدلال وعقده 187
19.3 LiteRT على الأجهزة الطرفية 188
19.4 المتصفح والخدمات 189
19.5 حالة واقعية: فحص عيب دون إنترنت 189
19.6 أمثلة تنفيذية إضافية 189
19.7 عقد الإدخال وإصدارات النموذج 190
19.8 معايرة التكميم وقرار القبول 190
19.9 تشغيل الخدمة وطرح تدريجي 191
19.10 أمان سلسلة النشر 191
19.11 تشغيل TensorFlow Serving 191
19.12 تطبيق طرفي: مسار اختبار كامل 191
19.13 المراقبة بعد النشر 192
19.14 خطة rollback جاهزة 192
19.15 مثال استدلال يطابق العقد 192
19.16 TFLite/LiteRT signatures 193
19.17 المتصفح وخصوصية المستخدم 193
19.18 اختبار الطرف إلى الطرف 193
19.19 خدمة API قابلة للتوسع 194
19.20 اختبار الحمل والفشل 194
19.21 تقييم الطرف إلى الطرف 194
19.22 مصفوفة توافق الإصدار 194

19.23 التوثيق للمستخدم النهائي 194
19.24 أخطاء شائعة 195
19.25 تمارين 195
19.26 ملخص 195
20 MLOps: TFX والمراقبة 196
20.1 أهداف الفصل 196
20.2 خط إنتاج له نسب بيانات وإصدارات 196
20.3 المراقبة ترى التغير لا الحقيقة وحدها 197
20.4 دورة إصدار قابلة للرجوع 198
20.5 حالة واقعية: تغير سلوك العملاء 198
20.6 أمثلة تنفيذية إضافية 199
20.7 خطوط التشغيل ليست مجرد رسم مربعات 199
20.8 مراقبة متعددة الطبقات 199
20.9 دورة إصدار تحمي من التحسن الزائف 200
20.10 إعادة التدريب والاستدامة 200
20.11 نموذج أولي وتدرج الاحتراف 200
20.12 قائمة قبول لنموذج جديد 201
20.13 تتبع التجارب والآثار 201
20.14 تصميم مراقبة الانجراف 201
20.15 مسؤوليات الفريق والاستدامة 202
20.16 مؤشرات تشغيلية وبوابات قرار 202
20.17 فشل pipeline والتعافي 202
20.18 اتفاقية تشغيل بين الفرق 202
20.19 أمان البيانات والنماذج 202
20.20 مستوى نضج تدريجي 203
20.21 حادث انجراف: سيناريو استجابة 203
20.22 تخزين النسب والاحتفاظ 203
20.23 بوابة قبول مرشحة 203
20.24 أخطاء شائعة 204
20.25 تمارين 204

20.26 ملخص 204
21 التعلم المعزز 205
21.1 أهداف الفصل 205
21.2 عملية قرار ماركوف 205
21.3 من Q-learning إلى DQN 206
21.4 تجربة آمنة في CartPole 207
21.5 حالة واقعية: روبوت المستودع 207
21.6 أمثلة تنفيذية إضافية 207
21.7 صياغة Bellmang MDP 208
21.8 Q-learning وDQN عمليًا 208
21.9 REINFORCE وPPO 209
21.10 تقييم قبل الواقع 209
21.11 متى لا تستخدم RL؟ 209
21.12 درس CartPole كتجربة علمية 209
21.13 ضبط المكافأة دون خلق تحايل 209
21.14 Replay وحالات نهاية الحلقة 210
21.15 قياس المتانة وقرار الانتقال للواقع 210
21.16 بيانات بشرية كخط أساس 210
21.17 انحياز وتغطية replay 210
21.18 وثائق بيئة قابلة للتكرار 211
21.19 فحص سياسة قبل نشرها 211
21.20 Offline RL والحذر من البيانات 211
21.21 خريطة تنقل للمبتدئ والباحث 211
21.22 مؤشرات فشل التدريب 211
21.23 التعلم من بيانات ثابتة 212
21.24 أخطاء شائعة 212
21.25 تمارين 212
21.26 ملخص 213
22 البيانات الجدولية والتوصيات والرسوم البيانية 214
22.1 أهداف الفصل 214

22.2 اعرف العمود قبل ترميزه 214
22.3 توصية على مرحلتين 215
22.4 الرسوم ومرور الرسائل 216
22.5 حالة واقعية: اقتراح المنتج التالي 216
22.6 أمثلة تنفيذية إضافية 216
22.7 معالجة الجدول ضمن النموذج 217
22.8 ترتيب التوصية من الطرف إلى الطرف 217
22.9 الرسم البياني وتمرير الرسائل 217
22.10 التعميم على فئات وعناصر جديدة 218
22.11 شجرة قرار مقابل شبكة عميقة 218
22.12 منع تسرب السمات 218
22.13 تجربة توصية من التفاعل إلى العرض 218
22.14 بناء GNN قابلة للتوسع 219
22.15 تغطية العناصر الجديدة والنتائج 219
22.16 خدمة الميزات والفهارس 219
22.17 مراقبة عرض التوصية 219
22.18 واجهة fallback واضحة 219
22.19 Feature store ومزامنة الزمن 220
22.20 حذف المستخدم والخصوصية 220
22.21 سجلات الدلالة وجودة الوسم 220
22.22 تحديث نظام التوصية 220
22.23 الاختبار بحسب السيناريو 220
22.24 أخطاء شائعة 221
22.25 تمارين 221
22.26 ملخص 222
23 الأخلاق والخصوصية والتفسير والأمان 223
23.1 أهداف الفصل 223
23.2 العدالة سؤال عن القرار والسياق 223
23.3 التفسير أداة فحص 224
23.4 أمان الخصومة والحوكمة 225

23.5 حالة واقعية: نموذج توظيف 225
23.6 أمثلة تنفيذية إضافية 225
23.7 تقييم عدالة القرار خطوةً خطوة 226
23.8 الخصوصية التفاضلية والتعلم الاتحادي 226
23.9 التفسير والتحقق من متانته 226
23.10 التهديدات: FGSM والتسميم وإساءة الاستعمال 227
23.11 بطاقة نموذج ومراجعة إطلاق 227
23.12 دراسة تدقيق: نموذج توظيف 227
23.13 الخصوصية في جمع البيانات والتسجيل 227
23.14 شرح النموذج وتدخل المشغل 228
23.15 نموذج تهديد مختصر 228
23.16 دورة الحوكمة بعد الإطلاق 228
23.17 مراجعة إطلاق موثقة 228
23.18 إدارة حادث أو اعتراض 229
23.19 مراجعة مستقلة للتغييرات الكبيرة 229
23.20 مقاييس متانة وخصوصية مفيدة 229
23.21 التوثيق بلغة المستخدم 229
23.22 مصفوفة مخاطر قبل الجمع والنشر 229
23.23 تصميم تفسير يتجنب الإفراط في الثقة 230
23.24 موازنة الخصوصية والفائدة 230
23.25 أخطاء شائعة 230
23.26 تمارين 231
23.27 ملخص 231
24 مشاريع ختامية وخارطة الطريق 232
24.1 أهداف الفصل 232
24.2 ما الذي يفرّق المشروع الختامي عن دفتر تجارب؟ 232
24.3 تصنيف الصور: حين تكون الدقة أسهل ما في المشروع 234
24.4 تنبؤ الطلب: الزمن لا يُخلط 236
24.5 تحليل المشاعر العربي: اللغة أصعب من النموذج 238
24.6 كشف الاحتيال: حين يكون الخطأ مكلفًا 240

24.7 ست حالات واقعية تكمل الأمثلة الأربعة 242
24.8 خارطة الطريق ومعرض الأعمال 247
24.9 أخطاء شائعة 249
24.10 تمارين 250
24.11 ملخص 251

الملاحق

مراجع وأدوات

A الرياضيات العملية 252
A.1 المتجهات والمصفوفات 252
A.2 التفاضل وقاعدة السلسلة 252
A.3 الاحتمال والتوقع 252
A.4 حساب يدوي صغير 253
A.5 أحجام الأشكال الشائعة 253
A.6 قائمة مراجعة 253
B دليل البيئة والأعطال 255
B.1 إعداد نظيف 255
B.2 التحقق 255
B.3 جدول الأعطال 256
B.4 GPU وWSL2 والحاويات 256
B.5 تقرير مشكلة مفيد 256
C المصطلحات: عربي ↔ إنجليزي 258
D مرجع TensorFlow/Keras السريع 269
D.1 التنسورات والتدرج 269
D.2 نموذج Keras 269
D.3 خسائر ومقاييس 270
D.4 خطوط البيانات والنشر 270
D.5 تذكير 271
E حلول التمارين المختارة 272
E.1 حل تمرين الأبعاد 272

E.2 حل تمرين الانحدار 272
E.3 حل تمرين مصفوفة الالتباس 272
E.4 حل تمرين عدم توازن الفئات 272
E.5 حل تمرين تقسيم السلسلة الزمنية 272
E.6 حل تمرين نقل التعلم 273
E.7 حل تمرين النشر الطرفي 273
E.8 حل تمرين المراجعة الأخلاقية 273
F المراجع والمصادر 274
F.1 الوثائق الرسمية 274
F.2 كتب تأسيسية 275
F.3 أوراق محورية 275
F.4 استخدام مجموعات البيانات 276
G فهرس الموضوعات 277

الجزء الأول

الأساسات: كيف تفكّر في TensorFlow؟

من التنسور الأول إلى أول نموذج كامل

01 TensorFlow: الحكاية والمنظومة والمكانة

02 بيئة 2026: Python 3.14 وTensorFlow 2.21

03 التنسورات: التفكير بالأبعاد

04 الاشتقاق التلقائي والرسوم البيانية

05 Keras: من الفكرة إلى النموذج

الفصول 1-5 · 5 فصول

الجزء الأول · الأساسات: كيف تفكّر في TensorFlow؟

الفصل الأول

01

TensorFlow: الحكاية والمنظومة والمكانة

> **حكاية الفصل**
>
> في مستشفى جامعي، أمضى فريقٌ من المهندسين ثلاثة أشهر في بناء نموذجٍ يكتشف الالتهاب الرئوي من صور الأشعة. على دفتر Jupyter، كانت الدقة 94%، وكانت الرسوم البيانية جميلة. ثم جاء يوم التركيب: جهازُ الأشعة لا يملك GPU، وشبكةُ المستشفى معزولة، وحجمُ النموذج أكبر من أن يُحمَّل على الجهاز الطرفي. سألني رئيسُ الفريق بهدوء: «نحن نعرف كيف نُدرّب. لكن هل نعرف كيف نُنشر؟». في تلك اللحظة أدركتُ أنّ TensorFlow ليس مكتبةً لتدريب النماذج فقط؛ إنّه **سلسلةُ أدواتٍ تُوصل الفكرةَ من المختبر إلى سرير المريض**. فما الحكاية؟ وكيف وصلنا إلى هنا؟

1.1 أهداف الفصل

- [ ] التمييز بين الذكاء الاصطناعي وتعلّم الآلة والتعلّم العميق.
- [ ] فهم مكان TensorFlow في المنظومة، وما الذي يميّزه عن PyTorch وJAX.
- [ ] معرفة مكوّنات المنظومة: Keras، tf.data، TFX، LiteRT، TF Serving، TF.js.
- [ ] بناء أول نموذج كامل بـ Keras وفهم دورة حياته.
- [ ] اتخاذ قرارٍ واعٍ: متى تختار TensorFlow، ومتى تختار غيره؟

1.2 ما الذكاء الاصطناعي؟ ثلاث دوائر متداخلة

تخيّل ثلاث دوائر متداخلة. الكبرى اسمها **الذكاء الاصطناعي (AI)**: كلُّ محاولةٍ لجعل الآلة تؤدّي مهمةً نعدّها «ذكية»، من برنامج الشطرنج القديم إلى روبوت المحادثة. داخل هذه الدائرة تأتي **دائرة تعلّم الآلة (ML)**: بدلًا من كتابة القواعد يدويًا، نُعطي الخوارزميةَ أمثلةً تستخرج منها القواعد. ثم تأتي الدائرة الصغرى، **التعلّم العميق (DL)**: تعلّمُ الآلة بشبكاتٍ عصبيةٍ متعدّدة الطبقات، قادرةٍ على اكتشاف أنماطٍ هرمية: الحافة، ثم الشكل، ثم الوجه، ثم الشخص.

لماذا هذا التمييز مهم؟ لأنّ كثيرًا من الأسئلة تختلط: «هل نحتاج TensorFlow؟» الجواب يعتمد على الدائرة التي تقف فيها. إن كانت مشكلتك جدولَ مبيعاتٍ بسيطًا، فقد يكفيك الانحدار الخطي أو شجرة قرار. وإن كانت صورًا أو نصوصًا أو إشاراتٍ معقّدة، فالتعلّم العميق يصبح خيارًا قويًا. **الأداةُ تتبعُ المشكلة، لا العكس.**

> ◆ **ملاحظة**
>
> التعلّم العميق ليس سحرًا. إنّه تقريبُ دوالٍ معقّدة عبر تحسين وسائط كثيرة. كلّما زادت البيانات والوسائط، زادت قدرة التقريب – وزاد خطر الحفظ الأعمى (overfitting).

1.3 من DistBelief إلى TensorFlow 2.21: لماذا كل هذه الأسماء؟

في بداية العقد الثاني من الألفية، كان باحثو Google يبنون شبكاتٍ عصبيةً ضخمة على أنظمةٍ داخلية. كان المشروعُ يُسمّى **DistBelief**، وكانت فكرتُه الأساسية: وزّعِ الحسابَ على آلاف الآلات. لكنّه كان صلبًا، مرتبطًا ببنيةٍ داخلية، ويصعبُ على الخارجيين استخدامُه.

ثم جاءت لحظةٌ فارقة: أراد الفريقُ لغةً واحدةً تصفُ الحسابَ الرياضيَّ كرسمٍ بيانيّ، ثم تُنفّذه على CPU أو GPU أو هاتف. في عام 2015 أُعلن **TensorFlow** مفتوحَ المصدر. كان الاسمُ مستوحى من «تدفّق التنسورات» عبر الرسم البياني. في الإصدار الأول، كانت الكتابةُ أشبهَ ببناء معبد: `graph` ، `placeholder` ، `Session` . قويةٌ جدًا، لكنّها قاسيةٌ على المبتدئ.

في 2019 جاء **TensorFlow 2.0**، فقلبَ المعادلة: التنفيذُ الفوري (eager execution) صار الافتراضي، وKeras صارت الواجهةَ الرسميةَ العليا، واختفتْ جلساتُ `Session` من الحياة اليومية. صار بإمكانك أن تكتب:

```python
import tensorflow as tf
x = tf.constant([1.0, 2.0, 3.0])
print(x * 2)   # tf.Tensor([2. 4. 6.], shape=(3,), dtype=float32)
```

هذه البساطةُ الظاهرة تخفي تحتها نظامًا عميقًا: يمكنك أن تُغلّفَ الدالةَ بـ `tf.function` فتُحوَّل إلى رسمٍ بيانيّ عالي الأداء، وأن تُصدّرَ النموذجَ إلى SavedModel، ثم إلى LiteRT ليعملَ على هاتف. منذ 2020 وحتى **2.21**، تراكمتْ طبقاتُ النضوج: دعمٌ أفضل للمحوّلات، وتكاملٌ أعمق مع Keras 3، وتحسيناتٌ في الأداء الموزّع، وأدواتُ LiteRT وTF Servingg.

الشكل التالي يضع المنظومةَ في صورةٍ واحدة:



شكل 1-1 خريطة منظومة TensorFlow: من البيانات إلى النشر

1.4 المنظومة: سبعة أجزاء تعمل معًا

حين تقول «TensorFlow»، فأنت لا تقصد ملفًا واحدًا. أنت تقصد عائلةً من المشاريع، لكلٍّ منها دور:

المكوّن	الدور	متى تستخدمه؟
TensorFlow Core	التنسورات، العمليات، التدرّج التلقائي، `tf.function`	حين تحتاج تحكّمًا منخفض المستوى
Keras 3	واجهة بناء النماذج والتدريب	في 90% من المشاريع
tf.data	خطوط بيانات عالية الأداء	عند التعامل مع صور/نصوص/ملفات كبيرة
TensorBoard	مراقبة التدريب والرسوم	لرؤية ما يحدث داخل النموذج
TFX	خطوط إنتاج النماذج وMLOps	عند الانتقال إلى الإنتاج
LiteRT (TFLite)	تشغيل النماذج على الأجهزة الطرفية	هاتف، Raspberry Pi، متحكّم دقيق
TF Serving / TF.js	خدمة النموذج على الخادم أو المتصفح	APIs، تطبيقات ويب، نشر واسع

هذه المنظومةُ هي ميزة TensorFlow الكبرى: **يمكنك أن تبدأ صغيرًا وتكبرَ دون أن تغيّر أدواتك**. النموذجُ الذي درّبتَه في دفترٍ يمكن أن يُصدَّرَ ويُخدمَ على خادم، ثم يُصغَّرَ ليعملَ على هاتف. لكنّها أيضًا مصدرُ إرباكٍ للمبتدئ: أسماءٌ كثيرة، وإصداراتٌ متداخلة، ودروسٌ قديمة. لذلك سنُبني الفهمَ طبقةً طبقة، ولن نستخدم أداةً قبل أن نعرفَ لماذا.

> ✦ **نصيحة**
>
> لا تُحاولْ تعلّمَ المنظومة كلها في أسبوع. ابدأ بـ `tf.constant` و `keras.Sequential` و `model.fit`، ثم أضفْ أداةً واحدةً كلّما احتجت. الأدواتُ تُتعلَّمُ بالألم، لا بالحفظ.

1.5 لماذا TensorFlow؟ خمسة أسباب بلا مبالغة

1. **النشرُ من اليوم الأول.** ليست الفكرةُ أن تُدرّبَ فقط، بل أن تُخرجَ نموذجًا يعملُ في بيئةٍ حقيقية. SavedModel وLiteRT وTF Serving يغطّون سلسلةً كاملة.
2. **نظامٌ بيئيٌّ ناضج.** توثيقٌ ضخم، أدواتُ مراقبة، تكاملٌ مع السحابات، ومجتمعٌ واسع.
3. **الأداءُ على المدى الطويل.** `tf.data` و `tf.function` وXLA والتدريبُ الموزّع مصمّمةٌ للمشاريع الكبيرة.
4. **الاستقرارُ في الإنتاج.** واجهاتُ الحفظ والخدمة مُختبرةٌ في أنظمةٍ ضخمة، مع دعمٍ للإصدارات.

5. **الجسرُ إلى الأجهزة الطرفية.** من الخادم إلى الهاتف إلى المتحكّم الدقيق، بلا إعادة كتابة النموذج من الصفر.

لكنّ هذه الأسبابَ نفسَها تحملُ ثمنًا: سطحٌ واسعٌ قد يُشتّت، وتغييراتٌ بين الإصدارات، وكودٌ منخفضُ المستوى أحيانًا. ولا عيبَ في الاعتراف بذلك.

1.6 TensorFlow أم PyTorch أم JAX؟

لا يوجد «أفضل» مطلق. يوجد **سياق**. هذا جدولٌ يختصرُ الفروق العملية:

المعيار	TensorFlow 2.21	PyTorch 2.x	JAX
سهولة التعلّم الأولى	متوسطة (Keras تُسهّلها)	عالية	منخفضة (نمط وظيفي)
مرونة البحث	جيدة	ممتازة	ممتازة جدًا
النشر على الأجهزة الطرفية	ممتاز (LiteRT)	جيد (ExecuTorch)	محدود
التدريب الموزّع	ناضج (Strategy)	ناضج	قوي جدًا (pmap/shard_map)
النظام البيئي للإنتاج	الأوسع	واسع	أصغر
التوثيق العربي	نادر	نادر	نادر جدًا

قاعدة القرار: إن كان هدفك نشرَ نموذجٍ على هاتفٍ أو خدمةٍ إنتاجيةٍ واسعة، فTensorFlow خيارٌ طبيعي. إن كنتَ في مختبر أبحاثٍ تُجرّب أفكارًا جديدة كلَّ يوم، فقد يكون PyTorch أو JAX أسرع. وإن كنتَ تحتاجُ تفاضلًا تلقائيًا متقدّمًا وتوازيًا هائلًا، فJAX يستحقُّ التجربة. والأهم: **المفاهيمُ تنتقل**. من يُتقنُ الانتشارَ الخلفيَّ والالتفافَ والانتباهَ يستطيعُ الانتقالَ بين الأدوات في أسابيع، لا سنوات.



شكل 1-2 المسار الزمني لتطوّر TensorFlow من 2015 إلى 2026

1.7 أول نموذج: خطٌّ مستقيم يتعلّم

لنبدأ بأصغر مثالٍ ممكن: نريدُ نموذجًا يتعلّم العلاقة بين سعرٍ بالدولار وسعرٍ بالريال. العلاقةُ خطية:

$$y = wx + b$$

حيث x السعرُ بالدولار، وy السعرُ بالريال، وw معاملُ التحويل (سعر الصرف)، وb التحييز. لا نعرفُ w وb؛ نريدُ أن يتعلّمهما النموذجُ من أمثلة. نُعرّفُ الخسارةَ بمتوسط مربع الخطأ:

$$L(w, b) = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)^2$$

حيث N عددُ الأمثلة، و$\hat{y}_i$ التنبؤ، وy_i القيمةُ الحقيقية. كلّما صغرتْ L، اقتربَ الخطُّ من النقاط. يحدّثُ المحسّنُ الوسائطَ في اتجاه عكس التدرّج:

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}, \qquad b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

وη هو معدّلُ التعلّم: حجمُ الخطوة. صغيرٌ جدًا = تعلّمٌ بطيء، كبيرٌ جدًا = قفزٌ فوق الحل.

والآن الكود الكامل:

```python
import numpy as np
import tensorflow as tf
import keras

keras.utils.set_random_seed(42)          # نتائج قابلة للتكرار قدر الإمكان

# 1) بيانات صغيرة: سعر بالدولار → سعر بالريال (علاقة خطية تقريبية)
x = np.array([[1.0], [2.0], [3.0], [4.0], [5.0]], dtype="float32")
y = np.array([[3.75], [7.50], [11.25], [15.00], [18.75]], dtype="float32")

# 2) النموذج: طبقة خطية واحدة (Dense) بلا دالة تنشيط
model = keras.Sequential([
    keras.Input(shape=(1,)),
    keras.layers.Dense(units=1),
])

# 3) التجميع: خسارة MSE، محسّن SGD بمعدل تعلّم صغير
model.compile(
    optimizer=keras.optimizers.SGD(learning_rate=0.05),
    loss=keras.losses.MeanSquaredError(),
    metrics=["mae"],
)

# 4) التدريب: 200 دورة على خمس نقاط
history = model.fit(x, y, epochs=200, verbose=0)

# 5) الفحص
print("W =", model.layers[0].get_weights()[0].ravel()[0])
print("b =", model.layers[0].get_weights()[1][0])
print("التنبؤ بسعر 7$ =", model.predict(np.array([[7.0]]), verbose=0)[0][0])
# النتيجة المتوقعة: W قريب من 3.75، b قريب من 26.25، والتنبؤ قريب من 0
```

قفْ عند كل سطر. `Input(shape=(1,))` تعني: كلُّ عيّنةٍ رقمٌ واحد. `Dense(1)` تعني: تحويلٌ خطيٌّ واحد. `compile` لا يُدرّب، بل يُجهّز. `fit` هو الذي يُشغّلُ الحلقة: تمريرٌ أمامي، حسابُ خسارة، تمريرٌ خلفي، تحديثُ أوزان. `predict` يُعيدُ القيمةَ المتوقّعة.

هذا المثالُ الصغيرُ يحتوي، في جوهره، كلَّ ما ستفعله في الشبكات العملاقة: بيانات، نموذج، خسارة، محسّن، تدريب، تقييم. الفرقُ أن الشبكةَ العميقةَ تضيفُ طبقاتٍ وتنشيطاتٍ وتنظيمًا، لا مبادئَ جديدة.

> ⚠ **تحذير**
>
> **عرض:** يبدأ المبتدئ بـ `model.fit` على بياناتٍ لا يفهمها، ثم يتفاجأ بأنّ الخسارة لا تنخفض. **السبب:** شكلُ البيانات أو نوعُها أو تطبيعُها غير مناسب. **الحل:** اطبعْ `y.shape` ، `x.dtype` ، `x.shape` ، وأولَ خمس قيم قبل أي تدريب.

راقب مجموعة تحقق واحفظ أفضل نسخة

بعد أن يعمل النموذج، اجعل قرار التوقف والحفظ جزءًا من الدورة نفسها بدل اختيار آخر حقبة يدويًا:

```python
callbacks = [
    keras.callbacks.EarlyStopping(monitor="val_loss", patience=3,
                                  restore_best_weights=True),
    keras.callbacks.ModelCheckpoint("best.keras", save_best_only=True),
]
history = model.fit(x_train, y_train, validation_split=0.2,
                    epochs=30, callbacks=callbacks)
```

1.8 متى لا تختار TensorFlow؟

هذا سؤالٌ لا يطرحُه المتحمّسون عادةً، لكنّه علامةُ النضوج:

- **بياناتٌ جدوليةٌ صغيرة:** غالبًا شجرةُ قرارٍ أو XGBoost/LightGBM أسرعُ وأدقُّ من شبكةٍ عميقة.
- **نموذجٌ كلاسيكي:** الانحدارُ اللوجستي أو SVM قد يكفي، وبلا تعقيد.
- **فريقٌ يعتمدُ PyTorch بالأساس:** لا تُغيّرِ الأداةَ لمجرد الذوق؛ التكلفةُ البشريةُ أكبرُ من الفرق التقني.
- **مهمةٌ تحتاجُ مكتبةً بحثيةً حديثةً جدًا:** بعض الأوراق تُنشرُ أولًا بـPyTorch/JAX، وقد يستغرقُ نقلُها وقتًا.
- **جهازٌ محدودٌ جدًا بلا دعمٍ للنشر:** راجعْ بدائلَ LiteRT وONNX أولًا.

1.9 حالة واقعية: عندما أنقذت LiteRT المشروع

عاد فريقُ المستشفى إلى النموذج. كان حجمُه 90 ميغابايت، ودقتُه 94%. جرّبوا تشغيلَه على جهاز الأشعة: بطيءٌ جدًا. الحلُّ لم يكن شبكةً أفضل، بل **طريقةَ نشرٍ أفضل**:

1. تصديرُ النموذج إلى SavedModel.
2. تحويلُه إلى LiteRT.
3. ضغطُ الأوزان إلى `float16`.
4. اختبارُ الدقة بعد الضغط: 93.6% – فرقٌ مقبول.
5. قياسُ الزمن: من 4.2 ثانية إلى 0.6 ثانية للصورة.

لم يتغيّر النموذجُ علميًا، لكنّه صار قابلًا للاستخدام. هذه هي قيمةُ المنظومة: **الفكرةُ الجيدةُ لا تكفي؛ يجب أن تصل**. في الفصل التاسع عشر ستُعيدُ هذه الرحلةَ بيدك، من `model.export` إلى خدمة FastAPI.

1.10 أخطاء شائعة

خطر

الخطأ: تعلّمُ TensorFlow بشراء كتابٍ وحفظِ الأوامر دون تشغيل. **النتيجة:** معرفةٌ هشّة تنهار عند أول خطأ شكل. **البديل:** اكتبْ كلَّ مثال، وأفسدْه عمدًا، ثم أصلحْه. الخطأُ المُفتعَلُ في المختبر أرحمُ من الخطأ الحقيقي في الإنتاج.

تحذير

الخطأ: استخدامُ `tf.keras` و `keras` بالتبادل دون معرفة الفرق. **التوضيح:** في TensorFlow 2.16+ أصبح Keras 3 هو الافتراضي، و `tf.keras` واجهةَ توافق. اكتبْ `import keras` في الكتاب، واعرفْ أن `tf.keras` قد يختلف في الإصدارات.

تحذير

الخطأ: توقّعُ أن GPU سيجعل كلَّ شيء أسرع. **الحقيقة:** إن كان خطُّ البيانات هو عنقَ الزجاجة، فـGPU سينتظر. حسّنْ `tf.data` أولًا.

1.11 تمارين

1. **تصنيف الدوائر:** صنّفْ خمسة تطبيقات (كشف spam، قيادة ذاتية، توصية أفلام، تشخيص طبي، ترجمة إلى AI/ML/DL، مع تبرير. (المتوقع: جدولٌ يوضّح أن كلَّها ML، وبعضها DL، مع تفسير.)
2. **النسخة:** شغّلْ `print(tf.__version__)` و `print(keras.__version__)` . (المتوقع: 2.21.0 و3.15.1 أو أحدث.)
3. **تعديل المثال:** غيّرْ معدّلَ التعلّم إلى 0.5 ثم 0.001. ماذا يحدث للخسارة؟ (المتوقع: 0.5 قد يتذبذب/ينفجر، و0.001 يتعلّم ببطء.)
4. **بلا تحييز:** احذفِ التحييز من النموذج باستخدام `use_bias=False` . هل يناسب البيانات؟ (المتوقع: نعم هنا، لأن الخط يمرّ بالأصل تقريبًا، لكن ليس دائمًا.)
5. **تحليل:** لماذا لا تُستخدم دقة التصنيف وحدها في بياناتٍ غير متوازنة؟ (المتوقع: لأن نموذجًا يتنبأ بالأغلبية قد يحقق دقة عالية دون أن يكتشف الأقلية.)
6. **بحث:** ابحثْ عن مكوّنٍ من منظومة TensorFlow لم يُذكر في الفصل، واكتبْ عنه ثلاثة أسطر. (المتوقع: مثل TF-GNN، TF Decision Forests، TF Privacy، TF Hub.)
7. **قرار:** فريقٌ يريدُ نموذجًا للهاتف خلال شهر. اكتبْ توصيتك في خمسة أسطر. (المتوقع: TensorFlow + LiteRT، مع خطة بيانات وتقييم ونشر.)

1.12 ملخص

- الذكاء الاصطناعي ⊃ تعلّم الآلة ⊃ التعلّم العميق، والأداةُ تتبعُ المشكلة.
- TensorFlow منظومةٌ كاملة: Core، Keras، tf.data، TensorBoard، TFX، LiteRT، Serving، TF.js.
- أقوى ما فيه هو **الجسر من التدريب إلى النشر**، لا مجرّد بناء النماذج.
- لا يوجد إطارٌ «أفضل» مطلقًا؛ يوجد سياقٌ ومعايير.
- أول نموذجٍ يكشف الحلقةَ الأساسية: بيانات ← نموذج ← خسارة ← محسّن ← تقييم.
- النجاحُ في الإنتاج يبدأ من فهم البيانات وطريقة النشر، لا من تعقيد المعمارية.

الجزء الأول · الأساسات: كيف تفكّر في TensorFlow؟

02

الفصل الثاني

بيئة 2026: Python 3.14 وTensorFlow 2.21

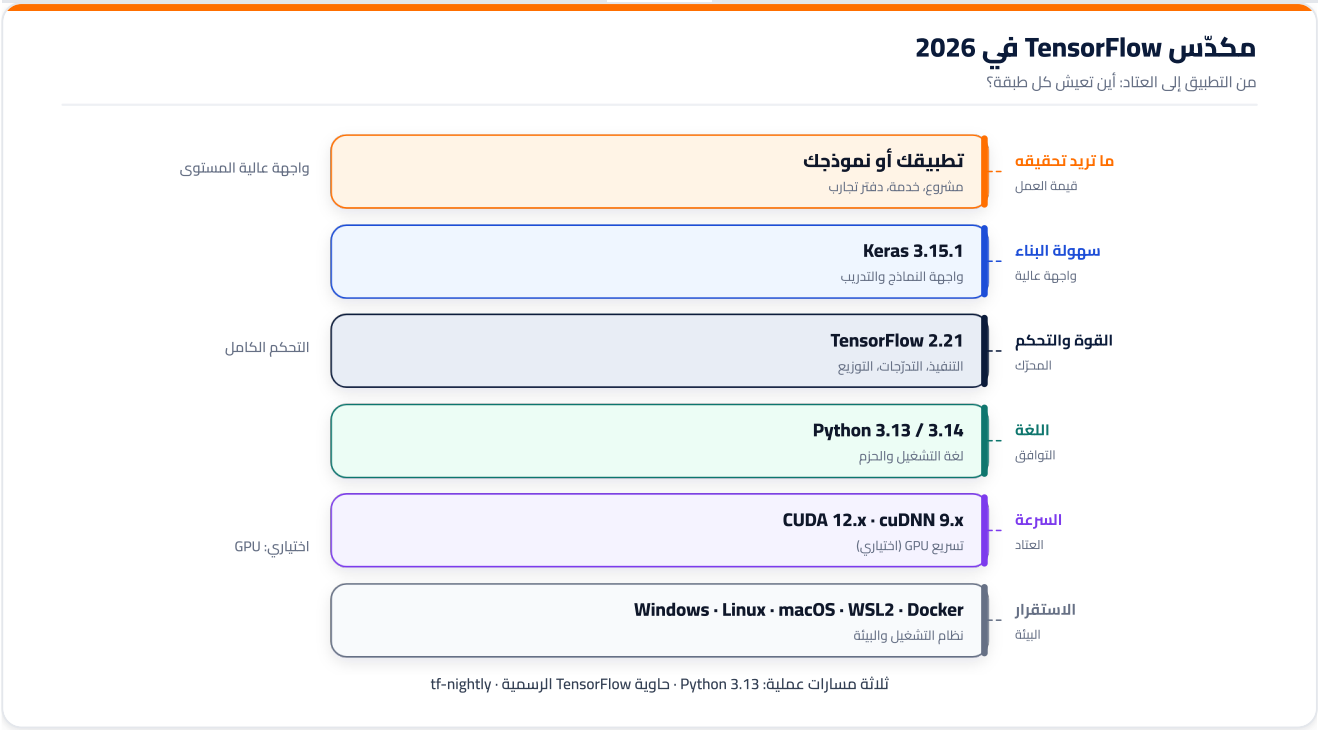
> **حكاية الفصل**
>
> وصل مهندس جديد إلى فريق الرؤية، وثبّت Python الأحدث على حاسوبه قبل يوم العرض. نجح تثبيت المحرّر، لكن حزمة TensorFlow لم تجد عجلة متوافقة. لم تكن المشكلة في بطاقة الرسوم ولا في الأمر الذي كتبه: كان إصدار Python نفسه خارج مصفوفة الدعم. كيف نختار بيئة قابلة لإعادة الإنتاج بدل مطاردة أحدث رقم؟

2.1 أهداف الفصل

- ☐ قراءة توافق Python وTensorFlow قبل التثبيت.
- ☐ إنشاء بيئة معزولة وتثبيت الحزم والتحقق منها.
- ☐ اختيار مسار CPU أو GPU أو حاوية أو بيئة سحابية.
- ☐ تشخيص أخطاء المكتبات والتعريفات الشائعة.

2.2 التوافق يسبق أمر التثبيت

لقطة توافق الطبعة — 24 أكتوبر 2026: الإصدار المستقر المرجعي هنا TensorFlow 2.21.0، وتدعم عجلاته الرسمية Python 3.10-3.13. أُعلن أيضًا عن TensorFlow 2.22.0-rc0 بوصفه إصدارًا مرشحًا، لكن الإصدار المرشح ليس اختيارًا افتراضيًا لبيئة إنتاج. مرجع Keras المستقر في أمثلة الكتاب 3.15.1؛ توافق Keras مع Python 3.14 لا يعني أن TensorFlow يوفّر عجلة رسمية لهذا المفسر. راجع تثبيت TensorFlow ومصفوفة Python، وإصدار TensorFlow 2.22.0-rc0، وإصدار Keras 3.15.1 قبل تثبيت بيئة جديدة.



شكل 1-2 طبقات بيئة TensorFlow من مفسر Python إلى المسرّع

حزمة TensorFlow ليست ملف Python خالصًا؛ تحتوي أجزاء مترجمة ترتبط بإصدار المفسر ونظام التشغيل والمعمارية. بحسب مرجع هذا الكتاب، TensorFlow 2.21.0 يوفر عجلات رسمية لـ Python 3.10-3.13. أما Python 3.14.6 فلا تعني تلقائيًا توفر عجلة متوافقة. افحص مصفوفة الإصدار على الموقع الرسمي وقت إعداد الجهاز، لأن التوافق يتغير أسرع من الكتب.

المسار	متى تختاره؟	ما الذي تتحقق منه؟
Python 3.13 داخل venv	تطوير محلي اعتيادي	عجلة TensorFlow ونسخة pip
حاوية رسمية	فريق يحتاج بيئة موحّدة	وسم الصورة ونسخة CUDA
WSL2 على Windows	عمل GPU ضمن بيئة Linux	تعريف NVIDIA وتوافق CUDA
Kaggle أو Colab	تجربة سريعة أو موارد مؤقتة	إصدار الحزمة ونوع المسرّع
tf-nightly	اختبار ميزة أو توافق مبكر	أنه إصدار ليلي متغيّر لا نشر إنتاج

المعادلة العملية هي أن البيئة القابلة للتكرار = إصدار مفسر معلوم + إصدارات حزم مثبتة + بيانات وإعدادات موثّقة. لا يكفي أن يعمل الاستيراد على جهاز شخص واحد.

2.3 أنشئ بيئة واعرف ما ثبّتّ

يمكن تلخيص شرط صلاحية التثبيت بأن يتقاطع دعم المفسّر والمنصة مع العجلات المنشورة للحزمة:

$$Environment_valid = Python_supported \wedge Platform_supported \wedge Dependencies_compatible$$

كل حد شرط مستقل: نجاح اثنين لا يعوض غياب الثالث.

```powershell
# على Windows: استخدم مفسرًا مدعومًا بدل الاعتماد على Python الافتراضي.
py -3.13 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install tensorflow==2.21.0 keras==3.15.1
python -m pip freeze > requirements-lock.txt
```

على Linux وmacOS يُفعّل المفسّر عبر `source .venv/bin/activate`. ويمكن استخدام `uv` لإنشاء البيئة وتثبيت الاعتمادات، بشرط تثبيت إصدارات صريحة للمشروع. لا تخلط بين `pip` الذي يعود إلى مفسّر آخر وبين البيئة النشطة؛ نفّذ `python -m pip` كي يكون الارتباط واضحًا.

```python
import sys
import tensorflow as tf

print(sys.version)
print(tf.__version__)
print(tf.config.list_physical_devices("GPU"))
# إصدار Python المتوقع: 2.21.0، ثم قائمة الأجهزة التي اكتشفها TensorFlow.
```

2.4 CPU وGPU: افصل بين التعريف والمكتبة

إن لم تظهر GPU، افحص أولًا أن نظام التشغيل يرى البطاقة وأن تعريف NVIDIA حديث ومدعوم. بعد ذلك طابق إصدار CUDA وcuDNN الذي تتطلبه نسخة TensorFlow؛ تثبيت CUDA عشوائيًا قد يجعل مكتبات النظام تتعارض. على Windows، المسار الشائع للتدريب المحلي ببطاقات NVIDIA هو WSL2 مع بيئة Linux، أو استخدام

حاوية موثّقة. لا تثبّت ملفات DLL من مصادر مجهولة لمعالجة رسالة تحميل مكتبة.

قبل الاعتماد على المسرّع، نفّذ عملية صغيرة عليه وقِس الزمن بعد الإحماء. ظهور اسم GPU يثبت الاكتشاف فقط، ولا يثبت أن النموذج أسرع: نقل البيانات، وصغر الدفعة، والعمليات غير المدعومة قد تجعل CPU أفضل.

2.5 تشخيص الأعطال

العرض	السبب المرجّح	خطوة التشخيص
`No matching distribution`	Python أو المعمارية غير مدعومة	افحص `python --version` ومصفوفة العجلات
`DLL load failed`	مكتبة أصلية أو runtime مفقود/متعارض	أنشئ venv نظيفة وافحص متطلبات النظام
استيراد NumPy يفشل	إصدار ABI غير متوافق	ثبّت نسخًا متوافقة من ملف القفل
GPU غير ظاهرة	تعريف أو بيئة تشغيل غير مكتملة	افحص الجهاز خارج TensorFlow ثم داخل الحاوية/WSL
نفاد VRAM	دفعة أو نموذج أكبر من الذاكرة	خفّض batch size وراقب الذاكرة

لا تعالج كل خطأ بإعادة تثبيت Python على النظام كله. سجّل نص الخطأ، والمفسّر النشط، ونسخ الحزم، ثم أعد التجربة في بيئة نظيفة يمكن حذفها دون إفساد مشاريع أخرى.

2.6 حالة واقعية: تجهيز فريق قبل بدء التدريب

يحتاج فريق جامعي إلى تدريب نموذج صور على حاسوبين مختلفين. يثبت الفريق Python 3.13 وTensorFlow 2.21g في بيئتين افتراضيتين، ويحفظ `requirements-lock.txt` ونسخة بيانات صغيرة وبذرة العشوائية. يخصص أحد الجهازين لـ CPU والآخر لـ GPU، ويقارن عدد الصور في الثانية بعد الإحماء. يصبح اختلاف الأداء معلومًا وقابلًا للتفسير، بدل أن يختلط بفرق إصدارات غير مسجل.

2.7 مصفوفة التثبيت العملية

لا توجد وصفة GPU واحدة صحيحة لكل أنظمة التشغيل. ثبّت نظام الأساس أولًا، ثم اتبع جدول الدعم الخاص بالإصدار الدقيق من TensorFlow. مرجع هذا الفصل TensorFlow 2.21.0 وKeras 3.15.1. يدعم TensorFlow 2.21 Python من 3.10 إلى 3.13 وفق مصفوفة الإنشاء؛ وجود دعم Python 3.14 في Keras لا يعني أن TensorFlow نفسه يملك عجلة رسمية لهذا المفسر. هذه نقطة يسهل أن تختلط على المبتدئ لأن `import keras` قد يعمل بينما `import tensorflow` لا يملك حزمة متوافقة. ملاحظات إصدار TensorFlow 2.21 وإعدادات البناء المدعومة.

النظام	CPU	GPU	قرار عملي
Windows أصلي	حزمة CPU على نسخ حديثة	دعم GPU الأصلي توقف بعد TensorFlow 2.10	استخدم WSL2 أو جهازًا سحابيًا لـ GPU
Linux x86-64	`tensorflow`	إعداد CUDA المعتمد لذلك الإصدار	طابق التعريف والمكتبات مع الدليل الرسمي
macOS	CPU	لا يوجد دعم CUDA رسمي لبطاقات Apple	استخدم CPU أو backend مناسبًا للعتاد
Colab/Kaggle	بيئة جاهزة	موارد سحابية بحسب الجلسة	لا تعتمد على بقاء القرص المؤقت

هذه مصفوفة قرار وليست قائمة كل الاستثناءات. الحزم المرتبطة بالعتاد تتغير؛ لا تنسخ أرقام CUDA من مقال قديم إلى إصدار جديد. على Windows، يجب التفريق بين تشغيل CPU على Windows وبين تسريع CUDA عبر Linux/WSL2. كما أن ظهور GPU في النظام لا يضمن أن كل العمليات أو كل أنواع البيانات تستخدمها.

2.8 ملف اعتماد قابل لإعادة الإنتاج

ملف `pip freeze` مفيد لتسجيل حل بيئة موجودة، لكنه ليس دائمًا أفضل ملف اعتماد بشري لمكتبة عامة؛ فقد يسجل حزمًا انتقالية خاصة بجهازك. للمشروع الصغير، اكتب الاعتمادات المباشرة بإصدارات مثبتة ثم أنشئ ملف قفل بأداة المشروع. سجّل نسخة Python ونظام التشغيل في README، لأن سطر الحزمة وحده لا يقفل العجلات الثنائية أو تعريف GPU.

```text
# requirements.in: الاعتمادات التي اختارها المشروع مباشرة
tensorflow==2.21.0
keras==3.15.1
numpy
```

في بيئة بحث، أضف تاريخ بناء البيئة، الجهاز، إعداد البذور، وطريقة الوصول إلى البيانات. لا تضع رموز الوصول أو كلمات المرور في ملف القفل. خزّن نسخة آمنة من ملف الاعتماد مع نتيجة التجربة، وأعد إنشاءها في بيئة نظيفة قبل مشاركة الورقة أو النموذج.

2.9 فحص GPU بطريقة منصفة

يتطلب تقرير الاستخدام أكثر من `list_physical_devices` . اجعل العملية كبيرة بما يكفي لإظهار عمل المسرّع، نفذها عدة مرات بعد الإحماء، واجمع زمنًا متزامنًا. بعض التنفيذات غير متزامنة؛ قياس وقت إرسال الأمر فقط يعطي زمنًا مضللًا. افحص كذلك ذاكرة الجهاز، وتأكد أن البيانات لا تُنسخ بين CPU وGPU في كل خطوة. ابدأ بـ CPU لاختبار صحة الشفرة، ثم قارن السرعة على حجم عمل واقعي.

2.10 التخطيط بين مبتدئ وباحث وفريق إنتاج

للمبتدئ، ابدأ ببيئة CPU نظيفة ومثال Keras صغير؛ لا تجعل إعداد CUDA شرطًا لفهم loss أو gradient. للباحث، ثبّت إصدارًا دقيقًا واحتفظ بملف البيئة وإعداد الجهاز مع الورقة أو الدفتر. لفريق الإنتاج، ابنِ صورة حاوية محددة بالـ digest، وافصل build environment عن runtime، واختبر تحميل النموذج من الصفر في بيئة معزولة. لكل مرحلة احتياج مختلف؛ لا تبدأ بخدمة Kubernetes عند حل واجب تصنيف.

إذا استخدمت Docker، مرر GPU من المضيف إلى الحاوية عبر إعداد runtime موثّق، وثبت tag للصورة لا `latest` . لا تخلط صورة TensorFlow التي تتضمن CUDA بإصدار تعريف غير متوافق على المضيف. سجّل `nvidia-smi` , نسخة الحاوية، `tf.__version__` , ومخرجات اختبار GPU؛ عند إعادة الإنتاج، لا تكفي عبارة «لدي بطاقة RTX». إذا كانت بياناتك خاصة، افحص mount والصلاحيات ومسار cache وملفات checkpoints قبل نشر الحاوية.

2.11 إدارة البيئة عبر عدة مشاريع

لا تضع TensorFlow في Python النظام. استخدم venv أو uv أو Conda وفق سياسة الفريق، ولا تخلط حزمًا ثنائية من قنوات مختلفة. أنشئ اسم بيئة صريحًا لكل مشروع، ونفذ التثبيت في CI على مفسر مدعوم. يمكن أن يثبت `pip check` تناقض metadata، لكنه لا يضمن أن كل kernel GPU يعمل. أضف smoke test قصيرًا في CI يستورد الحزم، ويبني نموذجًا صغيرًا، ويحفظه ثم يعيد تحميله.

عند ترقية TensorFlow، شغّل مقارنة على مجموعة تحقق ثابتة وسجل تغييرات API والحزم الملحقة.
TensorFlow 2.21 أزال الاعتماد الافتراضي على TensorBoard؛ إذا كان المشروع يستخدمه في callbacks أو تحليل ملفات، أضفه اعتمادًا مباشرًا وبنسخة معلومة بدل الاعتماد العرضي. وتراجع دعم Python 3.9 في هذا الإصدار. ملاحظات الإصدار الرسمية.

2.12 التثبيت ليس نهاية التحقق

بعد `pip install` , تحقق من أربع طبقات: مسار المفسر، إصدار TensorFlow/Keras، عملية حساب على CPU، ثم اكتشاف/استخدام GPU إن كان مطلوبًا. جرب تحميل ملف نموذج وحفظه في مسار يمكن للبرنامج الوصول إليه. نفذ تجربة صغيرة من ملف CSV أو صورة؛ مشكلات codecs ومكتبات الصور لا تظهر في `import` `tensorflow`. وإذا كان العمل متعدد الأجهزة، اختبر `strategy.num_replicas_in_sync` على الجهاز الحقيقي قبل تنفيذ تدريب طويل.

2.13 أمثلة تنفيذية إضافية

مفسّر Python

```python
import sys
print(sys.executable)
print(sys.version.split()[0])
```

عملية على CPU

```python
import tensorflow as tf
with tf.device("/CPU:0"):
    result = tf.reduce_sum(tf.ones((128, 128)))
print(float(result))  # المتوقع: 16384.0
```

2.14 أخطاء شائعة

⚠ تحذير

أحدث Python لا يعني أنه مدعوم من كل حزمة ذات امتدادات أصلية. تحقّق من العجلات قبل إنشاء المشروع. [DANGER!] لا تخلط CUDA وcuDNN من مصادر وإصدارات مختلفة اعتمادًا على رسالة خطأ منفردة؛ قد يبدو الاستيراد ناجحًا ثم يفشل التدريب.

2.15 تمارين

1. اختر إصدار Python مدعومًا في المصفوفة الحالية وثبّت البيئة. (المتوقع: تقرير نسخ يمكن إعادة بنائه.)
2. قارن `python -m pip --version` قبل تفعيل البيئة وبعده. (المتوقع: مساران مختلفان واضحان.)
3. افحص الأجهزة المرئية لـ TensorFlow. (المتوقع: CPU على الأقل، وGPU إن كان الإعداد مكتملًا.)
4. شخّص رسالة `No matching distribution` دون حذف Python العام. (المتوقع: تحديد عدم توافق المفسّر أو المنصة.)
5. قارن CPU وGPU بعد إحماء عمليتين متطابقتين. (المتوقع: قياس منصف لا يعتمد على أول تشغيل.)
6. اكتب ملف قفل وشارك خطوات إعادة إنشاء البيئة. (المتوقع: إعداد موثّق لفريق آخر.)

2.16 ملخص

- افحص توافق TensorFlow والمفسّر قبل التثبيت؛ مرجع هذا الكتاب لا يَعُد Python 3.14 مدعومًا بعجلة رسمية لـ 2.21.
- اعزل الاعتمادات وسجّلها، ثم تحقق من الأجهزة والإصدارات من داخل البيئة النشطة.
- GPU المكتشفة لا تضمن تدريبًا أسرع، وبيئة نظيفة أفضل من ترقيع تثبيت النظام.

الجزء الأول · الأساسات: كيف تفكّر في TensorFlow؟

الفصل الثالث

03

التنسورات: التفكير بالأبعاد

حكاية الفصل

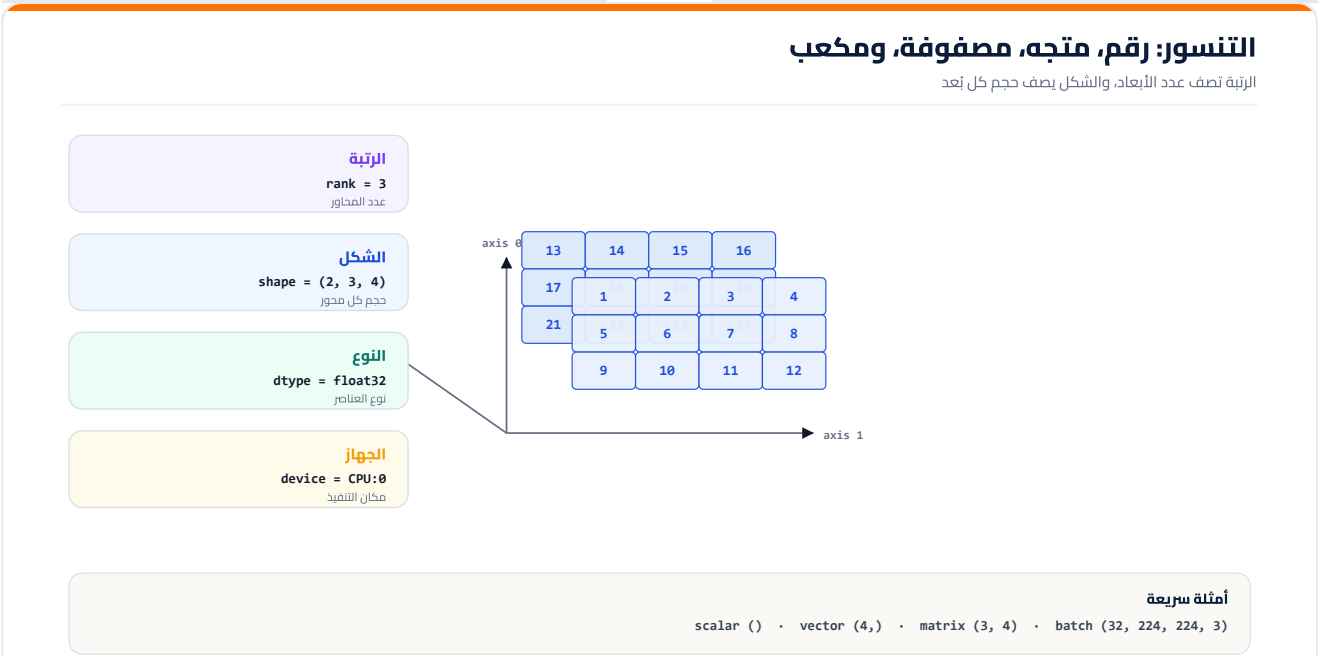
أرسل فريق متجرٍ صور المنتجات على هيئة دفعة، لكن النموذج رفض الإدخال. كل صورة منفردة كانت مقبولة، والبيانات لم تتغير. اختلف ترتيب الأبعاد فقط: توقّع النموذج دفعة ثم ارتفاعًا وعرضًا وقنوات، بينما وصلته القنوات في الموضع الثاني. كيف نقرأ شكل التنسور قبل أن نتهم النموذج؟

3.1 أهداف الفصل

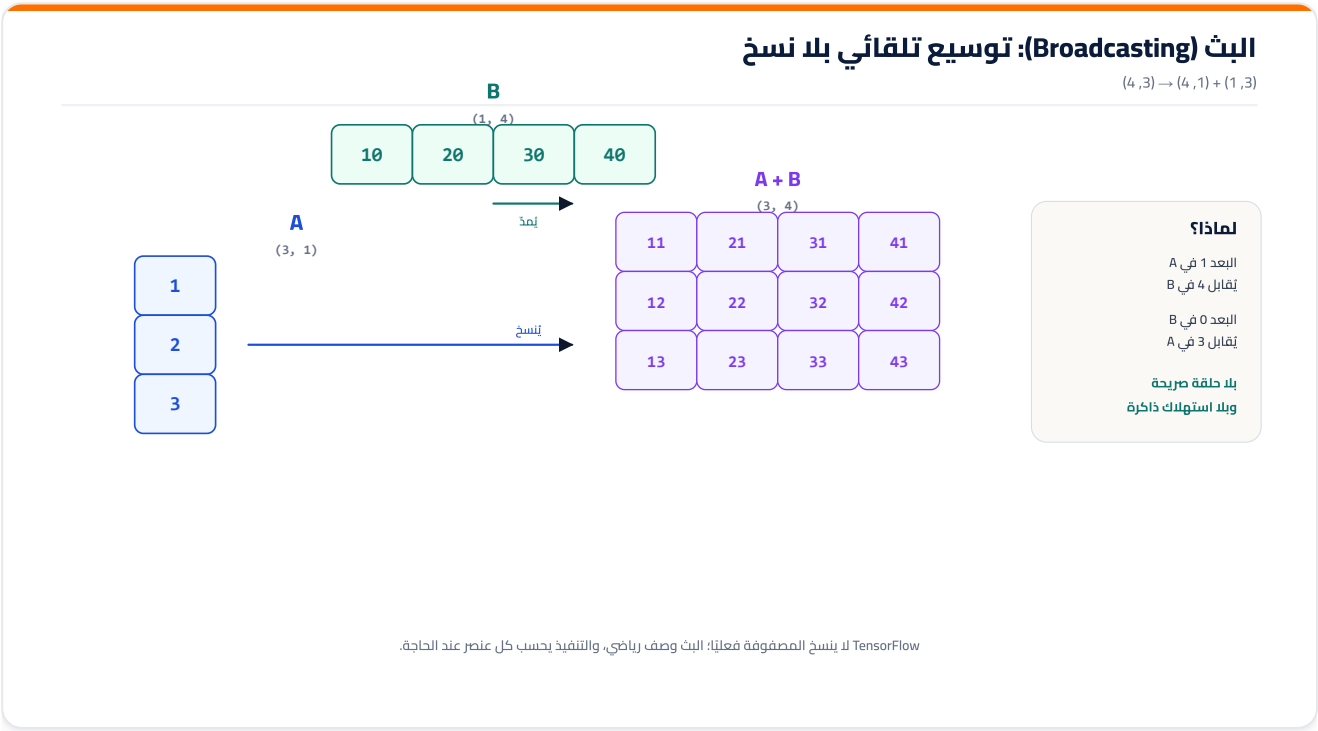
- ☐ التمييز بين الرتبة والشكل والنوع والجهاز.
- ☐ إنشاء التنسورات وفهرستها وإعادة تشكيلها ودمجها.
- ☐ فهم البث ومتغيرات التدريب والبيانات المتناثرة.
- ☐ تشخيص عدم توافق الأبعاد قبل التدريب.

3.2 أربعة أوصاف للتنسور

التنسور مصفوفة متعددة الأبعاد. الرتبة عدد محاوره، والشكل حجم كل محور، والنوع يحدد دقة التخزين والعمليات، والجهاز يحدد مكان التنفيذ. المتجه `[2, 5, 7]` رتبته 1 وشكله `(3,)`؛ ومصفوفة ثلاث صور رمادية قد يكون شكلها `(3, 28, 28, 1)`. البعد الأول هنا عدد الأمثلة، لا جزءًا من كل صورة.



شكل 3-1 الرتبة والشكل والنوع في التنسور



شكل 3-2 أمثلة توافق الأبعاد في البث

$$N_{elements} = \prod_{i=1}^{r} d_i$$

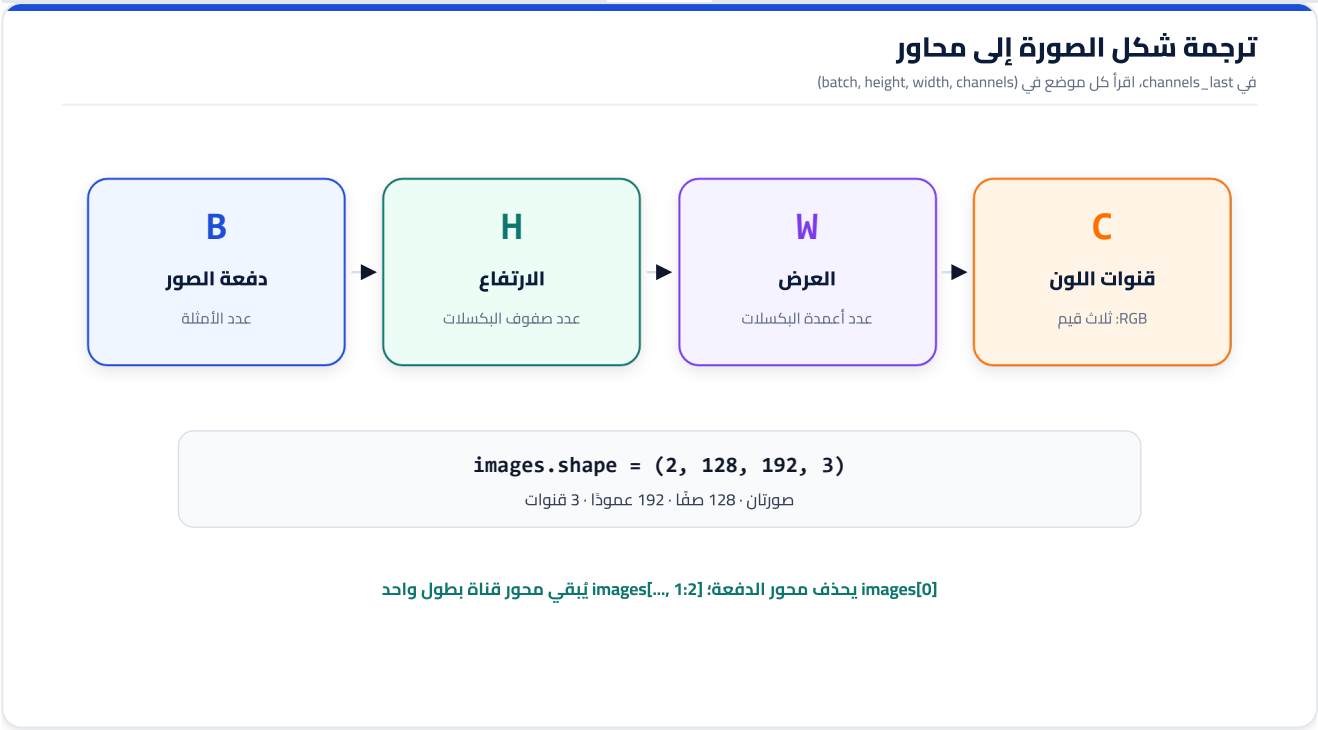
حيث `r` الرتبة و`d_i` طول المحور رقم `i`. يساعد عدد العناصر على اكتشاف إعادة تشكيل لا تغيّر البيانات: يجب أن يبقى حاصل الضرب نفسه.

```python
import tensorflow as tf

x = tf.constant([[1, 2, 3], [4, 5, 6]], dtype=tf.float32)
print(x.shape, x.dtype, x.ndim)
print(x[:, 1])
# المتوقع: (2, 3) و float32 ثم [2. 5.]، و2
```

3.3 اقرأ شكل الصورة قبل أن تدخل النموذج

ما معنى `(2, 128, 192, 3)`؟ لا تحفظه كسلسلة أرقام؛ اسأل عن كل محور: هل هذا عدد الأمثلة، أم ارتفاع الصورة، أم عرضها، أم قنوات اللون؟ وفق ترتيب `channels_last` الشائع في Keras، لدينا صورتان، ارتفاع كل منهما 128 بكسل، وعرضها 192، وفي كل بكسل ثلاث قيم للأحمر والأخضر والأزرق. إذا اخترنا صورة واحدة بقيت أبعادها الثلاثة، وإذا أخذنا القناة الخضراء وحدها تغيّر الشكل مرة أخرى.



شكل 3-3 ترجمة شكل الصورة إلى محاور مفهومة قبل تمريرها إلى النموذج

جرّب القراءة على بيانات صغيرة قبل تحميل مجموعة صور كبيرة. يطبع المثال شكل الدفعة والصورة المفردة والقناة الخضراء؛ وانظر كيف يحافظ التقطيع `[..., 1:2]` على محور قناة بطول واحد، بينما `[..., 1]` يحذفه.

```python
import tensorflow as tf

# محاكاة دفعة صور RGB بأعداد صحيحة بين 0 و255.
raw = tf.random.uniform((2, 128, 192, 3), maxval=256, dtype=tf.int32)
raw = tf.cast(raw, tf.uint8)
images = tf.image.convert_image_dtype(raw, tf.float32)

one_image = images[0]
green_without_axis = images[..., 1]
green_with_axis = images[..., 1:2]
print(images.shape, one_image.shape)
print(green_without_axis.shape, green_with_axis.shape)
# المتوقع: (3 ,192 ,128 ,2) و(3 ,192 ,128)
# ثم (192 ,128 ,2) و(1 ,192 ,128 ,2).
```

لماذا نهتم بمحور واحد؟ لأن طبقة `Conv2D` تحتاج أن تعرف أي محور يمثل القنوات. وإذا وصلها `(batch, channels, height, width)` بدل `(batch, height, width, channels)` فالأعداد نفسها موجودة، لكن معانيها تبدلت. افحص `element_spec` للدفعة، واطبع شكل عينة، واختبر ناتج التحويل قبل بدء التدريب. هذه الخطوات الصغيرة تمنع ساعات من البحث في خطأ سببه ترتيب الأبعاد.

3.4 إعادة تشكيل دون تغيير الترتيب

`reshape` يغيّر طريقة عرض العناصر في ترتيبها الحالي. أما `transpose` فيبدّل محاورها. إذا كان `images` بالشكل `(batch, height, width, channels)`، فإن التحويل إلى `(batch, channels, height, width)` هو تبديل محاور، لا إعادة تشكيل عمياء. استخدم الأسماء أو اطبع الأشكال عند كل حدّ بين خط البيانات والنموذج.

```python
import tensorflow as tf

x = tf.reshape(tf.range(12), (3, 4))
column = tf.expand_dims(x[:, 0], axis=-1)
joined = tf.concat([column, column], axis=1)
stacked = tf.stack([column, column], axis=0)
print(x.shape, column.shape, joined.shape, stacked.shape)
# المتوقع: (3, 4)، (3, 1)، (2, 3)، (1, 3, 2)
```

الدمج `concat` يزيد محورًا موجودًا، أما `stack` فينشئ محورًا جديدًا. اختيار المحور يحدد معنى الناتج: هل أضفنا ميزات إلى كل مثال أم أضفنا مجموعة أمثلة جديدة؟

3.5 البث: محاذاة المحاور من اليمين

يجوز تنفيذ عملية عنصر بعنصر عندما تتطابق الأبعاد أو يكون أحدها 1، بدءًا من المحور الأيمن. لذلك يمكن جمع متجه انحياز شكله `(features,)` إلى كل صف في دفعة شكلها `(batch, features)`. لكن الشكل `(batch,)` لا يطابق تلقائيًا `(batch, features)` كما قد يتوقع المرء؛ أضف محورًا صريحًا عند الحاجة.

```python
import tensorflow as tf

images = tf.zeros((8, 32, 32, 3))
mean = tf.constant([0.5, 0.4, 0.3])
normalized = images - mean
print(normalized.shape)
# المتوقع: (8, 32, 32, 3)، إذ بُثّ متوسط القنوات على المحاور السابقة.
```

3.6 قيم ثابتة وأوزان قابلة للتعلم

`tf.constant` مناسب للبيانات التي لا تتغير أثناء التدريب. `tf.Variable` يحمل حالة متغيرة مثل وزن طبقة، ويحدّثه المحسّن. لا تُنشئ وزنًا جديدًا داخل كل استدعاء للدالة؛ عند بناء طبقة Keras استخدم `add_weight` كي تتعقّب المنظومة الحالة وتحفظها.

البنية	تمثيلها	مثال الاستخدام
Tensor كثيف	تخزين كل الخانات	صورة أو دفعة سمات
RaggedTensor	أطوال صفوف متفاوتة	جمل بعدد كلمات مختلف
SparseTensor	قيم غير صفرية مع فهارس	مصفوفة تفاعلات قليلة الكثافة
Variable	Tensor ذو حالة قابلة للتحديث	أوزان النموذج

البيانات المتناثرة لا تصبح أصغر دائمًا في كل عملية؛ بعض العمليات تحوّلها ضمنيًا إلى تمثيل كثيف. اختبر الذاكرة والدعم الفعلي قبل اختيارها.

3.7 حالة واقعية: دفعة صور لا تدخل النموذج

نموذج `Conv2D` الشائع في Keras يتوقع `(batch, height, width, channels)` افتراضيًا. إذا كان الملف يأتي من نظام آخر بترتيب `(batch, channels, height, width)`، افحص `shape` أولًا ثم بدّل المحاور بـ `tf.transpose`. لا تستخدم `reshape` لإخفاء اختلاف ترتيب البيانات؛ ستتغير دلالة البكسلات حتى لو بقي عددها ثابتًا.

3.8 فحص الأشكال كعقد بين مكونات النظام

أخطاء الأبعاد ليست مجرد رسائل مزعجة؛ إنها إخفاق عقد بين المحمّل والتحويل والنموذج. وثّق لكل مدخل اسمه ونوعه وشكله ووحداته ومداه. على سبيل المثال، الصورة ليست `(224,224,3)` فقط: هل القنوات RGB أم BGR؟ هل القيم `[0,255]` أم `[0,1]`؟ هل يعني `None` حجم دفعة متغيرًا؟ كتابة هذه التفاصيل تمنع أن تقبل الخدمة Tensor صحيح الشكل لكنه يمثل بيانات مختلفة.

المحور المجهول مفيد للدُفعات المتغيرة، لكن كل الأبعاد ليست ديناميكية تلقائيًا. داخل `tf.function` قد يتخصص الرسم لشكل أول استدعاء، ثم يعيد التتبع عند شكل جديد. استخدم `TensorSpec` أو `input_signature` عندما تعرف عقدًا مرنًا، وراقب عدد عمليات التتبع إذا كان الأداء يتذبذب. لا تضع عمليات تحويل إلى NumPy داخل المسار المطلوب تتبعه؛ ذلك قد يكسر التدرج أو يمنع الرسم.

```python
import tensorflow as tf

signature = tf.TensorSpec(shape=(None, 4), dtype=tf.float32)
@tf.function(input_signature=[signature])
def normalize_batch(values):
    tf.debugging.assert_all_finite(values, "المدخل يحتوي NaN أو Inf")
    return tf.math.l2_normalize(values, axis=-1)

print(normalize_batch(tf.ones((3, 4)))).shape)
print(normalize_batch(tf.ones((12, 4)))).shape)
# كلا الحجمين يعيدان رسمًا واحدًا يقبل دفعة متغيرة.
```

3.9 النوع العددي والدقة والذاكرة

`float32` مناسب لمعظم أمثلة التدريب، لكن `float16` و`bfloat16` قد يسرعان العمليات على عتاد يدعمهما. الأعداد الصحيحة مناسبة للفئات والفهارس، لا للضرب الحسابي مع أوزان عائمة دون تحويل صريح. عند الانتقال بين الأنواع، افحص مجال القيم: تحويل float إلى int يقتطع الجزء الكسري، والتحويل إلى دقة أقل قد يقرب الرقم أو يفيض. لا تعتمد على تحويل ضمني عند حدود الخدمة.

يتناسب تخزين Tensor كثيف تقريبًا مع عدد العناصر مضروبًا في بايتات العنصر، لكن الذاكرة أثناء التدريب تشمل التفعيلات والتدرجات وحالة المحسن. لهذا قد ينفد VRAM مع Tensor دخل يبدو صغيرًا. عند التخطيط، احسب حدًا أوليًا ثم قس الذروة باستخدام Sparseg profiler. Ragged يقللان تمثيل الخانات غير المستخدمة في حالات مناسبة، لكن بعض العمليات قد تنشئ تمثيلًا كثيفًا داخليًا.

3.10 قناع الحشو في التسلسلات

حين تجمع جملًا متفاوتة الطول في Tensor مستطيل، تملأ الذيل برمز padding. هذا الرمز ليس كلمة حقيقية؛ إذا دخل في المتوسط أو الانتباه دون قناع، يتعلم النموذج طول الدفعة أو محتوى الحشو بدل النص.

`Embedding(mask_zero=True)` يمكن أن يولد قناعًا في بنى Keras التي تمرره، لكن عليك التحقق من أن الطبقات اللاحقة تحافظ عليه. اختبر جملة قصيرة وحدها ومع حشو إضافي، وقارن التمثيل قبل النشر.

3.11 أدوات تشخيص سريعة

العرض	الفحص	التصحيح المحتمل
InvalidArgumentError في الجمع	اطبع الشكلين والنوعين	أضف محورًا مقصودًا أو أصلح المصدر
عدد عناصر مختلف في reshape	احسب حاصل ضرب الأبعاد	استعمل `-1` لمحور واحد فقط
تتبع متكرر	ثبات نوع وشكل الإدخال	حدد `input_signature` أو قلل التنوع
GPU ذاكرة ممتلئة	حجم الدفعة والتفعيلات	خفّض الدفعة أو استخدم mixed precision
أداء يتغير مع الحشو	اختبر القناع	مرر mask وتأكد من استهلاكه

3.12 البث بالتفصيل: محور مفقود لا يساوي قيمة مفقودة

في البث، نقارن الأبعاد من اليمين إلى اليسار. `(5,1,3)` مع `(1,4,1)` ينتجان `(5,4,3)`: كل محور إما متساوٍ أو أحد جانبيه واحد. أما `(5,3)` مع `(5,)` فيحاذي 3 مع 5 من جهة اليمين، ويفشل غالبًا رغم أن المستخدم قصد جمع قيمة لكل صف. الحل هو إعادة شكل المتجه إلى `(5,1)` أو تحديد محور الميزات المقصود. لا تضف `expand_dims` قبل أن تشرح معنى المحور.

البث يوفر الذاكرة لأنه لا يكرر القيمة فعليًا في Tensor جديد بالضرورة، لكنه يوسع الحساب على عناصر كثيرة. عملية تطبيع غير مقصودة قد تولد مصفوفة هائلة أو تعطي نتيجة صحيحة الشكل وخاطئة المعنى. اكتب اختبارًا بأعداد صغيرة تستطيع حسابه يدويًا، ثم انتقل لحجم الإنتاج.

3.13 المتغيرات وسجل الحالة

`tf.Variable` يحتفظ بقيمة قابلة للتحديث، وله خصائص مثل `trainable` و`dtype` و`shape`. أوزان Keras تتبعها الطبقة وتحفظها، لكن Tensor عادي محفوظ محليًا لا يصبح وزنًا مدربًا. في حلقة مخصصة، مرر قائمة المتغيرات القابلة للتدريب إلى `tape.gradient`، وتأكد أن التدرج ليس `None`. للمتغيرات غير القابلة للتدريب، أضفها عبر واجهة الطبقة المناسبة لكي تدخل checkpoint.

المورد `RaggedTensor` يخزن صفوفًا متفاوتة الطول بوضوح، بينما `SparseTensor` يمثل الفهارس والقيم والعقد الكلي. استخدم Ragged للنصوص غير المحشوة أو التسلسلات المتغيرة عندما تدعم الطبقات العملية، واستخدم Sparse لمصفوفة تفاعلات قليلة كثافة إن كانت العمليات اللاحقة تدعمه. إنشاء بنية متفرقة لا يضمن أن Dense أو accelerator سينفذها بكفاءة؛ قد يحدث تحويل داخلي إلى dense.

3.14 الجهاز والنقل

يمكن أن يكون Tensor على CPU أو GPU أو TPU. `tf.device` يطلب جهازًا، لكن TensorFlow يختار تلقائيًا عند عدم تحديده. نقل موتر كبير بين CPU وGPU يتطلب وقتًا وذاكرة؛ جمع البيانات الصغيرة متقطعة من CPU داخل كل خطوة قد يهدر فائدة GPU. جهز دفعة متصلة عبر `tf.data` وحافظ على تدفق نقل واضح. لا تثبت مكان كل عملية يدويًا إلا لسبب قياس، فقد تمنع المجدول من اختيار تنفيذ أفضل.

3.15 اختبار خصائص الأشكال

اكتب assertions عند حدود البيانات: عدد الصفوف، ترتيب المحاور، النوع، المجال، وخلو القيم من NaN. اختبار property بسيط يجرب أكثر من حجم batch ويضمن أن النموذج لا يعتمد على حجم ثابت. عند خلط Ragged وسلاسل padded، افحص mask في كل طبقة. هذه الاختبارات أرخص من اكتشاف خطأ shape بعد ساعات تدريب أو داخل API.

3.16 مثال حساب يدوي للبث

لنفترض مصفوفة قياسها `(2,3)` وصف متوسط `(3,)`. يجمع TensorFlow المتوسط مع كل صف لأن آخر محور متوافق؛ كل عمود يمثل ميزة واحدة. إذا كان لكل صف عتبة مختلفة، فلا تستخدم `(2,)` مباشرة، بل شكل `(2,1)` كي يمد العمود على محور الميزات. عينة تحقق صغيرة تكشف المقصود:

```python
import tensorflow as tf

scores = tf.constant([[0.2, 0.8, 0.4], [0.9, 0.1, 0.5]])
row_thresholds = tf.constant([[0.5], [0.3]])
decisions = scores ⩾ row_thresholds
print(decisions.numpy())
```

3.17 أمثلة تنفيذية إضافية

تبديل محاور الصورة

```python
import tensorflow as tf
x = tf.zeros((4, 3, 32, 32))
y = tf.transpose(x, (0, 2, 3, 1))
print(y.shape)  # (4, 32, 32, 3)
```

تسلسلات متباينة الطول

```python
import tensorflow as tf
tokens = tf.ragged.constant([[4, 7, 2], [9, 1]])
print(tokens.row_lengths().numpy())  # [3 2]
```

3.18 توثيق عقد البيانات

احفظ مثال Tensor مرجعيًا وقيمته المتوقعة، واختبره على CPU وGPU. وثق التحويلات والنوع والمجال، وتحقق من أن preprocessing لا تستعمل label أو معلومة مستقبلية. في مشروع فريق، العقد المكتوب يوفر على كل مستخدم استنتاج ما يعنيه كل محور.

3.19 فهرسة آمنة للأمثلة

فرّق بين `x[i]` الذي يزيل محور batch لأول مثال و `x[i:i+1]` الذي يحافظ عليه. عند إرسال دفعة واحدة إلى خدمة، يبقى الشكل `(1,features)` بينما المثال المنفرد `(features,)`. اختبر الدالتين عند حدود التدريب والاستدلال حتى لا ينجح مسار ويفشل آخر.

3.20 سجلات المتغيرات

عند تغيير `Variable.assign`، تأكد من أن الدلالة توافق نوع الحالة. `tf.Variable` قد يكون scalar أو مصفوفة، ولا يحفظ وحده على القرص إلا ضمن checkpoint أو Keras layer. سجل القيم الابتدائية وإصداراتها إذا كانت جزءًا من reproducibility.

3.21 توحيد أنواع البيانات

عند جمع صورة float مع mean من int، حوّل النوع بقرار معلن. استخدم `tf.cast` فقط بعد تحديد معنى التحويل، وافحص القيم بعده. خلط `float16` و `float32` قد يسبب تحويلًا ضمنيًا أو خطأ dtype؛ صرّح بالنوع عند حد كل عملية.

3.22 أخطاء شائعة

> ⚠ **تحذير**
>
> `(32, 784)` دفعة من 32 مثالًا، بينما `(784,)` مثال واحد مسطّح. البعد الأول قد يمثل الدفعة ولا يدخل في [!DANGER] . `Input(shape=...)` تحويل نوع الأعداد بصمت قد يرفع الذاكرة أو يسبب عدم توافق. صرّح بـ `dtype` وافحصه عند حدود الإدخال.

3.23 تمارين

1. احسب رتبة وشكل Tensor يحوي 12 عنصرًا أعيد تشكيله إلى `(3, 2, 2)`. (المتوقع: الرتبة 3 و12 عنصرًا.)
2. اكتب فهرسة تستخرج القناة الثانية من دفعة صور. (المتوقع: شكل `(batch, height, width)`.)
3. اجمع انحيازًا من 4 ميزات على دفعة من 10 أمثلة. (المتوقع: `(10, 4)`.)
4. فسّر الفرق بين `stack` و `concat` على مصفوفتين. (المتوقع: يزداد عدد المحاور في الأولى.)

5. صحّح ترتيب محاور صورة قادمة بقنوات أولًا. (المتوقع: `(N, H, W, C)`.)
6. اختر Ragged أو Sparse لثلاث حالات وعلّل. (المتوقع: قرار مرتبط ببنية البيانات وكثافتها.)

3.24 ملخص

- اقرأ الرتبة والشكل والنوع والجهاز معًا.
- استخدم `transpose` لتبديل المحاور و `reshape` لتغيير التقسيم مع حفظ ترتيب العناصر.
- البث مفيد لكنه قد يخفي خطأً دلاليًا: افحص الأشكال عند حدود المكونات.

الجزء الأول · الأساسات: كيف تفكّر في TensorFlow؟

الفصل الرابع

04

الاشتقاق التلقائي والرسوم البيانية

حكاية الفصل

في شركةٍ للتوصيل، بنى فريقُ التوقّعات نموذجًا يتنبّأ بعدد الطلبات في الساعة. في الأسبوع الأول، كانت الخسارة تنخفض بانتظام، ثم توقّفت عند رقمٍ ثابت لا يتحرّك. أضاف مهندسٌ شابٌّ سطرًا واحدًا يطلب مراقبة المدخل، فعاد النموذج إلى التعلّم. بعد أيام، اشتكى الفريق من أنّ الحقبة الواحدة صارت تستغرق ثماني دقائق بدلًا من دقيقة، مع أنّ النموذج والبيانات لم يتغيّرا. كان الخطأ في مكانٍ آخر تمامًا: دالةٌ تُعرَّف من جديد في كل خطوة. فكيف يحسب TensorFlow المشتقة أصلًا؟ ولماذا يتحوّل الكود نفسه من سريعٍ إلى بطيء؟

4.1 أهداف الفصل

- [ ] فهم المشتقة وقاعدة السلسلة بلغةٍ عملية، لا برموزٍ مجرّدة.
- [ ] استخدام `tf.GradientTape` بثقة: المراقبة، والاشتقاق، والاشتقاق الثاني، والجاكوبي.
- [ ] بناء حلقة تدريب يدوية، ثم اشتقاق دالةٍ مركّبة والتحقق من النتيجة عدديًا.
- [ ] فهم التتبّع (tracing) في `tf.function`، وضبط `input_signature` و `reduce_retracing`.
- [ ] التمييز بين التنفيذ الفوري (eager execution) والرسم البياني (graph)، ومعرفة متى يفيد XLA.

4.2 المشتقة: لغة التغيير

تخيّل أنك تدير أسطولَ درّاجاتٍ للتوصيل، وتسأل: إذا أضفتُ سائقًا واحدًا، كم تزيد تكلفةُ الساعة؟ المعدّلُ الذي يجيب عن هذا السؤال هو **المشتقة (derivative)**: مقدارُ تغيّر المخرَج مقابل تغيّرٍ صغير جدًا في المدخل. لا يهمّنا التغيّرُ الكبير، لأنّ سائقًا واحدًا قد يُغيّر جدولَ العمل كله؛ يهمّنا الميلُ اللحظي عند نقطةٍ محدّدة.

رياضيًا، إذا كانت f قابلةً للاشتقاق عند x_0، فمشتقتها هناك هي نهاية معدّل التغيّر:

$$\frac{df}{dx}(x_0) = \lim_{h \to 0} \frac{f(x_0 + h) - f(x_0)}{h}$$

هنا x_0 النقطةُ التي نقيس عندها، وh إزاحةٌ صغيرة، و$f(x_0 + h) - f(x_0)$ التغيّرُ في المخرَج. القسمةُ على h تحوّل التغيّرَ إلى «تغيّرٍ لكل وحدة»، والنهايةُ تُصغّر h حتى يختفي أثرُ التقريب.

لكن الشبكةَ العصبية ليست دالةً واحدة، بل دالةٌ داخل دالة: طبقةٌ تُغذّي طبقة، وخسارةٌ تجمع مخرجات الطبقة الأخيرة. هنا نحتاج **قاعدة السلسلة (chain rule)**:

$$\frac{d}{dx} f\big(g(x)\big) = f'\big(g(x)\big) \cdot g'(x)$$

$g(x)$ هي القيمةُ الداخلية، وf' مشتقةُ الدالة الخارجية محسوبةً عند تلك القيمة، و$g'(x)$ مشتقةُ الدالة الداخلية. القاعدة تقول باختصار: **مشتقة الخارج × مشتقة الداخل**. وإن كانت $y = f(g(h(x)))$، كرّرنا الضربَ ثلاث مرات. الشبكةُ العميقة ليست إلا هذا التكرار مئات المرات.

لماذا لا نحسب المشتقات يدويًا؟ لأن عدد الوسائط قد يبلغ الملايين، ولكلّ وسيطٍ مشتقةٌ جزئية، ثم نجمعها في متجهٍ واحد هو **التدرّج (gradient)** الذي يشير إلى اتجاه أسرع زيادةٍ في الخسارة. السيرُ عكسه هو جوهر التدريب. الحسابُ اليدوي ممكن لشبكةٍ من طبقتين، ومستحيل لشبكةٍ من مئة طبقة. لذلك يحتاج TensorFlow إلى آلةٍ تحسب المشتقة نيابةً عنك، بدقّة الرموز وسرعة الأرقام.

مثال عددي: لماذا يشير التدرّج إلى تعديل الوزن؟

لنفترض أن النموذج يقدّر أجرة توصيل خطية: `prediction = weight × distance` . لمسافة مقدارها 2، ووزن حالي يساوي 3، وتكلفة صحيحة تساوي 10، يكون التنبؤ 6 والخطأ `6 - 10 = -4` . نستخدم مربع الخطأ، فتكون الخسارة 16. مشتقة الخسارة بالنسبة إلى الوزن هي `2 × (-4) × 2 = -16` ؛ والإشارة السالبة تقول إن زيادة الوزن قليلًا تخفض الخسارة عند هذه النقطة.

```python
import tensorflow as tf

weight = tf.Variable(3.0)
distance = tf.constant(2.0)
target = tf.constant(10.0)

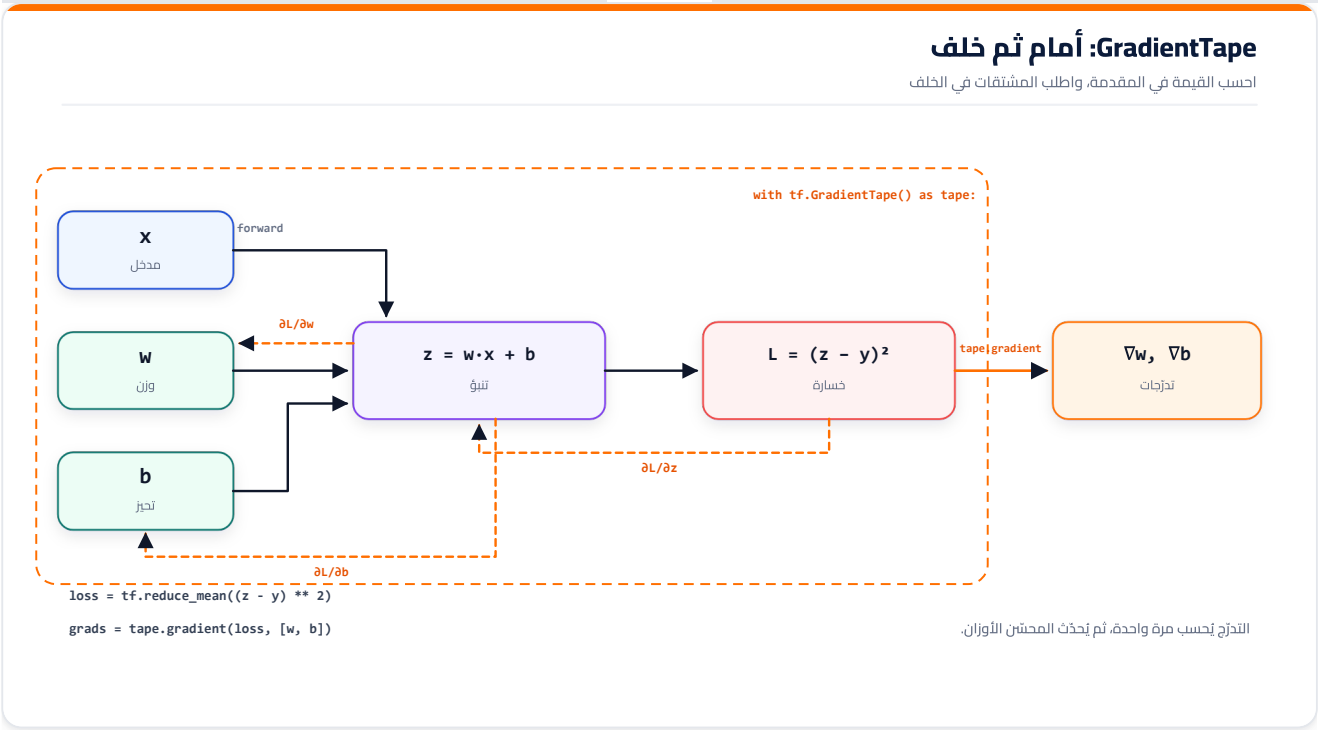
with tf.GradientTape() as tape:
    prediction = weight * distance
    loss = tf.square(prediction - target)

gradient = tape.gradient(loss, weight)
print("prediction:", prediction.numpy())  # 6.0
print("loss:", loss.numpy())              # 16.0
print("d(loss)/d(weight):", gradient.numpy())  # -16.0
```

لو طبقنا خطوة تحديث بمعدل تعلم `0.1`، يصبح الوزن `3 - 0.1 × (-16) = 4.6`، أي يتحرك في اتجاه يقلل الخسارة المحلية. لكن هذا المعدل كبير نسبيًا وقد يتجاوز أفضل قيمة؛ لذلك يضبط المحسّن حجم الخطوة، ويتكرر التحديث على أمثلة كثيرة بدل الاعتماد على عينة واحدة.

4.3 الشريط: كيف يتذكّر TensorFlow؟

في كل مرة تُنفّذ عمليةٌ على تنسور، لا يحسب TensorFlow المشتقةَ فورًا. بل يسجّل العمليةَ ومدخلاتها على **شريط (tape)**، تمامًا كما يسجّل محاسبٌ كلَّ حركةٍ في دفتره. بعد انتهاء الحساب الأمامي، يمرّ على الدفتر بالعكس: من الخسارة إلى الأوزان، ويطبّق قاعدة السلسلة عند كل خطوة. هذه الطريقة تُسمّى **الاشتقاق العكسي (reverse-mode automatic differentiation)**، وكلفتُها قريبةٌ من كلفة تمريرةٍ أمامية واحدة، مهما كان عدد الوسائط. لهذا تستطيع تدريبَ نموذجٍ بملايين المعاملات بحسابٍ واحدٍ للتدرّج.



شكل 4-1 الاشتقاق التلقائي: تمرير أمامي وخلفي عبر الرسم البياني

للشريط قواعد قليلة، لكنّها مصدرُ معظم الأخطاء:

1. **المتغيّرات تُراقَب تلقائيًا.** أي `tf.Variable` تدخل في الحساب يُسجَّل تدرّجها دون طلب.
2. **الثوابت لا تُراقَب.** إن أردت الاشتقاق بالنسبة إلى تنسورٍ عادي، فعليك `tape.watch(x)` صراحةً قبل الحساب.
3. **الشريط غير الدائم يُستهلك مرة واحدة.** استدعاء `tape.gradient` مرةً ثانية يرفع خطأً، إلا إذا أنشأته بـ `persistent=True` ثم حرّرته بـ `del`.
4. **الاشتقاق الثاني يحتاج شريطين متداخلين.** الخارجي يراقب حسابَ التدرّج نفسه.

والآن لنرَ ذلك في كودٍ واحد:

```python
import tensorflow as tf
import keras

keras.utils.set_random_seed(42)

# (1) متغيّر: الشريط يراقبه تلقائيًا
x = tf.Variable(2.0)
with tf.GradientTape() as tape:
```

```
    y = x ** 3 + 2.0 * x            # y = 12 عند x = 2
print("dy/dx =", tape.gradient(y, x).numpy())   # 14.0  (3x² + 2)

# (2) ثابت: لا يُراقَب إلا بطلبٍ صريح
c = tf.constant(3.0)
with tf.GradientTape() as tape:
    y2 = c ** 2
print("بلا watch:", tape.gradient(y2, c))       # None

with tf.GradientTape() as tape:
    tape.watch(c)
    y2 = c ** 2
print("مع watch:", tape.gradient(y2, c).numpy())  # 6.0

# (3) شريط دائم: يسمح بأكثر من اشتقاق
with tf.GradientTape(persistent=True) as tape:
    a = tf.Variable(1.5)
    f = a ** 2
    g = a ** 3
print("df/da =", tape.gradient(f, a).numpy())   # 3.0
print("dg/da =", tape.gradient(g, a).numpy())   # 6.75
del tape                                        # حرّر الموارد

# (4) المشتقة الثانية: شريطان متداخلان
x2 = tf.Variable(2.0)
with tf.GradientTape() as outer:
    with tf.GradientTape() as inner:
        y3 = x2 ** 3
    first = inner.gradient(y3, x2)              # 12.0
second = outer.gradient(first, x2)
print("d²y/dx² =", second.numpy())              # 12.0

# (5) جاكوبي: دالة من R² إلى R³
v = tf.Variable([2.0, 3.0])
with tf.GradientTape() as tape:
    out = tf.stack([v[0] ** 2, v[0] * v[1], v[1] ** 3])
J = tape.jacobian(out, v)
print("shape =", J.shape)                       # (3, 2)
print(J.numpy())
# [[ 4.  0.]
#  [ 3.  2.]
#  [ 0. 27.]]
```

لاحظ الفرق بين `gradient` و`jacobian`: الأول يعيد متجهًا بحجم المدخل عندما يكون المخرَج رقمًا واحدًا (خسارةً مثلًا)، والثاني يعيد مصفوفةً كاملة عندما يكون المخرَج متجهًا. في التدريب نستخدم الأول غالبًا، لأنّ الخسارة عددٌ واحد؛ أمّا الثاني فيظهر في حساب مصفوفة المعلومات أو في تحليل حساسية مخرجاتٍ متعدّدة.

4.4 من الاشتقاق إلى التدريب

الآن لنُغلق الحلقة. أمامك مهمةٌ بسيطة: شركةُ شحنٍ تريد معادلةً تربط وزنَ الطرد بتكلفة توصيله. العلاقة خطية تقريبًا، لكنّ القياسات تحمل ضجيجًا. نموذجنا:

$$\hat{y} = wx + b$$

حيث $\hat{y}$ التكلفةُ المتوقّعة، وx الوزن، وw سعرُ الكيلوغرام، وb رسمٌ ثابت. نُدرّب النموذج بتقليل متوسط مربع الخطأ:

$$L(w, b) = \frac{1}{N} \sum_{i=1}^{N} \left(w x_i + b - y_i \right)^2$$

N عددُ الطرود، وy_i التكلفةُ الحقيقية للطرد i. الأسُّ التربيعي يعاقب الأخطاء الكبيرة بشدّة، والمتوسط يجعل الخسارةَ مستقلةً عن حجم الدفعة.

في Keras المعتاد، يتولّى `fit` كلَّ شيء. لكن لفهم ما يجري، سنكتب الحلقةَ بأيدينا:

```python
import numpy as np
import tensorflow as tf
import keras

keras.utils.set_random_seed(42)

# بيانات: تكلفة الشحن تزيد خطيًا مع الوزن، مع ضجيج بسيط
rng = np.random.default_rng(42)
x = rng.uniform(0.5, 5.0, size=(200, 1)).astype("float32")
y = 4.0 * x + 2.5 + rng.normal(0.0, 0.8, size=(200, 1)).astype("float32")

# وسطان يُتعلّمان: نبدأ من الصفر
w = tf.Variable(0.0, name="w")
b = tf.Variable(0.0, name="b")
optimizer = keras.optimizers.SGD(learning_rate=0.02)

for epoch in range(200):
    # كل خطوة تحتاج شريطًا جديدًا: الشريط يُستهلك بعد gradient
```

```
    with tf.GradientTape() as tape:
        y_pred = w * x + b
        loss = tf.reduce_mean(tf.square(y_pred - y))

    # التدرّج بالنسبة إلى الوسيطين معًا
    dw, db = tape.gradient(loss, [w, b])

    # المحسّن يطبّق: variable -= learning_rate * gradient
    optimizer.apply_gradients(zip([dw, db], [w, b]))

    if epoch % 50 == 0:
        print(f"epoch {epoch:3d} | loss = {loss.numpy():.3f}")

print("w =", float(w.numpy()), "| b =", float(b.numpy()))
# المتوقع: loss ينخفض من نحو 200 إلى أقل من 1،
# w يقترب من 4.0، و b يقترب من 2.5
```

ثلاثة أسطر تستحقّ التوقّف. `tape.gradient(loss, [w, b])` يعيد قائمةً بنفس ترتيب الطلب. `zip` يزاوج كلَّ تدرّجٍ مع متغيّره. و`apply_gradients` لا يعرف شيئًا عن الشبكات؛ إنّه فقط يقرأ التدرّج ويُحدّث المتغيّرات. هذا هو التدريب في أعمق صوره: لا أكثر.

والآن دالةٌ مركّبة، حيث تظهر قاعدة السلسلة بوضوح. لتكن $h(x) = \sin(x^2)\, e^{x/3}$. لا نعرف مشتقّتها بالحدس، لكن يمكننا التحقق من الإجابة بطريقتين: الشريط، والفرق العددي:

python

```
import tensorflow as tf

def h(x):
    return tf.sin(x ** 2) * tf.exp(x / 3.0)

x0 = tf.Variable(1.2)
with tf.GradientTape() as tape:
    value = h(x0)
dy = tape.gradient(value, x0)

# تحقق عددي: (f(x+ε) - f(x-ε)) / 2ε
eps = 1e-3
numeric = (h(tf.constant(1.2 + eps)) - h(tf.constant(1.2 - eps))) / (2.0 * eps)

print("autodiff =", round(float(dy), 6))      # 0.95999
print("numeric  =", round(float(numeric), 6))  # 0.95999 تقريبًا
# المتوقع: القيمتان متطابقتان إلى خمس منازل
```

التطابق ليس صدفة: الشريط طبّق قاعدة السلسلة على $\sin$ وx^2 و$\exp$ بالترتيب، والفرقُ العددي قرّب النهايةَ نفسها. الفرقُ أنّ الأول دقيقٌ وسريع، والثاني يعاني خطأَ التقريب ويتطلّب استدعاءين لكلّ مشتقة. في الشبكات الكبيرة، الفرقُ بين الطريقتين هو الفرق بين دقائق وأسابيع.

4.5 tf.function: الرسم البياني الذي يُبنى مرة واحدة

حتى الآن، كان TensorFlow ينفّذ كلَّ سطرٍ فور كتابته. هذا مريح للتصحيح، لكنه مكلف: كلُّ عمليةٍ صغيرة تُرسَل إلى المحرّك على حدة، فيضيع وقتٌ كبير في التنسيق بدل الحساب. الحلُّ هو `tf.function`: تُغلّف الدالة، فينفّذها TensorFlow مرةً واحدة لتسجيل عملياتها في **رسم بياني (graph)**، ثم يعيد استخدام الرسم في الاستدعاءات التالية.



شكل 4-2 tf.function: من التنفيذ الفوري إلى الرسم البياني

تلك المرّة الأولى تُسمّى **التتبّع (tracing)**. أثناءها يمرّ بايثون على أسطر الدالة، ويُحوّل كلَّ تنسورٍ إلى موضعٍ في الرسم. وبعدها لا يعود بايثون إلى العمل؛ الرسمُ وحدَه هو الذي يُنفَّذ. من هنا تأتي قاعدةٌ ذهبية: **الأشكال المختلفة تعني تتبّعًا جديدًا**. استدعِ الدالة بدفعةٍ من 32 صفًا، ثم بـ64، فتحصل على رسمين. استدعِها بألف شكلٍ مختلف، فتحصل على ألف رسم، وذاكرةٌ تنتفخ بلا فائدة.

المعيار	التنفيذ الفوري (eager)	الرسم البياني داخل `tf.function`
سهولة التصحيح	عالية: كل سطر يُنفَّذ ويمكن طباعته	أقل: الأخطاء تظهر وقت التتبّع
سرعة الحلقات الكثيفة	أبطأ بسبب كلفة الإرسال لكل عملية	أسرع: العمليات تُدمج وتُنفَّذ كوحدة
المرونة في الأشكال	كل استدعاء حرّ	شكل جديد = تتبّع جديد
آثار بايثون (`print` مثلًا)	تُنفَّذ كل مرة	تُنفَّذ مرة واحدة وقت التتبّع

هذا الجدول يفسّر مفارقةً يقع فيها كثيرون: يضعون `print(loss)` داخل دالةٍ مُغلّفة، فيرون الرقم مرةً واحدة ثم يتوقّفون عن رؤيته. ليست الخسارة ثابتة؛ الطباعةُ هي التي نُفّذت مرةً واحدة. للحصول على طباعةٍ في كل خطوة، استخدم `tf.print` التي تتحوّل إلى عمليةٍ داخل الرسم.

ولضبط التتبّع، هناك أداتان: `input_signature` تُعلن مسبقًا شكلَ المدخل ونوعه، فيُبنى رسمٌ واحد يقبل أي حجمٍ في البُعد المعلن `None` . و `reduce_retracing=True` تسمح لـ TensorFlow بتخفيف الأشكال تلقائيًا بدل بناء رسمٍ لكل حجم. أمّا **AutoGraph** فهو المحوّل الذي يجعل حلقات `for` وشروط `if` في بايثون تعمل داخل الرسم، ويمكنك رؤية الناتج بـ `tf.autograph.to_code(my_function)` .

python

```python
import tensorflow as tf
import keras

keras.utils.set_random_seed(42)

traces = {"count": 0}

@tf.function
def scale(x):
    traces["count"] += 1                # يُنفَّذ وقت التتبّع فقط
    tf.print("trace:", traces["count"])  # tf.print يُنفَّذ وقت التشغيل
    return x * 2.0 + 1.0

print(scale(tf.constant([1.0, 2.0])).numpy())        # [3. 5.]
print(scale(tf.constant([1.0, 2.0, 3.0])).numpy())   # [3. 5. 7.]
print("عدد التتبعات =", traces["count"])             # 2

# بصمة ثابتة: شكل مجهول في البُعد الأول، فتتبع واحد يكفي
traces2 = {"count": 0}

@tf.function(input_signature=[tf.TensorSpec(shape=(None,), dtype=tf.float32)])
def scale_fixed(x):
```

```
    traces2["count"] += 1
    return x * 2.0 + 1.0

scale_fixed(tf.constant([1.0]))
scale_fixed(tf.constant([1.0, 2.0, 3.0]))
print("عدد التتبعات =", traces2["count"])            # 1

# reduce_retracing: سمح بتخفيف الأشكال بدل رسمٍ لكل حجم
@tf.function(reduce_retracing=True)
def normalize(x):
    return (x - tf.reduce_mean(x)) / (tf.math.reduce_std(x) + 1e-7)

print(normalize(tf.constant([1.0, 2.0, 3.0])).numpy())
# المتوقع: [-1.2247  .0  1.2247]
```

إن احترت في سلوكٍ غريب، فعّل التنفيذ الفوري داخل الدوال مؤقتًا بـ `tf.config.run_functions_eagerly(True)`. سيتوقّف التتبّع، وسترى كلَّ سطرٍ يعمل، لكنّ الأداء سينخفض. استخدمها للتشخيص ثم أعدها إلى `False`؛ لا تتركها في كود الإنتاج.

4.6 XLA: حين يُعاد تشكيل الرسم

بعد أن صار لدينا رسمٌ بياني، يمكننا أن نطلب من مُصرّف **XLA** (اختصار Accelerated Linear Algebra) أن يُعيد كتابته. الفكرةُ أنّ العمليات الصغيرة المتتالية، مثل `x * 2 + 1` ثم `reduce_mean` ثم `square`، تُدمج في نواةٍ واحدة تُنفَّذ دفعةً واحدة، فتنخفض زياراتُ الذاكرة. هذا هو **اندماج العمليات (operator fusion)**، وهو أكبر مكسبٍ يأتي من XLA.

تفعيلُه بسيط: `tf.function(jit_compile=True)`، أو `model.compile(..., jit_compile=True)` في Keras. لكنّه ليس مفتاحًا سحريًا:

- **يفيد** عندما تكون العمليات كثيرة وصغيرة، أو عندما تكون الذاكرة هي عنق الزجاجة، أو على مسرّعاتٍ تدعم XLA جيدًا.
- **لا يفيد** عندما يكون الوقت ضائعًا في قراءة البيانات أصلًا، أو عندما تتغيّر الأشكال كلَّ دفعة فتُعاد الترجمة، أو عندما تحتوي الدالة عمليةً لا يدعمها XLA.
- **أول استدعاء يكون أبطأ** لأنّ الترجمة تحدث مرةً واحدة. لذلك لا تحكم على الأداء من قياسٍ واحد.

```python
import time
import tensorflow as tf

# دالة كثيفة: ضرب مصفوفتين كبيرتين
@tf.function
def matmul_graph(a, b):
    return tf.linalg.matmul(a, b)

@tf.function(jit_compile=True)
def matmul_xla(a, b):
    return tf.linalg.matmul(a, b)

a = tf.random.normal((1024, 1024))
b = tf.random.normal((1024, 1024))

# تسخين: أول استدعاء يبني الرسم ويُصرّف النواة
matmul_graph(a, b)
matmul_xla(a, b)

def bench(fn, repeats=20):
    start = time.perf_counter()
    for _ in range(repeats):
        fn(a, b)
    return (time.perf_counter() - start) / repeats

print("graph:", round(bench(matmul_graph) * 1000, 2), "ms")
print("xla  :", round(bench(matmul_xla) * 1000, 2), "ms")
# المتوقع: رقمان متقاربان على CPU؛ الفرق يظهر أوضح
# على GPU أو مع سلاسل طويلة من العمليات الصغيرة
```

لاحظ أنّ القياس يشمل كلفة استدعاء بايثون، وهي جزءٌ من الحقيقة العملية لكنها تحجب الفرق أحيانًا. في القياس الجدّي، استخدم `tf.profiler` أو كرّر العملية داخل الدالة نفسها حتى يصبح الحساب هو الغالب.

4.7 حالة واقعية: ثماني دقائق بدل دقيقة

عاد فريقُ التوصيل إلى الكود. كانوا قد كتبوا حلقةً مخصّصة على نمط ما رأيناه، ثم قرّروا الاستفادة من الرسم البياني، فوضعوا `@tf.function` على دالة الخطوة **داخل** الحلقة:

```python
for batch in dataset:
    @tf.function                # خطأ: زخرفة جديدة في كل تكرار
    def train_step(batch):
        ...
```

كلُّ تكرارٍ يُنشئ كائنًا جديدًا، فيُبنى رسمٌ جديد، وتُستهلك الذاكرة، ويصبح الزمنُ تراكميًا. الحلُّ كان نقلَ الدالة خارج الحلقة، وإضافة `input_signature` لتثبيت شكل الدفعة. عاد الزمنُ إلى دقيقة، ثم انخفض أكثر بعد إزالة `print` من داخل الخطوة واستبداله بـ `tf.print` كلّ مئة خطوة. لم يتغيّر النموذج؛ تغيّر مكانُ التعريف فقط.

هذه هي القاعدة: **الرسم يُبنى مرة، والحلقة تدور كثيرًا**. كلُّ ما يُنشأ داخل الحلقة من دوال أو طبقات أو أوزان يُنشأ آلاف المرات. ضعْ التعريف خارجها، واتركْ داخلها البياناتَ فقط.

4.8 أخطاء شائعة

⚠ تحذير

العرض: `tape.gradient` يعيد `None` بلا رسالة واضحة. **السبب:** التنسور المُشتق بالنسبة إليه ثابتٌ لم يُمرَّر إلى `tape.watch`، أو أنّه أُنشئ بعد انتهاء التسجيل. **الحل:** مرّر المدخلات كـ `tf.Variable`، أو استدعِ `tape.watch(x)` داخل سياق الشريط وقبل الحساب.

⛔ خطر

العرض: `RuntimeError: A non-persistent GradientTape can only be used to compute one set of gradients`. **السبب:** استدعيتَ `tape.gradient` مرتين على الشريط نفسه، أو حسبتَ تدرّجًا داخل حلقةٍ من نفس الشريط. **الحل:** أنشئ شريطًا جديدًا في كل خطوة، أو استخدم `persistent=True` مع `del tape` بعد آخر استخدام.

⚠ تحذير

العرض: الطباعة داخل `tf.function` تظهر مرةً واحدة، والزمن يتضاعف مع كل شكلٍ جديد. **السبب:** الخلط بين زمن التتبّع وزمن التشغيل، وبناء رسمٍ لكل حجم دفعة. **الحل:** استخدم `tf.print` للطباعة وقت التشغيل، و`input_signature` أو `reduce_retracing` لتثبيت عدد الرسوم.

4.9 تمارين

1. **اشتقاق يدوي:** احسب مشتقة $f(x) = 3x^2 + 5x - 2$ عند $x = 4$، ثم تحقّق بالشريط. (المتوقع: 29، وهو ما يعيده `tape.gradient`.)
2. **الثوابت:** اشتقّ $y = x^2$ بالنسبة إلى `tf.constant(3.0)` بلا `watch`. ماذا يعيد؟ أصلح الخطأ. (المتوقع: `None`، ثم 6.0 بعد المراقبة.)
3. **المشتقة الثانية:** احسب المشتقة الثانية لـ $f(x) = x^4$ عند $x = 1.5$ بشريطين متداخلين. (المتوقع: 27.0، لأنّ $12x^2$.)
4. **الجاكوبي:** احسب مصفوفة جاكوبي لـ $f(x, y) = (x + y,\ xy)$ عند $(2, 3)$. (المتوقع: مصفوفة 2×2 قيمتها `[[1, 1], [3, 2]]`.)
5. **التتبّع:** غلّف دالةً تطرح متوسط المدخل، واستدعها بثلاثة أحجام مختلفة، ثم أضف `input_signature`. قارن عدد التتبعات. (المتوقع: ثلاثة رسوم قبل التوقيع، ورسم واحد بعده.)
6. **XLA:** شغّل كود ضرب المصفوفات مع وبدون `jit_compile` خمس مرات، وسجّل أول استدعاء و subsequent. (المتوقع: أول استدعاء لـXLA أبطأ، ثم نتائج متقاربة أو أفضل قليلًا حسب العتاد.)
7. **تشخيص:** دالةٌ داخل `tf.function` تطبع الخسارة مرةً واحدة ثم تصمت. اشرح السبب واقترح حلّين. (المتوقع: الطباعة حدثت وقت التتبّع؛ الحل `tf.print` أو `tf.config.run_functions_eagerly` للتشخيص.)

4.10 ملخص

- المشتقة تقيس الميل اللحظي، وقاعدة السلسلة تضرب مشتقات الطبقات المتتالية.
- `tf.GradientTape` يسجّل العمليات ثم يمرّ عليها عكسيًا، فتصبح كلفة التدرّج قريبة من تمريرةٍ أمامية.
- المتغيّرات تُراقَب تلقائيًا، والثوابت تحتاج `watch`، والشريط غير الدائم يُستهلك مرةً واحدة.
- `tf.function` يبني رسمًا واحدًا ويعيد استخدامه؛ اختلاف الأشكال يعني تتبّعًا جديدًا.
- `input_signature` و`reduce_retracing` يضبطان عدد الرسوم، و`tf.print` يطبع وقت التشغيل.
- XLA يدمج العمليات الصغيرة، ويفيد حين يكون الحساب كثيفًا والذاكرة عنق الزجاجة.
- القاعدة العملية: عرّف الدوال والطبقات خارج الحلقة، وضعْ البيانات داخلها.

الجزء الأول · الأساسات: كيف تفكّر في TensorFlow؟

الفصل الخامس

05

Keras: من الفكرة إلى النموذج

حكاية الفصل

احتاج فريقٌ إلى تقدير زمن توصيل الطلب. بدأت الفكرة بثلاث طبقات، ثم ظهرت حاجة إلى فرع يتعامل مع نوع المركبة وآخر مع المسافة. تحوّل السؤال من «كيف أكتب طبقة؟» إلى «كيف أنظم النموذج وأقيسه وأحفظه؟». هنا تصبح Keras واجهة دورة العمل كاملة.

5.1 أهداف الفصل

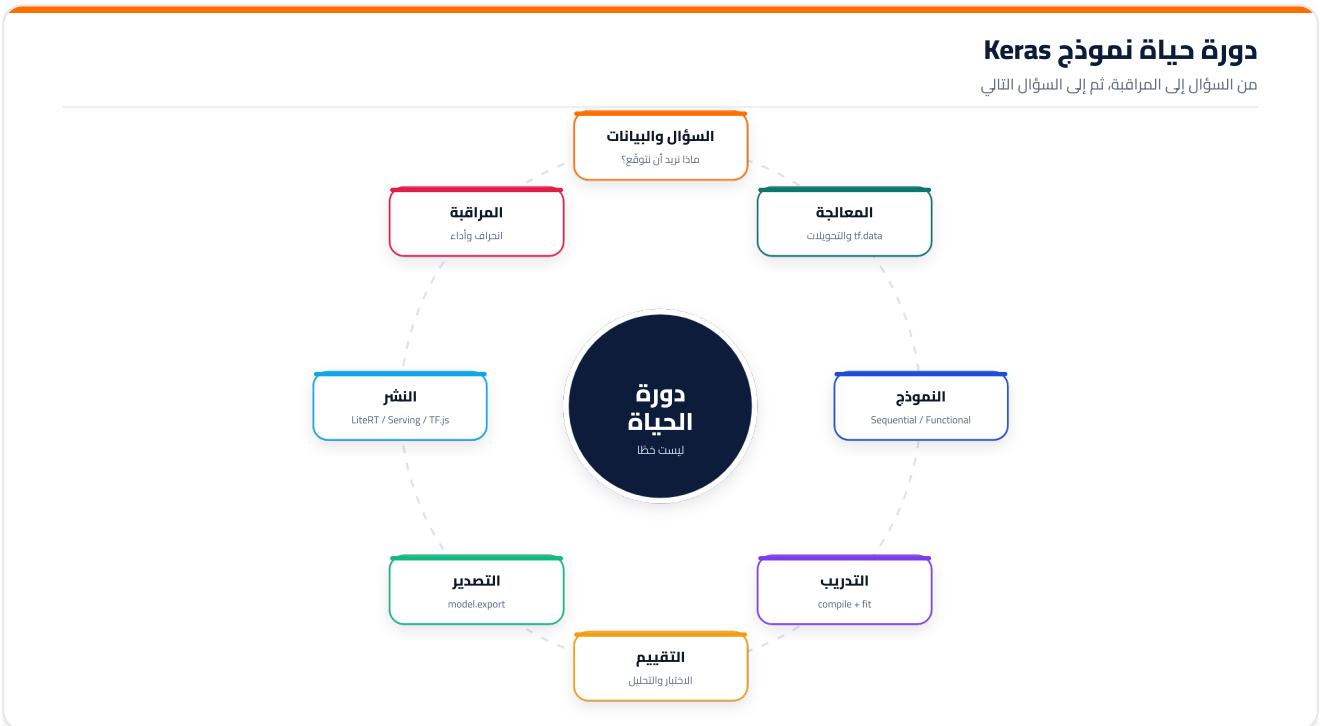
- [ ] اختيار Sequential أو Functional أو subclassing وفق شكل الحساب.
- [ ] ربط طبقات النموذج بالأشكال والخسارة والمقياس.
- [ ] تنفيذ compile وfit وevaluate والحفظ.
- [ ] التمييز بين ملف `.keras` وواجهة export.

5.2 أي بنية تناسب الحساب؟

تسلسل خطي من المدخل إلى المخرج يناسبه `Sequential`. إذا كان هناك مدخلان أو تخطٍّ بين الطبقات أو أكثر من مخرج، يصف Functional API الرسم صراحة. أما subclassing فيفيد حين يحتاج النموذج سلوكًا مخصصًا، لكنه يزيد مسؤولية البناء والتسلسل والتصحيح.



شكل 5-1 مقارنة واجهات بناء نموذج Keras



شكل 5-2 دورة النموذج من الإنشاء إلى الحفظ

الأسلوب	نقاط القوة	الكلفة
Sequential	قصير وواضح للتتابع	لا يصف تفرعًا عامًا
Functional	رسم صريح متعدد المدخلات والمخرجات	يتطلب فهم تدفق الأشكال
Subclassing	تحكم كامل في السلوك	يتطلب عناية بالبناء والحفظ

```python
import keras
from keras import layers

inputs = keras.Input(shape=(8,), name="order_features")
x = layers.Dense(32, activation="relu")(inputs)
x = layers.Dropout(0.1)(x)
outputs = layers.Dense(1, name="delivery_minutes")(x)
model = keras.Model(inputs, outputs)
model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss="mse", metrics=[keras.metrics.MeanAbsoluteError()])
model.summary()
```

5.3 الخسارة والمقياس يجيبان عن سؤالين مختلفين

في الانحدار البسيط، متوسط مربع الخطأ يكتب على الصورة التالية:

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

`N` عدد الأمثلة، و `y_i` الهدف الحقيقي، و `ŷ_i` التنبؤ. تضاعف العقوبة عند تضاعف مقدار الخطأ.

الخسارة توجه التحسين بالتدرج، أما المقياس فيلخص جودة نحتاج إلى قراءتها. في الانحدار، متوسط مربع الخطأ يعاقب الخطأ الكبير أكثر من متوسط الخطأ المطلق. في تصنيف متعدد الفئات مع logits، استخدم خسارة تقبل `from_logits=True` بدل تطبيق softmax مرتين. شكل المخرج وعدد الوحدات جزء من العقد بين البيانات والخسارة.

```python
import numpy as np
import keras
from keras import layers

keras.utils.set_random_seed(42)
x = np.random.default_rng(42).normal(size=(256, 8)).astype("float32")
y = (2 * x[:, :1] - x[:, 1:2]).astype("float32")
model = keras.Sequential([layers.Input((8,)), layers.Dense(16, activation="relu"), layers.De
model.compile(optimizer="adam", loss="mse", metrics=["mae"])
history = model.fit(x, y, validation_split=0.2, epochs=5, batch_size=32, verbose=0)
print(model.evaluate(x[:32], y[:32], verbose=0))
# خسارة و MAE المتوقع: عدديان، وتتغير القيم مع التدريب.
```

يفصل `validation_split` جزءًا من البيانات التي مررتها، لكنه لا يصنع تقسيمًا زمنيًا أو طبقيًا مناسبًا تلقائيًا. جهّز `validation_data` صراحة عندما يعتمد تصميم المسألة على ترتيب زمني أو مجموعات متشابهة.

5.4 دورة الحفظ والنشر

ينشئ Keras 3 ملف `.keras` واحدًا يحفظ النموذج القابل لإعادة التحميل بصيغة `model.export(path)`. في SavedModel للاستدلال؛ لا تخلط الملفين. احتفظ أيضًا بإعداد المعالجة المسبقة، وإصدارات الحزم، وعتبة القرار، لأن وزن النموذج وحده لا يحدد سلوك النظام.

```python
model.save("delivery.keras")
loaded = keras.models.load_model("delivery.keras")
loaded.export("delivery_savedmodel")
```

5.5 حالة واقعية: الطلبات بفرعين

قد يستخدم تقدير التوصيل مدخلًا عدديًا للمسافة ومدخلًا فئويًا لنوع المركبة. يبني Functional API فرعين، ثم يدمجهما قبل المخرج. يحفظ الفريق النموذج مع عقد المدخلات، ويرفض في الخدمة قيمًا غير محددة بدل تمريرها بصمت.

5.6 خط أساس كامل من البيانات إلى القياس

النموذج الذي يصنف قيمة لا يصبح تجربة قابلة للتكرار لمجرد أن `fit` انتهى. يجب تعريف تقسيم يطابق شكل الاستخدام، واختيار baseline واضح، وتسجيل معيار قبول قبل البحث في الطبقات. في تصنيف غير متوازن، قد يضلل `accuracy`؛ أضف precision وrecall أو AUC، واحسبها على التحقق لا على بيانات التدريب فقط. عند استخدام `validation_split`، تذكر أن Keras يأخذ جزءًا من المصفوفات المقدمة ولا يضمن تقسيمًا طبقيًا أو زمنيًا.

الـ callbacks تنظم أحداث التدريب. `EarlyStopping` يوقف عندما يتوقف المقياس عن التحسن، و`ModelCheckpoint` يحفظ أفضل حالة، و`TensorBoard` يسجل المنحنيات. راقب المقياس في الاتجاه الصحيح: `val_auc` يحتاج تعظيمًا بينما `val_loss` يحتاج تقليلًا. لا تحفظ «آخر نموذج» فقط إن كنت ستختار على أفضل حقبة، ولا تستخدم الاختبار لاختيار عدد epochs.

5.7 مدخلان ومخرجان في Functional API

عندما يكون تدفق الحساب رسمًا لا سلسلة، سمّ المدخلات والمخرجات. هذا يجعل توقع الخدمة أوضح ويقلل أخطاء تبديل القيم:

```python
import keras
from keras import layers

distance = keras.Input(shape=(1,), name="distance_km")
vehicle = keras.Input(shape=(), dtype="string", name="vehicle_type")
lookup = layers.StringLookup(output_mode="one_hot")
lookup.adapt(["bike", "car", "van"])
category = lookup(vehicle)
x = layers.Concatenate()([distance, category])
x = layers.Dense(32, activation="relu")(x)
minutes = layers.Dense(1, name="minutes")(x)
late_probability = layers.Dense(1, activation="sigmoid", name="late")(x)
model = keras.Model({"distance_km": distance, "vehicle_type": vehicle},
                    {"minutes": minutes, "late": late_probability})
```

في التطبيق الفعلي يجب تكييف `StringLookup` على قيم التدريب أو توفير مفرداته الثابتة، وتحديد معالجة قيم OOV. إذا كان هناك مخرجان، اكتب خسارة ومقياسًا لكل واحد، وافحص مساهمة كل هدف. أوزان الخسائر ليست مجرد معاملات تجميل؛ قد تجعل التحسين يركز على مهمة على حساب الأخرى.

5.8 الحفظ الآمن ودورة النشر

صيغة `.keras` تحفظ إعداد النموذج والأوزان وحالة المحسن عند توفرها. لا تحتوي شيفرة Python لتعريف الكائنات المخصصة؛ سجل الطبقات أو مرر `custom_objects` من شيفرة موثوقة عند التحميل. Keras الحديثة تستخدم وضع تحميل آمن افتراضيًا لرفض deserialization غير الموثوق لبعض الكائنات. لا تعطّل `safe_mode` لمجرد إسكات خطأ ملف مجهول.

افصل ملف إعادة التدريب عن artifact الاستدلال، وسجل بصمة النموذج ونسخة المعالجة. قبل إتاحة الإصدار، أعد تحميله في عملية جديدة وتحقق من أن مدخلات حقيقية تعطي نتائج مناظرة. النموذج المحفوظ دون أداة تحويل السمات أو ترتيب الفئات ليس منتجًا قابلًا للتشغيل.

5.9 عقود التدريب: خسارة، مقياس، ووزن عينة

حدد ما إذا كانت labels فهارس صحيحة أو متجهات one-hot، ثم اجعل شكل المخرج وخسارته مطابقين. Sparse categorical loss تقبل رقم فئة واحدًا لكل مثال، أما categorical loss فتتوقع توزيعًا على الفئات. عند تمرير logits أوقف softmax في المخرج واستعمل `from_logits=True`؛ وعند إخراج احتمالات استخدم خسارة متوافقة معها. هذا الاختيار يؤثر في الاستقرار العددي ويمنع تكرار softmax.

`sample_weight` يغير مساهمة الأمثلة في loss وmetrics إذا كانت المقاييس تدعمه. استخدمه لأهمية موثقة أو سياسة وزن، لا كحل آلي لكل عدم توازن. راقب مؤشرات الفئة النادرة منفصلة؛ متوسط الدقة قد يخفي collapse إلى الأغلبية. لو عندك عدة مخرجات، حدد `loss_weights` علنًا وسجل خسارة كل مخرج كي تعرف أي هدف لم يعد يتعلم.

5.10 بناء input pipeline مع Keras

`fit` يقبل NumPy arrays و`tf.data.Dataset` وPyDataset وغيرها. عند ملف بيانات كبير أو معالجة متوازية، Dataset يعطيك تحكمًا في القراءة والخلط والدفعة. حدد `steps_per_epoch` فقط عند حاجة معروفة؛ الخطأ فيه قد يوقف التدريب قبل مرور البيانات أو يعيد أمثلة غير محدودة. callback لا يحل محل تقسيم validation؛ جهز تحققًا مستقلاً يتوافق مع توزيع الاستخدام.

5.11 اكشف انهيار التعلم بالتشخيص

ارسم loss وmetric للتدريب والتحقق. إذا كان كلاهما سيئًا، افحص labels، شكل المخرج، learning rate، وfeature scaleg. إذا كان التدريب ممتازًا والتحقق ضعيفًا، افحص overfitting والتسرب واختلاف المجموعتين. إذا ظلت loss ثابتة، جرّب overfit على 16 مثالًا؛ يفترض أن يستطيع نموذج مرن تقليل خسارتها. فشل هذا الاختبار غالبًا خلل في تدفق gradients أو target mapping.

5.12 قابلية إعادة الإنتاج

ثبت البذور عبر `keras.utils.set_random_seed`، لكن وثق أن التكرارية قد تظل متأثرة بنوع الجهاز والعمليات غير الحتمية. احفظ config لكل تجربة: بنية النموذج، optimizer وbatch sizeg، learning rate، تقسيم البيانات، preprocessing، seed، ومقياس الاختيار. لا تخزن history بلا ربطه بإصدار الكود. الباحث يحتاج مقارنة عادلة؛ المطور يحتاج rollback وإعادة إنشاء.

5.13 Model subclassing ومتى تحتاجه

ابدأ بـ Sequential أو Functional لأنهما يقدمان config ورسمًا يسهل فحصه. انتقل إلى subclassing عندما تحتاج تدفقًا يعتمد حالة أو سلوكًا متكررًا لا يعبر عنه الرسم المعتاد. نفذ `get_config` إذا أمكن، وابنِ جميع المتغيرات مرة واحدة، واختبر serialization. المرونة الزائدة تجعل الرسم والتحويل إلى LiteRT أو Serving أصعب؛ لا تستخدم subclassing لمجرد أنه يبدو أكثر احترافًا.

5.14 فصل البحث عن التقييم النهائي

استخدم train لتعلم الأوزان، validationg لاختيار حجم الشبكة والعتبة والحقبة، testg للتقدير النهائي مرة بعد تثبيت الخطة. إذا جربت عشرات إعدادات النموذج على test، فهو لم يعد test مستقلًا. في بيانات زمنية أو طبية، قد يكون تقسيم الأمثلة عشوائيًا غير صالح؛ استخدم تاريخًا مستقبليًا أو GroupKFold بحسب وحدة الاستقلال. اذكر عدد الأمثلة والأشخاص والمصادر، لا نسبة التقسيم فقط.

5.15 Keras callbacks تعاقد وليست سحرًا

يحتاج EarlyStopping إلى اسم مقياس موجود في logs؛ خطأ `monitor="val_accuracy"` مع عدم تعريف metric قد يمنع الإيقاف المقصود. استخدم `mode="max"` لمقياس أعلى أفضل، و`restore_best_weights=True` كي لا تستعيد آخر أوزان بعد تدهور. ModelCheckpoint يحفظ الملف وفق

save format المطلوب. TensorBoard callback يكتب event logs؛ منذ TensorFlow 2.21 لم يعد TensorBoard اعتمادًا ضمنيًا في الحزمة، فثبته مباشرة في بيئة المشروع عند استعماله.

5.16 فحص قبل الإنتاج

اختبر inference على حالات حدية: دفعة من مثال واحد، دفعة كبيرة، قيمة OOV، عنصر مفقود، وطول تسلسل عند الحد. تحقق أن ترتيب أسماء المخرجات ثابت. قارن الأصل والمحمّل من `.keras`، ثم صدّر inference وقارن النتائج بعد التحويل ضمن tolerance مسجل. احفظ أيضًا tokenizer أو lookup vocabulary، feature schema، threshold، ونسخة preprocessing. الأثر الصحيح هو «نموذج + عقد + معالجة + إعداد قرار».

5.17 دورة تعلم بحسب المستوى

المبتدئ يطبق `fit` ويتعلم شكل البيانات والخسارة. المتوسط يقارن Functional/Sequential وcallbacks وmetrics على مشكلة معلومة. المحترف يخصص data pipeline وserialization وCI. الخبير يفحص contract عبر backend أو runtime ويحدد دلالات القياس وتكلفته. لا تنتقل إلى custom loop قبل أن تفسر سلوك `fit` على baseline.

5.18 أمثلة تنفيذية إضافية

إدارة التدريب عبر callbacks

```python
callbacks = [
    keras.callbacks.EarlyStopping(monitor="val_loss", patience=2, restore_best_weights=True),
    keras.callbacks.ModelCheckpoint("best.keras", save_best_only=True),
]
# مرّر callbacks=callbacks إلى model.fit(...)
```

توقع دفعة

```python
predictions = model.predict(x[:4], verbose=0)
print(predictions.shape)
```

5.19 مواصفات التشغيل والتسليم

أرفق model summary واسم المدخل وشكل الدفعة والتحويلات thresholdg mappingg الفئات ونسخ الحزم ومقاييس validation. أنشئ smoke inference بمدخل اصطناعي fixtureg منزوع الحساسية. عند تحديث النموذج، قارن الإصدارات على مجموعة ثبات. المستهلك يحتاج عقدًا صغيرًا واضحًا لا دفتر التدريب كاملًا.

5.20 تحقق من التسمية والتوزيع

اطبع label وتوقعًا واحدًا بجانب المدخل قبل تشغيل fit. عند sparse labels، تأكد أن المعرفات تبدأ من المجال المتوقع ولا تحتوي قيمة خارج عدد الوحدات. اختبر mapping الفئات بعد reload، لأن ترتيب القائمة قد يتغير بين train وAPI.

5.21 استدلال بدفعات صغيرة

خدمة التنبؤ قد ترسل مثالًا أو مجموعة؛ اختبر الحالتين، وراقب إن كانت المخرجات تحتفظ بمحور batch. لا تستخدم `squeeze` بلا محور محدد، فقد يزيل بعدًا صحيحًا عند batch size=1.

5.22 اختيار metric مرتبط بالتكلفة

قد يكون false negative أغلى من false positive. عرّف مصفوفة تكلفة القرار، ثم اختر threshold على validation مع إبقاء test لتقدير نهائي. إذا تغيرت prevalence في الإنتاج، أعد تقييم calibration والعتبة عبر إجراء مراقب، لا تعديلها آليًا على مجموعة test.

5.23 احتفظ بعقد الخدمة مع النموذج

عند إصدار endpoint جديد، وثق ما إذا كانت الدرجات logits أو احتمالات، وترتيب الفئات ووحداتها. عميل يفسر logit على أنه probability قد يتخذ قرارًا خاطئًا دون استثناء برمجي. أضف versioned schema ومثال request/response إلى بطاقة التسليم.

5.24 أخطاء شائعة

⚠ تحذير

`model.compile()` لا يدرّب النموذج؛ هو يحدد طريقة التحديث والمقاييس، ويبدأ التدريب مع `fit()`. [!DANGER] `save()` و`export()` يخدمان غرضين مختلفين في Keras 3. اختبر التحميل والتوقيع الفعلي قبل تسليم النموذج.

5.25 تمارين

1. مثّل مصنفًا خطيًا بـ Sequential. (المتوقع: طبقات مرتبة ومخرج بعدد الفئات.)
2. صمّم Functional API بمدخلين. (المتوقع: فرعان ودمج موثق.)
3. اختر خسارة لمخرج logits ثنائي. (المتوقع: BinaryCrossentropy مع `from_logits=True`.)
4. أضف EarlyStopping لمراقبة `val_loss`. (المتوقع: إيقاف واستعادة أفضل أوزان.)
5. احفظ `.keras` ثم أعد التحميل. (المتوقع: تنبؤ متقارب على المدخل نفسه.)
6. فسّر متى تصدّر SavedModel. (المتوقع: عند تجهيز مسار الاستدلال والتكامل.)

5.26 ملخص

- اختر بنية النموذج وفق تدفق البيانات.
- اربط شكل المخرج بنوع الخسارة، وافصل مقياس التحسين عن مقياس القراءة.
- وثّق المعالجة والعتبة، واحفظ بصيغة توافق غرض الاستخدام.

الجزء الثاني

بناء الشبكات وتدريبها

الهندسة، التحسين، التقييم، والبيانات

06 الشبكات العصبية من الصفر

07 المحسّنات والتنظيم: فنّ التدريب

08 التقييم والضبط: كيف تعرف أن نموذجك جيد؟

09 tf.data: هندسة خطوط البيانات

10 الرؤية الحاسوبية: CNN من الالتفاف إلى الإنتاج

الفصول 6-10 · 5 فصول

الجزء الثاني · بناء الشبكات وتدريبها

الفصل السادس

06

الشبكات العصبية من الصفر

> **حكاية الفصل**
>
> يريد مصنع فرز قطعًا معدنية من صورة منخفضة الدقة. يبدأ المهندس بقياس سطوع البكسلات، ثم يبني طبقات تتعلم أنماطًا أكثر تعقيدًا. ما الذي تتعلمه الأوزان فعلًا، وكيف ينتقل أثر قرار التصنيف إلى ملايين المعاملات؟

6.1 أهداف الفصل

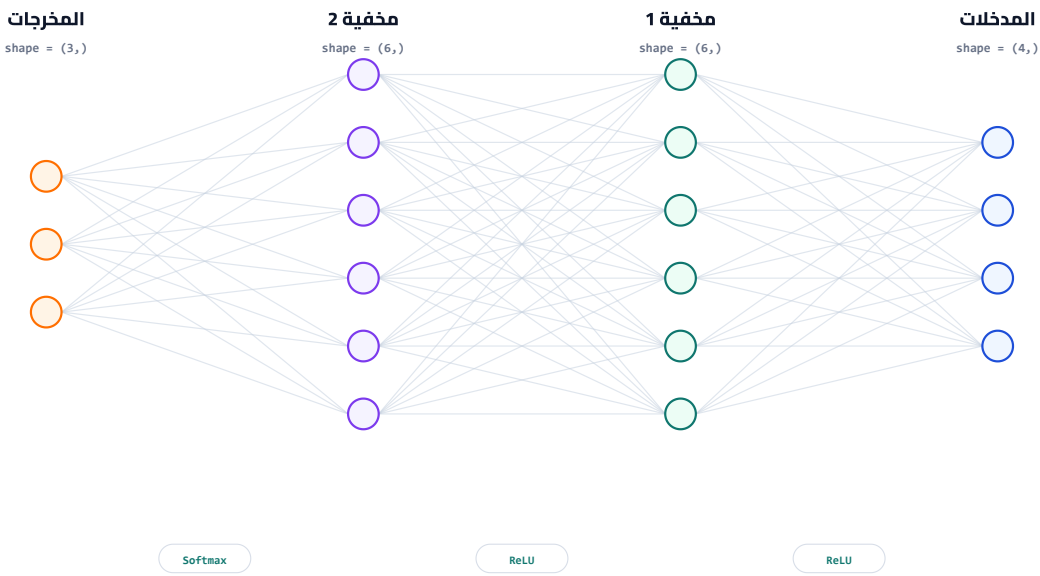
- ☐ تفسير الإدراك والطبقات والانتشار الأمامي.
- ☐ مقارنة التنشيطات والتهيئات الشائعة.
- ☐ اشتقاق تحديث وزن عبر قاعدة السلسلة.
- ☐ كتابة حلقة تدريب بسيطة وفهم حدودها.

6.2 من جمع موزون إلى قرار

تحسب الوحدة العصبية $z = \sum_i w_i x_i + b$، ثم تحول القيمة عبر تنشيط $a = \phi(z)$. المدخل `x_i` سمة، والوزن `w_i` مقدار تأثيرها، والانحياز `b` إزاحة قابلة للتعلم. شبكة متعددة الطبقات تكرر هذه العملية، فتكوّن تمثيلات وسيطة بدل كتابة قواعد السمات يدويًا.

الشبكة العصبية متعددة الطبقات

كل خطّ هو وزن، وكل دائرة عصبون

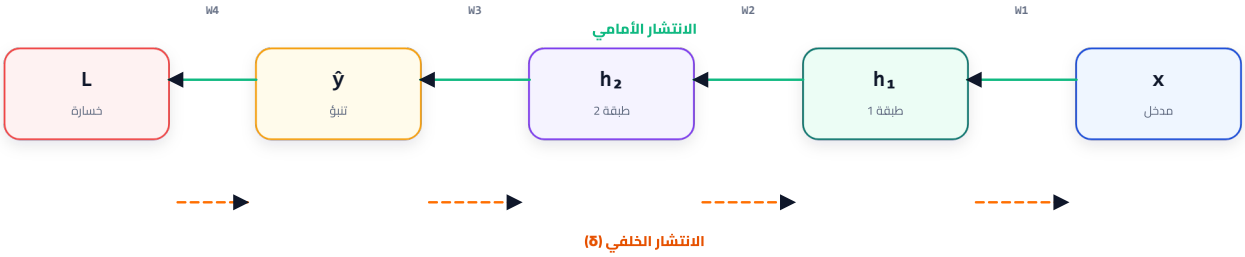


الاتجاه من اليمين (المدخلات) إلى اليسار (المخرجات).

شكل 6-1 الانتشار الأمامي عبر شبكة متعددة الطبقات

الانتشار الخلفي: من الخسارة إلى الأوزان

الأمام يحسب القيمة، والخلف يحسب نصيب كل وزن منها



ماذا يعني؟

الخطأ ينتقل للخلف، وكل وزن يأخذ جزءًا يتناسب مع مساهمته في الخطأ.

قاعدة السلسلة

$$\delta_2 = (W_3^T\ \delta_3) \odot \sigma'(z_2)$$

$$\partial L/\partial W_2 = \delta_2\ h_1^T$$

شكل 6-2 عودة التدرج لتحديث الأوزان

التنشيط	السلوك	استعمال شائع
Sigmoid	يضغط إلى (0,1)	احتمال ثنائي في المخرج
tanh	متمركز حول الصفر	حالات متكررة تقليدية
ReLU	يصفّر السالب	طبقات مخفية بسيطة
GELU	انتقال ناعم	محولات حديثة
Softmax	يوزع الاحتمال عبر الفئات	مخرج متعدد الفئات

6.3 الانتشار الخلفي قاعدة سلسلة منظمة

إذا كانت الخسارة تعتمد على المخرج، والمخرج يعتمد على وزن، فإن قاعدة السلسلة تفك هذا الاعتماد إلى مشتقات محلية. في طبقة واحدة تكون المشتقة بالنسبة للوزن حاصل ضرب خطأ المخرج والمدخل، وتُحدّث الأوزان بخطوة معاكسة للتدرج. لا تحتاج إلى كتابة المشتقة يدويًا في كل نموذج، لكن فهمها يفسر لماذا يجب أن تبقى العملية داخل الرسم الذي يراقبه GradientTape.

$$w_{t+1} = w_t - \eta \frac{\partial L}{\partial w_t}$$

حيث `w_t` الوزن الحالي، و `L` الخسارة، و `η` معدل التعلّم. الخطوة الكبيرة قد تتجاوز الحد الأدنى، والصغيرة جدًا تبطئ التعلم.

```python
import tensorflow as tf

x = tf.constant([[0.0], [1.0], [2.0], [3.0]])
y = tf.constant([[1.0], [3.0], [5.0], [7.0]])
w = tf.Variable([[0.0]])
b = tf.Variable([0.0])
optimizer = tf.keras.optimizers.SGD(learning_rate=0.05)

for _ in range(100):
    with tf.GradientTape() as tape:
        loss = tf.reduce_mean(tf.square(x @ w + b - y))
    dw, db = tape.gradient(loss, [w, b])
    optimizer.apply_gradients([(dw, w), (db, b)])
print(w.numpy(), b.numpy())
# المتوقع: w تقترب من 2 وb تقترب من 1.
```

6.4 التهيئة والاستقرار

إذا بدأ كل وزن بالقيمة نفسها، تتلقى الوحدات المتناظرة تدرجات متطابقة وتبقى متشابهة. تهيئة Glorot توازن التباين تقريبًا بين الطبقات، وHeg تراعي ReLU. أما التدرج المتلاشي فيضعف إشارة التعلم في طبقات سابقة، والانفجار يجعل التحديثات ضخمة؛ التطبيع والقص واختيار المعمارية تساعد، لكنها لا تصلح بيانات أو هدفًا سيئًا.

نظرية التقريب الشامل تقول إن شبكة واسعة بما يكفي يمكنها تقريب دوال مستمرة ضمن شروطها، لكنها لا تقول إن التدريب سيجد الحل أو أن البيانات كافية أو أن الحل يعمم. القدرة التعبيرية ليست ضمانًا لجودة النموذج.

6.5 حالة واقعية: مصنف القطع

يُسطّح الإدخال الصغير إلى متجه، وتُطبّع قيم البكسلات، ثم يخرج النموذج logits لفئتي «سليم» و«معيب». يُقاس الاستدعاء للفئة المعيبة لأن تكلفة تمرير قطعة تالفة أعلى من فحص قطعة سليمة. تبدأ التجربة بخط أساس لوجستي؛ لا تضف طبقات إلا إذا أظهر تحليل الأخطاء نمطًا لا يلتقطه الخط الأساس.

6.6 اشتقاق رجوع الخطأ من طبقة واحدة

خذ طبقة كثيفة لمثال واحد: `z=wx+b` و `a=φ(z)`، وخسارة مربع `L=(a-y)^2/2`. باستخدام قاعدة السلسلة:

$$\frac{\partial L}{\partial w} = (a - y)\phi'(z)x$$

الخطأ `a-y` يقيس اتجاه الفرق، و `φ'(z)` يحول أثره عبر التنشيط، و `x` يحدد كيف يؤثر الوزن في المجموع. في شبكة ذات طبقتين يتكرر هذا التركيب: الخطأ عند المخرج يعود إلى الطبقة السابقة عبر أوزانها ومشتقة تنشيطها. هذا الاشتقاق يشرح لماذا توقف `GradientTape` عن إعطاء تدرج إذا قطعت الحساب بتحويل NumPy أو خرجت من نطاق عملياته.

6.7 التنشيط مشتقةً وسلوكًا

Sigmoid وtanh تتشبعان عند قيم كبيرة موجبة أو سالبة، فتصغر مشتقتهما ويضعف التدرج في الطبقات العميقة. ReLU مشتقتها ثابتة في النصف الموجب وصفر في السالب، ما يسرع كثيرًا من الشبكات لكنه قد يجعل وحدة لا تستجيب. LeakyReLU تحتفظ بميل صغير للجهة السالبة، وGELU انتقال سلس يستخدم كثيرًا في المحولات. Softmax ليست عادة تنشيطًا للطبقات المخفية؛ هي تطبيع متنافس لمجموعة logits في مخرج متعدد الفئات.

التنشيط	مجال المخرج	أثر جانبي	موضع مألوف
Sigmoid	(0,1)	تشبع، وغير متمركز حول الصفر	احتمال ثنائي أو بوابة
tanh	(-1,1)	تشبع عند الأطراف	حالة متكررة
ReLU	[0,∞)	وحدات خاملة في جهة السالب	طبقات كثيفة/تلافيفية
GELU	غير محدود قليلًا	كلفة/سلوك أنعم	Transformer blocks
Softmax	توزيع على الفئات	يعتمد على جميع logits	تصنيف متعدد الفئات

6.8 التهيئة وحجم الطبقة

تحدد التهيئة نطاق قيم الأوزان قبل رؤية البيانات. تهيئة Glorot (Xavier) توازن التباين وفق fan-in وfan-out، وHe تراعي وحدات ReLU. بدء كل الأوزان بصفر يمنع كسر التناظر؛ وحدات الطبقة الواحدة ستتلقى التدرج نفسه وتتعلم النسخة نفسها. أما قيمة عشوائية ضخمة، فقد تمرر تفعيلات وتدرجات ضخمة. اترك اختيار التهيئة الافتراضي الملائم للطبقة ما لم يكن لديك سبب تجريبي لتغييره.

عدد المعاملات في Dense يساوي تقريبًا `input_dim × units + units`. لذلك طبقة بـ 10,000 سمة و1,000 وحدة تخفيّة تحمل أكثر من عشرة ملايين وزن وانحياز قبل حساب التفعيلات. راقب الحجم والذاكرة وحجم البيانات؛ زيادة عدد المعاملات دون بيانات كافية تسهّل حفظ الأمثلة.

6.9 نموذج رقمي كامل للمبتدئ

لبيانات MNIST، المدخل صورة 28×28 رمادية. الخطوات هي تثبيت البذرة، تحميل البيانات، تطبيع البكسل، تعريف المخرج، اختيار خسارة تتوافق مع logits، ثم تقييم الاختبار مرة بعد انتهاء القرارات:

```python
import keras
from keras import layers

keras.utils.set_random_seed(42)
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
model = keras.Sequential([
    layers.Input((28, 28)), layers.Flatten(),
    layers.Dense(128, activation="relu"), layers.Dense(10),
])
model.compile(optimizer=keras.optimizers.Adam(1e-3),
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=["accuracy"])
model.fit(x_train, y_train, validation_split=0.1, epochs=5, batch_size=128)
print(model.evaluate(x_test, y_test, verbose=0))
```

هذا نموذج تعليمي كامل، لكنه ليس نتيجة بحث منشورة: الاختيار العشوائي لتقسيم validation قد يخفي اختلاف المصدر، و`accuracy` لا يصف كل الأخطاء، ولا توجد معالجة لتغير المجال. وسّعه فقط بعد تسجيل baseline وتحليل صور الخطأ.

6.10 المقارنة بين التنشيطات على رسم واحد

عند مقارنة ReLU وGELU وtanh، ثبت باقي المتغيرات. قارن منحنى المشتقة، ونسبة الوحدات التي خرجت بصفر، وgradient norm في الطبقات الأولى. قد تتفوق دالة على مجموعة صغيرة بسبب معدل تعلم مناسب بالصدفة؛ كرر التجربة على عدة بذور ثم اعرض mean وstandard deviation. التحسين ليس جدولًا عالميًا منفصلًا عن العمق والتهيئة والتطبيع.

Softmax تحسب احتمالًا لكل فئة وتربط المخرجات بعضها ببعض: زيادة logit واحدة تقلل الحصص النسبية للآخرين. لا تضفها إلى كل مخرج تلقائيًا إذا كانت المهمة multilabel حيث يمكن أن تكون عدة فئات صحيحة معًا؛ استعمل sigmoid مستقلًا لكل label وخسارة ثنائية. هذا الفرق بين multiclass وmultilabelg مصدر خطأ شائع في التصنيف الطبي متعدد المؤشرات.

6.11 من المشتقة إلى تحديث الدفعة

في batch، تحسب الخسارة غالبًا متوسطًا على الأمثلة، فيكون gradient متوسط مساهماتهم. زيادة حجم الدفعة تقلل ضوضاء تقدير التدرج لكنها تستهلك ذاكرة وقد تتطلب تعديل learning rate. SGD العشوائي يعيد تقدير الاتجاه من أمثلة مختلفة كل مرة؛ لذلك يتعرج على سطح الخسارة، بينما momentum ينعّم الحركة. لا تقارن optimizer مع batch size مختلف ثم تنسب كل فرق الخسارة إلى optimizer.

التدرج العددي finite difference يصلح للتحقق من مشتقة بسيطة: غيّر الوزن بمقدار صغير واحسب فرق الخسارة مقسومًا على مقدار التغيير. التحقق العددي مكلف وغير دقيق جدًا مع epsilon صغير أو كبير، لكنه مفيد في طبقة مخصصة ذات معادلة يدوية. TensorFlow يوفر GradientTape للمشتقات العملية؛ يجب أن يكون المدخل أو متغير النموذج قابلًا للتتبع.

6.12 كيف تقرأ model.summary؟

العمود Output Shape يبين حجم التفعيل بعد الطبقة، Paramg # عدد المعاملات. في Dense يرتبط العدد بحجم الإدخال والوحدات. في CNN يتأثر بعدد القنوات لا مساحة الصورة بذاتها، لكن الذاكرة الوسيطة تتأثر بالارتفاع والعرض والدفعة. المعاملات القابلة للتدريب يجب أن تقارب مجموع أوزان الطبقات، وأي layer بها صفر معاملات قد تكون مقصودة مثل pooling أو دليلًا أن فرعًا لم يُبنَ.

6.13 حدود نظرية التقريب الشامل

النتيجة الرياضية تقرر وجود شبكة بقدرة كافية تقرب دالة مستمرة ضمن شروط ونطاق محدود. لا تحدد عدد أمثلة التدريب المطلوب، ولا تضمن أن gradient descent يجد الأوزان، ولا تمنع النموذج من حفظ الضوضاء. الانتظام والتقسيم والاختبار الخارجي تظل واجبة. استخدم النظرية لتفهم القدرة التمثيلية، لا لتسويق تعميم مؤكد.

6.14 تشخيص كل طبقة على حدة

في تجربة مصغرة، اطبع mean/std للتفعيلات وتدرجات كل طبقة، وعدد الأوزان التي تساوي صفرًا بعد ReLU. تفعيل متضخم عبر طبقات متعاقبة قد يدل على تهيئة أو معدل تعلم؛ تدرجات قريبة من صفر في البداية قد تشير إلى التشبع أو مسار قطع. لا تفسر histogram وحده؛ قارنه مع loss وحجم الخطوة وتوزيع السمات.

Gradient clipping يحد مقدار التحديث عندما تتجاوز norm قيمة، لكنه لا يعالج عدم توافق الأهداف أو إدخالًا متضخمًا. إذا احتجته، سجل نسبة الخطوات التي قُصت؛ القص الدائم إشارة أن معدل التعلم أو التصميم يحتاج مراجعة. في الشبكات ذات skip connections، المسار الأقصر يساعد مرور التدرج، لكنه لا يغني عن مراقبة أعداد دقيقة.

6.15 منطق اختيار عدد الطبقات

أضف طبقة أو وحدات فقط عندما يكشف الخطأ الحالي نقصًا قابلًا للتعلم. لو كان النموذج لا يفصل فئتين بسبب تمثيل خطي، أضف nonlinearity. لو تطابق train وانخفض validation، خفّض السعة أو حسن البيانات. لو الاثنين ضعيفان، زد السعة بعد التحقق من الملصقات والخط الأساس. هذا التشخيص يمنع البحث العشوائي في مئات architectures.

6.16 مثال انحدار متعدد الميزات

في مثال الطلب، ميزات مثل المسافة والطقس والساعة تدخل Dense. لكل ميزة مقياس مختلف؛ طبقة Normalization تحفظ إحصاءات train وتطبقها عند inference. لا تستخدم طبقة تتعلم الإحصاءات من كامل البيانات قبل split. قارن مع linear baseline لتعرف هل طبقات غير خطية تضيف فائدة فعلية. للانحدار بمتغير موجب مثل زمن الشحن، افحص هل مخرج خطي يعطي قيمًا سالبة؛ تصميم output مناسب أو تحويل الهدف قد يحل المشكلة.

6.17 ماذا تعرض في التقرير؟

اعرض شكل النموذج، عدد المعاملات، train/validation curves، تعريف loss، عتبة القرار، وconfusion matrix. في بحث عن تعميم، أضف عدة runs أو cross-validation. اذكر صراحة ما لم تختبره: مجال صور مختلف، أو أجهزة منخفضة الدقة، أو فئات نادرة. تفاصيل هذه الحدود تجعل النتائج قابلة للثقة أكثر من رقم دقة بلا سياق.

6.18 أمثلة تنفيذية إضافية

مصنف يخرج logits

```python
import keras
from keras import layers
classifier = keras.Sequential([layers.Input((28, 28, 1)), layers.Flatten(),
    layers.Dense(64, activation="relu"), layers.Dense(10)])
classifier.compile(optimizer="adam",
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True))
```

تثبيت البذرة

```python
import keras
keras.utils.set_random_seed(42)
```

6.19 أخطاء شائعة

⚠ تحذير

لا تستخدم تنشيطًا خطيًا في كل الطبقات ثم تتوقع تمثيلًا غير خطي؛ تركيب التحويلات الخطية يبقى خطيًا.

[DANGER!] لا تخلط مخرجات softmax مع خسارة `from_logits=True` ؛ حدد عقدًا واحدًا بين المخرج والخسارة.

6.20 تمارين

1. احسب ناتج وحدة بمدخلين وأوزان معلومة. (المتوقع: `z` ثم تنشيط محدد.)
2. اشتق خسارة تربيعية بالنسبة لوزن واحد. (المتوقع: تطبيق مباشر لقاعدة السلسلة.)
3. قارن ReLU وSigmoid على قيم سالبة وموجبة. (المتوقع: ملاحظة التشبع والتصفير.)
4. بدّل SGD بـ Adam في المثال وقارن التقارب. (المتوقع: مقارنة لا حكم مطلق.)

5. اشرح لماذا لا تضمن نظرية التقريب تعميمًا جيدًا. (المتوقع: فصل التمثيل عن التعلم والبيانات.)
6. صمّم مصنفًا وحدد مقياس الفئة الحرجة. (المتوقع: قرار مرتبط بكلفة المصنع.)

6.21 ملخص

- الشبكة تركيب وحدات خطية ولاخطية تتعلم تمثيلات.
- الانتشار الخلفي يطبق قاعدة السلسلة، والمحسّن يحدّث الأوزان باتجاه يخفض الخسارة.
- التهيئة والتنشيط يؤثران في استقرار التدريب، والقدرة على التقريب لا تضمن التعميم.

الجزء الثاني · بناء الشبكات وتدريبها

الفصل السابع

07

المحسّنات والتنظيم: فنّ التدريب

حكاية الفصل

حفظ نموذجٌ بيانات مرضى التدريب بدقة ممتازة، لكنه أخفق في مستشفى آخر. لم تكن إضافة طبقات الحل؛ كان النموذج قد حفظ ضوضاء المستشفى الأول. كيف نفرّق بين تعلّم نمط مفيد وحفظ الأمثلة؟

7.1 أهداف الفصل

- [ ] فهم أثر معدل التعلم والزخم وخوارزميات التحسين.
- [ ] التمييز بين التطبيع والتنظيم.
- [ ] تشخيص فرط التعلّم وعدم استقرار الخسارة.
- [ ] بناء تدريب يستخدم الإيقاف المبكر والقص عند الحاجة.

7.2 المحسّن يحدد طريقة الحركة

SGD يخطو وفق التدرج الحالي. الزخم يجمع اتجاهات سابقة لتخفيف التعرج، RMSProp يطبع الخطوة وفق متوسط مربعات التدرج، Adam يجمع تقديري المتوسط والتباين. AdamW يفصل اضمحلال الوزن عن تحديث التدرج. لا توجد خوارزمية تفوز دائمًا؛ ثبّت تقسيم البيانات وقارن على المقياس الذي يخدم الاستخدام.

ثلاثة مسارات إلى القاع

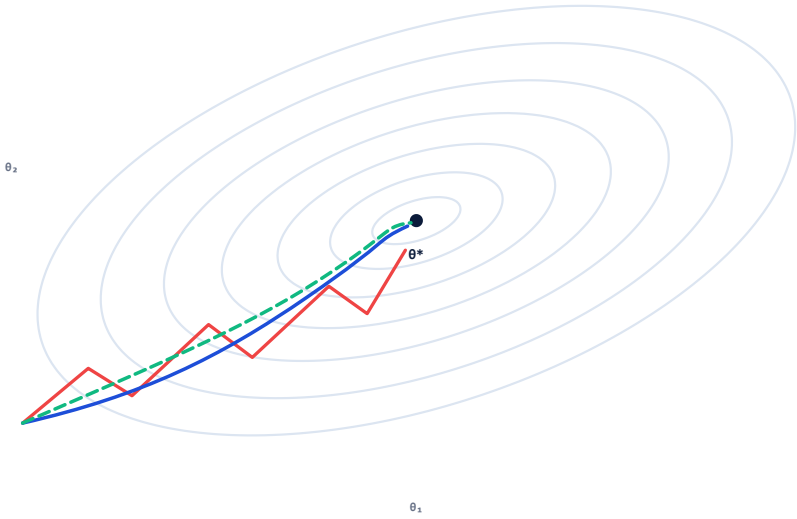
نفس سطح الخسارة، وسلوك مختلف في الاقتراب من الحل

SGD
خطوات بسيطة، تذبذب عالٍ

Momentum
ذاكرة للاتجاه، سلاسة أكبر

Adam
معدّل تعلّم تكيّفي لكل وزن

$$\theta \leftarrow \theta - \eta \nabla L(\theta)$$
القاعدة المشتركة



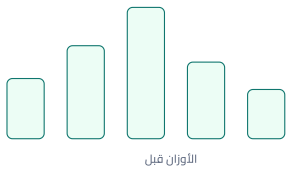
شكل 7-1 اتجاه المحسنات على سطح الخسارة

أربع أدوات لترويض الإفراط في التعلّم

كل أداة تعالج عرضًا مختلفًا؛ لا تستخدمها كلها بلا سبب

L2 / Weight Decay
عقوبة على الأوزان الكبيرة

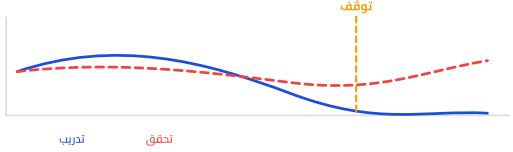
$$L = L_0 + \lambda \|w\|^2$$
λ كبيرة ← تقليص أقوى



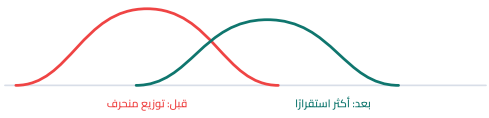
Dropout
تعطيل عصبونات عشوائيًا أثناء التدريب



EarlyStopping
أوقف التدريب قبل أن يبدأ الحفظ



BatchNorm
تطبيع التفعيلات داخل كل دفعة



شكل 7-2 أدوات التنظيم للحد من فرط التعلم

$$v_t = \beta v_{t-1} + (1-\beta) g_t$$

يمثل `g_t` التدرج، و`v_t` متوسطه المتحرك، و`β` معامل الزخم. يخفف المتوسط اهتزاز الإشارة في اتجاهات متضاربة، لكنه قد يحمل اتجاهًا لم يعد مناسبًا إن تغيرت طبيعة البيانات.

العرض	تفسير محتمل	تجربة أولى
خسارة تتذبذب بعنف	معدل تعلم مرتفع أو دفعات صغيرة	خفّض المعدل وراقب التدرج
تدريب جيد وتحقق ضعيف	فرط تعلّم أو انقسام غير ممثل	تنظيم وتحليل الانقسام
كلاهما ضعيف	نموذج محدود أو إعداد تدريب غير كاف	افحص خط الأساس والميزات
تحقق يتدهور بعد تحسن	بدء حفظ بيانات التدريب	إيقاف مبكر واستعادة الأفضل

7.3 التنظيم يحدّ من اعتماد النموذج على تفاصيل العينة

L1 يدفع بعض الأوزان نحو الصفر، وL2 يعاقب الأوزان الكبيرة. Dropout يعطّل وحدات عشوائيًا أثناء التدريب، ولا يعمل بالطريقة نفسها في التنبؤ. BatchNorm يطبّع وفق إحصاءات الدفعة ويملك سلوك تدريب واستدلال مختلفًا؛ LayerNorm يطبّع داخل المثال، ولهذا يناسب تسلسلات كثيرة. التنظيم أداة، وليس بديلًا عن تقسيم صحيح أو بيانات ممثلة.

```python
import keras
from keras import layers

model = keras.Sequential([
    layers.Input((64,)),
    layers.Dense(128, activation="relu", kernel_regularizer=keras.regularizers.l2(1e-4)),
    layers.Dropout(0.25),
    layers.Dense(1, activation="sigmoid"),
])
model.compile(optimizer=keras.optimizers.AdamW(learning_rate=3e-4, weight_decay=1e-4),
              loss="binary_crossentropy", metrics=[keras.metrics.AUC(name="auc")])
callbacks = [keras.callbacks.EarlyStopping(monitor="val_auc", mode="max",
                                           patience=3, restore_best_weights=True)]
```

7.4 معدل التعلم والقص والدقة المختلطة

ابدأ بمعدل تعلم معقول وسجل المنحنيات. جدولة Cosine تخفض المعدل تدريجيًا؛ `ReduceLROnPlateau` يتفاعل مع ثبات مقياس التحقق. قص معيار التدرج يحد من انفجار التحديث لكنه لا يعالج السبب الجذري. الدقة المختلطة تقلل كلفة الذاكرة والحساب على مسرّعات مناسبة، لكنها تتطلب الانتباه إلى دعم العتاد ودقة العمليات الحساسة.

7.5 حالة واقعية: تعميم نموذج المستشفى

قسّم البيانات بحسب المستشفى أو المريض قبل التدريب حتى لا تظهر صور الشخص نفسه في مجموعتين. راقب خسارة التحقق والاستدعاء لكل موقع. إذا تحسنت مجموعة التدريب وحدها، لا تعالج الأمر بمجرد رفع epochs؛ افحص التسرب والتحول بين المواقع، ثم جرّب augmentation أو تقليل سعة النموذج أو regularization.

7.6 مسارات ضبط منهجية

ابدأ بمعدل تعلم يحقق نزولًا واضحًا ومستقرًا، ثم غيّر عاملًا واحدًا في كل تجربة. مخطط الخسارة مقابل الزمن يكشف المعدل غير المناسب: انفجار متكرر يوحي بخطوة كبيرة، ونزول بطيء جدًا قد يشير إلى خطوة صغيرة أو تدرج ضعيف. لا تقارن runs على تقسيمات مختلفة. سجل البذرة، ونسخة البيانات، وحجم الدفعة، والتطبيع، والمعدل، والمقياس، ووقت التدريب.

SGD مع momentum قد يعمم جيدًا ويحتاج ضبط معدل وجدول بعناية. Adam يضبط الخطوات لكل معامل ويساعد على بدء سريع، لكنه قد يحتاج tuning للتعميم. AdamW يفصل اضمحلال الأوزان عن التحديث التكيفي. L2 regularization يضيف حدًا إلى الخسارة، في حين weight decay يضمحل المعامل ضمن قاعدة المحسن؛ تنفيذ Adam القديم قد يربطهما بخلاف AdamW. لا تفسر `weight_decay=...` على أنه مكافئ تلقائيًا لـ kernel L2 في كل محسن.

7.7 تطبيع داخل الشبكة لا يطبع البيانات الأصلية

BatchNormalization يستخدم إحصاءات الدفعة أثناء التدريب ويخزن متوسطًا وتباينًا متحركين للاستدلال. لذلك يجب اختبار المسارين `training=True` و `training=False`، مع الدفعات الصغيرة جدًا قد تكون الإحصاءات مشوشة. LayerNormalization يطبّع محاور المثال دون الاعتماد على بقية أمثلة الدفعة، ولهذا

يناسب كثيرًا من طبقات التسلسل. كلاهما يغيران optimization، لكنهما لا يغنيان عن تطبيع المدخلات عند اختلاف وحدات الميزات.

7.8 وصفة تشخيص عملية لفرط التعلّم

إذا انخفضت خسارة التدريب واستقرت خسارة التحقق ثم ارتفعت، فهذه إشارة إلى حفظ تفاصيل التدريب. افحص أولًا التسرب وتقسيم البيانات؛ التنظيم لا يصلح اختبارًا ملوثًا. بعد ذلك جرّب بالترتيب: تقليل عدد epochs مع استعادة الأفضل، augmentation لا يغير حقيقة الملصق، dropout أو weight decay معتدل، ثم نموذج أصغر. لكل خطوة سجل هل تحسنت فجوة التعميم والمقياس الذي يهم المجال.

7.9 الدقة المختلطة والاستقرار العددي

تضبط سياسة الدقة قبل بناء الطبقات. `mixed_float16` قد يسرع وحدات Tensor Core على GPU حديثة، و`mixed_bfloat16` مناسب لعتاد يدعم نطاق bfloat16. تحتفظ Keras بمتغيرات كثيرة بدقة float32 وتطبق loss scaling تلقائيًا في إعدادات `fit` المعتادة. افحص الخسارة وعدم وجود NaN، وقارن مقياس التحقق وذاكرة الذروة. إذا كان الجهاز CPU لا يملك عتادًا ملائمًا، قد يكون المسار أبطأ.

```python
import keras

# اضبط السياسة قبل بناء النموذج، ثم تحقق من dtype للطبقات والمخرجات.
keras.mixed_precision.set_global_policy("mixed_float16")
model = make_model()
print(model.layers[0].compute_dtype, model.layers[0].variable_dtype)
```

اجعل آخر طبقة إخراج حساسة بدقة مناسبة عند الحاجة، خصوصًا logits وخسائر ذات مجال عددي ضيق. لا تقارن نسخة مختلطة بأخرى مختلفة في batch أو عدد خطوات، وإلا لن تعرف مصدر فرق السرعة.

7.10 جداول معدل التعلم

جدول ثابت الخطوات يخفض المعدل بعد عدد محدد من الخطوات، Cosine يخفّضه بسلاسة نحو حد أدنى. `ReduceLROnPlateau` ينتظر ثبات metric لكنه يعتمد على ضجيج validation. patience حدد وحدة الزمن: هل `decay_steps` حقبة أو دفعة؟ في التدريب الموزع قد يتغير عدد الخطوات لكل حقبة. سجل المعدل الفعلي مع كل epoch، ولا تختَر جدولًا طويلًا يجعل التدريب لا يصل أبدًا إلى مرحلة التناقص.

اختبار range صغير يرفع معدل التعلم تدريجيًا ويراقب loss؛ قد يساعد على اقتراح نطاق آمن، لكنه heuristic لا يغني عن tuning. تجارب البحث العشوائي أو Hyperband يجب أن تستخدم تقسيمًا ثابتًا budgetg واضحًا، وتستبعد تجربة فشلت لأسباب بيئية بدل عدّها قيمة سيئة للنموذج.

7.11 Dropoutg BatchNorm في inference

BatchNorm يتبدل بين inferenceg training: أثناء التدريب يستخدم إحصاءات batch ويحدث moving averages، وأثناء الاستدلال يستخدم القيم المتراكمة. إذا استدعيت النموذج يدويًا بـ `training=True` عند التنبؤ، تتغير النتائج مع تركيب الدفعة. Dropout يعطل وحدات عشوائيًا أثناء التدريب ويصبح identity تقريبًا في inference. اختبر batch منفردًا ومجموعة مجمعة للتأكد من ثبات السلوك المنشور.

7.12 تنظيم يعكس فرضية

Dropout يحد من الاعتماد على وحدة مفردة، لكنه قد يبطئ تقارب نموذج صغير أكثر مما يحسن التعميم. L2 يشجع معاملات أصغر، وL1 قد ينتج أوزانًا صفرية لكنه لا يضمن تفسير السمات. Augmentation فرضية أن التحويل يحافظ على label؛ إذا كانت العبارة «لا» وانقلبت بالقص أو augmentation، فتعلم النموذج label غير صحيح. راجع عينات التحويل المرئي والملصقات مع خبير المجال.

7.13 خطة اختبار لعدم الاستقرار

إذا ظهرت loss=NaN، افحص أول دفعة تخرج فيها المشكلة، وقيم المدخلات والهدف، gradient normg، والدقة العددية. جرب خفض learning rate، إزالة عملية exponential غير مستقرة، أو تفعيل clipping إذا انفجر gradient. لا تعالج NaN بإخفائها عبر `nan_to_num` دون معرفة السبب. اكتب assertion للقيم finite عند مدخل pipeline ومخرج النموذج خلال التشخيص.

7.14 سجل المقارنة

التجربة	التغيير الوحيد	نتيجة التدريب	نتيجة التحقق	تكلفة
baseline	Adam، لا تنظيم إضافي	يسجل بعد التدريب	مرجع المقارنة	وقت/ذاكرة
A	خفض learning rate	يوضح استقرار الحركة	هل تحسن المقياس؟	زمن أطول محتمل
B	weight decay أو dropout	فجوة تعميم أصغر؟	تحسن على شرائح؟	قد يبطئ التقارب

تجنب اختبار كل التركيبات يدويًا بلا تسجيل. اكتب فرضية مسبقة لكل تغيير، واحتفظ بنتيجة فاشلة حين تفسر حدود المجال.

7.15 فرط التعلّم تحت المجهر

راقب فجوة التعميم `val_loss - train_loss` على امتداد epochs، لكن تذكر أن loss تعتمد مقياس الخسارة والتوزيع. افحص منحنى تعلم بزيادة كمية بيانات التدريب؛ إذا تحسن التحقق مع المزيد، فالعينة قد تكون غير كافية. إذا لم يتحسن، راجع جودة السمات أو label noise أو انحياز النموذج. سجل أخطاء يدوية بحسب الفئة والمصدر بدل الاعتماد على الخط البياني وحده.

7.16 وزن الفئات والـ focal loss

class weights تعظم أثر الأخطاء في الفئة الأقل تمثيلًا، لكنها تغير معايرة الاحتمال وقد ترفع false positives. Focal loss تقلل وزن الأمثلة السهلة وتركز على الصعبة، وملائمة لبعض كشف الأجسام، لكنها ليست الخيار الافتراضي لكل عدم توازن. قارنها مع baseline وقيّم precision-recall calibrationg على توزيع الإنتاج. إذا كان هناك resampling، نفذه في train فقط ولا تكرر أمثلة validation.

7.17 Early stopping ومشكلة epoch المختارة

patience يجب أن تسمح بتذبذب metric لا أن تطيل التدريب بلا داعٍ. اكتب `restore_best_weights=True`، وحدد min_delta وmonitorg حسب الهدف. إذا كانت validation صغيرة أو noisy، قد تكون نقطة الأفضلية عرضية؛ استخدم smoothing للتشخيص لا لتحويل score الرسمي، وراجع folds أو فترة تحقق أكبر. لا توقف على `loss` إذا كان قرار النجاح يستند إلى recall عند threshold.

7.18 تنظيم البيانات أهم من تنظيم الوزن

قبل L1/L2، تأكد من أن split يمنع تكرار أصل الصورة أو العميل. تسرب المعالجة المسبقة يجعل التحقق متفائلًا حتى مع نموذج بسيط. ثبت augmentations الصحيحة دلاليًا، واستخدم data collection لسد فجوات المجال. النموذج ينتظم وفق المعلومات المتاحة له؛ لا يستطيع dropout إزالة تحيز label موروث.

7.19 جدول قرار للتقنيات

العلامة المرصودة	أول تشخيص	تجربة محددة
train valg مرتفعان	underfit أو pipeline خطأ	overfit لدفعة صغيرة، ثم سعة أعلى
train منخفض valg مرتفع	overfit أو split مختلف	تحقق من التسرب ثم تنظيم/بيانات
loss انفجرت	LR أو أعداد غير مستقرة	خفض LR وفحص المدخل والgradient
recall منخفض لفئة نادرة	imbalance أو threshold	PR curve وأوزان على train
الجودة جيدة والزمن سيئ	input أو kernel بطيء	profiler ثم اختبار mixed precision

7.20 أمثلة تنفيذية إضافية

قص التدرج

```python
optimizer = keras.optimizers.Adam(learning_rate=3e-4, clipnorm=1.0)
model.compile(optimizer=optimizer, loss="binary_crossentropy")
```

جدول Cosine

```python
schedule = keras.optimizers.schedules.CosineDecay(
    initial_learning_rate=1e-3, decay_steps=1000, alpha=0.1)
optimizer = keras.optimizers.Adam(learning_rate=schedule)
```

7.21 أخطاء شائعة

⚠ تحذير

BatchNorm مع دفعات صغيرة جدًا قد يقدّر إحصاءات غير مستقرة؛ قارنه بـ LayerNorm أو إعدادات ملائمة.

[DANGER!] لا تضف Dropout وL1 وL2 وكل منظمات التدريب دفعة واحدة؛ عندها يصعب معرفة سبب التحسن أو التدهور.

7.22 تمارين

1. ارسم منحنيات تدريب وتحقق وحدد علامة فرط التعلّم. (المتوقع: فجوة متزايدة بعد تحسن التدريب.)
2. قارن SGD وAdam على إعداد ثابت. (المتوقع: تسجيل المنحنى والزمن لا الدقة النهائية وحدها.)
3. أضف EarlyStopping مع استعادة أفضل أوزان. (المتوقع: نموذج يطابق أفضل حقبة تحقق.)
4. اختر BatchNorm أو LayerNorm لتسلسل قصير وعلّل. (المتوقع: ربط القرار ببعد التطبيع والدفعة.)
5. اقترح سببًا لتذبذب خسارة التدريب. (المتوقع: اختبار معدل التعلم وحجم الدفعة.)
6. فسّر متى يفيد قص التدرج. (المتوقع: الحد من تحديثات منفجرة لا تصحيح بيانات خاطئة.)

7.23 ملخص

- المحسّن ومعدل التعلم يحددان مسار خفض الخسارة.
- التنظيم يقلل فرط التعلّم، لكن يجب تشخيص السبب باستخدام مجموعات مستقلة.
- سجّل المنحنيات، واستعد أفضل أوزان، ولا تضف تقنيات متعددة بلا تجربة واضحة.

الجزء الثاني · بناء الشبكات وتدريبها

الفصل الثامن

08

التقييم والضبط: كيف تعرف أن نموذجك جيد؟

> **حكاية الفصل**
>
> في شركة اتصالات، بنى فريقٌ نموذجًا للتنبؤ بفقدان العملاء (churn). أظهر التقرير دقة 92%، فاحتفل الجميع. بعد شهرٍ من التشغيل، لاحظ مدير التسويق أن النموذج لم يُنبّه إلى أي عميلٍ راحلٍ تقريبًا: كان يقول «سيبقى» للجميع، لأن 92% من العملاء يبقون فعلًا. لم يكن الحساب خاطئًا؛ كانت **الدقة** وحدها هي الكذبة. فكيف نقيس نموذجًا لا يكتفي بأن يبدو صحيحًا؟

8.1 أهداف الفصل

- [ ] تقسيم البيانات إلى تدريب وتحقق واختبار، وكشف التسريب قبل أن يفسد النتائج.
- [ ] استخدام التحقق المتقاطع الطبقي (stratified cross-validation) بدل الاعتماد على تقسيم واحد.
- [ ] قراءة مصفوفة الالتباس (confusion matrix) واشتقاق الإحكام والاستدعاء وF1 منها.
- [ ] تفسير منحنيات ROC وPR والمعايرة (calibration)، وربطها بقرارٍ عملي لا برقمٍ مجرّد.
- [ ] ضبط فرط الوسائط (hyperparameters) وتتبّع التجارب وتحليل الأخطاء بمنهجٍ قابل للتكرار.

8.2 ثلاثة صناديق: لماذا لا تكفي مجموعة واحدة؟

تخيّل طالبًا يدرس من أسئلة الامتحان نفسها، ثم يتفاجأ بعلامة متواضعة. النموذج يفعل الشيء ذاته حين نُقيّمه على البيانات التي تدرّب عليها. لذلك نقسم البيانات إلى ثلاثة صناديق: **التدريب (train)** الذي تُتعلَّم منه الأوزان، **التحقق (validation)** الذي نختار به فرط الوسائط ونوقف التدريب مبكرًا، و**الاختبار (test)** الذي لا نلمسه إلا مرة واحدة في النهاية. نسبة شائعة هي 70/15/15 أو 80/10/10، لكن النسبة تتبع حجم البيانات: كلما قلّت، زادت أهمية التحقق المتقاطع.

الاختبار أشبه بورقة مغلقة. إن نظرت إليها في كل تجربة، ثم اخترت النموذج الذي يناسبها، فقد حوّلتها إلى مجموعة تحقق ثانية، وصار تقديرك متفائلًا بلا أن تشعر. القاعدة: **الاختبار يُستخدم للحكم النهائي، لا للبحث.**

الخطر الأكبر هنا هو **التسريب (leakage)**: أن تصل معلومة من خارج مجموعة التدريب إليها. صورتُه الأشهر حساب متوسط وتطبيع البيانات على المجموعة كاملة قبل التقسيم، فيحمل كل مثال تدريبي أثرًا من عينات الاختبار. ومن صوره أيضًا:

- **تسريب الهدف (target leakage):** ميزة تُسجَّل بعد الحدث الذي تتنبأ به. نموذجٌ يتوقع إعادة دخول المريض للمستشفى يستخدم «عدد الزيارات اللاحقة»، وهي معلومة غير متاحة لحظة القرار.
- **تسريب المجموعات:** صور المريض نفسه أو معاملات العميل نفسه تتوزع بين التدريب والاختبار، فيبدو النموذج حافظًا وهو يتعرّف على الهوية.
- **تسريب زمني:** خلط صفوف سلسلة زمنية عشوائيًا، فيتدرّب النموذج على المستقبل ويتقيّم على الماضي.
- **تكرار العينات:** نسخةٌ من كل مثال في مجموعتين، أو إعادة عينات الأقلية قبل التقسيم.

الحل بنيوي لا لفظي: اقسم أولًا، ثم لائم كل معالجة داخل طية التدريب فقط. اجعل التطبيع والتشفير واختيار الميزات جزءًا من خط أنابيب يُعاد تركيبه في كل تقسيم. وإذا كان لديك معرفة بالمجموعات أو الزمن، فاستخدم تقسيمًا يحترمهما بدل التقسيم العشوائي.

8.3 التحقق المتقاطع: أن تختبر أكثر من مرة

تقسيم واحد قد يعطيك حظًا جيدًا أو سيئًا. **التحقق المتقاطع (cross-validation)** يعالج ذلك بتقسيم البيانات إلى K طيات (folds)، يتدرّب النموذج على $K-1$ ويتحقق على الطية المتبقية، ثم يدور الدور. النتيجة توزيعٌ من الدرجات، لا رقم واحد.

في التصنيف غير المتوازن نستخدم **التحقق المتقاطع الطبقي (StratifiedKFold)**، الذي يحافظ على نسبة الفئات في كل طية. وإذا كانت العينات مرتبطة بمجموعات، فـ `GroupKFold` يمنع تسرّب المجموعة. وللسلاسل الزمنية، `TimeSeriesSplit` يجعل كل تدريب يسبق تحققه زمنيًا.

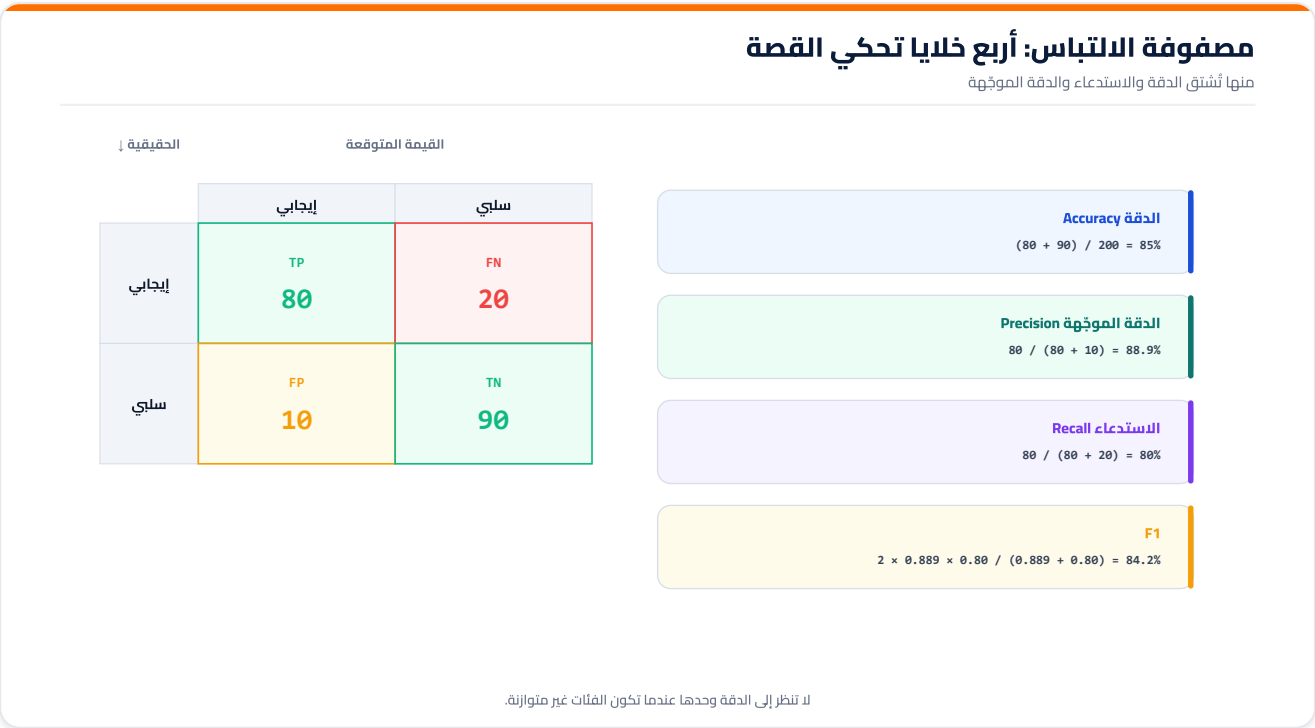
لماذا يهمّ التشتت؟ لأن متوسط الدرجات وحده لا يكفي. الخطأ المعياري للمتوسط يقارب:

$$\mathrm{SE} = \frac{s}{\sqrt{K}}$$

حيث s الانحراف المعياري للدرجات بين الطيات، وK عدد الطيات. كلما زاد K ضاق الخطأ، لكن زاد زمن التدريب. خمس طيات توازنٌ عملي شائع، وعشرٌ مفيدة عندما تكون البيانات صغيرة والزمن متاحًا. ولا تختر الطية التي أعطت أفضل رقم؛ فذلك تسريبٌ صغير لكنه حقيقي.

8.4 مصفوفة الالتباس: تشريح الخطأ

الدقة (accuracy) تجيب عن سؤال واحد: كم نسبة ما أصبت؟ لكنها لا تقول **أي** الأخطاء ارتكبت. مصفوفة الالتباس تفصّل ذلك في أربع خلايا: الموجب الصحيح (TP)، والموجب الخاطئ (FP)، والسالب الصحيح (TN)، والسالب الخاطئ (FN).



شكل 8-1 مصفوفة الالتباس: من أين تأتي الأخطاء؟

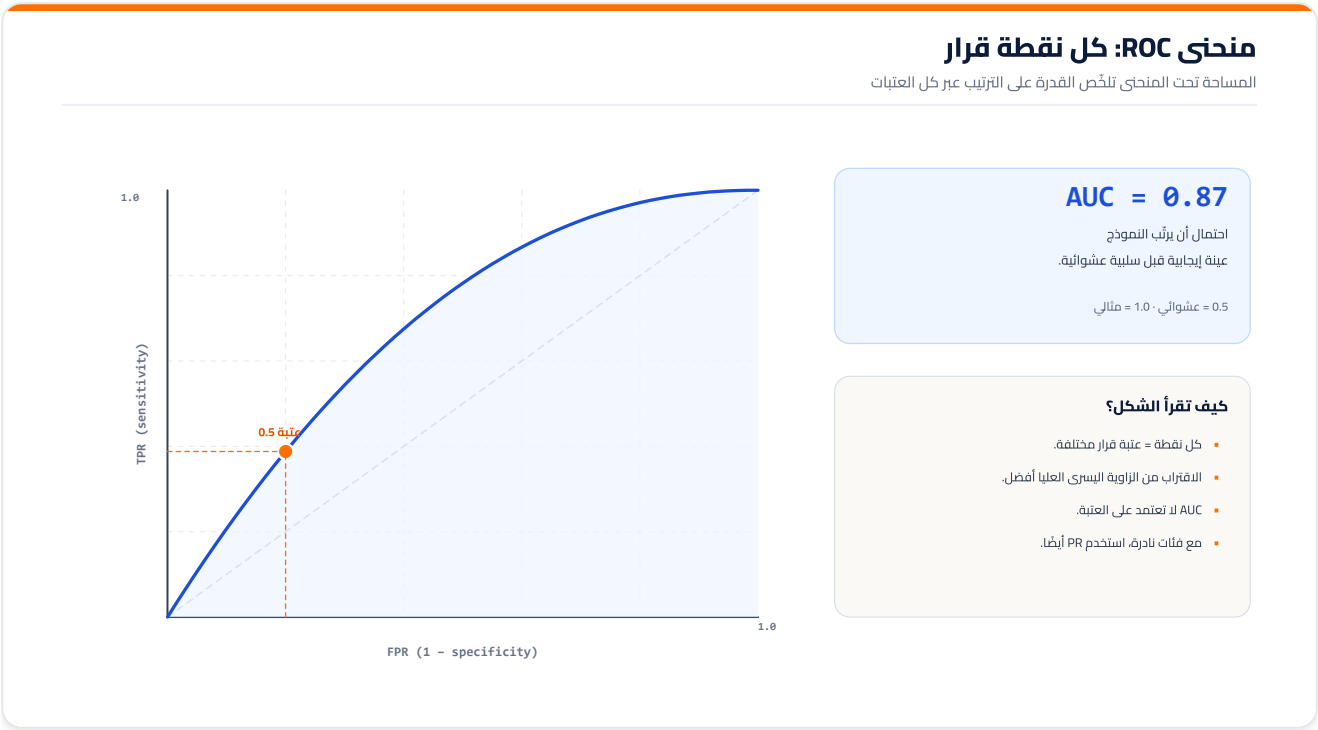
من هذه الخلايا نستخرج مقياسين متضادّين في الغالب. **الإحكام (precision)** هو نسبة ما تنبّأت به موجبًا وكان موجبًا فعلًا: $\mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$. و**الاستدعاء (recall)** هو نسبة الحالات الموجبة الحقيقية التي أمسكت بها: $\mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$. الأول يجيب: «حين أطلق الإنذار، كم مرة أكون محقًا؟». الثاني يجيب: «من كل الحالات التي كان يجب أن أمسكها، كم أمسكت؟».

يقيس **F1** التوازن بينهما:

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R}$$

حيث P الإحكام، وR الاستدعاء. إن كان أحدهما صفرًا، صار F_1 صفرًا: فهو لا يكافئ نموذجًا يضحّي بأحدهما كليًا. وهناك متوسطات أخرى مثل المتوسط الحسابي المرجّح، لكن F1 يبقى الأكثر شيوعًا حين تريد رقمًا واحدًا يلخّص التوازن.

منحنى ROC يرسم معدل الموجب الصحيح مقابل معدل الموجب الخاطئ عند كل عتبة ممكنة، و**AUC** هي المساحة تحته. قيمتها 0.5 تعني ترتيبًا عشوائيًا، و1 تعني ترتيبًا مثاليًا. هي مقياس لا يعتمد على العتبة، لذا تفيد في مقارنة قدرة النموذج على الترتيب، لكنها لا تخبرك بالعتبة التي تناسب عملك.



شكل 8-2 منحنى ROC ومساحة AUC

في البيانات النادرة، قد يبدو ROC متفائلًا لأن العدد الهائل من السلبيات يجعل معدل الموجب الخاطئ صغيرًا حتى مع أخطاء كثيرة. هنا يكون **منحنى الإحكام-الاستدعاء (PR)** أصدق: يركز على الفئة الموجبة وحدها. القاعدة العملية: استخدم PR عندما تكون الأقلية هي ما يهمّك، وROC عندما تريد صورة عن الترتيب عبر العتبات.

8.5 المعايرة: هل الثقة تعني الاحتمال؟

نموذجٌ يخرج 0.9 لا يعني بالضرورة أن تسعة من كل عشرة أمثلة كهذه موجبة. **المعايرة (calibration)** تسأل: هل تطابق الثقة المعلنة مع التكرار الفعلي؟ النموذج المُفرِط في الثقة (overconfident) يقول 0.95 ويصيب في 70% فقط، وهو خطير حين تُبنى القرارات على الاحتمال نفسه: حدّ الرفض، تسعير التأمين، ترتيب حالات المراجعة.

أبسط مقياس هو **درجة برير (Brier score)**: متوسط مربع الفرق بين الاحتمال المتوقع والنتيجة الحقيقية. ومقياسٌ آخر أسهل في التفسير هو **خطأ المعايرة المتوقع (ECE)**:

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left| \mathrm{acc}(B_m) - \mathrm{conf}(B_m) \right|$$

حيث M عدد حزم الثقة (مثل عشر حزم بين 0 و1)، وB_m مجموعة العينات التي وقعت ثقتها في الحزمة m، و$|B_m|$ عددها، وN العدد الكلي، و$\mathrm{acc}(B_m)$ نسبة الإجابات الصحيحة داخل الحزمة، و$\mathrm{conf}(B_m)$ متوسط الثقة فيها. الرمز $|\cdot|$ يعني القيمة المطلقة.

ارسم **مخطط الموثوقية (reliability diagram)**: الثقة على المحور الأفقي، والدقة الفعلية على الرأسي. النموذج المعاير يقترب من الخط القطري. ومن أبسط العلاجات **تدريج الحرارة (temperature scaling)**: اقسم لوغاريتمات المخرجات على ثابت $T > 1$ قبل softmax، فيلين التوزيع دون تغيير الترتيب. لاحظ أن أوزان الفئات وخسارة التركيز تُغيّر الاحتمالات عمدًا، لذا عاير النموذج بعد ضبطها إن كنت ستستخدم الاحتمال قرارًا.

8.6 عندما تكون الأقلية هي المهمة

في كشف الاحتيال، قد تكون نسبة الحالات الموجبة 0.5%. نموذجٌ يتنبأ «سليم» دائمًا يحقق دقة 99.5%، ويفشل في المهمة كلها. الحل ليس رقمًا سحريًا واحدًا، بل مجموعة أدوات.

أوزان الفئات (class weights) تكلّف النموذج أكثر عند الخطأ في الأقلية. وزن الفئة c يُحسب عادة:

$$w_c = \frac{N}{K \cdot N_c}$$

حيث N العدد الكلي للعينات، وK عدد الفئات، وN_c عدد عينات الفئة c. فإذا كان لديك 950 سالبًا و50 موجبًا، يصبح وزن الموجب 10 ووزن السالب نحو 0.53. مرّرها إلى `fit` عبر `class_weight`.

إعادة العينات (resampling) نوعان: تكرار الأقلية (oversampling) يحافظ على البيانات لكنه يضاعف خطر الحفظ، وحذف من الأكثرية (undersampling) يسرّع التدريب لكنه يرمي معلومة. والتوليد الاصطناعي مثل SMOTE يخلق عينات بين الجيران، وهو مفيد أحيانًا ومضرّ أحيانًا في البيانات عالية الأبعاد. الأهم: أعد العينات **بعد** التقسيم، وإلا تسرّبت نسخٌ من التدريب إلى الاختبار.

خسارة التركيز (focal loss) فكرة مختلفة: بدل وزن الفئة فقط، خفّض وزن الأمثلة السهلة التي صار النموذج يتقنها، وركّز على الصعبة.

$$\mathrm{FL}(p_t) = -\alpha_t \, (1 - p_t)^{\gamma} \log(p_t)$$

حيث p_t احتمال الفئة الصحيحة، وα_t وزنًا للفئة، وγ معامل التركيز: كلما كبر، قلّت مساهمة الأمثلة السهلة. عند $\gamma = 0$ تعود الخسارة إلى الانتروبيا المتقاطعة المعتادة.

الأسلوب	ما يغيّره	الخطر	متى يناسبك؟
أوزان الفئات	يكلف الخطأ في الأقلية أكثر	احتمالات غير معايرة	بداية سريعة وبلا تغيير في البيانات
تكرار الأقلية	يوازن الدفعات	حفظٌ وتكرار حرفي	بيانات قليلة ومعالجة بسيطة
حذف الأكثرية	يقلّل حجم البيانات	فقدان معلومات	بيانات ضخمة جدًا وزمن تدريب حرج
خسارة التركيز	يركّز على الأمثلة الصعبة	حساسية لـγ وα	تصنيف نادر وكثيف الأخطاء
تحريك العتبة	يوازن الإحكام والاستدعاء	لا يغيّر النموذج نفسه	عندما تعرف تكلفة كل خطأ

8.7 الضبط والتتبع وتحليل الأخطاء

بعد أن يصبح النموذج سليمًا، يبدأ السؤال: أي فرط وسائط نختار؟ **KerasTuner** يجرّب فضاءً محددًا: عدد الطبقات، الوحدات، معدل التعلّم، الإسقاط. **البحث العشوائي (RandomSearch)** أبسط من البحث الشبكي وأفضل منه غالبًا في الأبعاد الكثيرة. أما **Hyperband** فيبدأ تجارب كثيرة بدورات قصيرة، ويوقف الضعيف منها مبكرًا، ويُمنح الجيد مزيدًا من الدورات. هذا «التقليم المتتالي» يوفّر زمنًا كبيرًا حين يكون التدريب مكلفًا.

اختر هدفًا واحدًا يعبّر عن قرارك، مثل `val_auc` أو `val_precision` عند استدعاء ثابت. لا تُحسّن دقة خامّة في بيانات غير متوازنة، ولا تبحث على مجموعة الاختبار. سجّل مع كل تجربة: البذرة، وإصدار البيانات، والمعالجة، والمقياس، والزمن؛ فالتجربة التي لا يمكن إعادتها ليست نتيجة، بل ذكرى.

TensorBoard يجعل هذا مرئيًا: منحنيات الخسارة والمقاييس، توزيع الأوزان، الصور، ومنحنيات PR. اكتب سجلًّا لكل تشغيل تحت مجلد مختلف، ثم قارن. وإلى جانب الأدوات، يبقى **تحليل الأخطاء (error analysis)** أقوى خطوة: خذ مئة مثال خاطئ واقرأها بعينك. صنّفها إلى ضوضاء في الوسم، وحالات غامضة، وميزة ناقصة،

وانحراف في شريحة معينة. ثم اختبر الفرضية على شريحة أوسع. كثيرًا ما يكون التحسين الأكبر في البيانات أو المعالجة، لا في المعمارية.

تجربة صغيرة: من الاحتمال إلى مصفوفة الالتباس

يعيد النموذج احتمال الانتماء إلى الفئة الإيجابية، لكن مدير التشغيل يحتاج إلى قرارٍ واضح. نختار العتبة على مجموعة التحقق وفق كلفة الخطأ، ثم نثبتها قبل لمس الاختبار. المثال الآتي يحسب المقاييس مباشرة كي نرى أثر العتبة بدل الاكتفاء برقم الدقة:

```python
import numpy as np
from sklearn.metrics import confusion_matrix, precision_recall_fscore_support

# 1 تعني حالة تستلزم مراجعة بشرية.
y_true = np.array([1, 1, 1, 0, 0, 0, 0, 0])
prob = np.array([0.91, 0.62, 0.38, 0.81, 0.44, 0.31, 0.20, 0.05])
threshold = 0.60  # يختارها الفريق على التحقق وفق سعة المراجعة والكلفة
y_pred = (prob >= threshold).astype(int)

print(confusion_matrix(y_true, y_pred, labels=[0, 1]))
precision, recall, f1, _ = precision_recall_fscore_support(
    y_true, y_pred, average="binary", zero_division=0
)
print(f"precision={precision:.2f}, recall={recall:.2f}, F1={f1:.2f}")
# المتوقع: precision=0.67 وrecall=0.67 وF1=0.67 ثم [[2 ,1] ,[1 ,4]]
```

تُظهر المصفوفة أن صفوفها تمثّل الحقيقة وأعمدتها التنبؤ، بالترتيب الذي طلبناه `[0, 1]`. إذا كانت مراجعة الحالة السلبية مكلفة، فقد نرفع العتبة؛ وإذا كان تفويت حالة خطرة غير مقبول، نخفضها ونقبل بمراجعات أكثر. لا تُشتق العتبة من الاختبار، وإلا أصبح الاختبار جزءًا من التدريب.

8.8 حالة واقعية: حين صار قرار المراجعة أهم من AUC

شركة مدفوعات لديها 0.6% عمليات احتيال، وفريق مراجعة يستطيع فحص 300 تنبيه يوميًا. النموذج الأول حقق AUC مرتفعًا، لكن عند العتبة التي تعطي إحكامًا 0.95 لم يكتشف سوى 18% من الاحتيال. النموذج الثاني كان أقل في AUC، لكن منحنى PR لديه أفضل في المنطقة التي تهمّ: استدعاء 0.64 عند إحكام 0.28. بعد معايرة الاحتمالات على مجموعة تحقق، اختار الفريق عتبةً تُنتج نحو 280 تنبيهًا يوميًا، وربط تكلفة الاحتيال المفقود بتكلفة المراجعة. النتيجة لم تكن «نموذجًا أفضل»، بل **قرارًا أوضح**: كم حالة نريد أن نمسك، وبأي ثمن؟

8.9 أمثلة تنفيذية إضافية

اختيار العتبة

```python
import numpy as np
threshold = 0.60
y_pred = (validation_prob ⩾ threshold).astype("int32")
# Do not tune a threshold on the test set.
```

تقرير التصنيف

```python
from sklearn.metrics import classification_report
print(classification_report(y_true, y_pred, zero_division=0))
```

8.10 أخطاء شائعة

⛔ خطر

العرض: دقة ممتازة في التدريب والتحقق، ثم انهيار في الإنتاج. **السبب:** تطبيع أو اختيار ميزات حُسب على كل البيانات قبل التقسيم، أو ميزة تُعرف بعد الحدث. **الحل:** اقسم أولًا، ولائم المعالجة داخل طية التدريب، وراجع كل ميزة بسؤال: هل تتوفر فعلًا لحظة التنبؤ؟

⚠ تحذير

العرض: تتحسّن النتيجة في كل مرة تعيد فيها ضبط العتبة على الاختبار. **السبب:** الاختبار تحوّل عمليًا إلى مجموعة تحقق. **الحل:** اضبط على التحقق، وافتح الاختبار مرة واحدة، ثم سجّل النتيجة كما هي.

⚠ تحذير

العرض: نموذجان لهما أرقام مختلفة، وأحدهما «أفضل» بفارق 0.3%. **السبب:** تقسيمات مختلفة، أو معالجة مختلفة، أو بذرة واحدة بلا تكرار. **الحل:** وحّد التقسيم والبذور، وقارن المتوسط والتشتت عبر طيات متعددة، لا رقمًا واحدًا.

8.11 تمارين

1. **حساب يدوي:** مصفوفة تعطي TP=40، FP=10، FN=60، TN=890. احسب الإحكام والاستدعاء وF1. (المتوقع: الإحكام 0.8، الاستدعاء 0.4، F1 نحو 0.533.)
2. **تقسيم طبقي:** أنشئ بيانات بنسبة 8% موجبة، واقسمها بـ `train_test_split(stratify=y)`. اطبع نسبة الموجب في كل جزء. (المتوقع: نسبة قريبة من 8% في الثلاثة.)
3. **كشف تسريب:** مشروعٌ يطبّع كل الأعمدة قبل التقسيم، وفيه مرضى مكررون بين المجموعتين. اذكر التسريبين وخطة الإصلاح. (المتوقع: تسريب معالجة وتسريب مجموعات؛ الحل خط أنابيب داخل الطية وتقسيم بالمجموعة.)
4. **معايرة:** نموذج أعطى ثقة 0.9 لمئة عينة، وكان منها 70 موجبة. ما مساهمة هذه الحزمة في ECE؟ (المتوقع: 0.2 مضروبة في وزن الحزمة 0.1 إن كانت تمثل عُشر البيانات، أي 0.02.)
5. **أوزان:** بيانات فيها 950 سالبًا و50 موجبًا. احسب وزن كل فئة بالمعادلة. (المتوقع: نحو 0.526 للسالب و10 للموجب.)
6. **Hyperband:** لماذا قد يكون أسرع من البحث العشوائي بثلاثين تجربة كاملة؟ (المتوقع: لأنه يوقف التجارب الضعيفة بعد دورات قليلة ويخصص الزمن للواعد منها.)
7. **تحليل أخطاء:** لديك 25 حالة موجبة فاتت النموذج. اقترح ثلاث فرضيات وطريقة اختبار كل واحدة. (المتوقع: وسم خاطئ، ميزة ناقصة، انحراف شريحة؛ تُختبر بمراجعة العينات ومقارنة الشرائح.)
8. **اختيار مقياس:** بيانات احتيال بنسبة 1% وفريق مراجعة محدود. أتختار ROC أم PR؟ (المتوقع: PR، لأنه يوضّح الأداء في الفئة النادرة وقرار العتبة.)

8.12 ملخص

- مجموعة الاختبار تُفتح مرة واحدة؛ كل نظرة إضافية إليها تسرّب معلومة إلى قرارك.
- التسريب يبدأ من المعالجة والتكرار والزمن والمجموعات، لا من النموذج وحده.
- التحقق المتقاطع الطبقي يعطيك توزيعًا لا رقمًا، ويكشف هشاشة النموذج.
- الإحكام والاستدعاء يقيسان وجهين مختلفين للخطأ، وF1 يوازن بينهما بلا سياق التكلفة.
- ROC يقيس الترتيب، وPR أنسب للفئات النادرة، والمعايرة تقيس صدق الاحتمال.
- أوزان الفئات وإعادة العينات وخسارة التركيز تعالج عدم التوازن، لكنها تُغيّر الاحتمالات.
- الضبط بلا تتبّع تجارب وبلا تحليل أخطاء يعيدك إلى نفس النقطة بثقة أكبر.

الجزء الثاني · بناء الشبكات وتدريبها

الفصل التاسع

09

tf.data: هندسة خطوط البيانات

❝ حكاية الفصل

صار تدريب مصنف الصور سريعًا على GPU، لكن الجهاز ينتظر البيانات معظم الوقت. كانت الصور تُقرأ وتُفكّ واحدةً تلو الأخرى في Python. كيف نجعل خط الإدخال يواكب المسرّع دون تحميل البيانات كلها إلى الذاكرة؟

9.1 أهداف الفصل

- ☐ بناء Dataset من موترات أو مولّد.
- ☐ ترتيب shuffle mapg batchg cacheg prefetchg.
- ☐ فهم TFRecord والتوازي.
- ☐ قياس خط البيانات وتحديد اختناقه.

9.2 Dataset سلسلة تحويلات

يمكن النظر إلى إنتاجية الخط على أنها أقل إنتاجية بين مراحله المتوازية:



شكل 9-1 خط تدفق البيانات بواسطة tf.data

$$Throughput_{pipeline} \approx \min(r_{read}, r_{parse}, r_{augment}, r_{device})$$

`r` معدل العناصر في الثانية لكل مرحلة. لذلك لا يفيد تسريع القراءة إذا بقي فك الصور أبطأ مرحلة.

تخيّل خطًا في مصنع: لا تستطيع محطة التعبئة إنتاج أكثر مما تسمح به أبطأ محطة قبلها. خط البيانات كذلك؛ فـ `map` يصف ما تفعله بكل مثال، و `num_parallel_calls` يتيح تنفيذ أمثلة مستقلة في الوقت نفسه، و `batch` يجمعها لتقليل كلفة استدعاء النموذج، و `prefetch` يحضّر الدفعة التالية بينما تعمل البطاقة على الدفعة الحالية. إذا بقي GPU خاملًا، فالسؤال المفيد ليس «كيف أزيد حجم النموذج؟» بل «أي مرحلة تجعل المسرّع ينتظر؟».

تبدأ `tf.data.Dataset` بمصدر، ثم تحول العناصر على دفعات أو بالتوازي. `from_tensor_slices` يناسب بيانات موجودة في الذاكرة؛ `from_generator` يربط مولّد Python حين لا يناسب المصدر ذلك، لكنه قد يحد من قابلية التسلسل والأداء. للملفات الكبيرة، اقرأها كتدفق بدل تجميعها أولًا في مصفوفة عملاقة.

```python
import tensorflow as tf

x = tf.range(1000, dtype=tf.float32)
y = 3.0 * x + 1.0
ds = tf.data.Dataset.from_tensor_slices((x, y))
ds = ds.shuffle(1000, seed=42, reshuffle_each_iteration=True)
ds = ds.batch(32, drop_remainder=False).prefetch(tf.data.AUTOTUNE)
for xb, yb in ds.take(1):
    print(xb.shape, yb.shape)
# المتوقع: دفعة حتى 32 قيمة ودفعة أهداف بالشكل نفسه.
```

9.3 ترتيب التحويلات قرار ذاكرة وزمن

نفّذ shuffle على أمثلة التدريب قبل batch كي تختلط الأمثلة، واجعل prefetch قرب نهاية السلسلة ليتداخل تجهيز الدفعة مع التدريب. `cache` يخزن ناتج التحويل؛ إن وضعته قبل augmentation العشوائي حافظت على القراءة المكلفة وأبقيت augmentation متغيرًا في كل مرور. إن وضعت cache بعده، قد تثبت التحويل العشوائي لأول مرور. افهم ما الذي تخزنه قبل استخدامه.

التحويل	الأثر	ملاحظة
`map(..., AUTOTUNE)`	تحويل العناصر	دالة نقية أسهل للتوازي
`shuffle(buffer)`	يخلط ضمن المخزن	المخزن الأصغر خلطه أقل
`batch(size)`	يجمع الأمثلة	آخر دفعة قد تكون أصغر
`cache()`	يقلل القراءة اللاحقة	يحتاج مساحة ذاكرة/قرص
`prefetch(AUTOTUNE)`	يتداخل الإدخال والتدريب	غالبًا يوضع أخيرًا

هذا مثال صغير يربط أسماء التحويلات بوظيفتها. `AUTOTUNE` يسمح لـTensorFlow باختيار مستوى من التوازي، ثم تقيس زمن الحقبة واستهلاك الذاكرة على جهازك. لا تضع `cache()` بلا تقدير للحجم: صور مفكوكة بدقة كاملة قد تستهلك أضعاف مساحة الملفات المضغوطة.

```python
import tensorflow as tf

AUTOTUNE = tf.data.AUTOTUNE
ds = tf.data.Dataset.from_tensor_slices((features, labels))
ds = ds.shuffle(10_000, seed=42, reshuffle_each_iteration=True)
ds = ds.map(parse_and_augment, num_parallel_calls=AUTOTUNE)
ds = ds.batch(64, drop_remainder=False)
ds = ds.prefetch(AUTOTUNE)

for batch_x, batch_y in ds.take(1):
    print(batch_x.shape, batch_y.shape)
```

عقد التحويل هنا واضح: تبدأ كل عينة بمدخل ووسم، ثم تعيد `parse_and_augment` زوجًا من الشكل نفسه، ثم يضيف `batch` محور الدفعة. وإذا كان التحويل عشوائيًا، فاجعله بعد التخزين المؤقت كي لا تعيد التدريب على نسخة augmentation ثابتة كل حقبة. أما خط الاختبار فيبقى حتميًا عادةً؛ فالخلط فيه يصعّب مطابقة التنبؤات مع أمثلتها.

9.4 الملفات الكبيرة TFRecord

TFRecord صيغة سجلات متسلسلة مفيدة للتدفق والتوزيع، وليست ضغطًا سحريًا للصور. خزّن مخطط الحقول وإصدار المعالجة مع البيانات. استعمل `interleave` لقراءة عدة ملفات بالتوازي عندما يسمح التخزين بذلك. قارن زمن الحقبة واستهلاك الذاكرة، لأن التوازي المفرط قد يزاحم التدريب بدل تسريعه.

9.5 حالة واقعية: صور خط إنتاج

يقرأ الخط أسماء الملفات والملصقات، يخلط مسارات التدريب فقط، يفك الصور ويغيّر حجمها ضمن `map` متوازٍ، ثم يجمعها ويطبق augmentation أثناء التدريب. يبقى الاختبار حتميًا ومن دون تحويلات عشوائية. سجّل عدد الصور في الثانية على مراحل: القراءة، وفك الترميز، ثم التدريب؛ عندها تعرف موضع الاختناق.

9.6 من ملف صورة إلى دفعة مدربة

المسار القابل للصيانة يفصل تعداد الملفات عن فك الترميز وعن التحويل. خزّن قائمة المسارات والملصقات بإصدار معلوم، واخلط مسارات التدريب ببذرة محددة قبل القراءة، ثم طبّق augmentation بعد فك الصورة. عند تشغيل تقييم، لا تخلط ولا تستخدم تحولات عشوائية. يجب أن تكون كل خطوة محددة من اسم ملف أو

سجل إلى Tensor بمعنى معلوم.

```python
import tensorflow as tf

def read_image(path, label):
    encoded = tf.io.read_file(path)
    image = tf.io.decode_jpeg(encoded, channels=3)
    image = tf.image.resize(image, (224, 224))
    image = tf.cast(image, tf.float32) / 255.0
    return image, label

train_ds = tf.data.Dataset.from_tensor_slices((paths, labels))
train_ds = train_ds.shuffle(len(paths), seed=42, reshuffle_each_iteration=True)
train_ds = train_ds.map(read_image, num_parallel_calls=tf.data.AUTOTUNE)
train_ds = train_ds.batch(32).prefetch(tf.data.AUTOTUNE)
```

المثال يفترض أن الملفات JPEG وأن `paths` و`labels` متطابقان؛ أضف فحصًا للعناصر المفقودة والأبعاد والملصقات قبل تدريب مكلف. إذا كانت الصورة ملونة أو القناة اللونية مختلفة، ثبّت سياسة اللون في التحضير. لا تخلط التحويل إلى `[0,1]` مع `preprocess_input` الخاص بنموذج جاهز دون مراجعة ما يتوقعه النموذج.

9.7 متى تختار TFRecord؟

TFRecord حاوية لسجلات متسلسلة، و`tf.train.Example` يصف ميزات عددية أو بايتية أو قوائم أعداد. يفيد في تدفق ملفات مجزأة وتوزيعها على العمال، لكنه ليس قاعدة بيانات ولا يحسن الأداء تلقائيًا. ينبغي توثيق schema ونسخة المعالج، وفحص مجموعة عينة من السجلات، وقياس القراءة من التخزين المستهدف. تغيير صيغة التسجيل يتطلب قابلية ترحيل أو إعادة إنتاج المصدر.

9.8 الحتمية والتوزيع

لإعادة التجربة، ثبت البذور في مولد الخلط وفي مكتبات المعالجة، وسجل ما إذا كانت العمليات الحتمية مفعلة. قد تختلف بعض عمليات GPU أو الأداء بين العتاد والإصدار حتى مع بذرة واحدة؛ اذكر حدود إعادة الإنتاج بدقة. في التدريب الموزع، وزع shards بحيث لا يقرأ عاملان الأمثلة نفسها بالخطأ، واضبط batch size ليكون عالميًا أو لكل replica بوضوح. راقب عدد الأمثلة التي رأتها كل حقبة إذا كانت بياناتها غير قابلة للقسمة بالتساوي.

9.9 تشخيص اختناق البيانات

قِس إنتاجية النموذج على Tensor موجود في الذاكرة أولًا، ثم قِس الخط الكامل. إن كان النموذج سريعًا في الاختبار الأول وبطيئًا مع الملفات، فافصل القراءة، وفك الضغط، والتحويل، والخلط، والنسخ. `AUTOTUNE` يختار التوازي بناءً على الملاحظات لكنه لا يخلق مساحة تخزين أو CPU. كما أن `cache()` بعد تحويل مكلف قد يستهلك ذاكرة أكثر من المتاحة؛ يمكن التخزين إلى ملف مؤقت معلوم إذا كان مناسبًا.

9.10 الثبات وإعادة التشغيل

`Dataset.shuffle` يحتاج buffer بحجم مناسب للتوزيع؛ buffer صغير لا يخلط كامل المصدر، وإن كان يحسن التنوع عن الترتيب الأصلي. ثبّت seed وسجل `reshuffle_each_iteration` . الحتمية الكاملة قد تتطلب تفعيل عمليات deterministic وقد تقلل السرعة، كما تختلف بين إصدارات GPU. في تجربة علمية، سجل ما الذي ثُبّت بالفعل وما الذي بقي احتماليًا.

`repeat()` يجعل المصدر لا نهائيًا؛ عند استخدامه يجب تحديد `steps_per_epoch` وإلا لن تعرف نهاية epoch. لا تستخدم `repeat` بلا داعٍ مع `fit` على Dataset منتهٍ. `drop_remainder=True` يثبت حجم الدفعة ويساعد بعض المجمعات، لكنه يرمي أمثلة أخيرة كل حقبة؛ احسب عددها وقرر هل ذلك مقبول للفئات النادرة.

9.11 schemag TFRecord

سجل TFRecord يحتوي بايتات، و `tf.train.Example` رسالة features بأشكال bytes أو float أو int64. خزّن في وصف خارجي نسخة schema وكيفية فك كل ميزة. ابدأ بعدد ملفات يناسب التوازي؛ ملف ضخم واحد قد يقيد القراءة، وملفات صغيرة جدًا تضيف كلفة فتح metadata. استخدم checksum أو إحصاء عدد السجلات لتكشف ملفًا مبتورًا قبل التدريب. لا تخزن مسارات محلية مطلقة إذا ستنقل dataset إلى فريق آخر.

9.12 زيادة البيانات على مستوى batch

هناك augmentation في طبقات Keras مثل RandomFlip تعمل داخل النموذج أثناء التدريب، وهذا يسهل تضمين التحويل ضمن artifact ويجعل سلوك الاستدلال identity عند `training=False` . وهناك augmentation داخل `Dataset.map` ، وهذا قد يوازن الكلفة مع الإدخال. حدد هل عشوائية التحويل per-example أو per-batch. إذا نفذت تعديلًا على label كصندوق كشف أو قناع، طبّق الهندسة نفسها على الصورة والهدف؛ تحويل الصورة وحدها يفسد الإشراف.

9.13 مراجعة قبل التدريب

ارسم 16 عينة بعد كل خطوة: بعد القراءة، وبعد resize، وبعد normalization، وبعد augmentation. اطبع shape وdtype وmin وmax ومدى القيم. افحص عدة labels وclass counts، ومرر batch واحدة في النموذج. هذه المراجعة الصغيرة تلتقط خلط المسار بالملصق، القناة اللونية، وقناعًا فقد المحاذاة قبل صرف وقت GPU.

9.14 نمط reproducible لـ Dataset

إذا كان التحويل يستخدم عشوائية، فمرر seed صريحًا أو استخدم طبقات Keras ذات seed، وسجل ترتيب الخلط. لا تفترض أن `shuffle(seed=42)` يجعل كامل التدريب مطابقًا بين إصدارات النظام أو عدد العمال. عند الحاجة إلى تدقيق علمي، فعّل deterministic operations، واختبر أن dataset نفسها تعطي تسلسلًا ثابتًا، ثم قس تكلفة الحتمية.

في التوزيع، استفد من `Dataset.shard` أو استراتيجية التوزيع المعتمدة كي لا يعالج كل عامل كامل البيانات. احسب عدد الأمثلة الفعلي لكل عامل بعد drop remainder. إذا كانت الفئات مصنفة في مجلدات مرتبة، فاختلاط المسارات يزيل تحيز ترتيب الملفات؛ لكنه لا يصلح تقسيمًا مسربًا بين مصادر متشابهة.

9.15 مسألة cache عمليًا

`cache()` بلا مسار يحتفظ بالبيانات في ذاكرة العملية، ويكتمل التخزين بعد استهلاك dataset إلى النهاية. إذا أوقفت epoch قبل إكمال المرور، قد يعاد بناء cache في الحقبة التالية. إذا كان التحويل بعد cache عشوائيًا، يتكرر augmentation مع كل epoch؛ إذا قبله، قد تتجمد العينة. اختر موقع cache بناءً على ما هو مكلف وما ينبغي أن يبقى عشوائيًا، وراقب مساحة القرص عند استخدام اسم ملف.

9.16 معايير قبول خط البيانات

عند تسليم pipeline، قس وقت تحميل batch، إنتاجية الحقبة، نسبة تلف الملفات، عدد الأمثلة، توازن الفئات، ذاكرة cache، وثبات المعالجة بين train وinference. اكتب عقدًا مختصرًا لكل عنصر Dataset. إذا غيّر الفريق image size أو feature schema، زد data version وأعد التقييم؛ لا تحدث نفس الملف في مكانه بصمت.

9.17 هندسة CSV والنصوص

لـ CSV، ثبت header وأنواع الأعمدة وencoding، وتعامل مع missing values قبل batch. لا تستخدم `from_tensor_slices` على DataFrame ضخم إذا كان التحويل ينسخ البيانات كلها إلى RAM؛ استخدم reader متدفقًا أو صيغًا موزعة. للنص، حول bytes إلى UTF-8 ثم tokenize مع vocabulary محدد. وثق هل التقطيع يتضمن علامات ترقيم وتشكيلًا، لأن هذه المعالجة جزء من نموذجك.

`Dataset.from_generator` مفيد لتكامل مصدر Python خاص، لكنه يضع قيودًا على graph serialization والتوزيع، وقد يعيد التنفيذ في Python لكل عنصر. اكتب `output_signature` واضحًا، وقارن سرعة generator مع مصدر أصلي TensorFlow. إذا كان المصدر قابلًا للتحويل إلى TensorFlow I/O أو TFRecord، قد تحصل على أداء أكثر استقرارًا.

9.18 البيانات التي تتغير عند القراءة

إذا كان المصدر live أو متغيرًا، سجّل snapshot أو query boundary قبل التدريب. قراءة قاعدة بيانات عبر عدة عمال قد تنتج مجموعات مختلفة بين التجارب. استخدم pagination أو shards ثابتة ومعرّف dataset version. عند فشل ملف، لا تسقطه بصمت عبر `ignore_errors` إذا كانت نسبته مهمة؛ سجل العدد والمصدر، وضع حدًا للفشل يسمح بإيقاف التدريب بدل تقييم بيانات ناقصة.

9.19 أمثلة تنفيذية إضافية

تحويل متواز

```python
ds = ds.map(parse_example, num_parallel_calls=tf.data.AUTOTUNE)
```

فحص الدفعة

```python
for features, labels in ds.take(1):
    print(features.shape, labels.shape)
```

9.20 فحوص قبول قبل حساب مكلف

احتفظ بإحصاءات الخط قبل وبعد إعادة بناء المصدر. إذا اختلف العدد أو التوازن أو متوسط السمة، أوقف المقارنة حتى تفسر الفرق؛ قد تكون البيانات تغيرت لا النموذج. احسب تكلفة epoch كاملة مبكرًا، ثم قرر إن كانت cache أو صيغة تسجيل أخرى تستحق وقت التحويل.

9.21 حجم الملفات والمشاركة

جزّئ سجلات TFRecord حسب مصدر تخزين وتوازي القراءة، واستخدم أسماء ملفات تشمل نسخة schema. لا تضع الملفات المؤقتة في مسار cache غير مضمون ثم تعتبرها مصدرًا دائمًا. احسب checksums عند نقل dataset، وسجل وقت اكتمال النسخ.

9.22 فحص label mapping

استخرج خريطة الفئات مرة واحفظها مع نسخة النموذج. إذا ولّد loader معرفات بفئات مرتبة أبجديًا بينما API تستخدم ترتيبًا يدويًا، تكون التوقعات صحيحة رقميًا ومسمّاة خطأ. أضف اختبارًا على اسم معروف وفئة نادرة.

9.23 أخطاء شائعة

> ⚠ **تحذير**
>
> `shuffle` بعد `batch` يخلط الدفعات لا الأمثلة داخلها. [DANGER!] لا تستخدم `cache` في الذاكرة لبيانات تفوقها دون قصد؛ قد يتحول التسريع إلى نفاد ذاكرة.

9.24 تمارين

1. أنشئ Dataset من أزواج سمات وأهداف. (المتوقع: كل عنصر زوج متوافق.)
2. رتب التحويلات للتدريب والتقييم. (المتوقع: خلط للتدريب فقط.)
3. قارن prefetch مع بدونه. (المتوقع: قياس زمن الحقبة.)
4. فسّر أين تضع cache عند وجود augmentation عشوائي. (المتوقع: قبل التغيير العشوائي.)
5. اختر مصدرًا لبيانات أكبر من الذاكرة. (المتوقع: تدفق ملفات/TFRecord.)

6. حدّد اختناق خط صور. (المتوقع: قياس القراءة والفك والتدريب منفصلة.)

9.25 ملخص

- صمّم خطًا متدفقًا ومتوازيًا بدل قراءة عنصر في كل خطوة Python.
- ترتيب التحويلات يغير دلالة الخلط والعشوائية واستهلاك الذاكرة.
- قِس الإنتاجية ولا تفترض أن زيادة التوازي تفيد دائمًا.

الجزء الثاني · بناء الشبكات وتدريبها

الفصل العاشر

10

الرؤية الحاسوبية: CNN من الالتفاف إلى الإنتاج

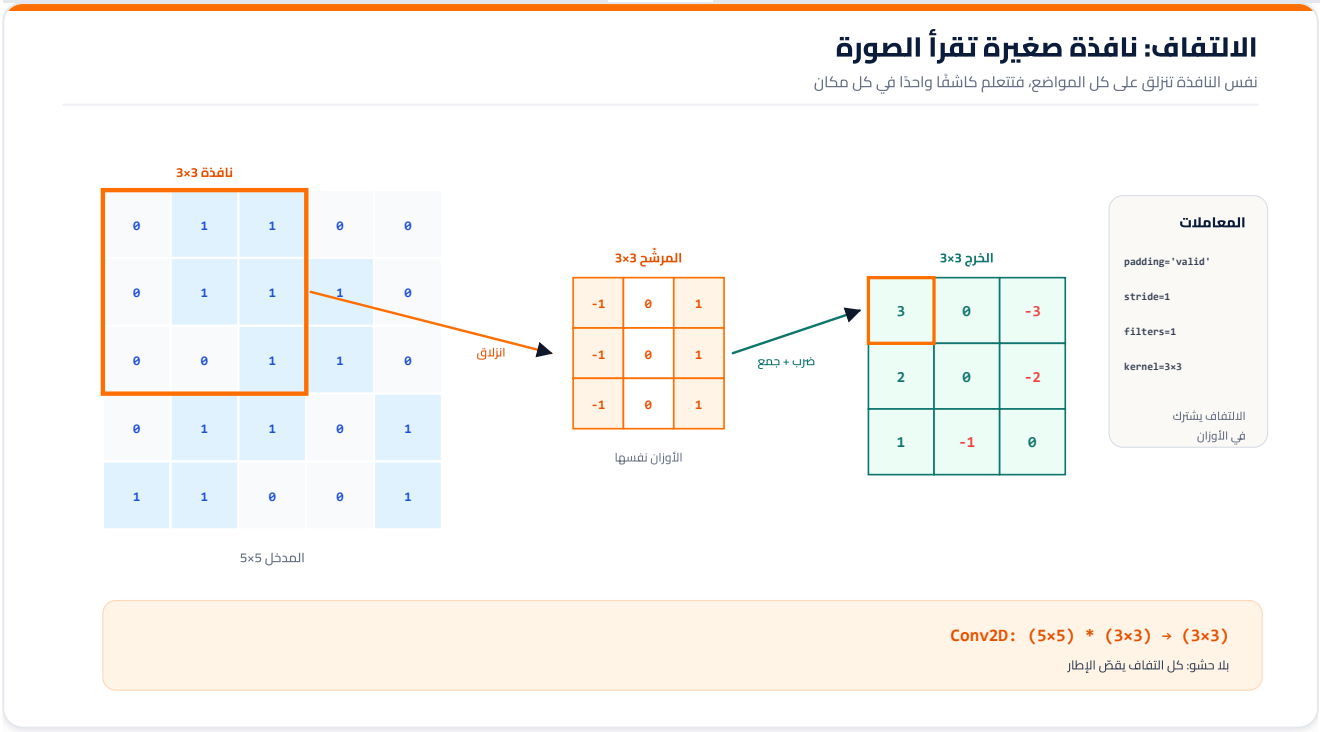
> **حكاية الفصل**
>
> طلبت ورشة صيانة فرز خدوش دقيقة على ألواح كبيرة. أعطى المصنف الكثيف نتائج ضعيفة، لأن تسطيح الصورة أزال الجوار المكاني ورفع عدد الأوزان. كيف تستفيد الشبكة من حقيقة أن الحافة القريبة تشبه الحافة نفسها في موضع آخر؟

10.1 أهداف الفصل

- ☐ حساب أبعاد الالتفاف وفهم القنوات والمرشحات.
- ☐ بناء CNN صغيرة وشرح pooling والتطبيع.
- ☐ مقارنة عائلات معمارية معاصرة.
- ☐ تفسير تنبؤ بصورة عبر Grad-CAM.

10.2 مرشح محلي يعاد استخدامه

تطبق طبقة الالتفاف مرشحات صغيرة على رقع متجاورة، فتتعلم حوافًا وأنماطًا محلية وتعيد استخدام الأوزان عبر المواضع. يحدد `stride` مقدار الحركة، ويضيف `padding="same"` حوافًا للحفاظ تقريبًا على الحجم المكاني. عدد المرشحات هو عدد قنوات الخرج.



شكل 10-1 حركة مرشح الالتفاف على الصورة



شكل 10-2 مكونات شبكة تلافيف للصور

$$H_{out} = \left\lfloor \frac{H + 2P - D(K-1) - 1}{S} \right\rfloor + 1$$

`H` ارتفاع الإدخال، و`K` حجم المرشح، و`P` الحشو، و`D` التمدد، و`S` الخطوة. تنطبق الصيغة على محور واحد، وتكرر للعرض.

```python
import keras
from keras import layers

model = keras.Sequential([
    layers.Input((64, 64, 3)),
    layers.Conv2D(32, 3, padding="same", use_bias=False),
    layers.BatchNormalization(), layers.Activation("relu"),
    layers.MaxPooling2D(),
    layers.Conv2D(64, 3, padding="same", activation="relu"),
    layers.GlobalAveragePooling2D(),
    layers.Dense(5, activation="softmax"),
])
model.summary()
```

يقلل pooling الدقة المكانية ويزيد مجال الرؤية النسبي، بينما `GlobalAveragePooling2D` يجمع كل خريطة سمات في قيمة واحدة ويحد من تضخم المعاملات مقارنة بـ Flatten.

10.3 من LeNet إلى ConvNeXt

العائلة	الفكرة التعليمية	ما الذي تراقبه؟
LeNet	التلافيف والتجميع للمشاهد الصغيرة	البساطة وعدد الطبقات
VGG	تتابع مرشحات صغيرة	كثرة الأوزان والذاكرة
ResNet	وصلات تخطٍ	تدريب شبكات أعمق
EfficientNet	موازنة العمق والعرض والدقة	كلفة الاستدلال
ConvNeXt	تحديث تصميم CNN الحديثة	السياق والموارد المطلوبة

لا تختَر النموذج باسم عائلته فقط. قارن الدقة والزمن والذاكرة على صورك، وافصل بيانات التحقق بحسب مصدر الصور كي لا يتسرب المشهد نفسه إلى التدريب والاختبار.

10.4 تفسير بصري لا يثبت السببية

Grad-CAM يجمع تدرجات فئة مستهدفة مع خرائط السمات في طبقة التفاف، فينتج خريطة تقريبية للمناطق المؤثرة في الدرجة. استخدمه لاكتشاف اعتماد النموذج على علامة مائية أو خلفية، لا لإثبات أن النموذج «فهم» الكائن. اختبر التفسير على أخطاء وصور مضادة وعلى طبقات مختلفة.

10.5 حالة واقعية: كشف خدش

تُقص صور الألواح وفق إجراء ثابت، وتُطبّع بالطريقة نفسها في التدريب والخدمة. يبدأ الفريق بخط أساس مدرب مسبقًا ثم يوازن الحساسية مع زمن الفحص. يسجل عدد الحالات التي فاتت كل وردية، ويعيد التقييم عند تغيير الكاميرا أو الإضاءة؛ فالتغير الفيزيائي قد يبدل التوزيع دون تغيير الشفرة.

10.6 مجال الرؤية وحساب الكلفة

ترى المرشحة الأولى رقعة محلية، ثم تزيد الطبقات والتجميع مجال الرؤية الفعلي. هذا يفسر كيف تنتقل الشبكة من الحواف إلى الأشكال، لكنه يفسر أيضًا لماذا لا تكفي طبقة صغيرة لعيب واسع. يمكن حساب مجال الرؤية نظريًا من تتابع أحجام المرشحات والخطوات؛ اكتب الحجم المكاني بعد كل طبقة على عينة صغيرة قبل تدريب المجموعة الكاملة.

معاملات التفاف 2D تحسب تقريبًا `K_h × K_w × C_in × C_out`، ويضاف انحياز لكل قناة إذا لم يُلغَ. تغيير عرض الشبكة يضاعف الحساب والذاكرة؛ مضاعفة عدد القنوات في الداخل والخارج قد ترفع عدد المعاملات بنحو أربعة أضعاف. قِس أزمنة inference على جهاز التنفيذ نفسه: بطاقات الخادم والهواتف تتفاوت في دعم المرشحات والدفعات.

10.7 زيادة البيانات يحافظ على معنى الملصق

التقليب الأفقي صالح لبعض صور المنتجات، لكنه قد يقلب معنى النص أو اتجاه قطعة غير متناظرة. الدوران الشديد قد يحول علامة اتجاه إلى عيب، والتباين المبالغ فيه قد يمحو خدشًا منخفض التباين. اربط كل زيادة بفرضية عن تغير العالم الحقيقي، واعرض لوحة عينات ناتجة قبل التدريب. طبّق التغيير على مجموعة التدريب فقط، واحتفظ بنسخة غير معدلة للاختبار.

10.8 النقل من نموذج مدرب مسبقًا

الأوزان المدربة على صور عامة تعلم سمات مفيدة غالبًا، لكن بيانات المجال تختلف في اللون والدقة وتوزيع الفئات. استخدم preprocessing الذي يطابق قاعدة النموذج، وجرب خط أساس مجمدًا ثم فك الطبقات العليا بمعدل أصغر. لا تخلط مصطلح `include_top=False` مع تطبيع الإدخال؛ الأول يزيل رأس التصنيف الأصلي، والثاني يحول الصورة إلى المقياس المتوقع.

10.9 تفسير Grad-CAM ومسؤولية التحقق

احسب تدرج logit الفئة بالنسبة إلى خرائط آخر طبقة تلافيفية، واجمع الخرائط بأوزان مشتقة من متوسط التدرجات مكانياً. طبّع الخريطة للعرض وارسمها فوق الصورة. اختبر حالات صحيحة وخاطئة، وصورة لا تحتوي الهدف، وصورة شُوّهت خلفيتها؛ إن بقيت الحرارة على إطار الصورة، فقد تعلم النموذج أثرًا جانبيًا. Grad-CAM لا يثبت سبب القرار ولا يكفي وحده للتصديق على نظام طبي أو صناعي.

10.10 من التدريب إلى القياس التشغيلي

لا تحسب الدقة على قصاصات متجاورة موزعة عشوائيًا إذا كانت كلها من اللوح أو الدفعة نفسها. اجمع التقييم على مستوى الوحدة التي سيواجهها النموذج: دفعة إنتاج، أو كاميرا، أو يوم عمل. اعرض confusion matrix وأداء كل نوع عيب والكمون عند حجم الدفعة المتوقع. إن كان على المفتش معالجة الصور في زمن محدد، فكمون p95 وحجم الذاكرة شرطان في تعريف النجاح.

10.11 وصفة تدريب قابلة للتكرار

قبل اختيار معمارية عميقة، أنشئ baseline: نموذج خطي أو CNN صغيرة، تطبيع ثابت، وتقسيم مجموعة بحسب مصدر الصور. ثم تحقق أن النموذج يمكنه overfit على دفعة واحدة؛ إذا لم يستطع، راجع labels وshapes وlossg. استخدم `ModelCheckpoint` لمراقبة `val_loss` أو مقياس مرتبط بالقرار، و `EarlyStopping` مع استعادة أفضل أوزان. خزّن تاريخ التدريب مع seed، وسجل الفئات وأي mapping.

في CIFAR أو صور عامة، augmentation معقول قد يحسن التعميم، لكن لا تقارن نسخة augmentation على مجموعة تحقق مختلفة. لا تستخدم augmentation على validation/test؛ يجب أن تظل هذه العينات ممثلة للقياس. عند عدم توازن الفئات، اعرض per-class recall وbalanced accuracy أو macro-F1 بجانب accuracy. إن كان هناك threshold، اختَره على validation ثم ثبّته قبل الاختبار.

10.12 لماذا يختلف أداء CNN بين المجالين؟

مرشحات ImageNet تتعلم قوامًا وأشكالًا مفيدة، لكن الصور الطبية قد تكون grayscale وأنماطها أدق من تصنيفات عامة. قص الصورة إلى 224×224 قد يمحو عيبًا صغيرًا، والضغط قد يغير texture. جرّب resolution وتنسيق الصورة والمعالجة كفرضيات، وافصل أثرها عن أثر العمق. في البيانات عالية الدقة، tiled inference قد يساعد لكنه يكرر الحواف ويتطلب دمج الصناديق أو الأقنعة.

10.13 ضوابط BatchNorm ودفعة صغيرة

BatchNorm يحدّث moving statistics من الدفعة. إذا كانت الدفعة صغيرة بسبب صور عالية الدقة، قد تصبح الإحصاءات noisy. يمكن إبقاء base المدرب مسبقًا في `training=False` خلال fine-tuning في بعض السيناريوهات، أو اختيار GroupNorm/LayerNorm إن كان التصميم يدعمها. لا تغير normalization في نموذج مدرب مسبقًا دون فحص؛ وزن الطبقات التالية يرتبط بتوزيع التفعيلات المتوقع.

10.14 تهيئة صور الخدمة

يجب أن ينفذ المعالج الصورّي نفس خطوة التدريب: decode، ترتيب القنوات، interpolation، resize/crop، نوع الدقة، normalization. اختبر صورة محفوظة من الإنتاج على pipeline التدريب وpipelineg الخدمة وقارن الموتر الناتج عنصرًا بعنصر. اختلاف BGR/RGB لا يسبب استثناء، لكنه يفسد التنبؤ بصمت. سجّل مكتبة decode وإصدارها عندما يؤثر على النتائج.

10.15 Grad-CAM: تطبيق وتفسير حذر

اختر طبقة التفاف أخيرة لها خريطة مكانية، واحسب gradient للـ class logit، ثم خذ متوسط gradient على المحاور المكانية كوزن لكل قناة. اجمع خرائط القنوات بعد قص الأوزان السالبة، ثم طبّع وأعد الحجم على الصورة الأصلية. يمكن لخريطة مرتفعة على الخلفية أن تكشف dataset shortcut، لكن خريطة تبدو معقولة ليست اختبارًا سببيًا. قارن على أمثلة خاطئة، وبدّل الخلفية عندما يمكن ذلك.

10.16 تصميم مشروع تصنيف صور كامل

ابدأ بجمع بيانات يوضح من التقطها وتحت أي ظروف، واكتب taxonomy فئات لا تتداخل. افحص أمثلة ملتبسة مع خبير واتفق على labeling guidelines. افصل train/validation/test على مستوى المصدر الذي تتوقع التعميم عليه. حدد baseline بسيطًا، ثم CNN صغيرة. احفظ confusions بحسب الفئة، واعرض أمثلة false positive

false negativeg، لأن صورة بعيدة عن التوزيع قد تشبه فئة أخرى.

في التدريب، استخدم augmentation مضبوطًا، learning rate schedule، checkpointg لأفضل metric تحقق. لا تغيّر validation مع كل تجربة. بعد اختيار القرار، افحص set خارجية من كاميرا أو دفعة إنتاج لم تستخدم للضبط. قد تكون الدقة الإجمالية عالية لكن الفئة التي تتطلب الصيانة هي الأضعف؛ استخدم decision threshold أو abstention للصور غير الواثقة بمراجعة بشرية.

10.17 الموازنة بين الدقة والكمون

ResNet أعمق لا يضمن أنه مناسب للكميرا. قس زمن preprocess + forward + postprocess على CPU وGPU الهدف. خفض resolution يقلل الحساب تقريبًا بحسب مساحة الصورة لكنه قد يمحو الأجسام الصغيرة. quantization أو pruning قد يصغر النموذج مع تغير الدقة. اختبر كل ضغط على benchmark ثابت، وسجل model size وكمون p95 وذاكرة الوصول العشوائي.

10.18 أخطاء من الواقع

من الأخطاء الشائعة تسميات غير متسقة بين annotators، قنوات لونية مقلوبة، صور مكررة، أو crop يقطع الكائن. كشف duplicate بصمة الصور يساعد على منع leak. عند استخدام إعادة التحجيم، حدد interpolation؛ nearest للأقنعة، bilinear أو bicubic للصورة وفق العملية. لا تتخلص من صور «غريبة» تلقائيًا؛ قد تكون الحالات الأصعب والأكثر أهمية للتعميم.

10.19 المراجعة العلمية

أبلغ البنية والوزن المدرب مسبقًا ومصدره، طريقة تقسيم البيانات وعدد الصور، سياسة التهيئة، optimizer وجدول التعلم، بذور، epochs، وعتبة القرار. قدم class-wise metrics وفواصل ثقة إن أمكن. اذكر نسخة TensorFlow/Keras والعتاد، وشارك كود preprocessing وmodel configg حتى يتمكن المختبر من تكرار القياس.

10.20 استكشاف feature maps

اعرض تفعيلات طبقة مبكرة وأخرى عميقة للصورة نفسها. الأولى قد تستجيب لحواف واتجاهات، والأخيرة لقوام أو أجزاء ذات صلة بالفئة. استخدام feature map تفسير وصفي لا حكم على فهم النموذج. إذا كانت كل القنوات صفرًا أو ثابتة، افحص تهيئة الطبقة ومقياس الإدخال وحالة training/inference.

10.21 التهيئة المسبقة والملصقات الصاخبة

أوزان pretrained تقلل احتياج البيانات لكنها قد تحمل تحيزًا من مجموعة المصدر. قارن evaluation على فئات أو خلفيات متباينة. في الملصقات غير المؤكدة، لا ترفع capacity قبل تدقيق annotation؛ قد يحفظ النموذج أخطاء الوسم. اجمع أمثلة خلاف المراجعين وحدد سياسة adjudication.

10.22 متطلبات اكتساب البيانات

ضع معيار تصوير يعالج مسافة الكاميرا والإضاءة والfocus والخصوصية. إذا تطورت كاميرات الخط، اجمع بيانات انتقال قبل الطرح الواسع. سجّل metadata مثل الجهاز والوردية في مكان مضبوط الوصول، واستخدمها للتقسيم والتحليل دون تمريرها للنموذج إن لم تكن متاحة وقت الاستخدام.

10.23 أمثلة تنفيذية إضافية

زيادة البيانات

```python
augment = keras.Sequential([
    layers.RandomFlip("horizontal"), layers.RandomRotation(0.05),
])
augmented = augment(images, training=True)
```

تطبيع قيم البكسل

```python
images = tf.cast(images, tf.float32) / 255.0
print(tf.reduce_min(images).numpy(), tf.reduce_max(images).numpy())
```

10.24 أخطاء شائعة

⚠ تحذير

`Flatten` المبكر قد يحول صورة صغيرة إلى طبقة كثيفة ضخمة ويهدر البنية المكانية. [DANGER!] لا تخلط صورًا متجاورة أو مشتقة من الصورة الأصلية بين التدريب والاختبار؛ هذا يرفع القياس دون تعميم حقيقي.

10.25 تمارين

1. احسب حجم خرج مرشح 3×3 بخطوة 2. (المتوقع: تطبيق الصيغة لكل محور.)
2. ابنِ CNN بأربع فئات. (المتوقع: مخرج احتمالات طوله 4.)
3. قارن Flatten وGlobalAveragePooling. (المتوقع: اختلاف عدد المعاملات.)
4. حدد تقسيمًا يمنع تسرب صور اللوح نفسه. (المتوقع: فصل بحسب اللوح أو الوردية.)
5. فسّر خريطة Grad-CAM مضللة. (المتوقع: فحص الخلفية وعلامات المصدر.)
6. اختر عائلة نموذج تحت قيد ذاكرة. (المتوقع: قياس النماذج على الجهاز المستهدف.)

10.26 ملخص

- الالتفاف يستفيد من المحلية ومشاركة الأوزان.
- الخطوة والحشو والمرشح تحدد الشكل والكلفة، وعمق الشبكة ليس هدفًا بذاته.
- Grad-CAM وسيلة فحص مفيدة وليست برهانًا على السببية.

الجزء الثالث

الرؤية والتسلسلات واللغة والتوليد

من الالتفاف إلى المحوّلات ونماذج الانتشار

11 الرؤية المتقدمة: النقل والكشف والتجزئة

12 التسلسلات: RNN/LSTM/GRU والسلاسل الزمنية

13 الانتباه والمحوّلات

14 معالجة اللغة الطبيعية والعربية

15 التوليد: GANs VAEs Autoencoder

16 نماذج الانتشار: التوليد الحديث

الفصول 11-16 · 6 فصول

الجزء الثالث · الرؤية والتسلسلات واللغة والتوليد

الفصل الحادي عشر

11

الرؤية المتقدمة: النقل والكشف والتجزئة

حكاية الفصل

لدى عيادة محلية مئات الصور فقط لتصنيف آفات نادرة، بينما دُرّبت نماذج عامة على ملايين الصور. هل نبدأ من الصفر، أم ننقل السمات، وكيف نمنع النموذج من حفظ اختلاف الأجهزة بدل المرض؟

11.1 أهداف الفصل

- [ ] تنفيذ استخراج السمات ثم الضبط الدقيق.
- [ ] تفسير IoU وNMS وmAP في الكشف.
- [ ] بناء U-Net للتجزئة وفهم Dice.
- [ ] تصميم تقييم لمجموعة بيانات صغيرة.

11.2 التعلم بالنقل مرحلتان

حمّل قاعدة مدربة مسبقًا بأوزان مناسبة للمجال، ثم جمّدها ودرب رأس التصنيف الجديد. بعد استقرار الرأس، افتح بعض الطبقات العليا وأعد `compile` بمعدل تعلم أصغر؛ فك التجميد دون إعادة التجميع لا يضمن تحديث الأوزان كما تتوقع. استخدم دالة المعالجة المسبقة التابعة للمعمارية، لأن ترتيب القنوات ومقياس البكسل جزء من نموذجها.



شكل 11-1 مراحل التعلم بالنقل والضبط الدقيق



شكل 11-2 الفرق بين التصنيف والكشف والتجزئة

```python
import keras
from keras import layers

base = keras.applications.EfficientNetB0(include_top=False, weights="imagenet",
                                         input_shape=(224, 224, 3), pooling="avg")
base.trainable = False
inputs = keras.Input((224, 224, 3))
x = keras.applications.efficientnet.preprocess_input(inputs)
x = base(x, training=False)
outputs = layers.Dense(3, activation="softmax")(x)
model = keras.Model(inputs, outputs)
model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss="sparse_categorical_crossentropy", metrics=["accuracy"])
```

بعد تدريب الرأس، جمّد معظم القاعدة وافتح آخر الكتل فقط. استخدم تقسيمًا بحسب الشخص أو الجهاز الطبي، وزيادات لا تغير الدلالة السريرية. في البيانات القليلة، أبلغ عن عدم اليقين وحجم العينة لا رقم الدقة وحده.

11.3 الكشف يحدد أين وماذا

$$IoU = \frac{|B_{pred} \cap B_{true}|}{|B_{pred} \cup B_{true}|}$$

`B_pred` الصندوق المتوقع و`B_true` الصندوق الحقيقي؛ القيمة تساوي صفرًا عند انعدام التقاطع وتقترب من الواحد مع التطابق.

يعيد كاشف الأجسام صناديق وفئات ودرجات ثقة. يقيس Intersection over Union مقدار تداخل الصندوق المتوقع مع الحقيقي: مساحة التقاطع مقسومة على مساحة الاتحاد. يخفض Non-Maximum Suppression الصناديق المتكررة للفئة نفسها بترتيب الثقة. يحسب mAP متوسط الدقة عبر الفئات وعتبات IoU، لذلك يجب وصف إعداد القياس لا ذكر الرقم منفردًا.

المهمة	خرج النموذج	مقياس أساسي
تصنيف	فئة للصورة	Recall أو F1 بحسب الكلفة
كشف	صناديق وفئات	mAP عند عتبات IoU معلنة
تجزئة	فئة لكل بكسل	IoU أو Dice لكل فئة

11.4 U-Net للتجزئة

يهبط مسار المشفر في الدقة المكانية ويزيد القنوات، ثم يصعد المسار المفكك ويستعيد التفاصيل عبر وصلات تخطٍ. عند تدريب الأقنعة، طابق حجمها ودلالات قيمها مع خرج النموذج. Dice يركز على التداخل، وIoU يمنع مكافأة المناطق الفارغة وحدها؛ اعرض القياس لكل فئة عند عدم التوازن.

11.5 حالة واقعية: صور من جهاز جديد

قسّم بيانات العيادة بحسب المريض، ثم احجز جهازًا أو فترة زمنية لاختبار التحول. قارن خط أساس مجمدًا مع ضبط دقيق. إذا تحسنت النتائج على الصور العشوائية وفشلت على الجهاز المحجوز، فالمشكلة اختلاف المجال لا نقص عمق الشبكة. راجع الإضاءة والدقة ومعالجة الصورة قبل إضافة بيانات مولدة.

11.6 الكشف: من الصندوق إلى قرار التشغيل

يتنبأ الكاشف بمواقع مرشحة وفئات ودرجات ثقة. بعض العائلات تفصل اقتراح المناطق عن تصنيفها، وأخرى تتنبأ مباشرة في مرور واحد؛ الاختيار يتأثر بالسرعة والمقياس والعتاد. بعد التنبؤ، تزيل NMS صناديق متداخلة للفئة نفسها وفق عتبة IoU، لكن ضبطها يعتمد على كثافة الأجسام: عتبة منخفضة جدًا قد تحذف جسمين متجاورين، ومرتفعة جدًا تترك تكرارات.

mAP يجمع منحنى precision-recall لكل فئة، ثم يحسب متوسطًا وفق بروتوكول محدد. اذكر مجموعة عتبات IoU، وطريقة interpolation، وأسماء الفئات. إذا كانت الفئة الحرجة نادرة، أبلغ AP لها مستقلًا وعدد أهدافها؛ المتوسط الميكروي قد يخفي فشلها. لا تقارن رقمًا من نظام يقيس عند IoU=0.5 بآخر يقيس متوسط عتبات متعددة.

11.7 التجزئة: قناع صحيح مكانيًا

يتوقع U-Net فئة لكل بكسل. يجب محاذاة الأقنعة والصور بعد القص والتغيير الحجمي؛ interpolation للأقنعة الفئوية يكون nearest-neighbor كي لا يخترع معرفات فئات بينية. في الفئات الثنائية، عرّف بوضوح هل النموذج يخرج sigmoid لقناة واحدة أم logits لفئتين. إذا كانت الخلفية تهيمن على الصورة، لا تجعل pixel accuracy المقياس الوحيد؛ اعرض Dice وIoU لكل فئة وقيّم حدود المنطقة ذات الصلة.

Dice يساوي ضعفي عدد البكسلات المشتركة مقسومًا على مجموع حجمي القناعين. أضف epsilon لتجنب القسمة على صفر، لكن قرر كيف تعامل فئة غائبة في الحقيقة والتنبؤ. استبعادها من المتوسط، أو احتسابها بدرجة كاملة، أو إظهارها منفصلة قرارات مختلفة ويجب التصريح بها.

11.8 التنبؤ بوجود هدف نادر

تقسيم train/validation/test عشوائيًا على الصور قد يسرّب صور المريض نفسه أو الفيديو نفسه. قسّم حسب المريض أو الموقع أو الالتقاط بحسب طريقة التعميم المستهدفة. عند فئات نادرة، استخدم الطبقية داخل حدود المجموعة المحجوزة، ولا تنقل صور مجموعة الاختبار لتوازن التدريب. augment الصور لا يعوّض غياب التنوع الحقيقي بين المؤسسات.

11.9 TTA وموازنة الكمون

Test-time augmentation يحول الصورة عدة مرات ويجمع التنبؤات، وقد يرفع ثبات النتيجة على حساب زمن الاستدلال. اذكر التحويلات وطريقة الدمج، وطبق الانعكاس العكسي على الأقنعة أو الصناديق قبل الجمع. لا تستخدم TTA إلا إذا تغلب التحسن على كلفته في الجهاز المستهدف؛ قياس GPU كبير ليس مؤشرًا على أداء هاتف أو كاميرا طرفية.

11.10 تقرير مشروع قابل للمراجعة

يحتوي التقرير: مصدر البيانات وترخيصها، وحدة التقسيم، مخطط baseline، annotation، المعالجة، إعداد التدريب، اختيار العتبة، مقياس كل فئة، مصفوفة الأخطاء، اختبار التحول، والكمون/الذاكرة. ضمّن مثالًا أو مثالين للفشل مع تفسير قابل للاختبار، واذكر الاستخدامات التي لم يختبرها المشروع. هذا التقرير يسمح للباحث الآخر بإعادة التجربة ومراجعة موضع عدم اليقين.

11.11 Transfer learning بميزانية بيانات صغيرة

استعمل augment يحافظ على الفئة، وابدأ بقاعدة مجمدة ورأس بسيط. افحص منحنيات train/validation، ولا تفتح القاعدة كلها فورًا؛ طبقات مبكرة تتعلم حوافًا عامة وقد لا تحتاج إلى تغيير. عند فك كتل عليا، استخدم معدل تعلم منخفضًا وتأكد أن BatchNorm لا يحدّث إحصاءات من بضع عشرات الصور إذا كان ذلك يضر. يمكن استخدام class weights، لكن عاير الاحتمالات لاحقًا لأن الأوزان تغير prior الفعال.

لا تكرر صور المريض إلى train وvalidation عبر augmentation ثم تعتبرها استقلالًا. الصور المتحولة من أصل واحد تبقى مجموعة مترابطة. عند وجود عدد قليل من المرضى، استخدم grouped cross-validation أو تحققًا خارجيًا بموقع مستقل إذا أمكن، واذكر عدد المرضى لا عدد الصور فقط. النتيجة لكل صورة قد تبالغ في دقة التقدير إذا كانت الوحدة الحقيقية هي المريض.

11.12 مكونات كاشف الأجسام

شبكة backbone تستخرج خرائط سمات، وneck يدمج مقاييس مكانية، وhead يتنبأ بفئة وصندوق أو توزيع صندوق. الصناديق قد تمثل `[x_min,y_min,x_max,y_max]` أو مركزًا وعرضًا وارتفاعًا؛ تحقق من convention قبل حساب IoU. عند resize letterbox، احفظ مقدار padding وscale لإرجاع الصندوق إلى إحداثيات الصورة الأصلية. أخطاء الإحداثيات قد تبدو كأن النموذج ضعيف بينما الحقيقة postprocessing معطوب.

قبل اختيار YOLO أو RetinaNet أو كاشف آخر، حدد حجم الأجسام، والكثافة، والكمون، والذاكرة، والعتاد. التقييم يشمل threshold الثقة وNMS وعتبات IoU. احسب latency شامل decode وpostprocessing، لأن NMS قد يصبح اختناقًا. عند مقارنة النماذج، استعمل نفس resize ودقة الصورة وطريقة حساب mAP.

11.13 أقنعة segmentation ومعالجة الحدود

القناع له دقة مكانية وربما عدة فئات. downsampling بقيمة interpolation خطية قد ينتج فئة 1.6 غير معرّفة؛ استخدم nearest للـ label maps. التنبؤ على tiled image يحتاج overlap كي لا يفشل قرب الحواف، ثم ادمج probability maps أو logits وفق قاعدة ثابتة. لا تجمع masks الثنائية قبل تنعيم الحدود دون وصف threshold.

11.14 تقييم مقاوم لتغير المجال

احجز جهازًا أو عيادة أو موسمًا كاملًا للاختبار، ثم اختبر المتانة بعد اختلاف اللون والحدة والقص والإضاءة. لا تستخدم augmentation صناعيًا مشابهًا للاختبار كبديل عن بيانات خارجية. أبلغ مقياس calibration إن كانت درجة confidence توجه قرارًا سريريًا، وافصل إحكام detection عن localization quality.

11.15 annotator agreement وإدارة الأقنعة

الوسوم الكثيفة قد تختلف بين خبراء حول الحدود. قس inter-annotator agreement أو راجع عينة مزدوجة، واحتفظ بنسخة تعليمات annotation. عند وجود خلاف، لا تحوله إلى mask حقيقة بلا سياسة معلنة. يمكن عرض Dice بين الخبراء كنقطة مرجعية؛ إذا كان توافق البشر أقل من النموذج المعلن، راجع بروتوكول المقارنة.

11.16 كشف الأجسام الصغيرة وتعدد المقاييس

الأجسام الصغيرة تختفي بعد downsampling. اختر resolution وFPN levels على توزيع أحجام الصناديق، وقارن الاستدعاء حسب مساحة الهدف. anchor-based detectors تحتاج ملاءمة anchors؛ أما anchor-free فتتجنب بعض هذه المعاملات لكنها لها تمثيل وpostprocessing خاص. عند ازدحام الأشياء، NMS التقليدي قد يحذف هدفًا مجاورًا صحيحًا؛ راجع Soft-NMS أو بديلًا مدعومًا ثم قس على مشاهد كثيفة.

11.17 تكلفة البيانات وتحديد الأولوية

حدد إن كانت أخطاء النموذج نتيجة نقص بيانات أو labels أو تحوّل مجال. uncertainty sampling قد يختار صورًا لمراجعة بشرية، لكن عينة غير عشوائية لا تصلح وحدها للتقدير النهائي. احتفظ بمجموعة audit عشوائية محجوزة، وسجل مصدر كل صورة، واستخدم active learning لإضافة بيانات التدريب فقط بعد مراجعة الوسم.

11.18 عتبات تشغيل حسب المخاطر

قد تقبل العيادة false positive أكثر لتجنب missed finding، لكن ذلك يزيد مراجعة الأخصائي. ارسم precision-recall curve لكل فئة، وحدد threshold مع مالك القرار. عند احتمال خارج مجال التدريب، أضف رفضًا أو إحالة بشرية بدل إجبار أحد الفئات. اختبر التغطية عند abstain rate مختلف، واذكر كلفة المراجعة اليدوية.

11.19 وثائق المشروع والقيود

أرفق data card ومخطط hardware، latency، class metrics، model card، preprocessing، classes، وأمثلة فشل. اذكر هل الكشف one-stage/ two-stage أو segmentation، والعتبات المستخدمة في mAP/IoU. لا تكتب «يصلح للتشخيص» من دون دراسة سريرية؛ صغ حدود الاستخدام والجهات التي لم تمثلها العينة.

11.20 حين لا توجد صناديق لكل البيانات

الوسم المربع مكلف؛ يمكن اختيار عينة active learning للمراجعة، لكن احتفظ بعينة عشوائية ممثلة لتقييم unbiased. إذا استخدمت pseudo-labels، افصلها عن labels البشرية وسجل confidence والمرشح الذي ولّدها. لا تقيّم النموذج على pseudo-labels الخاصة به ثم تعتبر ذلك ground truth. راجع خطأ الصندوق والملصق على عينات صعبة.

11.21 القياس الميداني وإدارة الإصدارات

توقع اختلاف نوع العدسة والإضاءة والدقة والضغط. احتفظ بمجموعة challenge تركز على ظروف ميدانية لا تظهر في التدريب، مع حوكمة بيانات مناسبة. سجل model version في نتيجة كل inference حتى يمكن نسبة الانحراف. تحديث قاعدة pretrained قد يغير تمثيل السمات؛ أعد اختبار مجموعة القبول قبل استبدالها.

11.22 إمكانية إعادة إنتاج المشروع

احتفظ بملفات config، أسماء الفئات والترتيب، طريقة إحداثيات boxes، حجم الصورة وcheckpoint، padding، ومكتبات postprocess. إذا لم توثق الإحداثيات normalized أو pixel coordinates، يمكن أن يكون التنبؤ نفسه صحيحًا لكن العرض خاطئًا. ضع عينة واحدة مع input/output متوقعين ضمن اختبارات التكامل.

11.23 أمثلة تنفيذية إضافية

تجميد القاعدة

```python
base.trainable = False  # درب رأس التصنيف أولًا
# افتح بعض الطبقات العلويا ثم أعد التجميع بمعدل أصغر
```

حساب IoU للصناديق

```python
intersection = 40.0
area_a, area_b = 100.0, 120.0
iou = intersection / (area_a + area_b - intersection)
print(round(iou, 3))
```

11.24 أخطاء شائعة

> ⚠ **تحذير**
>
> بعد تغيير `trainable` أعد `compile` حتى يلتقط إعداد التدريب الحالة الجديدة. [DANGER!] لا تفسر mAP بلا عتبات IoU ومجموعة فئات معلومة؛ أرقام إعدادات مختلفة ليست قابلة للمقارنة مباشرة.

11.25 تمارين

1. احسب IoU لصندوقين معلومين. (المتوقع: مساحة تقاطع/اتحاد.)
2. صمم مرحلتي استخراج السمات والضبط الدقيق. (المتوقع: تجميد ثم فتح محدود بمعدل أصغر.)
3. اختر تقسيمًا يمنع صور الشخص في مجموعتين. (المتوقع: split بحسب المريض.)
4. قارن Dice وIoU عند فئة نادرة. (المتوقع: قياس منفصل للفئة.)
5. اشرح لماذا يستخدم NMS. (المتوقع: حذف صناديق متكررة متداخلة.)
6. اقترح اختبارًا للتحول بين الأجهزة. (المتوقع: تقييم خارجي محجوز.)

11.26 ملخص

- يبدأ النقل بتجميد قاعدة مناسبة ثم ضبط محدود ومدروس.
- الكشف والتجزئة يخرجان مواقع أو أقنعة، ولكل مهمة عقد ومقاييس مختلفة.
- البيانات القليلة تتطلب تقسيمًا يحاكي اختلاف الأشخاص والمصادر الواقعية.

الجزء الثالث · الرؤية والتسلسلات واللغة والتوليد

الفصل الثاني عشر

12

التسلسلات: RNN/LSTM/GRU والسلاسل الزمنية

حكاية الفصل

طلب متجر موسمي يرتفع قبل الأعياد. درّب فريقٌ نموذجًا عشوائي التقسيم فحقق خطأً صغيرًا، ثم أخفق في التنبؤ بالشهر التالي. تسربت معلومات المستقبل إلى التدريب. كيف نمثل الزمن ونقيسه بإنصاف؟

12.1 أهداف الفصل

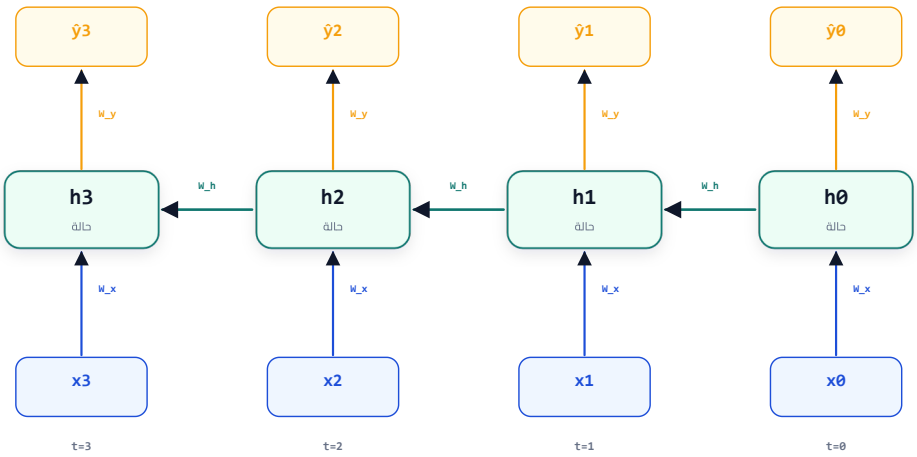
- ☐ تمثيل دفعة تسلسلات وشكل مخرجات RNN.
- ☐ شرح BPTT ومشكلة التدرج.
- ☐ مقارنة SimpleRNN وLSTM وGRU وConv1D.
- ☐ بناء نوافذ زمنية وتقسيم زمني وخط أساس.

12.2 ترتيب الزمن جزء من البيانات

شكل الدفعة الشائع `(batch, time, features)` : عدد النوافذ، وطول كل نافذة، وعدد المتغيرات في كل لحظة. عند التنبؤ بالمستقبل، درّب على الماضي وقيّم على فترة لاحقة؛ التطبيع يحسب من التدريب فقط. النوافذ المتداخلة قد تشارك نقاطًا، لذا قسّم زمنيًا قبل توليد النوافذ إذا كان ذلك يعكس الاستخدام.

الشبكة المتكررة: ذاكرة تمشي مع الزمن

نفس الخلية تُعاد في كل خطوة، والحالة تحمل السياق

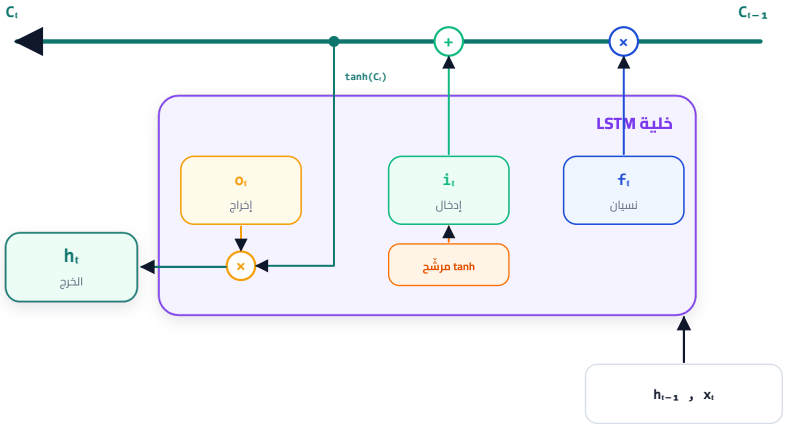


$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

شكل 12-1 حالة شبكة متكررة عبر خطوات الزمن

بوابة LSTM: ماذا ننسى، وماذا نحفظ؟

ثلاث بوابات تتحكم في خط الحالة، بدلًا من تحديث واحد



خط الحالة يحمل الذاكرة، والبوابات تقرر ما يُنسى وما يُحفظ وما يُخرج.

المعادلات

$$f_t = \sigma(W_f\,[h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i\,[h_{t-1}, x_t] + b_i)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

$$o_t = \sigma(W_o\,[h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \odot \tanh(C_t)$$

⊙ ضرب عنصري
σ دالة السيجمويد

شكل 12-2 أبواب الذاكرة في LSTM

$$h_t = \phi(W_x x_t + W_h h_{t-1} + b)$$

`x_t` مدخل الزمن الحالي، و`h_t` الحالة، و`W_x` و`W_h` مصفوفتا الأوزان، و`φ` التنشيط. تنتقل الحالة إلى الخطوة التالية، ويُشتق الخطأ عبر الزمن بعملية BPTT.

12.3 لماذا LSTM وGRU؟

في RNN بسيطة، تتكرر المصفوفة عبر الخطوات، وقد تضعف التدرجات أو تكبر. تضيف LSTM بوابات نسيان وإدخال وإخراج ومسار حالة، بينما تبسط GRU البوابات. لا يعني ذلك أن LSTM تتفوق دائمًا؛ نافذة قصيرة أو بيانات صغيرة قد تناسب Conv1D أو نموذجًا خطيًا أسرع وأسهل ضبطًا.

النموذج	علاقات يلتقطها	كلفة تقريبية
خطي منزلق	نمط ثابت في السمات المتأخرة	منخفضة جدًا
Conv1D	أنماط محلية متوازية	ملائمة للمقاطع
SimpleRNN	ترتيب متسلسل بسيط	قد يعاني التدرج
LSTM/GRU	اعتماد أطول عبر بوابات	ذاكرة وزمن أعلى

```python
import numpy as np
import keras
from keras import layers

series = np.sin(np.arange(1200, dtype="float32") * 0.03)
window, horizon = 48, 1
x = np.stack([series[i:i+window] for i in range(len(series)-window-horizon)])
y = np.stack([series[i+window:i+window+horizon] for i in range(len(series)-window-horizon)])
split = int(len(x) * 0.8)
model = keras.Sequential([layers.Input((window, 1)), layers.LSTM(16), layers.Dense(horizon)])
model.compile(optimizer="adam", loss="mae")
model.fit(x[:split, :, None], y[:split], validation_data=(x[split:, :, None], y[split:]),
          epochs=3, batch_size=32, verbose=0)
```

هذا المثال يوضح التشكيل فقط؛ في بيانات حقيقية، يجب فصل الفترة الزمنية قبل الملاءمة، وفحص خط أساس يحمل آخر قيمة أو متوسطًا موسميًا. لا تسمح بعشوائية النوافذ أن تخلط الماضي بالمستقبل.

12.4 حالة واقعية: تخطيط مخزون

يتنبأ الفريق بالطلب للأسبوع المقبل ويقارن LSTM بخط أساس موسمي وبمتوسط متحرك. يحسب MAE لكل متجر وشهر، ويعرض الأداء وقت العروض والعطل منفصلًا. إذا كان الخطأ النسبي مهمًا، يُضاف مقياس مناسب مع الانتباه إلى الأصفار. لا يُستخدم MAPE دون معالجة القيم الصفرية.

12.5 النافذة والهدف وخط الأساس

إذا كانت السلسلة بطول `T`، وطول النافذة `W`، وأفق التنبؤ `H`، فعدد النوافذ الكاملة يساوي `T-W-H+1`. اكتب هذا العقد قبل إنشاء Tensor حتى تعرف هل النموذج يتنبأ بالقيمة التالية أم بأفق متعدد. في multiple-horizon، قد ينتج رأس النموذج متجهًا لكل مستقبل مطلوب، أو تستخدم تنبؤًا autoregressive يعيد مخرج كل خطوة إلى المدخل؛ الثاني يسمح بتراكم الخطأ.

الخط الأساس البسيط يحدد قيمة التعقيد الإضافي. لسلسلة موسمية بفترة `m`، توقع الأسبوع الحالي بقيمة قبل `m` خطوة. قارن نموذجًا يتنبأ المتوسط أو آخر قيمة أو seasonal naive. إذا لم يتفوق النموذج العميق باستمرار في walk-forward validation، فبساطة baseline تقلل الكلفة وتزيد قابلية التفسير.

12.6 LSTM بواباتٌ وحالةٌ

تحتوي LSTM حالة الخلية `c_t` وحالة مخفية `h_t`. بوابة النسيان تحدد ما يبقى من الذاكرة السابقة، وبوابة الإدخال تحدد ما يكتب، وبوابة الإخراج تحدد ما يظهر للطبقة التالية. GRU تدمج بعض هذه الأدوار في بوابتي تحديث وإعادة ضبط. كلاهما يخفف مشكلة المسار المتكرر مقارنة بـ SimpleRNN، لكنهما لا يضمنان حفظ نمط بعيد إذا كانت البيانات والنافذة غير مناسبين.

Bidirectional RNN تقرأ التسلسل من الاتجاهين؛ تصلح لتصنيف مستند كامل حيث كل السياق متاح، لكنها غير صالحة لتنبؤ آني عند الزمن `t` إذا كان الاتجاه المعاكس يرى قيم `t+1`. هذا فرق استخدام جوهري، لا تفصيل معماري.

12.7 تقييم زمني واقعي

استخدم walk-forward: درب حتى تاريخ، توقع الفترة التي تليه، ثم حرّك نقطة القطع. لا تخلط الطيات عشوائيًا. اعرض التنبؤات مع فواصل زمنية للضوضاء أو عدة بذور عند الحاجة، وافصل الأداء حسب أفق التوقع. متوسط MAE عبر أفق طويل قد يخفي أن الخطأ يكبر بعد اليوم الأول. في سلاسل كثيرة، اذكر وحدة القياس وعدد السلاسل وحجم كل مجموعة.

MAPE يقسم الخطأ على القيمة الحقيقة، لذلك ينفجر أو يصبح غير معرف قرب الصفر؛ يمكن استخدام sMAPE أو WAPE وفق تعريف واضح، أو MAE إذا كانت وحدة الخطأ مناسبة. لا توجد مقياس واحد مثالي لكل بيانات demand. اختيار المقياس قرار أعمال: كم يكلف فائض المخزون مقارنة بنفاده؟

12.8 بيانات متغيرة وحالات مفقودة

تاريخ قياس مفقود يختلف عن قيمة صفرية: صفر المبيعات معلومة، والفراغ يعني عدم تسجيل. استبدال الاثنين بالصفر يعلم النموذج سلوكًا خاطئًا. يمكن إضافة قناع مفقود أو سمات زمنية (يوم الأسبوع، الموسم، عطلة) مع حسابها كما ستكون متاحة وقت التنبؤ. عالج إعادة ضبط الساعة والتوقيت الصيفي ووحدات الزمن إن كانت السجلات متعددة المناطق.

12.9 Stationarity والاتجاه والموسمية

السلسلة قد تحمل اتجاهًا طويلًا أو موسمية أو تغيير مستوى. لا يعني اشتراط stationarity أن كل تطبيق RNN يجب أن يفرق السلسلة؛ لكنه يذكرك بأن baseline والتطبيع قد يتغيران عبر الزمن. اعرض السلسلة، وموسمية أسبوعية/سنوية، وقارن نافذة تاريخية بأخرى حديثة. إذا استخدمت differencing، احتفظ بكيفية عكسه عند الاستدلال، ولا تحول التنبؤ إلى وحدة غير مفهومة للفريق.

12.10 التسرب الزمني وfeature availability

كل ميزة يجب أن تكون متاحة لحظة اتخاذ القرار. مبيعات اليوم النهائي لا تصلح feature لتوقع الصباح، حتى لو كانت موجودة في الجدول بعد اكتمال اليوم. استخدم as-of join يربط كل سجل بتاريخ توفره لا تاريخ الحدث فقط. خطأ التسرب قد يجعل كل النماذج تبدو قوية، ولا يصلحه تقسيم زمني وحده إن بقيت ميزة مستقبلية في الصف.

12.11 اختيار نافذة الإدخال

نافذة قصيرة قد تفقد موسمية مهمة، وطويلة قد تضيف ضوضاء وتستهلك ذاكرة. اختبر `window` و`horizon` على walk-forward folds، وقارن baseline موسميًا. لا تدمج سلاسل لمتاجر ذات scale مختلف دون تطبيع مدروس؛ التطبيع لكل متجر قد يحتاج معالجة متجر جديد. أبلغ المقاييس macro على المجموعات إذا كانت كل سلسلة لها أهمية مستقلة.

12.12 teacher forcing والتنبؤ المتكرر

في seq2seq، يدرب decoder أحيانًا على الرمز الحقيقي السابق (teacher forcing) بينما يستخدم عند الاستدلال تنبؤه السابق. الفرق يخلق exposure bias: الأخطاء المبكرة تتراكم في التوليد. في سلاسل الأرقام، التنبؤ autoregressive المتكرر قد ينجرف؛ التنبؤ المباشر لأفق كامل يزيل الحلقة لكنه يحتاج مخرجًا متعدد الخطوات. قارن الخيارين على نافذة مستقبلية حقيقية.

12.13 تفسير نموذج زمني

اعرض التوقع فوق السلسلة، وافصل فترات العرض والعطل، وافحص البواقي حسب الزمن والقيمة. إذا بقي نمط موسمي في residuals فالنموذج لم يلتقطه. Integrated gradients أو perturbation يمكن أن تفحص ميزات التأخير، لكنها ليست تفسيرًا سببيًا. استخدم مقياسًا مرتبطًا بقرار المخزون؛ خسارة الوحدات الناقصة وفائض الوحدات قد لا تكون متناظرة.

12.14 تدريب LSTM عمليًا

`return_sequences=True` يجعل الطبقة تخرج حالة لكل timestep؛ استخدمه عند تكديس طبقة متكررة أو بناء sequence labeling. الوضع الافتراضي يخرج حالة نهائية واحدة. `mask` يخبرها تجاهل padding في تسلسل متفاوت. `stateful=True` يعيد استخدام الحالة بين دفعات، ويشترط ثبات ترتيب السلاسل وعددها وطريقة reset؛ لا تفعله لتحسين الأداء بلا فهم.

قص التدرج قد يحد انفجار BPTT. يمكنك clipnorm في optimizer ومراقبة gradient norm. نافذة أطول تتيح سياقًا أكثر لكنها تزيد الذاكرة والحساب، وقد تزيد ضوضاء التدريب. أجرِ ablation على horizong window length بدل اختيار طول تقليدي عشوائي.

12.15 سلاسل متعددة ومتغيرات خارجية

توقع مبيعات عدة فروع قد يتعلم من تمثيل مشترك، لكن الفروع لها مقاييس مختلفة وموسمية محلية. global model قد يعمم بين المتاجر، بينما local model متخصص وقد يحتاج بيانات أكثر لكل فرع. أضف static features مثل المنطقة، وdynamic covariatesg مثل السعر فقط إن عرف المستقبل أو كانت متوقعة وقت التنبؤ. إدخال طقس تاريخي كأنه طقس مستقبلي حقيقي هو تسرب.

12.16 مقاييس الانحدار الزمني

MAE سهل التفسير بوحدة الهدف ومقاوم نسبيًا للقيم الكبيرة؛ RMSE يعاقب الأخطاء الكبيرة. WAPE يجمع absolute errors على إجمالي الحقيقة، لكنه قد يهيمن عليه حجم السلسلة الأكبر. Quantile loss يدعم توقع فاصل لعدم اليقين بدل رقم واحد، ويعين قرار المخزون عند كلفة عدم تماثلية. اشرح دائمًا وحدة القياس وما إذا كان المتوسط عبر سلاسل أو أمثلة.

12.17 اختبار انزياح الزمن

افحص الأداء على آخر فترة، ثم على مواسم مختلفة، ثم بعد تغييرات تشغيليه. مراقبة error على labels المتأخرة ضرورية؛ drift في توزيع التوقع لا يساوي drift في الدقة. عند حصول تغيير عملية، سجل تاريخ الحدث وقيّم قبل وبعد. إعادة التدريب قد تساعد، لكن ابدأ بفحص مصدر المشكلة وإمكانية تعديل التقويم أو feature pipeline.

12.18 عدم اليقين والتخطيط

نقطة توقع واحدة قد تكون مضللة حين تتغير السلسلة. Quantile regression يعطي حدودًا شرطية، ويمكن استخدام ensembles أو عدة seeds لتقدير تشتت نموذجي. تحقق من coverage: هل يقع نحو 90% من الحقيقة داخل فاصل 90%؟ الفواصل المنكمشة في موسم لم يرَه النموذج علامة overconfidence. استخدم توزيع التوقع لمعادلة نقطة إعادة الطلب أو المخزون الاحتياطي.

12.19 اختيار RNN أو Conv1D أو Transformer

RNN تدير تسلسلًا خطوة بخطوة وتناسب تحديث حالة تدريجيًا. Conv1D تعالج مقاطع محلية بالتوازي، dilationg يزيد مجال السياق دون زيادة kernel كبير. Transformer يربط مواضع بعيدة مباشرة لكنه مكلف مع طول السلسلة. جرّب نموذجًا أبسط وأسرع أولًا؛ إذا كانت البيانات قليلة ولا يوجد نمط طويل، قد يكون seasonal

baseline أقوى.

12.20 تقرير متانة زمني

اعرض عدة cutoffs للتقييم ونتيجة لكل أفق، وقياسًا لكل series group، ونتائج فترة العطل أو الانقطاع. اذكر كيف تعاملت مع missing values والوقت المحلي والتغير التشغيلي. شغّل backtest قبل إطلاق أي retraining schedule وتأكد أن الميزات المتاحة تاريخيًا هي نفسها المتاحة مستقبلاً.

12.21 أمثلة تنفيذية إضافية

إنشاء نوافذ زمنية

```python
window = 24
windows = tf.signal.frame(series, frame_length=window + 1, frame_step=1)
x, y = windows[:, :-1], windows[:, -1]
```

التنبؤ

```python
prediction = model(x_batch, training=False)
print(prediction.shape)
```

12.22 توقع احتمالي واحتياط المخزون

لا تحوّل عدم اليقين إلى حد ثقة من غير معايرة. قارن coverage على عدة فترات، وقس أثر quantile على نفاد المخزون ومخزون الفائض. اربط عتبة الطلب بكلفة العمل الفعلية، ثم اختبرها على فترة مستقبلية لم تدخل ضبط النموذج.

12.23 أخطاء شائعة

⚠ تحذير

لا تخلط ترتيب الزمن عند التقسيم ولا تحسب المتوسط والانحراف من كامل السلسلة. [DANGER!]

`stateful=True` يحمل حالة دفعة إلى التالية؛ استخدامه بلا ترتيب ثابت للعينات يربط سجلات مختلفة.

12.24 تمارين

1. فسّر محاور `(32, 24, 5)` . (المتوقع: 32 سلسلة، 24 خطوة، 5 ميزات.)
2. أنشئ نوافذ بطول 12 وتنبؤ خطوتين. (المتوقع: مدخل `(N,12,features)` وهدف `(N,2)`).
3. ابن خط أساس يحمل آخر قيمة. (المتوقع: توقع ساذج قابل للمقارنة.)
4. قارن LSTM وConv1D بزمن التدريب. (المتوقع: قياس دقة وزمن.)
5. وضح موضع تسرب المستقبل. (المتوقع: تقسيم عشوائي لنوافذ متداخلة.)
6. اختر MAE أو مقياسًا نسبيًا وعلّل. (المتوقع: اختيار مرتبط بوحدة القرار والأصفار.)

12.25 ملخص

- الزمن محور بنيوي، لذلك يسبق التقسيم الزمني التدريب وتوليد النوافذ.
- بوابات LSTM وGRU تساعد على الاعتماد الممتد مقابل كلفة أعلى.
- قارن دائمًا بخط أساس واحسب المقاييس على فترات مستقبلية محجوزة.

الجزء الثالث · الرؤية والتسلسلات واللغة والتوليد

الفصل الثالث عشر

13

الانتباه والمحوّلات

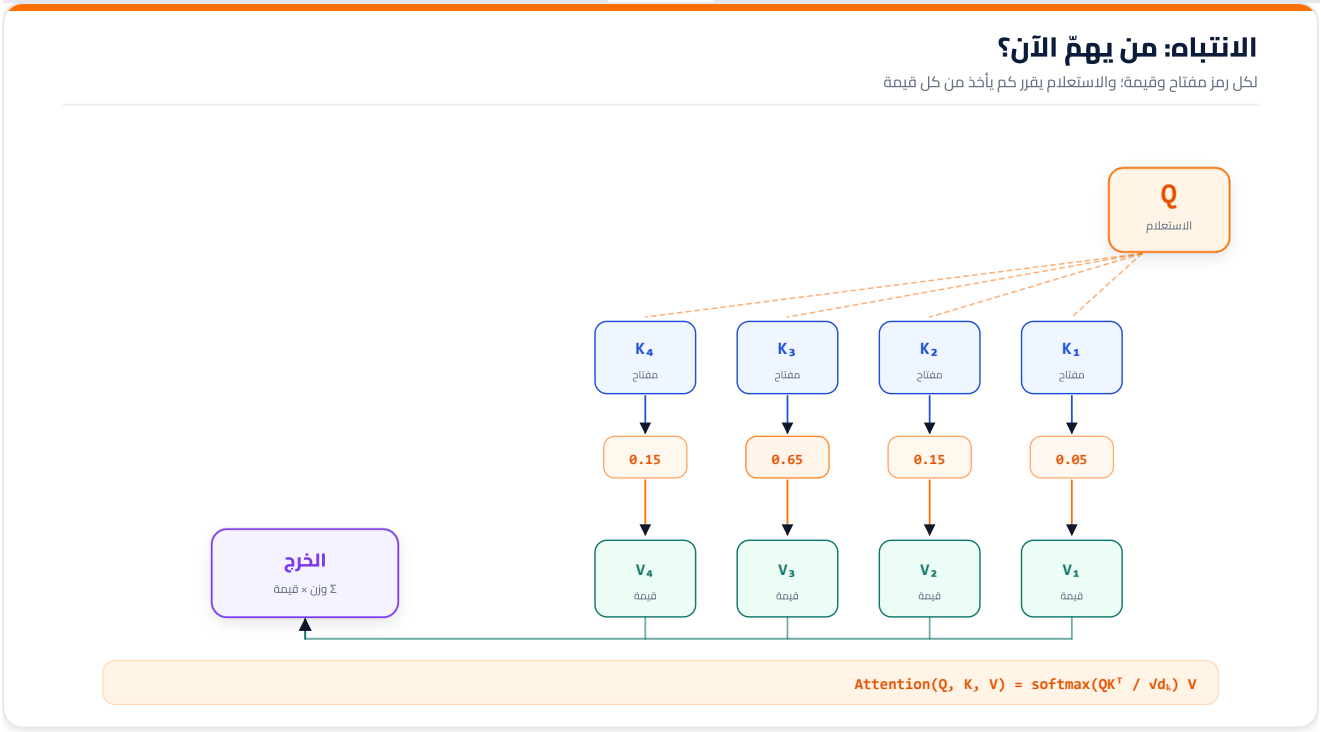
> **حكاية الفصل**
>
> كان نموذج ترجمة قديم ينسى بداية الجملة قبل أن يصل إلى نهايتها. ضغط كل المصدر في متجه واحد، ولم يعرف أي الكلمات ينبغي الرجوع إليها عند إخراج الكلمة التالية. يتيح الانتباه للنموذج أن يزن أجزاء السياق مباشرة. كيف يعمل ذلك حسابيًا؟

13.1 أهداف الفصل

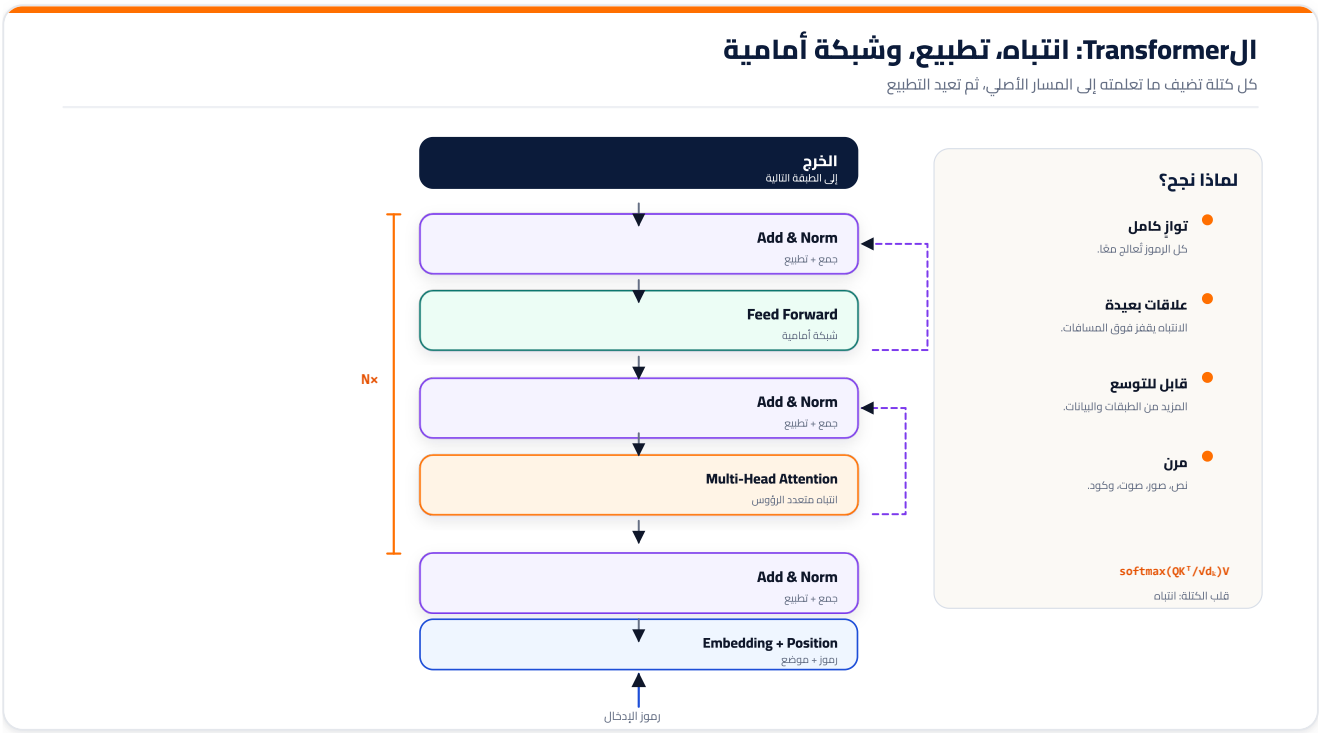
- [ ] اشتقاق الانتباه بالاستعلام والمفتاح والقيمة.
- [ ] شرح الرؤوس المتعددة والترميز الموضعي.
- [ ] فهم أقنعة الحشو والسببية.
- [ ] تركيب كتلة Transformer صغيرة ومعرفة كلفة الانتباه.

13.2 من السياق المضغوط إلى أوزان الانتباه

لكل موضع استعلام `Q` يقارن مفاتيح `K`، ثم يوزع الأوزان على القيم `V`. القسمة على الجذر التربيعي لعمق المفتاح تحد من كبر الضرب النقطي. يتيح هذا اختيار سياق مختلف لكل موضع، وتُحسب المواضع بالتوازي بدل السير التكراري الذي يميز RNN.



شكل 13-1 الانتباه يوزع الاهتمام على السياق



شكل 13-2 مسار مرمّز المحوّل ومفكّكه

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

`d_k` عمق المفتاح، و `M` قناع يضيف قيمة سالبة كبيرة للمواقع الممنوعة. ينتج softmax أوزانًا مجموعها واحد عبر محور المفاتيح.

13.3 لماذا رؤوس متعددة ومعلومة موضع؟

تقسم Multi-Head Attention التمثيل إلى إسقاطات فرعية، فيمكن للرؤوس تعلم علاقات مختلفة. لكن الانتباه وحده لا يعرف ترتيب الكلمات؛ لذلك يضاف تمثيل موضعي إلى embeddings. يتبع ذلك اتصال متبقٍ وتطبيع وطبقة أمامية، ما يساعد على تحسين تمثيلات أعمق.

```python
import keras
from keras import layers

tokens = keras.Input((None,), dtype="int32")
x = layers.Embedding(input_dim=5000, output_dim=64, mask_zero=True)(tokens)
attention = layers.MultiHeadAttention(num_heads=4, key_dim=16)(x, x)
x = layers.LayerNormalization()(x + attention)
ffn = layers.Dense(128, activation="gelu")(x)
ffn = layers.Dense(64)(ffn)
x = layers.LayerNormalization()(x + ffn)
pooled = layers.GlobalAveragePooling1D()(x)
outputs = layers.Dense(3, activation="softmax")(pooled)
model = keras.Model(tokens, outputs)
```

يوضح المثال كتلة ترميز للتصنيف، ولا يشكل Transformer توليديًا كاملًا. في التوليد نحتاج قناعًا سببيًا يمنع الرمز من رؤية المستقبل، وفي دفعات بأطوال متفاوتة نحتاج قناع الحشو أيضًا. جرّب أن تمرر جملة أطول وتحقق من عدم تأثر التمثيل بحشو الذيل.

13.4 الكلفة والبدائل

الانتباه الكثيف يقارن كل موضع بكل موضع، فتزداد ذاكرة مصفوفة الأوزان تربيعيًا مع طول التسلسل. للرؤوس المتعددة فائدة تمثيلية، لكنها لا تلغي هذه الكلفة. CNN مناسبة للأنماط المحلية، وRNN قد تكون أبسط للتدفق خطوة بخطوة، أما Transformer فيناسب التوازي والسياق الواسع إذا سمحت الذاكرة والبيانات.

الخيار	ميزة	قيد
RNN	تدفق متسلسل وحالة	التوازي محدود
Conv1D	محلية وكلفة مضبوطة	مجال السياق يحتاج عمقًا/تمددًا
Transformer	علاقات بعيدة وتوازي	ذاكرة تربيعية للتسلسلات الطويلة

13.5 حالة واقعية: تصنيف تذاكر الدعم

تُحوّل التذاكر إلى رموز وتُقص وفق حد مبني على توزيع الطول. يتعلم المصنف علاقات بين كلمات المشكلة والحل، ويقاس macro-F1 كي لا تطغى فئة «أخرى». افحص ما إذا كان الحشو أو طول التذكرة يحمل إشارة مصطنعة، وقارن بالـ TF-IDF وخط أساس لوجستي قبل استخدام شبكة عميقة.

13.6 بناء الكتلة خطوةً خطوة

أدخل الرموز إلى embedding، وأضف تمثيلًا للموضع، ثم مررها عبر attention متعدد الرؤوس. اجمع الناتج مع الوصلة المتبقية، وطبّق LayerNorm، ثم مرر كل موضع إلى شبكة أمامية صغيرة وأعد وصلة وتطبيعًا. الترتيب pre-norm أو post-norm يغير استقرار التدريب؛ لا تخلط تكوينًا من ورقة مع كود من تكوين آخر دون توثيق.

قناع الحشو يخفي المواضع التي أضيفت لمعادلة أطوال الجمل. القناع السببي يخفي جميع المفاتيح المستقبلية، ويستخدم في التوليد autoregressive. في encoder لتصنيف جملة كاملة، لا نريد القناع السببي عادة لأن كل كلمة يمكنها استخدام السياق اللاحق. اختبار عملي: غيّر رموز الحشو وحدها، وتحقق أن تنبؤات الجملة لا تتغير.

13.7 موضعية مطلقة ونسبية

يمكن إضافة embeddings متعلمة لمواضع الجملة أو ترميز sinusoidal ثابت. الترميز المطلق يميز موقعًا بعينه، بينما الترميزات النسبية تعبر عن بعد أو اتجاه بين الرموز، وقد تعمم على أطوال أخرى وفق التصميم. لا توسع طول السياق لمجرد أن Tensor يقبله؛ تدريب النموذج على أطوال معينة واستقراره عند أطول كثيرًا أمر مختلف.

13.8 طول السياق والذاكرة

لو كان التسلسل بطول `n`، فمصفوفة الدرجات لكل رأس تشمل `n×n` أزواج مواضع. مضاعفة `n` تقارب أربعة أضعاف حجم هذه المصفوفة. Flash Attention يحسب الأوزان في كتل لتقليل حركة الذاكرة عندما يدعمه backend والعتاد، لكنه لا يغير الكلفة الحسابية النظرية للمقارنة الكاملة. Keras 3.15 أضاف تحسينات على fused/flash attention، كما أضاف `sliding_window` لـ MultiHeadAttention في حالات محددة؛ تحقق من إصدار backend والقيود قبل الاعتماد عليه. إصدارات Keras.

13.9 اختيار بنية المحول

Encoder-only يناسب فهم النص والتصنيف، decoder-only يتنبأ بالرمز التالي، encoder-decoderg مناسب لتحويل سلسلة إلى سلسلة. هذه ليست حدودًا مطلقة، لكنها نقطة بداية لاختيار القناع وهدف التدريب. عند بناء نموذج صغير للتعلم، قلل المفردات وطول السياق وعدد الطبقات، ثم اختبر overfit على دفعة واحدة قبل تشغيل مجموعة كاملة. إذا لم يستطع النموذج حفظ دفعة صغيرة، فابحث في masking أو labels أو loss.

13.10 أساليب تدريب ومعايرة

التدريب من الصفر يحتاج بيانات كثيرة، بينما النموذج المدرب مسبقًا يبدأ من تمثيل لغوي. في fine-tuning اختبر معدلًا صغيرًا، warmup، وجدول تناقص، وقارن تجميد الطبقات الدنيا بتحريرها. قيّم طول الإدخال الحقيقي والتقطيع، لأن خطأ tokenization قد يكون أهم من حجم النموذج. في التوليد، عتبة top-k أو top-p ودرجة الحرارة تغير التنوع، لكنها لا تصلح نموذجًا غير ملائم.

13.11 ضبط شكل الاستعلام والمفتاح والقيمة

إذا كان الإدخال `(batch, sequence, embed_dim)`، فإن projections تنتج Q و K و V. `num_heads × key_dim` يحدد أبعاد الإسقاط، وليس بالضرورة مساويًا لـ `embed_dim`؛ طبقة Keras تعيد إسقاط الناتج إلى بعد query افتراضيًا. هذا التفصيل يمنع الاعتقاد أن كل رأس يقسم بعد الإدخال بالتساوي. شغّل `model.summary()`

واطبع shape scores عند تفعيل `return_attention_scores=True` في نموذج صغير.

لا تفسر مصفوفة attention باعتبارها شرحًا كافيًا لقرار النموذج؛ الوزن يدل على مسار حساب، ويتأثر بالإسقاطات والقيم والطبقات التالية. يمكن للرؤوس المختلفة إعطاء خرائط متباينة وتظل المخرجات متقاربة. استخدم attention visualization لاستكشاف الأخطاء، وقارنه بتدخلات حذف الكلمات وال attribution عند الحاجة.

13.12 قناع padding في دفعة لغوية

الرمز `0` قد يملأ مواضع إضافية، لكنه ليس قيمة صفرية محايدة لأن embedding له وزن. `mask_zero=True` في طبقة Embedding يمكنه إنتاج mask، لكن custom layers يجب أن تمرره صراحة أو تعرّف دعم `compute_mask`. قناع انتباه صحيح يخفي key positions المحشوة. إذا كان pooling المتوسط يشمل padding، فإما استخدم mask-aware pooling أو اجمع القيم المقنعة ثم اقسم على عدد الرموز الحقيقية.

13.13 لماذا انتشر Flash Attention؟

إصدارات Keras 3.15 تتضمن تحسينات في attention مدمجًا وواجهات نافذة منزلقة في أنماط backendsg مدعومة؛ افحص ملاحظات الإصدار الرسمية وملف تعريف العتاد قبل اعتمادها في تجربة منشورة.

الانتباه التقليدي يكتب مصفوفة `n×n` إلى الذاكرة ويقرأها مرارًا. Flash Attention يجزئ الحساب ويقلل نقل الذاكرة، مع مراعاة softmax المستقرة، فيسرع على عتاد مدعوم دون تغيير الصيغة المقصودة رياضيًا. لكن kernels المدمجة تعتمد نمط القناع وdtype وbackend. قِس ما فعّله Keras فعليًا، وتأكد من تطابق المخرجات ضمن tolerance، ولا تفترض أن مجرد تثبيت Keras أحدث يستخدم kernel أسرع لكل شكل.

13.14 التحليل الحسابي والذاكرة

لطول `n` sequence وبعد `d` , الكلفة attention تقريبًا من رتبة `n²d` للحساب و `n²` لمصفوفة الدرجات في التنفيذ المباشر. Feed-forward يضيف كلفة تقارب `ndd_ff` . في نصوص طويلة قد يهيمن attention، وفي نماذج أخرى تهيمن MLP أو embedding. profiler يفصل ذلك. sliding window يحد السياق بكل token بمحيط محلي، لكنه يحرم علاقة مباشرة بين أطراف بعيدة إلا عبر طبقات متعددة.

13.15 فحوص قبل fine-tuning

تحقق من tokenizer وإصدار vocabulary، وأن special tokens وpadding IDs مطابقة للرأس المدرب. جرّب جملًا قصيرة وطويلة ونصًا عربيًا مع punctuation وOOV. افحص attention mask فعليًا. على مجموعة صغيرة، تأكد أن training loss ينخفض. ثم قارن checkpoint الأصلي والنسخة المضبوطة على benchmark ثابت؛ نسخ الأوزان الخاطئة أو tokenizer مختلف قد يفسد الأداء دون crash.

13.16 تصميم fine-tuningg pretraining

تعلم masked language modeling يخفي رموزًا ويتنبأ بها من السياق، بينما causal language modeling يتنبأ بالرمز التالي فقط. الهدف يحدد القناع ومعمارية encoder أو decoder. Fine-tuning لمهمة تصنيف يضيف رأسًا يقرأ pooled representation أو رمزًا مخصصًا، واختيار representation يجب أن يوافق checkpoint. لا تفترض أن `[CLS]` مدرب بالطريقة نفسها في كل نموذج؛ راجع بطاقة النموذج والـ tokenizer.

13.17 التعامل مع طول السياق

اقتطع النص وفق سياسة لا تمحو الإجابة المهمة. head truncation مناسب إذا كانت البداية أهم، وtail truncation في سجلات قد تضع القرار في النهاية. sliding windows تغطي مستندًا طويلًا، لكن تحتاج تجميع scores وتعريف label على مستوى المستند. dynamic padding داخل batch يقلل الحساب على الحشو، لكنه قد يسبب أشكالًا متغيرة وإعادة تتبع إذا لم تضبط pipeline.

13.18 التحسين والتدرج

Fine-tuning لنموذج كبير يحتاج learning rate صغيرًا وwarmup في كثير من الحالات، مع AdamW weight decay مضبوط. فصل معدل الرأس عن encoder قد يفيد عند تباين الدرجات. استخدم gradient accumulation عند محدودية الذاكرة، وmixed precision على GPU مدعومة، مع مراجعة loss scale والدقة. لا تحول المثال على GPU صغيرة إلى وعد أن نفس batch تناسب accelerator آخر.

13.19 المعايرة والتقييم

Macro-F1 يعطي كل فئة وزنًا متساويًا، وmicro-F1 يهيمن عليه مجموع الأمثلة؛ اختر ما يتوافق مع هدف الدراسة. في بحث NLP، أبلغ accuracy وprecision/recall للفئات، confusion matrix، التباين عبر seeds، ونسخة tokenizer. إذا كانت الثقة تدخل triage، قيّم calibration وcoverage عند abstain. أخطاء tokenization العربية

والهجات والتشكيل تحتاج slice tests مستقلة.

13.20 إدارة checkpoints

احفظ اسم النموذج الأصلي والhashg revision الملفات. KerasHub presets أو Hugging Face checkpoints قد تختلف في أسماء أوزان ورخصة الاستخدام. اقرأ model card وسياسة الاستخدام التجارية والقيود اللغوية. حمّل الأوزان من مصدر موثوق، وتحقق من checksum، ولا تتجاوز أمان deserialize. عند نشر النموذج، صِل tokenizer وإعدادات التطبيع والحد الأقصى للسياق بالنسخة نفسها.

13.21 تجربة استئصال (ablation) سليمة

عند تقييم positional encoding أو attention heads، أزل عنصرًا واحدًا كل مرة مع تثبيت بيانات التدريب والميزانية وعدد المعلمات قدر الإمكان. اختلاف compute قد يمنح نموذجًا وقتًا أطول في التدريب؛ اذكر معيار المقارنة. كرر عدة seeds حين تكون المهمة صغيرة، واعرض الفروق لا أفضل نتيجة فقط.

13.22 Transformer للتصنيف على بيانات صغيرة

لا تبدأ بمحول ضخم إذا كانت لديك مئات الأمثلة. قارن TF-IDF مع logistic regression، وembedding + pooling، ثم transformer pretrained. اضبط طول sequence على quantile أطوال النصوص والذاكرة، لا على أكبر مستند فقط. استخدم class weights أو threshold عند عدم التوازن، وافحص macro-F1 ومعايرة probabilities.

13.23 حدود الانتباه كشرح

attention weights تفسر وزن القراءة الداخلي في طبقة، لكنها ليست بالضرورة أثرًا سببيًا على prediction. قد تتغير attention مع بقاء prediction ثابتًا. لذلك اربط visualization باختبارات حذف أو counterfactual، وأجب عن سؤال محدد: هل رمز معين ضروري للتصنيف؟ هل النموذج يتبع النفي؟ راجع أمثلة لغوية يضع فيها ترتيب الكلمات المعنى على المحك.

13.24 أمثلة تنفيذية إضافية

انتباه متعدد الرؤوس

```python
attention = keras.layers.MultiHeadAttention(num_heads=4, key_dim=16)
context = attention(query=x, value=x, key=x)
```

قناع سببي

```python
attention = keras.layers.MultiHeadAttention(num_heads=4, key_dim=16)
context = attention(x, x, use_causal_mask=True)
```

13.25 أخطاء شائعة

⚠ تحذير

من دون معلومة موضعية لا يميز الانتباه ترتيب تسلسل الرموز. [DANGER!] قناع سببي مفقود يسمح للنموذج برؤية الرموز اللاحقة أثناء التدريب، فيبدو التقييم ممتازًا ويتعطل التوليد.

13.26 تمارين

1. احسب أوزان softmax لاستعلام ومفتاحين صغيرين. (المتوقع: مجموع الأوزان واحد.)
2. اشرح دور `sqrt(d_k)` . (المتوقع: ضبط مقياس الدرجات قبل softmax.)
3. أضف قناعًا للحشو. (المتوقع: تجاهل مواضع الرمز صفر.)
4. قارن كلفة طول 128 وطول 512. (المتوقع: مصفوفة الانتباه تكبر بنحو 16 مرة.)
5. ميّز encoder عن decoder سببي. (المتوقع: رؤية كاملة مقابل منع المستقبل.)
6. اختر خط أساس لتذاكر الدعم. (المتوقع: TF-IDF مع مصنف تقليدي.)

13.27 ملخص

- يوزن الانتباه القيم وفق توافق الاستعلام والمفاتيح.
- الرؤوس المتعددة والترميز الموضعي والقناع عناصر مختلفة لكل منها وظيفة.
- طول التسلسل يرفع الذاكرة تربيعيًا، لذا تقارن المحولات ببدائل أبسط.

الجزء الثالث · الرؤية والتسلسلات واللغة والتوليد

الفصل الرابع عشر

14

معالجة اللغة الطبيعية والعربية

حكاية الفصل

في مركز اتصال ببنك خليجي، دُرّب نموذجٌ لتصنيف شكاوى العملاء، فحقق دقة 92% على مراجعاتٍ فصيحة جُمعت من الموقع. في أول أسبوعٍ للإنتاج، انهارت الدقة إلى 61%: فالتغريدة التي تقول «ما في خدمة» لا تعني ما تعنيه «ما في الخدمة؟»، والكلمة نفسها تتبدّل دلالتها بين اللهجات. لم يكن العيب في الشبكة، بل في الطريقة التي حوّلنا بها اللغة إلى أرقام. فكيف نُعلّم الآلة أن تقرأ النص العربي كما يقرأه الإنسان: بسياقه، لا بحروفه فقط؟

14.1 أهداف الفصل

- بناء خط معالجة نص عربي يمتد من التنظيف إلى التقطيع وبناء المعجم.
- فهم الحشو والتقنيع، والفرق بين التمثيل الثابت والتمثيل السياقي.
- شرح Word2Vec وGloVe وBERT دون الخلط بين أهدافها التدريبية.
- ضبط نموذج BERT عبر KerasHub وتقييم تصنيف المشاعر بمقاييس مناسبة.
- تشخيص التسريب وعدم تطابق التطبيع والتحيّز اللهجي قبل النشر.

14.2 من الحرف إلى الرمز: لماذا لا يرى النموذج نصك؟

حين تكتب «الخدمة ممتازة»، يرى المحرّر سلسلةً من الحروف والمسافات وعلامات الترقيم. أما النموذج فلا يرى إلا أعدادًا صحيحة تشير إلى مواقع في معجم. بين هذين العالمين يقوم التنظيف (Cleaning) والتقطيع (Tokenization): قراراتٌ صغيرة يترتّب عليها أثرٌ كبير. احذف علامة النفي «لا» لأنها «علامة ترقيم»، فينقلب معنى الجملة. احذف التشكيل كله، فقد تختفي معلوماتٌ يحتاجها نموذجٌ يقرأ الشعر أو النص القرآني. والعكس صحيح: الإبقاء على كل شيء يجعل المعجم متضخّمًا ومليئًا بأشكالٍ لا تتكرّر.

التطبيع (Normalization) هو تثبيت الأشكال المختلفة للكلمة الواحدة. في العربية، تُكتب الهمزة على ألف أو واو أو ياء، وتأتي الألف مقصورة أو ممدودة، وتظهر التاء المربوطة في نهاية بعض الكلمات. توحيد أ وإ وآ إلى ا، وى إلى ي، وحذف التطويل، يقلّل التنوّع الشكلي ويجعل الكلمة أقرب إلى نظيراتها. لكنّه ليس بلا ثمن:

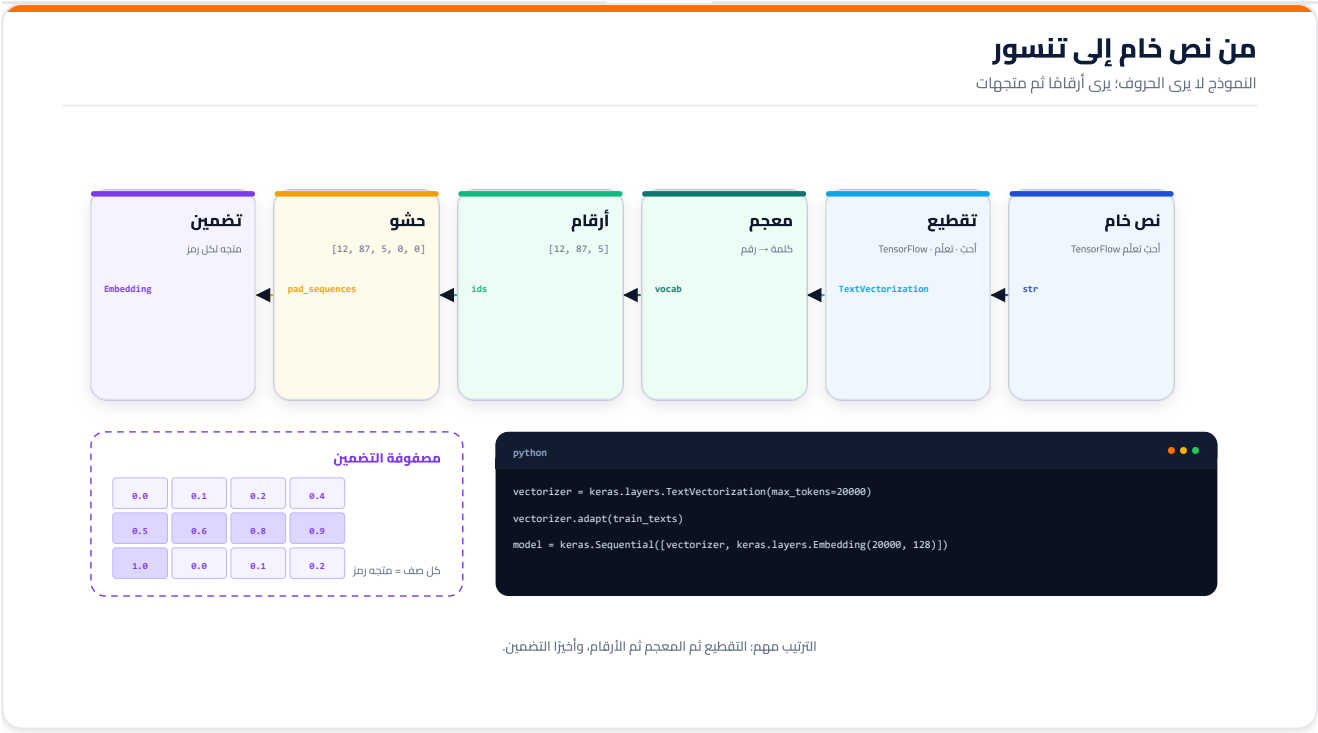
توحيد `ة` إلى `ه` قد يجمع كلمتين مختلفتين، وحذف التشكيل قد يخلط بين «عَلِم» و«عَلَم». القاعدة العملية: طبّق أصغر تطبيعٍ يخدم المهمة، واختبر أثره على مجموعة تحقّق، لا على حدسك.

مستويات التقطيع

التقطيع على مستوى الكلمة (Word-level) بديهي، لكنّه هشٌّ في العربية. اللغة غنية بالاشتقاق واللواحق والضمائر المتصلة: «وبكتابهم» كلمةٌ واحدة تحمل واو العطف والباء والكتاب وضميرهم. معجم الكلمات يتضخّم، والكلمات الجديدة تصبح خارج المعجم (Out-of-Vocabulary, OOV). التقطيع على مستوى الحرف (Character-level) يقلّل OOV لكنه يطيل التسلسل ويُفقد النموذج وحداتٍ دلالية واضحة. التقطيع تحت-الكلمي (Subword) يوسّط بينهما: يقسم الكلمة إلى مقاطع متكرّرة، مثل `ال + خدم + ة`، فيغطّي كلماتٍ لم يرها أثناء التدريب.

المستوى	الوحدة	حجم المعجم	OOV	الأنسب لـ
حرف	حرف واحد	صغير جدًا	شبه معدوم	لهجات، أسماء، نصوص قصيرة
كلمة	كلمة كاملة	ضخم	مرتفع في العربية	نصوص فصيحة محدودة المجال
تحت-كلمة	مقطع	متوسط	منخفض	BERT والمحوّلات عمومًا

أشهر خوارزميات التقطيع تحت-الكلمي: ترميز أزواج البايت (Byte Pair Encoding, BPE) الذي يدمج أكثر الأزواج تكرارًا، وWordPiece الذي يختار الدمج وفق احتمال اللغة، وSentencePiece الذي يعمل على النص الخام بلا افتراض مسافات. الفرق العملي بينها أقلّ أهميةً من سؤالين: ما حجم المعجم؟ وما متوسط عدد المقاطع لكل كلمة عربية؟ كلّما زادت المقاطع، زاد طول التسلسل وتكلفة الانتباه، وقد تُقتطع نهاية الجملة قبل أن تصل إلى النموذج.



شكل 1-14 من نص خام إلى تنسور: التقطيع ثم المعجم ثم التضمين

المعجم والكلمة الخارجة عنه

المعجم (Vocabulary) هو الجدول الذي يربط كل رمزٍ بعددٍ صحيح. تُحجز عادةً مواقع خاصة: `""` للحشو، و`[UNK]` للكلمة المجهولة، و`[CLS]` و`[SEP]` و`[MASK]` في BERT. حين تظهر كلمة خارج المعجم، تُستبدل بـ`[UNK]`، فتخسر هويتها. لهذا لا يُقاس جودة المعجم بحجمه، بل بمعدّل تغطية النص: كم كلمة من مجموعة الاختبار ستتحوّل إلى `[UNK]`؟ في نصٍّ عربي متخصّص، قد يبدو المعجم كبيرًا ومع ذلك يغطّي 85% فقط.

```python
import tensorflow as tf
import keras

keras.utils.set_random_seed(42)

# تطبيع عربي بسيط: نزيل التشكيل والتطويل ونوحّد الألف والياء والتاء المربوطة
def normalize_arabic(text):
    text = tf.strings.regex_replace(text, "[\u064B-\u065F\u0670]", "")
    text = tf.strings.regex_replace(text, "[أإآ]", "ا")
    text = tf.strings.regex_replace(text, "ي" ,"ى")
    text = tf.strings.regex_replace(text, "ه" ,"ة")
    text = tf.strings.regex_replace(text, "\u0640", "")
    text = tf.strings.regex_replace(text, r"[.,!?؛؟:\(\)\[\]\"'«»…\-]", " ")
    return tf.strings.regex_replace(text, r"\s+", " ")
```

```
corpus = [
    "الخِدمةُ مُمتازة والموظفون لُطفاء",
    "التوصيل تأخّر لكن المنتج جيّد",
    "الأسعار مرتفعة والجودة ضعيفة",
    "التطبيق سهل الاستخدام وسريع",
]

# نحوّل الجمل إلى أعداد صحيحة بطول ثابت؛ الحشو في النهاية
vectorizer = keras.layers.TextVectorization(
    max_tokens=50,
    standardize=normalize_arabic,
    output_mode="int",
    output_sequence_length=8,
)
vectorizer.adapt(corpus)

vocab = vectorizer.get_vocabulary()
print("أول عشرة رموز:", vocab[:10])
# المتوقع: ['', '[UNK]', 'ممتازه' ,'الخدمه',...  ] بحسب تكرار الكلمات

print("موقع [UNK]:", vocab.index("[UNK]"))   # المتوقع: 1

encoded = vectorizer(tf.constant([
    "الخِدمةُ ممتازة",    # كلمتان موجودتان في المعجم
    "الكتابُ رائع",        # كلمتان جديدتان
]))
print(encoded.numpy())
# المتوقع: مصفوفة 2×8؛ الصف الأول يبدأ بمعرّفات معروفة،
# والصف الثاني يبدأ بالرمز 1 مرتين، وبقية الأعمدة أصفار حشو
```

لاحظ أنّ `adapt` بنى المعجم من أربع جمل فقط. في مشروع حقيقي، يُبنى المعجم من مجموعة التدريب وحدها، لا من كل البيانات. هذا التفصيل الصغير هو أول بابٍ من أبواب التسريب (Leakage) الذي سنعود إليه.

14.3 التضمين: حين تصبح الكلمة متجهًا

بعد التقطيع، كل كلمة رقمٌ لا يحمل معنى. الترميز الواحد الساخن (One-hot) يجعل كل كلمة متجهًا طوله حجم المعجم، وفيه واحدٌ واحد وأصفارٌ كثيرة؛ متجه ضخم لا يعبّر عن تشابه. التضمين (Embedding) يستبدل ذلك بجدولٍ قابل للتعلّم: لكل رمزٍ متجهٌ كثيفٌ من بُعدٍ صغير، مثل 64 أو 256. الكلمات المتشابهة في الاستخدام تقترب متجهاتها، فتصبح المسافة معنى.

$$\mathcal{L}_{\text{SG}} = -\sum_{t=1}^{T}\sum_{j=-m}^{m} \log p\left(w_{t+j} \mid w_t\right)$$

هنا T عدد الرموز في النص، وm نصف نافذة السياق، وw_t الكلمة المركزية، وw_{t+j} كلمة مجاورة، وp احتمال التنبؤ بالمجاور من المركز (مع استثناء $j = 0$). هذا هدف Skip-gram في Word2Vec. يُحسب الاحتمال عادةً بـsoftmax على المعجم كله:

$$p\left(w_{t+j} \mid w_t\right) = \frac{\exp\left(\mathbf{v}_{w_t}^{\top}\mathbf{u}_{w_{t+j}}\right)}{\sum_{w' \in V}\exp\left(\mathbf{v}_{w_t}^{\top}\mathbf{u}_{w'}\right)}$$

$\mathbf{v}$ متجه الكلمة حين تكون مركزًا، و$\mathbf{u}$ متجهها حين تكون سياقًا، وV المعجم. حساب المقام على كل الكلمات مكلّف، لذا يستخدم Word2Vec أخذ العينات السلبية (Negative Sampling): يكتفي ببعض الكلمات السلبية بدل مجموع المعجم. أما GloVe فيبني متجهاته من مصفوفة التزامن، ويحاول أن يجعل حاصل الضرب النقطي بين متجهين قريبًا من لوغاريتم احتمال ظهورهما معًا. كلاهما ينتج تمثيلًا ثابتًا (Static): للكلمة متجهٌ واحد مهما تغيّر سياقها، ولهذا تختلط معاني «عين» و«سحابة» و«بنك».

التمثيل	سياقي؟	يحتاج تدريبًا؟	يتعامل مع OOV؟	الاستخدام النموذجي
TF-IDF	لا	لا	يتجاهل الكلمة	تصنيف خطي سريع
Word2Vec / GloVe	لا	نعم أو تحميل مسبق	بـ `[UNK]`	تشابه، تجميع، ميزات
BERT	نعم	نعم مع ضبط دقيق	بمقاطع تحت-كلمية	فهم سياقي وتصنيف

الحشو والتقنيع

الشبكات تحتاج دفعاتٍ مستطيلة، والجمل تتفاوت في الطول. الحشو (Padding) يضيف أصفارًا حتى يصل كل تسلسل إلى الطول المطلوب. لكنّ الأصفار ليست كلمات؛ إن مرّت على الطبقات كأنها كذلك، فستؤثر في المتوسط والانتباه. التقنيع (Masking) يخبر الطبقات أن تتجاهلها. في Keras، `Embedding(mask_zero=True)` يجعل الرمز صفرًا موقعًا للحشو، فتحمل الطبقات التالية قناعًا يمنع تأثيره.

```python
import numpy as np
import keras

keras.utils.set_random_seed(42)

# جملتان: الأولى ثلاثة رموز حقيقية، والثانية خمسة
sequences = np.array([
    [5, 9, 2, 0, 0],
    [7, 4, 11, 3, 8],
], dtype="int32")

model = keras.Sequential([
    keras.Input(shape=(5,), dtype="int32"),
    keras.layers.Embedding(input_dim=20, output_dim=4, mask_zero=True),
    keras.layers.GlobalAveragePooling1D(),
])

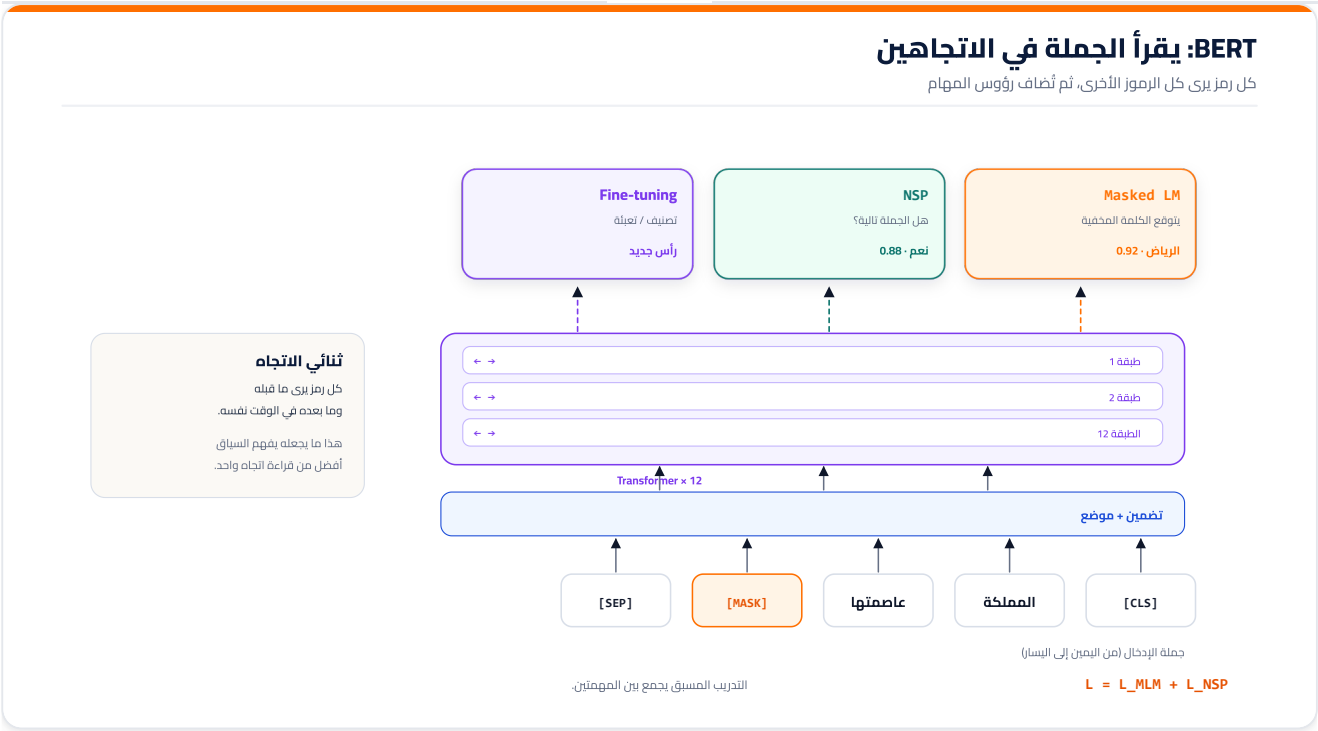
pooled = model(sequences)
print("شكل الخرج:", pooled.shape)   # المتوقع: (2 ,4)

# نقارن متوسط الجملة الأولى بمتوسط الرموز الثلاثة فقط، ثم بالخمسة مع الحشو
embeddings = model.layers[0](sequences).numpy()
mean_real = embeddings[0, :3].mean(axis=0)
mean_padded = embeddings[0].mean(axis=0)
print("تجاهل الحشو:", np.allclose(pooled.numpy()[0], mean_real, atol=1e-6))  # True
print("احتساب الحشو:", np.allclose(pooled.numpy()[0], mean_padded, atol=1e-6))  # False
```

هذا الفرق الصغير يفسّر كثيرًا من النتائج الغريبة: نموذجٌ يبدو أنه «يفهم» الجملة، لكنه في الحقيقة يحسب متوسط أصفار الحشو. في المحوّلات، لا يكفي `mask_zero` وحده؛ يجب تمرير قناع الانتباه (Attention Mask) أيضًا، وإلا انتبه النموذج إلى الحشو.

14.4 BERT: قراءة الجملة في اتجاهين

كل نموذجٍ لغوي تقليدي يقرأ من اليسار إلى اليمين، فيرى نصف السياق فقط. BERT (تمثيلات المحوّل ثنائية الاتجاه) يقرأ الجملة كاملةً في الاتجاهين معًا، لكنه لا يستطيع استخدام هدف «توقّع الكلمة التالية» لأنه سيرى الإجابة. الحلّ هو نموذج اللغة المقنّع (Masked Language Modeling, MLM): تُخفى نسبة 15% من الرموز، ويتعلّم النموذج إعادة بناءها من السياقين. من هذه الرموز المخفية، 80% تُستبدل بـ `[MASK]`، و10% برمزٍ عشوائي، و10% تبقى كما هي؛ هذا التوزيع يمنع النموذج من الاعتماد على وجود `[MASK]` وحده.



شكل 2-14 BERT: يقرأ الجملة في الاتجاهين

$$\mathcal{L}_{\text{MLM}} = -\sum_{i \in \mathcal{M}} \log p_\theta\left(x_i \mid \tilde{x}\right)$$

$\mathcal{M}$ مجموعة المواقع المخفية، وx_i الرمز الأصلي، و$\tilde{x}$ الجملة بعد الإخفاء، وθ أوزان النموذج. لا تُحسب الخسارة على كل الرموز، بل على المخفية فقط، وهذا ما يجعل التدريب مكلفًا لكنه يعلّم تمثيلًا سياقيًا غنيًّا.

في التصنيف، يُضاف رمز `[CLS]` في بداية الجملة، ويُدرَّب رأسه ليختصر تمثيل الجملة كلها. بعد التدريب المسبق، نستبدل رأس MLM برأس تصنيف صغير، ونضبط النموذج كله أو جزءًا منه بمعدّل تعلّم منخفض (Fine-tuning). معدّل التعلّم هنا أصغر بكثير من تدريب شبكة من الصفر: قيم شائعة بين `2e-5` و `5e-5`، لأن الأوزان تحمل معرفة لغوية ثمينة، والخطوة الكبيرة تدمّرها. عدد الدورات قليل، وغالبًا تكفي دورتان إلى أربع.

KerasHub هو الخليفة الرسمي لـKerasNLP وKerasCV. يوفّر الأوزان والمعالجة المسبقة والرؤوس الجاهزة، بدل إعادة بناء BERT يدويًا. المسار التالي يستخدم `BertTextClassifier` مع preset متعدّد اللغات يغطّي العربية، لكنه **لا يعمل إلا بعد تثبيت الحزمة وتنزيل الأوزان**، ولهذا تركته متوقفًا افتراضيًا حتى يبقى الملف قابلًا للتشغيل دون اتصال:

```python
```

```python
import keras

# pip install -U keras-hub
RUN_KERAS_HUB = False   # اجعلها True بعد التثبيت وتوفّر اتصال بالإنترنت

train_texts = [
    "الخدمة ممتازة وسريعة", "التطبيق سهل ومفيد",
    "الموظفون متعاونون ولطفاء", "المنتج وصل بحالة جيدة",
    "الأسعار مناسبة والجودة عالية", "التجربة رائعة وسأكررها",
    "الخدمة سيئة ومملة", "التطبيق بطيء ويغلق كثيرا",
    "الموظفون غير مهتمين", "المنتج تالف وغير جيد",
    "الأسعار مرتفعة والجودة ضعيفة جدا", "التجربة مخيبة ولن أكررها",
]
train_labels = [1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0]
test_texts = ["الخدمة سريعة والموظفون محترمون", "الطلب وصل متأخرا والمنتج تالف"]
test_labels = [0, 1]

if RUN_KERAS_HUB:
    import keras_hub

    # متعدد اللغات (104 لغات) بنحو 178 مليون معامل؛ التنزيل مئات الميغابايتات preset
    classifier = keras_hub.models.BertTextClassifier.from_preset(
        "bert_base_multi",
        num_classes=2,
    )
    classifier.compile(
        loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
        optimizer=keras.optimizers.Adam(learning_rate=5e-5),
        metrics=["accuracy"],
    )
    classifier.fit(
        train_texts, train_labels,
        batch_size=4, epochs=2,
        validation_data=(test_texts, test_labels),
    )
    print(classifier.predict(test_texts, batch_size=2))
else:
    print("مسار KerasHub جاهز لكنه متوقف: فعّله بعد تثبيت keras-hub وتنزيل الأوزان.")
    # المتوقع عند التعطيل: رسالة توضيحية فقط، بلا أي تنزيل
```

لماذا لا ندرّب BERT من الصفر في كل مشروع عربي؟ لأن التدريب المسبق يحتاج مليارات الرموز وقدرة حسابية هائلة. الضبط الدقيق يوفّر ذلك، لكنه يورث النموذج تحيّزات بياناته: نصٌّ فصيح أكثر من اللهجات، ومحتوى إخباري أكثر من محادثات العملاء. إذا كان مجالك بعيدًا عن توزيع التدريب المسبق، ففكّر في نموذج عربي مخصّص أو في استمرار التدريب المسبق (Continued Pretraining) على نصوصك.

14.5 العربية: لغةٌ تتحدى المعجم

العربية ليست لغةً واحدةً في الممارسة. بين الفصحى ولهجات الخليج ومصر والشام والمغرب فروقٌ في المفردات والتراكيب وحتى اتجاه النفي. جملة «ما في خدمة» تعني «لا توجد خدمة» في لهجاتٍ عديدة، بينما قد تُقرأ في الفصحى سؤالًا عن محتوى الخدمة. نموذجٌ دُرّب على الفصحى سيعامل «ما» و«في» ككلمتين مألوفتين، لكنه لن يرى التركيب السالب كوحدة دلالية. هذه ليست مشكلة معجمٍ فقط، بل مشكلة توزيع.

التشكيل يمثّل تحديًا آخر. معظم النصوص العربية اليوم بلا تشكيل، والنموذج يتعلّم من غيابه. إضافته في التدريب قد تحسّن النطق أو التحليل الصرفي، لكنها تخلق فجوةً بين التدريب والاستدلال إن كانت بيانات الإنتاج بلا تشكيل. كذلك تختلف رسم الهمزات والتاء المربوطة والألف المقصورة بين كاتبٍ وآخر، وقد يظهر الحرف نفسه بأشكال يونيكود متعدّدة. التطبيع يحلّ جزءًا، لكنّه قد يمحو فروقًا دلالية. الحلّ ليس «أفضل تطبيع»، بل تطبيعٌ موثّق ومُختبَر على شرائح البيانات المختلفة.

الجذور والصرف يضاعفان المشكلة. كلمة «استخدام» تحمل جذر «خدم» ولواحق عديدة؛ التقطيع تحت-الكلمي يجزّئها إلى مقاطع قد لا تحافظ على الجذر. بعض الأنظمة تستخدم الجذر أو الجذع (Stemming) لتقليص المعجم، لكنّ ذلك قد يخلط بين كلماتٍ متشابهة الجذر ومختلفة المعنى. مع BERT، غالبًا لا تحتاج إلى تجذيعٍ يدوي؛ النموذج يتعلّم التجزئة الأنسب من البيانات. مع النماذج الخطية أو TF-IDF، قد يفيد الجذع لأنه يقلّل التنوّع.

هناك أيضًا تحديات هندسية: النص العربي يُخلط بالإنجليزية والأرقام ورموز الهاشتاغ، وقد يحتوي إيموجي يحمل مشاعر لا تعبّر عنها الكلمات. اتجاه النص من اليمين إلى اليسار لا يغيّر حساب النموذج، لكنه يغيّر عرض البيانات وتفسير الأخطاء. والأهمّ: اللهجة ليست سمةً واحدة؛ قد تختلف داخل البلد نفسه بحسب المدينة والجيل والوسط. تقييم نموذجٍ على عيّنة واحدة من «العربية» يعطي طمأنينة زائفة.

حالة واقعية: بنكٌ يقرأ لهجات عملائه

بعد أن انهارت دقة النموذج في الإنتاج، راجع الفريق بيانات التدريب: 88% منها مراجعات فصيحة من الموقع، و12% فقط من تغريدات العملاء. بنى الفريق مجموعة اختبارٍ جديدة موزّعة على ثلاث لهجات، ثم أضاف بياناتٍ لهجية موثّقة، واستخدم تقطيعًا تحت-كلمي متعدّد اللغات بدل معجم الكلمات. غيّر أيضًا هدف التقييم: لم يعد «الدقة الكلية» كافيًا، بل أضاف الدقة والاستدعاء لكل لهجة على حدة. النتيجة لم تكن نموذجًا مثاليًا، لكنها كشفت الفشل مبكرًا: لهجةٌ واحدة بقيت ضعيفة، فصار قرار جمع البيانات مبنيًّا على قياس، لا على انطباع.

14.6 من النص إلى نموذج: تصنيف المشاعر خطوة بخطوة

لنبنِ مصنّفًا صغيرًا على بيانات عربية مضمّنة. الهدف تعليمي: نريد أن نرى الخط كاملًا، لا أن ننافس نموذجًا مدرّبًا مسبقًا. البيانات أربعٌ وعشرون زوجًا من الجمل الموجبة والسالبة، نصفها للتدريب ونصفها للاختبار. لاحظ أنّ التطبيع والبناء يتمّان على نصوص التدريب فقط، ثم يُطبّقان على الاختبار.

```python
import numpy as np
import tensorflow as tf
import keras

keras.utils.set_random_seed(42)

def normalize_arabic(text):
    text = tf.strings.regex_replace(text, "[\u064B-\u065F\u0670]", "")
    text = tf.strings.regex_replace(text, "[أإآ]", "ا")
    text = tf.strings.regex_replace(text, "ي" ,"ى")
    text = tf.strings.regex_replace(text, "ه" ,"ة")
    text = tf.strings.regex_replace(text, "\u0640", "")
    text = tf.strings.regex_replace(text, r"[.,!?؛؟:\(\)\[\]\"'«»…\-]", " ")
    return tf.strings.regex_replace(text, r"\s+", " ")

# 24 زوجًا: جملة موجبة تليها جملة سالبة، ليبقى التقسيم متوازنًا
pairs = [
    ("الخدمة كانت ممتازة والموظفون لطفاء جدا", "1"), ("الخدمة سيئة والموظفون غير مهتمين", "0"),
    ("أعجبني المنتج وجودته عالية", "1"), ("المنتج تالف وجودته منخفضة", "0"),
    ("تجربة رائعة وسأكررها بكل تأكيد", "1"), ("تجربة مخيبة ولن أكررها", "0"),
    ("التوصيل كان سريعا والطلب وصل بحالة جيدة", "1"), ("التوصيل تأخر كثيرا والطلب وصل متأخرا", "0"),
    ("الفيلم ممتع والقصة مشوقة", "1"), ("الفيلم ممل والقصة ضعيفة", "0"),
    ("الطعام لذيذ والمطعم نظيف", "1"), ("الطعام بارد والمطعم غير نظيف", "0"),
    ("التطبيق سهل الاستخدام وسريع", "1"), ("التطبيق بطيء ويغلق كثيرا", "0"),
    ("الرحلة كانت مريحة والطاقم محترم", "1"), ("الرحلة كانت متعبة والطاقم غير محترم", "0"),
    ("أنصح الجميع بهذا الكتاب المفيد", "1"), ("لا أنصح بهذا الكتاب الممل", "0"),
    ("الغرفة واسعة والإطلالة جميلة", "1"), ("الغرفة ضيقة والإطلالة سيئة", "0"),
    ("المعلم يشرح بوضوح ويجيب عن الأسئلة", "1"), ("المعلم لا يشرح بوضوح ولا يجيب عن الأسئلة", "0"),
    ("شعرت بالسعادة بعد قراءة الرسالة", "1"), ("شعرت بالحزن بعد قراءة الرسالة", "0"),
]

train_pairs, test_pairs = pairs[:20], pairs[20:]   # 40 تدريبًا و8 اختبارًا
train_texts = [p[0] for p in train_pairs]
test_texts = [p[0] for p in test_pairs]
train_labels = np.array([p[1] for p in train_pairs], dtype="float32")
test_labels = np.array([p[1] for p in test_pairs], dtype="float32")

# المعجم يُبنى من التدريب فقط، لا من الاختبار
```

```
vectorizer = keras.layers.TextVectorization(
    max_tokens=300,
    standardize=normalize_arabic,
    output_mode="int",
    output_sequence_length=10,
)
vectorizer.adapt(train_texts)
print("حجم المعجم:", len(vectorizer.get_vocabulary()))   # المتوقع: نحو 100 رمز

model = keras.Sequential([
    keras.Input(shape=(1,), dtype="string"),
    vectorizer,
    keras.layers.Embedding(
        input_dim=len(vectorizer.get_vocabulary()),
        output_dim=16,
        mask_zero=True,
    ),
    keras.layers.GlobalAveragePooling1D(),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(1, activation="sigmoid"),
])

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss=keras.losses.BinaryCrossentropy(),
    metrics=["accuracy", keras.metrics.Precision(name="precision"), keras.metrics.Recall(nam
)

history = model.fit(
    train_texts, train_labels,
    validation_data=(test_texts, test_labels),
    epochs=40, batch_size=8, verbose=0,
)

test_prob = model.predict(tf.constant(test_texts), verbose=0).ravel()
test_pred = (test_prob >= 0.5).astype("int32")
tp = int(np.sum((test_pred == 1) & (test_labels == 1)))
fp = int(np.sum((test_pred == 1) & (test_labels == 0)))
fn = int(np.sum((test_pred == 0) & (test_labels == 1)))
precision = tp / max(tp + fp, 1)
recall = tp / max(tp + fn, 1)
print(f"دقة الاختبار: {(test_pred == test_labels).mean():.2f}")   # غالبًا بين 0.75 و1.00
print(f"الدقة الموجبة: {precision:.2f} | الاستدعاء: {recall:.2f}")

samples = ["الخدمة رائعة والموظفون متعاونون", "الطلب تأخر والمنتج تالف"]
probs = model.predict(tf.constant(samples), verbose=0).ravel()
for text, p in zip(samples, probs):
```

```
    print(f"{text} → احتمال الإيجاب: {p:.2f}")
# المتوقع: الجملة الأولى فوق 0.5 والثانية تحته؛ القيم تتغيّر مع البذرة
```

النتيجة ليست مقياسًا حقيقيًا لجودة النموذج؛ فثماني جمل اختبار لا تكفي. لكنّ الكود يوضّح ثلاثة قرارات مهمة: بناء المعجم بعد التقسيم، وتطبيق التطبيع نفسه على التدريب والاختبار، واستخدام قناعٍ يمنع الحشو من التأثير. أغلب أخطاء مشاريع NLP لا تحدث في المعمارية، بل في هذه القرارات الصغيرة.

14.7 التقييم: ما الذي نقيسه فعلًا؟

الدقة الكلية (Accuracy) هي أول ما نطبعه، وهي أيضًا أكثر ما يخدعنا. في بياناتٍ غير متوازنة، نموذجٌ يتنبأ دائمًا بالفئة الأغلب يحقّق دقةً عالية وهو لا يعرف شيئًا عن الأقلية. لذلك نسأل: ما تكلفة الخطأ؟ في تصنيف الشكاوى، تفويت شكوى غاضبة (نقص في الاستدعاء) قد يكون أخطر من تصنيفٍ خاطئ إضافي (نقص في الدقة). في تحليل المشاعر، قد نريد توازنًا عبر F1، أو متوسطًا ماكرو (Macro) يعطي كل فئة وزنًا متساويًا بغضّ النظر عن حجمها.

للمهام المختلفة مقاييس مختلفة. في وسم التسلسلات (Sequence Labeling) مثل الكيانات المسماة، تُقاس الدقة على مستوى الرمز وعلى مستوى الكيان؛ الأولى قد تبدو عالية بينما النموذج يفشل في استخراج الكيان كاملًا. في التوليد، تُستخدم BLEU وROUGE وchrF، لكنها تقيس التشابه السطحي مع مرجعٍ واحد وقد تكافئ نصوصًا ركيكة. في الاسترجاع، نستخدم Recall@k وMRR وnDCG. لا يوجد مقياس واحد «أفضل»؛ يوجد مقياسٌ يطابق قرارًا عمليًا.

المقياس	يجيب عن	متى يخدعك؟
الدقة	كم نسبة التنبؤات الصحيحة؟	بيانات غير متوازنة
Precision	من بين ما قلت إنه موجب، كم كان صحيحًا؟	عندما تكون الإيجابيات نادرة
Recall	كم من الإيجابيات الحقيقية التقطت؟	عندما يكون تفويت الموجب مكلفًا
F1	توازن بين الدقة والاستدعاء	يفترض تساوي أهميتهما
ROC-AUC	قدرة الترتيب عبر العتبات	عدم توازن شديد
PR-AUC	أداء الفئة النادرة	يتأثر بالعتبة والمجال

التسريب وعدم تطابق التطبيع

أخطر خطأ في NLP لا يظهر في الكود، بل في تقسيم البيانات. إذا كرّرت جملةً متطابقة أو شبه متطابقة بين التدريب والاختبار، فالنموذج يحفظها ويبدو ذكيًا. هذا شائع في تغريدات أعيد نشرها، وفي مراجعاتٍ من المستخدم نفسه، وفي ترجماتٍ متعدّدة لنصٍّ واحد. الحلّ: إزالة التكرار قبل التقسيم، والتقسيم بحسب المستخدم أو المستند أو الزمن عندما يكون ذلك ممكنًا، لا بحسب الصف.

التسريب الثاني يخصّ المعجم والتطبيع: إذا بنيت `TextVectorization` على كل البيانات ثم قسّمت، فقد رأى المعجم كلمات الاختبار. التسريب الثالث يخصّ المقياس: إذا اخترت العتبة أو عدد الدورات من مجموعة الاختبار نفسها، فأنت تُضبط على الاختبار. استخدم مجموعة تحقّق منفصلة، واحتفظ بالاختبار لمرة واحدة.

أما عدم تطابق التطبيع (Normalization Mismatch) فيحدث حين يُطبّق تنظيفٌ مختلف في التدريب والخدمة. في التدريب حذفت التشكيل ووحّدت الألف، وفي الخدمة نسيت: فينهار النموذج على نصوصٍ تبدو له جديدة. الحلّ: دالة تنظيف واحدة تُستخدم في المسارين، وتُحفظ مع النموذج أو تُستدعى قبله، مع اختباراتٍ لنصوصٍ من كل لهجة.

> ⚠ **تحذير**
>
> **العرض:** دقة 98% على الاختبار، ثم 55% في الإنتاج. **السبب:** تكرار نصوص بين التدريب والاختبار، أو بناء المعجم على كل البيانات، أو تطبيع مختلف في الخدمة. **الحل:** أزل التكرار قبل التقسيم، ابنِ المعجم على التدريب وحده، ووحّد دالة التطبيع بين المسارين.

14.8 أخطاء شائعة

> ⛔ **خطر**
>
> **العرض:** حذف كل علامات الترقيم و«لا» و«ليس» باسم التنظيف، ثم انقلاب تصنيف المشاعر. **السبب:** معالجة النص كأنه حروف بلا نحو، وإزالة النفي الذي يحمل المعنى. **الحل:** احتفظ بأدوات النفي والضمائر، واختبر أثر كل خطوة تنظيف على مجموعة تحقّق.

> ⚠ **تحذير**
>
> **العرض:** قصّ كل الجمل إلى 16 رمزًا لأن النموذج أسرع، ثم انخفاض الدقة. **السبب:** القصّ يمحو نهاية الجملة، وقد تحمل النهاية رأيًا أو نفيًا. **الحل:** ادرس توزيع أطوال النصوص، واختر طولًا يغطّي 95% منها، أو استخدم تجميعًا هرميًّا.

> ⚠ **تحذير**
>
> **العرض:** نموذج BERT يعطي نتائج ممتازة على الفصحى وضعيفة على اللهجات. **السبب:** توزيع بيانات الضبط الدقيق لا يمثّل جمهور الإنتاج. **الحل:** قِس الأداء لكل لهجة، وأضف بيانات لهجية موثّقة، وفكّر في نموذج متعدّد اللهجات.

14.9 تمارين

1. **تقطيع يدوي:** خذ جملة «وبكتابهم الجديد» وحدّد اللواحق والضمائر والجذر. (المتوقع: و + ب + كتاب + هم، والجذر «كتب».)
2. **حجم المعجم:** قارن معجم الكلمات ومعجم المقاطع في 100 جملة عربية. (المتوقع: معجم الكلمات أكبر، ومعدّل OOV فيه أعلى.)
3. **التطبيع:** شغّل الكود مرة بإزالة التشكيل ومرة بدونه على 20 جملة. (المتوقع: فرقٌ في عدد الرموز، وربما فرقٌ في الدقة.)
4. **التقنيع:** غيّر `mask_zero` إلى `False` وأعد التجربة. (المتوقع: يتغيّر متوسط التجميع، وقد تنخفض الدقة.)
5. **BERT:** اختر معدّل تعلّم وعدد دورات لمجموعة من 2000 تغريدة. (المتوقع: `5e-5` - `2e-5`، ودورتان إلى أربع، مع EarlyStopping.)
6. **تدقيق تسريب:** راجع خطًّا يستخدم `vectorizer.adapt(all_texts)` ثم `train_test_split`. (المتوقع: تسريب معجمي؛ الحلّ: التقسيم أولًا ثم `adapt` على التدريب.)
7. **تحليل أخطاء:** صنّف 50 خطأً حسب اللهجة والطول والفئة. (المتوقع: جدولٌ يكشف شريحةً ضعيفة، لا مجرّد نسبة كلية.)
8. **مقياس:** لماذا لا تكفي الدقة في بيانات 95% سلبية؟ (المتوقع: نموذج يتنبأ بالسلب دائمًا يحقّق 95% دون استدعاء موجب.)

14.10 ملخص

- النص لا يدخل الشبكة كما هو؛ التنظيف والتقطيع والمعجم قراراتٌ تصميمية تؤثر في النتيجة.
- التقطيع تحت-الكلمي يقلّل OOV ويطيل التسلسل، والموازنة بينهما تخدم المهمة.
- التضمين يحوّل الرموز إلى متجهات، والحشو والتقنيع يمنعان الأصفار من إفساد التمثيل.
- Word2Vec وGloVe تمثيلات ثابتة، وBERT تمثيل سياقي ثنائي الاتجاه يُضبط دقيقًا.
- العربية تفرض تحديات التشكيل واللهجات والجذور، والحلّ يبدأ من بياناتٍ ممثّلة لا من معماريةٍ أعقد.
- KerasHub يختصر путь الأوزان الجاهزة، لكنّه لا يعفي من فهم التطبيع والتقييم.
- التقييم الصادق يبدأ بتقسيمٍ نظيف، ومقاييسَ تناسب القرار، وتحليل أخطاءٍ بحسب الشرائح.

الجزء الثالث · الرؤية والتسلسلات واللغة والتوليد

الفصل الخامس عشر

15

التوليد: GANg VAEg Autoencoder

حكاية الفصل

أراد فريق صيانة توليد صور عيوب نادرة لتدريب مفتشين جدد. أنتج النموذج صورًا واضحة، لكنها كانت نسخًا متقاربة من عيب واحد. جودة الصورة وحدها لم تكشف أن التوليد غطى جزءًا ضيقًا من الاحتمالات. كيف نمثل التوزيع ونقيس تنوعه؟

15.1 أهداف الفصل

- [ ] شرح عنق الزجاجة في Autoencoder.
- [ ] تفسير ELBO وإعادة المعلمة في VAE.
- [ ] وصف لعبة Generator وDiscriminator في GAN.
- [ ] تشخيص الانهيار ومقاييس التوليد.

15.2 Autoencoder يتعلم إعادة البناء

يضغط المشفر `x` إلى تمثيل كامن `z`، ثم يحاول المفكك إعادة بناء `x`. تقاس المسافة بين الأصل والناتج؛ لا تعني إعادة البناء الجيدة أن التمثيل مفيد لمهمة خارجية. إذا كان فضاء التمثيل غير منظم، فقد يؤدي أخذ نقطة عشوائية منه إلى مخرج بلا معنى.

15.3 VAE يعلّم فضاءً احتماليًا

يتنبأ المشفر بمعلمات توزيع كامن، عادة متوسطًا وانحرافًا معياريًا، ثم تؤخذ عينة منه ويعيد المفكك بناءها. يعاقب الهدف خطأ الإعادة والابتعاد عن prior معياري. مصطلح ELBO هو حد أدنى للأدلة اللوغاريتمية؛ وموازنة الحدين تحدد مقدار ضغط التمثيل مقابل دقة الإعادة.

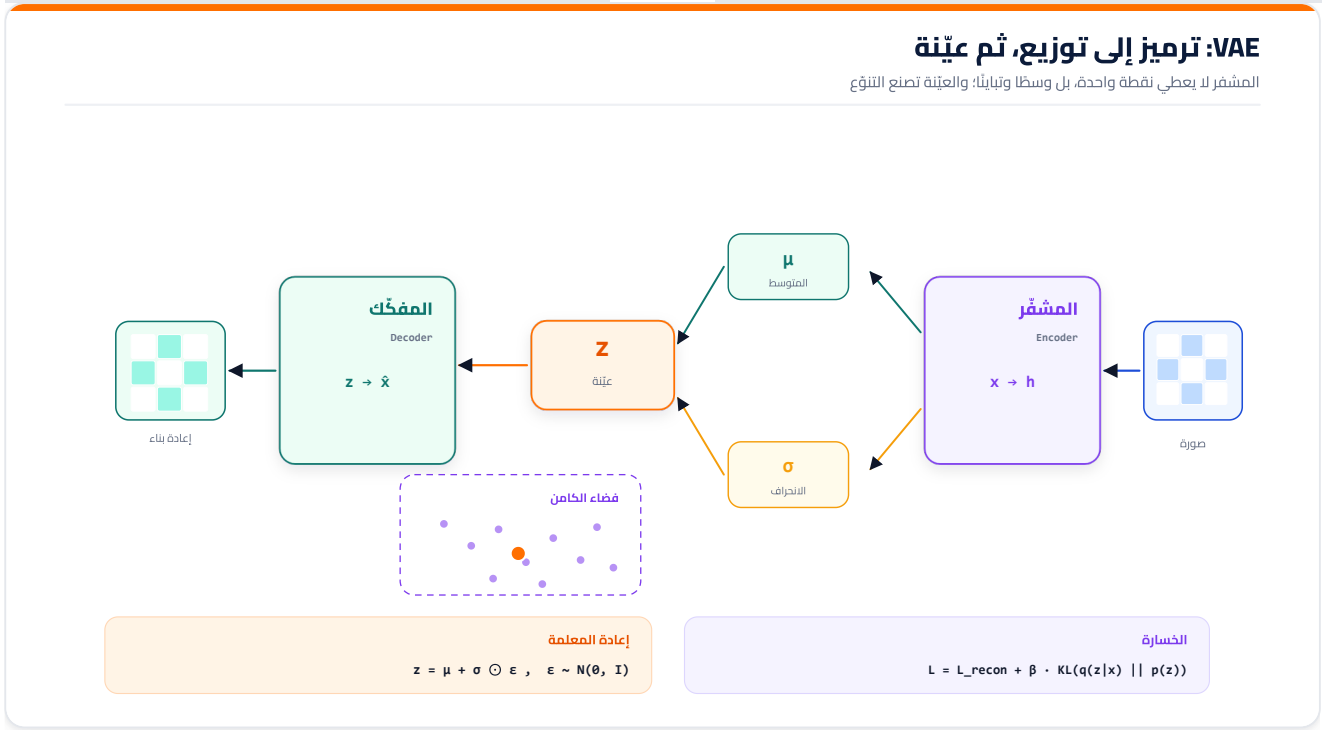
$$L = L_{reconstruction} + \beta D_{KL}(q(z|x) \| p(z))$$

الحد الأول يقيس الإعادة، و `D_KL` يقيس اختلاف التوزيع الكامن عن prior، و `β` يوازن بينهما. إعادة المعلمة تكتب `z=μ+σ·ε` حيث `ε` ضوضاء طبيعية مستقلة، كي تمر التدرجات إلى `μ` و `σ`.

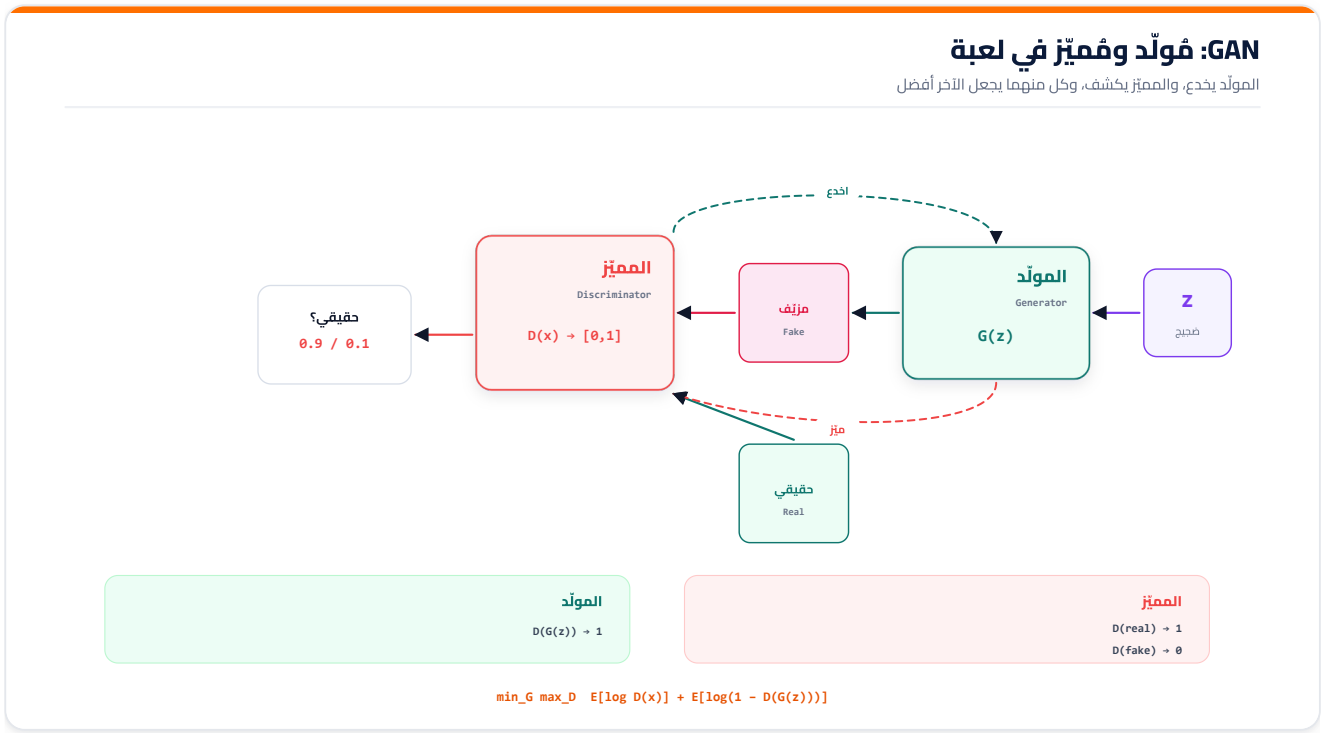
15.4 GAN لعبة خصومية لا مقياس جودة وحيد

ينتج المولد عينة من ضوضاء، ويحاول المميّز التمييز بين الحقيقي والمولد. يتدرب الطرفان بالتناوب؛ إذا صار المميز أقوى بكثير اختفى التدرج المفيد للمولد، وإذا تكرر المولد في عينة نمطية واحدة ظهر انهيار الأنماط. WGAN-GP يبدل صياغة الهدف ويضيف عقوبة تدرج لتحسين الاستقرار، لكنه لا يلغي الحاجة إلى ضبط التدريب.

النموذج	ما الذي يتعلمه؟	نمط الفشل
AE	إعادة بناء مدخل	تمثيل لا يصلح لأخذ عينات حرة
VAE	توزيع كامن منظم	صور ضبابية عند أهداف بسيطة
GAN	توزيع عبر لعبة خصومية	عدم استقرار أو انهيار أنماط



شكل 15-1 فضاء VAE الكامن وإعادة المعلمة



شكل 15-2 توازن المولد والمميّز في GAN

15.5 تقييم العينات

افحص عينات عشوائية متنوعة، وقارن تغطية الأنماط وجودة التفاصيل مع بيانات حقيقية. FID يقارن إحصاءات سمات عميقة بين مجموعتين؛ يتأثر بحجم العينة وبالشبكة المرجعية، فلا تقارن رقمين بإعدادين مختلفين. IS لا يقيس كل جوانب التنوع. في تطبيقات العيوب، مراجعة خبير المجال ضرورية حتى لا تدخل صور غير واقعية في التدريب.

15.6 حالة واقعية: عيوب صناعية نادرة

يدرّب الفريق VAE أو GAN على بيانات التدريب فقط، ويحتفظ بمجموعة حقيقية لاختيار الإعدادات. يقيس تنوع التوليد ويطلب من مفتشين تمييز الصور الاصطناعية، ثم يختبر ما إذا كان إدخالها يحسن استدعاء عيب نادر على صور حقيقية محجوزة. إن لم يحسن ذلك، فالتوليد ليس نجاحًا لمجرد إنتاج صور جذابة.

15.7 أمثلة تنفيذية إضافية

عنق زجاجة للمشفّر الذاتي

```python
inputs = keras.Input((28, 28, 1))
z = keras.layers.Flatten()(inputs)
z = keras.layers.Dense(16, activation="relu")(z)
outputs = keras.layers.Dense(784, activation="sigmoid")(z)
outputs = keras.layers.Reshape((28, 28, 1))(outputs)
autoencoder = keras.Model(inputs, outputs)
```

ضوضاء

```python
noise = tf.random.normal(tf.shape(images), stddev=0.1)
noisy_images = tf.clip_by_value(images + noise, 0.0, 1.0)
```

15.8 ورشة عملية: ما الذي نراقبه أثناء التدريب؟

يرجع مشفر VAE المتوسط واللوغاريتم للتباين. استعمل log-variance لتجنب انحراف معياري سالب، ثم خذ عينة عبر الضوضاء:

```python
import tensorflow as tf

def sample_latent(mean, log_var):
    epsilon = tf.random.normal(tf.shape(mean))
    return mean + tf.exp(0.5 * log_var) * epsilon

def kl_per_example(mean, log_var):
    term = 1.0 + log_var - tf.square(mean) - tf.exp(log_var)
    return -0.5 * tf.reduce_sum(term, axis=-1)
```

خسارة الإعادة تعتمد على نموذج البيانات: binary cross-entropy تناسب بكسلات مطبعة إلى `[0,1]` مع افتراض Bernoulli تقريبي، وMSE يناسب خطأ مستمر بسيط. طريقة `sum` أو `mean` على البكسلات تغير وزن KL الفعلي؛ لذلك اكتب reduction بوضوح. راقب متوسط KL لكل بعد وصور إعادة البناء، لا الخسارة الإجمالية فقط. KL warm-up قد يخفف posterior collapse لكنه يقدم معاملًا وجدولًا إضافيين يجب تسجيلهما.

في GAN التقليدي، يحدّث المميّز على عينات حقيقية ومولدة، ثم يحدّث المولد عبر مخرج المميّز. راقب خسارة الطرفين وشبكة عينات ثابتة من بذرة ثابتة. WGAN-GP يستخدم critic بلا sigmoid ويضيف عقوبة معيار التدرج على عينات interpolated؛ لا تخلط معه صيغة binary cross-entropy كما هي. تكرار الصور قد يدل على نمط منهار أو نقص تنوع البيانات أو عدم توازن التحديثات.

تقييم التوليد يجمع جودة فردية، وتنوعًا، وتغطية أنماط، وفحص تشابه مع بيانات التدريب، وأثرًا على المهمة اللاحقة. FID يقارن إحصاءات سمات شبكة صور عامة وقد لا يناسب صورًا طبية أو صناعية. اذكر عدد العينات ونسخة الشبكة ومجموعة الحقيقة المرجعية. افحص كذلك احتمال إعادة إنتاج صورة تدريب حساسة قبل مشاركة الأوزان.

15.9 بناء تجربة توليد يمكن للآخرين مراجعتها

حدد سؤال التوليد قبل اختيار البنية: هل نحتاج ضغطًا واستعادة، توليدًا متنوعًا من prior، أم زيادة بيانات؟ Autoencoder العادي يختبر ضغط المعلومات، لكنه ليس مولد عينات عشوائية بمجرد وجود latent space. VAE ينظم هذا الفضاء ويجعل interpolation أكثر معنى في كثير من الحالات، لكنه قد يضحي بحدة تفاصيل الإعادة.

GAN ينتج مخرجات حادة أحيانًا لكنه يحتاج توازن تدريب ومراقبة أكثر صعوبة. قرار البنية يتبع الهدف والبيانات والميزانية.

15.10 بروتوكول عملي على صور صغيرة

1. راجع نطاق الصور وحجمها، ونظف التكرار والملفات التالفة.
2. افصل train/validation/test بحسب المصدر أو المريض أو الأصل قبل augmentation.
3. ابن نموذجًا صغيرًا overfits على دفعة للتحقق من خسارة الإعادة.
4. ثبت seed لأخذ العينات، واحفظ grid ثابتة كل عدد خطوات.
5. غيّر معاملًا واحدًا في كل تجربة، مثل latent dimension أو learning rate.
6. افحص جودة الصور والتنوع وأقرب جيران للتدريب.
7. قس أثر العينات الاصطناعية على مصنف يقيّم على حقيقي محجوز.

يحد هذا البروتوكول من الخطأ الشائع المتمثل في تفضيل نموذج لأنه ينتج صورًا جذابة بصريًا. إذا كان المشروع إعادة بناء، اعرض PSNR/SSIM حيث يناسبان مع حدود كل مقياس. إذا كان توليدًا، اعرض مجموعة عينات عشوائية لا أفضل مجموعة فقط. اجعل مقياسًا إحصائيًا قابلاً لإعادة الإنتاج مع مراجعة خبير.

15.11 اختيار latent dimension وتفسيره

الفضاء الكامن الأصغر يضغط أكثر وقد يمحو تفاصيل صغيرة؛ الأكبر قد يسمح بسعة أعلى لكنه لا يضمن تنوعًا مفيدًا. في VAE افحص متوسط التباين وقيمة KL لكل بعد؛ أبعاد كثيرة ذات KL قريب من صفر قد تكون مهملة. interpolation الخطي بين تمثيلين يكشف إن كان الفضاء سلسًا، لكنه لا يثبت أن كل النقاط الوسيطة تقع على manifold البيانات.

15.12 الاستخدام البحثي والأخلاقي

اذكر أن الصور مولدة ولا تستخدمها لتوثيق ظاهرة واقعية. في المجال الطبي، لا تستبدل بيانات مرضى حقيقية بعينات مولدة دون دراسة أثر التمثيل والتحيز. احصل على إذن بيانات المصدر، وقيّم membership inference أو حفظ نسخ قريبة عند الحاجة. إذا سيشارك model weights، ضع معلومات الاستخدام المقصود والمخاطر والقيود مع النموذج.

15.13 دراسات ablation ومقارنة عادلة

اختبر latent dimension ووزن KL وحجم decoder على split ثابت وميزانية تدريب مماثلة. إذا ضاعفت عمق شبكة وقللت خطواتها، فأنت تقارن عاملين. احفظ كود seed الذي يولد عينة ثابتة ومجموعة samples عشوائية منفصلة. أبلغ زمن التدريب وحجم checkpoint ومقدار الذاكرة؛ نموذج مولد قد يكون ذا جودة جيدة لكنه غير عملي إذا تجاوز قيد التطبيق.

في VAE، اعرض reconstruction loss وKL كلًا على حدة. loss إجمالية أقل قد تعني أن أحد الحدين هيمن بسبب اختلاف reduction. راقب posterior collapse عبر قدرة decoder على إعادة البناء دون استخدام z، وتحقق من KL لكل بعد. في GAN، سجل الصور والخسائر وعينة تنويع، لأن loss المميّز لا تترجم مباشرة إلى جودة بشرية.

15.14 التوليد المشروط بالمعلومات

يمكن ربط latent أو طبقات وسيطة بفئة أو وصف. في GAN، condition embedding يصل إلى المولد والمميز؛ في VAE يمكن إدخال الشرط إلى المشفر والمفكك. اختبار الشرط لا يكون بجودة الصور وحدها: هل يتغير اللون أو النوع المقصود عند تغيير الشرط مع بقاء العوامل الأخرى؟ إذا لم يتغير، فالنموذج تجاهل الشرط. قس conditional fidelity وتنوع العينات داخل كل فئة.

15.15 اختيار مقياس الصورة

PSNR يقيس خطأ pixel ويتأثر بمحاذاة البكسلات، وSSIM يحاكي بعض جوانب التشابه البنيوي. FID وKID يقارنان سمات عميقة لتوزيع العينات. لكل مقياس افتراضات، وقد يتعارض مع رأي المختص. اعرض grid ومقاييس متعددة وبروتوكول مقارنة ثابت. لا تختَر أفضل الصور بالعين ثم تعرضها على أنها توزيع النموذج.

15.16 صلاحية استخدام البيانات

المولد يتعلم من بيانات المصدر حتى لو لم يحتفظ بها حرفيًا. راجع تراخيص الصور وحقوق الاستخدام، وقيّم إمكانية استرجاع شخص أو وثيقة حساسة. حدد هل مخرجاته آمنة للمشاركة، وقلل وصول المستخدم إلى latent/interpolation إذا زاد خطر استرداد المثال. استخدم مراجعة privacy لمنع نشر صور مولدة توحي خطأً بحدث حقيقي.

15.17 تصميم دراسة ومقارنة عادلة

اختبر latent dimension ووزن KL وحجم decoder على split ثابت وميزانية تدريب مماثلة. إذا زدت العمق وقللت خطوات التدريب، فقد غيرت عاملين. احفظ عينات ببذرة ثابتة ومجموعة random منفصلة. أبلغ زمن التدريب وحجم checkpoint والذاكرة؛ جودة النموذج وحدها لا تصف ملاءمته للتطبيق.

في VAE اعرض reconstruction loss وKL منفصلين، وافحص KL لكل بعد. في GAN اعرض خسائر الطرفين وشبكة صور ثابتة، لأن loss المميّز ليست مقياسًا مباشرًا لجودة الصور. اختبر conditional fidelity: هل يتغير المخرج مع تغيير الفئة المطلوبة؟

PSNR يقيس الخطأ البكسلي، وSSIM يصف تشابهًا بنيويًا، وFID/KID يقارنان سمات توزيع مجموعتين. قد لا تكون شبكة صور طبيعية مناسبة لمجال طبي أو صناعي. استخدم عينات عشوائية ومراجعة خبراء ومقياس المهمة اللاحقة. تحقق من أقرب الجيران إلى train لاحتمال الحفظ، وراجع ترخيص وخصوصية الصور قبل مشاركة الأوزان.

15.18 سياسة الاستخدام

صرح أن الصور مولدة ولا تصلح دليلًا لحدث واقعي. حدد الاستخدام المقصود والقيود في بطاقة النموذج، خصوصًا عندما يمكن أن تشبه صورة شخص أو وثيقة. إذا كان المشروع augmentation، فالنجاح هو تحسن نموذج لاحق على بيانات حقيقية محجوزة، لا مظهر مولد مقنع.

15.19 نشر النتائج

انشر ملفات config ومصدرًا موثوقًا للبيانات المسموح بها، وبذورًا وعينات متنوعة وبروتوكول تقييم. لا ترفق أفضل صورة وحدها؛ أرفق فشلًا معلومًا وقيود الاستخدام، ليتمكن قارئ آخر من تقدير ما ينجح وما لا ينجح.

15.20 أخطاء شائعة

> **⚠ تحذير**
>
> لا تقِس نجاح المولد بدقة المميّز وحدها؛ قد تعني الدقة العالية أن اللعبة مختلة. [DANGER!] لا تخلط بيانات الاختبار الحقيقي في تدريب المولد ثم تدّعِ تقييمًا مستقلاً.

فك الترميز من نقطة كامنة

يحوّل المفكك متجهًا كامنًا إلى عينة؛ في VAE يجب أن يكون المتجه من prior الذي تعلمه النموذج:

```python
z = tf.random.normal((16, latent_dim))
samples = decoder(z, training=False)
print(samples.shape)  # عينات مولدة 16
```

15.21 تمارين

1. ارسم مشفرًا ومفككًا وحدد الخسارة. (المتوقع: مقارنة دخل ومخرج.)
2. اشرح حد KL في VAE. (المتوقع: تنظيم التوزيع الكامن.)
3. نفذ إعادة المعلمة رمزيًا. (المتوقع: ضوضاء مستقلة وتدرج إلى المعلمات.)
4. شخّص انهيار الأنماط من شبكة عينات. (المتوقع: تكرار قلة من الأنماط.)
5. صمم اختبارًا لتوليد عيوب صناعية. (المتوقع: تحقق خبير وقياس على حقيقي.)
6. ناقش حدود FID. (المتوقع: حجم العينة والشبكة المرجعية.)

15.22 ملخص

- يضغط AE المدخل ويعيد بناءه، بينما يفرض VAE توزيعًا كامنًا قابلًا لأخذ العينات.
- GAN لعبة خصومية قد تكون غير مستقرة أو قليلة التنوع.
- جودة التوليد تتطلب تقييمًا كميًا وبشريًا واختبار أثره على المهمة النهائية.

الجزء الثالث · الرؤية والتسلسلات واللغة والتوليد

الفصل السادس عشر

16

نماذج الانتشار: التوليد الحديث

> **حكاية الفصل**
>
> احتاج مصمم إلى صور صناعية متنوعة لجزء لا يمكن تصويره في كل ظروف الإضاءة. تلوثت صور التدريب بضوضاء مدروسة تدريجيًا، ثم تعلم النموذج عكس العملية خطوة خطوة. لماذا يساعد هدف إزالة الضوضاء، ولماذا يكون أخذ العينات مكلفًا؟

16.1 أهداف الفصل

- [ ] وصف الانتشار الأمامي والعكسي.
- [ ] فهم جدول الضوضاء وهدف توقعها.
- [ ] تتبع حلقة أخذ عينات DDPM ومبدأ DDIM.
- [ ] تقدير الكلفة والتقييم والتحكم.

16.2 إضافة ضوضاء وفق جدول

في الانتشار الأمامي نضيف ضوضاء Gaussian تدريجيًا حتى يقترب التمثيل من ضوضاء خالصة. تحدد معاملات الجدول مقدار الإشارة المتبقية في كل خطوة. أثناء التدريب يختار النموذج زمنًا وعينة وضوضاء، ويتعلم توقع الضوضاء المضافة. التوزيع على الأزمنة مهم؛ لا تدرب على خطوة واحدة فقط.

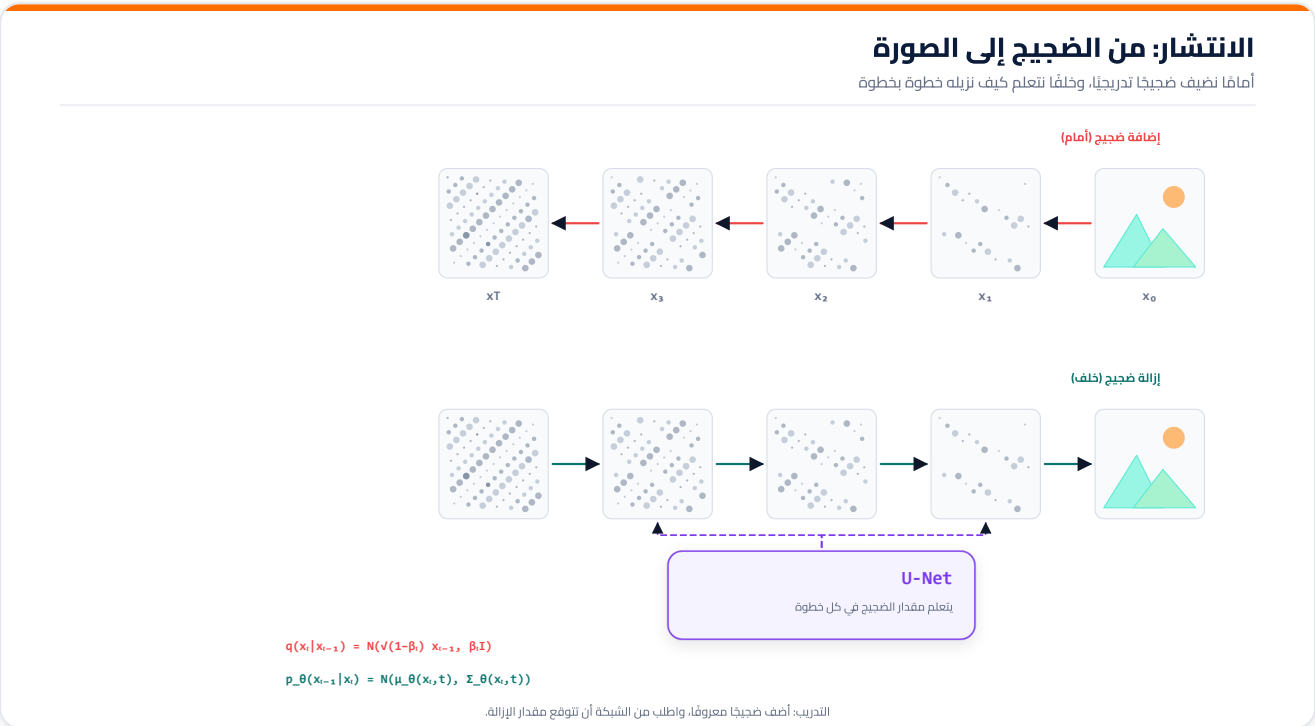
$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon$$

`x_0` الصورة الأصلية، و`ε` ضوضاء طبيعية، و`ᾱ_t` حاصل الاحتفاظ التراكمي بالإشارة حتى الزمن `t`. عندما تقل قيمة `ᾱ_t` تضعف تفاصيل الأصل.

16.3 إزالة الضوضاء مشروطة بالزمن

تستقبل U-Net الصورة المشوشة وتمثيلًا لخطوة الزمن، ثم تتنبأ بالضوضاء. تكرار التحديثات في الاتجاه المعاكس يحول ضوضاء البداية إلى صورة. يمكن إضافة شرط نصي أو فئوي لتوجيه التوليد. DDPM يستخدم سلسلة عشوائية من الخطوات، بينما يختار DDIM مسارًا أسرع أو أقل عشوائية وفق إعداداته.

العنصر	دوره	أثر الخطأ
جدول `β_t`	مقدار الضوضاء لكل خطوة	انهيار إشارة أو عينات رديئة
تضمين الزمن	إخبار U-Net بمرحلة العكس	خطوة إزالة غير ملائمة
هدف الضوضاء	إشارة تعلم مستقرة	مقياس خسارة مضلل
عدد الخطوات	توازن الزمن والجودة	بطيء أو منخفض التفاصيل



شكل 16-1 مسار الانتشار من الصورة إلى الضوضاء ثم العودة

16.4 التوجيه والتكلفة

التوجيه غير المشروط المصنّف يمكن أن يجمع توقعًا مشروطًا وغير مشروط؛ زيادة معامل التوجيه قد تقوي الالتزام بالشرط على حساب التنوع أو الجودة. تكلفة أخذ العينات تزداد مع عدد الخطوات، ولذلك توجد مسارات مختصرة ونماذج تقطير. قيّم الجودة والتنوع معًا، واستخدم FID ضمن بروتوكول ثابت، لا بوصفه حكمًا شاملًا.

16.5 حالة واقعية: صور قطع تحت إضاءة مختلفة

يُوحّد مدى الصورة وتُفصل بيانات التحقق قبل التدريب. تختبر العينات على ظروف إضاءة لم تستخدم في التكييف، وتراجعها جهة تصنيع. ثم يقارن المصنف المدرب بصور حقيقية فقط وآخر أضيفت إليه عينات اصطناعية؛ الهدف تحسن كشف العيب الحقيقي لا تحسن جمالية الصور.

16.6 أمثلة تنفيذية إضافية

عينة مشوشة

```python
alpha_bar = 0.7
epsilon = tf.random.normal(tf.shape(x0))
xt = alpha_bar**0.5 * x0 + (1.0-alpha_bar)**0.5 * epsilon
```

ضوضاء loss

```python
with tf.GradientTape() as tape:
    predicted_noise = denoiser([xt, timesteps], training=True)
    loss = tf.reduce_mean(tf.square(epsilon - predicted_noise))
```

16.7 جدول الضوضاء والتدريب

اختر `β_t` بين الصفر والواحد، ثم احسب `α_t=1-β_t` وحاصل الضرب التراكمي `ᾱ_t`. جدول يبدأ قويًا جدًا يمحو الإشارة بسرعة، وجدول ضعيف جدًا لا يعلم إزالة مستويات متنوعة. في كل دفعة اسحب خطوة زمنية `t` عشوائية وضوضاء مستقلة، وابن `x_t` بالصيغة المغلقة بدل تنفيذ جميع الخطوات الأمامية.

تنبؤ الضوضاء يعطي خسارة MSE مباشرة. توجد أهداف بديلة مثل توقع `x_0` أو v-parameterization؛ إذا غيرت صيغة الهدف، يجب أن تتوافق معها معادلة أخذ العينات. خزّن جدول الضوضاء والمعاملات ضمن artifact، فهي جزء من النموذج وليست تفصيل إعداد منفصلًا.

16.8 U-Net مشروطة بالزمن

حوّل رقم الخطوة إلى embedding ثم مرر الشرط إلى طبقات U-Net عبر جمع أو scale/shift أو cross-attention. وصلات التخطي تحفظ التفاصيل المكانية التي يفقدها المسار الهابط. عند الدقة العالية ترتفع ذاكرة خرائط التفعيلات، وقد تحتاج قصاصات أصغر أو gradient checkpointing. تحقق من مطابقة أبعاد الزمن وميزات الصورة قبل تشغيل تدريب طويل.

16.9 أخذ عينات DDPM وDDIM

ابدأ بضوضاء Gaussian، ثم نفذ انتقالات عكسية متتالية باستخدام النموذج والجدول نفسيهما اللذين تدرب عليهما. DDIM يختار مجموعة خطوات فرعية ويمكنه توليد عينة بوقت أقل، لكنه يغير المسار العكسي. قارن جودة العينات عند ميزانية زمن محددة، لا بعدد الخطوات وحده. التوجيه المصنف الحر يوازن بين التوقع المشروط وغير المشروط؛ الرفع المفرط للمعامل قد يقلل التنوع ويولد آثارًا غير طبيعية.

16.10 تقييم ومخاطر بيانات التوليد

ابدأ بدقة صغيرة وبيانات تسمح بها الرخصة، وثبت بذرة التجربة. اعرض مسار إضافة الضوضاء عند عدة أزمنة، وتأكد أن الإشارة تخفت تدريجيًا. راقب loss وتنوع شبكة عينات محفوظة دوريًا، وقس الوقت والذاكرة. افحص أقرب الجيران إلى بيانات التدريب لاحتمال نسخها. في الصور الطبية والصناعية، لا تدخل الاصطناعي إلى مجموعة تدريب نهائية قبل مراجعة خبير وقياس التحسن على بيانات واقعية محجوزة.

16.11 اختبار forward process قبل التدريب

اكتب اختبارًا يولد `x_t` لعدة قيم `t` من صورة واحدة، واعرض الصورة الأصلية والنتائج المتدرجة. ينبغي أن يزداد التشويش بانتظام. تحقق أن `ᾱ_t` متناقص، وأنه لا يصبح سالبًا أو NaN. هذا الاختبار يكشف أخطاء cumulative product أو نطاق `β_t` بسرعة أكبر من انتظار انهيار التدريب.

16.12 أخذ عينات مرئي ومقاييس هندسية

نفذ sampler منخفض الأبعاد أولًا، وسجل `x_t` في نقاط مختارة بدل كل خطوة. إذا انفجرت القيم، افحص تطبيع الصور ومعادلة posterior variance. قس زمن كل مرحلة من الشبكة، لأن U-Net قد تستدعى مئات المرات لكل عينة. عند عرض grid، ثبت seeds وشروط النص حتى تقارن الإصدارات على المدخل نفسه.

FID ليس بديلًا عن تقييم المجال؛ شبكة ImageNet قد تعتبر اختلافًا طبيًا مهمًا غير مهم والعكس. عند عدم توفر مقياس موثوق، صمم بروتوكول مراجعة عمياء لمتخصصين، وقيّم الواقعية والتنوع وصحة الشرط منفصلة. أبلغ inter-rater agreement أو تباين أحكام الخبراء بدل ادعاء حقيقة مطلقة.

16.13 اختيار دقة التدريب

تضاعف مساحة الصورة من 64×64 إلى 128×128 يرفع عدد البكسلات أربعة أضعاف قبل حساب قنوات النموذج. لذلك قد ينفد الذاكرة عند زيادة الدقة، وتقل الدفعة، وتزداد ضوضاء gradient. ابدأ بدقة صغيرة، تحقق من المفهوم والـ schedule، ثم ارفعها مع قياس. يمكن progressive resizing أو patch training، لكن اختلاف المجال بين قصاصة وصورة كاملة يحتاج فحصًا.

16.14 تلخيص دراسة نموذج

يجب أن يصف القسم المنهجي: عدد صور التدريب، normalization، resolution، جدول `β`، هدف التنبؤ، طريقة time embedding، عدد خطوات sampler، أي hardware-hours، guidance، والبذور. أبلغ مقارنة baseline ومجموعة التحقق. إن استعملت بيانات مفتوحة، راجع الترخيص والقيود، واذكر نطاقات لا يعمل عندها النظام. هذه التفاصيل تسمح لباحث آخر بإعادة إنتاج التكلفة لا النتيجة وحدها.

16.15 اشتقاق معاملات الجدول

تتحدد `α_t=1-β_t` , و `ᾱ_t=∏_{s=1}^{t} α_s` . يجعل هذا إعادة صياغة forward process مباشرة، فيمكن تدريب الشبكة على خطوة عشوائية من دون تكرار جميع الخطوات. افحص أول وآخر `ᾱ_t` ، وارسم SNR عبر الزمن `ᾱ_t/(1-ᾱ_t)` . جدول خطي أو cosine يوزع مستويات التشويش بشكل مختلف؛ اختيار الجدول يؤثر على ما يتعلمه النموذج.

في loss، انتبه إلى reduction وموازنة الأزمنة. إذا كان الخطأ أعلى عند أزمنة معينة، اعرض loss مقابل t، وفكر بوزن زمني وفق هدف معروف. لا توازن الإشارات على مجموعة الاختبار. أزل التكرارات الدقيقة في مجموعة الصور وحافظ على split المصدر.

16.16 تشخيص artifacts

خطوط أو checkerboard قد تنتج عن upsampling/convolution، وتشبع أبيض قد ينتج عن normalization أو guidance عالٍ، وطمس قد ينتج عن sampler أو تدريب غير كافٍ. اعرض الخرج قبل وبعد كل تحويل في postprocess. اختبر أن `x_0` noiseg في المجال العددي نفسه الذي افترضه التدريب. ضع noise prediction ground truth في لوحة تشخيص، واحسب خطأ الشبكة على عينة ثابتة عبر checkpoints.

16.17 التقطير والتسريع

خيارات السرعة تشمل sampler بخطوات أقل، distillation، وتعديل kernel. قارنها عند جودة مستهدفة لا عند عدد steps وحده. قِس زمن إنشاء batch كامل ووقت U-Net لكل خطوة. classifier-free guidance قد يتطلب تمريرًا مشروطًا وغير مشروط، فيضاعف الاستدلال تقريبًا ما لم يدمج batch. اذكر هذا القيد عند تقدير serving throughput.

16.18 تسجيل نموذج بحثي

احفظ نسخة EMA weights، parameterization، schedule، U-Net، إن استخدمت، sampler، optimizer state، seed. Exponential moving averageg للأوزان قد يعطي عينات أفضل، لكن قارن EMA الخاص بالشرط، config raw checkpointg على validation. لا تحفظ أوزانًا فقط ثم تعتمد على notebook مؤقت لا يعرف كيف ينفذ reverse process.

16.19 حدود التطبيق

النموذج المدرب على صور منخفضة الدقة لا يصبح مولدًا عالي الدقة بتغيير شكل الإدخال. كل تكبير يتطلب تدريبًا أو بنية متوافقة. في الاستخدام الطبي أو الجنائي، لا تعرض مخرجات اصطناعية بوصفها دليلًا واقعيًا. اذكر احتمالية hallucination، وآلية وسم الصور الاصطناعية، واستخدامات ممنوعة في model card.

16.20 مقارنة sampler وتقليل الكلفة

قارن DDPM وDDIM على الصور والبذور والشروط نفسها، وسجل عدد تقييمات الشبكة لا عدد الخطوات فقط. بعض المسارات تستخدم أكثر من تمرير عند التوجيه المصنف الحر. اختبر الجودة/السرعة عند ميزانية زمن ثابتة. mixed precision قد يساعد على جهاز مدعوم، لكن راقب الاستقرار العددي. ابدأ بدقة صغيرة للتحقق من الجدول وهدف إزالة الضوضاء قبل توسيع الصورة.

سجل أول وآخر `alpha_bar`، وارسم SNR على الزمن، وتحقق أن القيم finite. اعرض `x_t` عند أزمنة مختارة؛ التشويش ينبغي أن يتزايد تدريجيًا. إذا ظهر artifact، راجع normalization والجدول ومدخل U-Net وال postprocess قبل تغيير المعمارية.

16.21 الأثر البحثي والبيئي

وثق عدد الصور ودقتها parameterizationg scheduleg وطريقة sampling وال hardware-hours والبذور. احتفظ بإعدادات النموذج والخط العكسي مع checkpoint، لا الأوزان منفردة. أبلغ تقدير الذاكرة والكلفة، لأن diffusion قد يتطلب مئات استدعاءات U-Net للعينة. اختبر synthetic data على مهمة واقعية قبل نشرها، وراجع ترخيص المصدر ومخاطر حفظ بيانات حساسة.

16.22 اختيار تطبيق مسؤول

أعلن بوضوح أن المخرج مولد، واحتفظ بمعرف إصدار النموذج وجدول الضوضاء وإعداد التوجيه. لا تخلط صورة اصطناعية بوثائق حقيقة، وحدد سياسة منع الاستخدام الخادع والمحتوى الحساس.

16.23 التحقق من شرط التوليد

أنشئ جدولًا يبدل شرطًا واحدًا مع تثبيت noise والبذرة. إذا لم يتغير الجانب المقصود في المخرج، فالنموذج لا يتبع الشرط بما يكفي. قارن نماذج بلا شرط ومشروطة عند guidance=1 ثم عند قيم أخرى، واحفظ كل hyperparameter.

16.24 استقرار التحديث

استخدم EMA للأوزان كتجربة منفصلة، وقارنها مع checkpoint الخام على عينات validation ثابتة. سجّل decay وعدد تحديثات EMA؛ لا تستبدل وزن النموذج دون توثيق أي نسخة استعملت في sampler.

16.25 أخطاء شائعة

> ⚠ **تحذير**
>
> تطبيع الصور في التدريب وأخذ العينات يجب أن يتطابق؛ الاختلاف يغير معنى جدول الضوضاء. [!DANGER] المزيد من خطوات أخذ العينات لا يضمن جودة أفضل ضمن ميزانية زمنية محدودة.

تضمين الزمن

تصل خطوة الانتشار إلى U-Net كمدخل صريح حتى تميز مستويات الضوضاء:

```python
time_embedding = keras.layers.Embedding(input_dim=num_steps, output_dim=64)(timesteps)
conditioned = keras.layers.Concatenate()([pooled_image_features, time_embedding])
```

16.26 تمارين

1. احسب نسبة الإشارة والضوضاء عند قيمة $\bar{\alpha}_t$ معلومة. (المتوقع: تطبيق المعادلة.)
2. وضح بيانات هدف تدريب إزالة الضوضاء. (المتوقع: صورة وزمن وضوضاء.)
3. اشرح وظيفة تضمين الزمن. (المتوقع: تكييف التنبؤ مع مستوى التشويش.)
4. قارن DDPM وDDIM من حيث أخذ العينات. (المتوقع: مسار تقليدي مقابل خيار أسرع.)

5. صمم تقييمًا لعينات مشروطة. (المتوقع: جودة وتنوع ومراجعة مجال.)
6. احسب أثر مضاعفة خطوات الاستدلال. (المتوقع: زمن أعلى تقريبًا مع ثبات كلفة الخطوة.)

16.27 ملخص

- يتعلم الانتشار توقع الضوضاء ثم يستخدمها لإزالة التشويش تدريجيًا.
- جدول الضوضاء وتضمين الزمن والتطبيع أجزاء من العقد التدريبي.
- الاستدلال متكرر ومكلف؛ توازن السرعة والتنوع والجودة وفق التطبيق.

الجزء الرابع

الاحتراف والإنتاج

الأداء، النشر، MLOps، الأخلاق، والمشاريع

17 الطبقات والنماذج المخصصة والتدريب المتقدم

18 الأداء والتدريب الموزّع

19 النشر: LiteRT وTF.jsg وTF Servingg

20 MLOps: TFX والمراقبة

21 التعلم المعزز

22 البيانات الجدولية والتوصيات والرسوم البيانية

23 الأخلاق والخصوصية والتفسير والأمان

24 مشاريع ختامية وخارطة الطريق

الفصول 17-24 · 8 فصول

الجزء الرابع · الاحتراف والإنتاج

الفصل السابع عشر

17

الطبقات والنماذج المخصصة والتدريب المتقدم

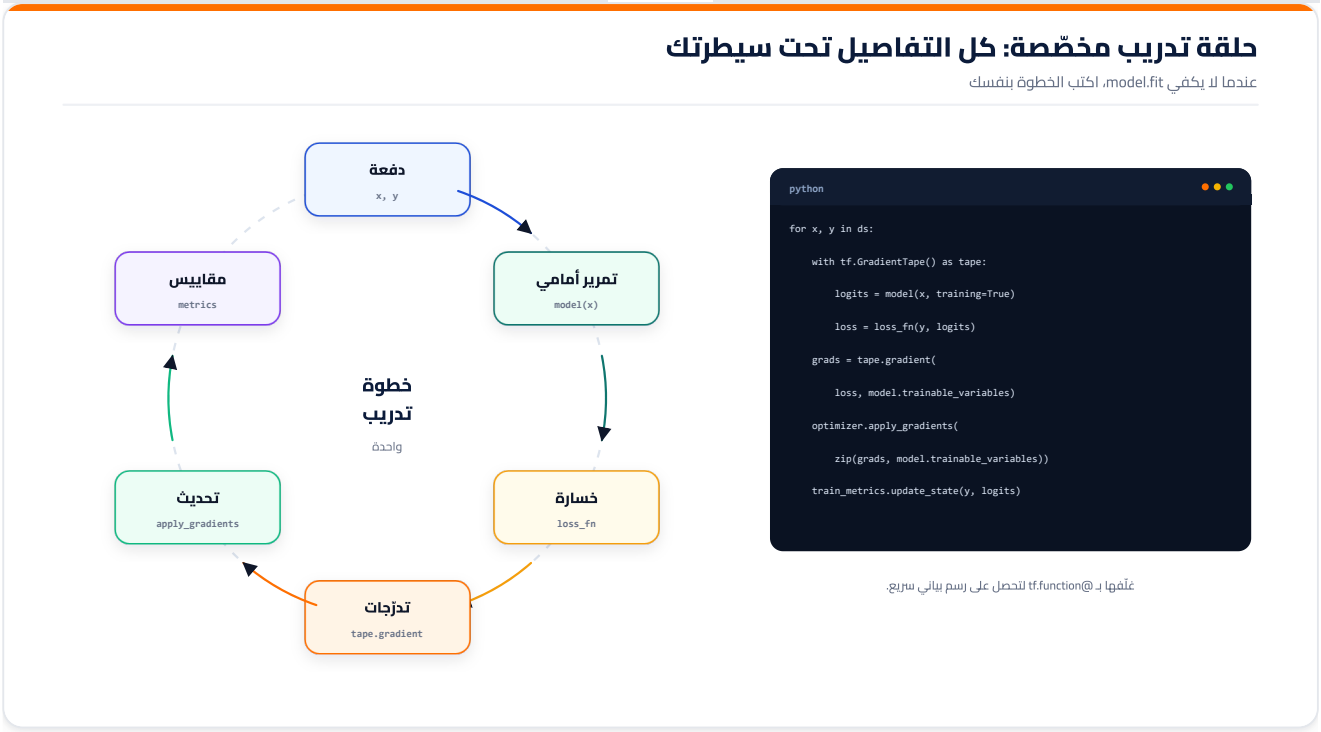
> **حكاية الفصل**
>
> طلب فريق بحث أن تحتفظ طبقةٌ مخصصة بحالة داخلية وتحسب خسارة إضافية. عملت الطبقة على دفعة واحدة، ثم فشل تحميل النموذج على جهاز زميل. كان الحساب صحيحًا لكن واجهة البناء والحفظ ناقصة. كيف نصمم امتدادًا يشارك دورة حياة Keras؟

17.1 أهداف الفصل

- [ ] تنفيذ Layer عبر `build` و `call` و `get_config`.
- [ ] إنشاء Model مخصص وتسلسله.
- [ ] كتابة `train_step` يتعامل مع المقاييس والأوزان.
- [ ] فصل السلوك المخصص القابل للاختبار عن واجهات Keras.

17.2 وزن ينشأ مرة واحدة

يهيئ `__init__` الإعدادات، وينشئ `build(input_shape)` الأوزان المعتمدة على شكل الإدخال، ثم يصف `call` الحساب الأمامي. ينشئ `add_weight` متغيرًا تتبعه Keras وتحفظه. أعد الإعدادات القابلة للتسلسل من `get_config`، وسجل الفئة باسم واضح عند الحاجة إلى إعادة تحميلها.



شكل 17-1 تفاعل الطبقة المخصصة مع حلقة التدريب

```python
import keras
from keras import ops

@keras.saving.register_keras_serializable(package="Book")
class ScaledResidual(keras.layers.Layer):
    def __init__(self, initial_scale=0.1, **kwargs):
        super().__init__(**kwargs)
        self.initial_scale = initial_scale

    def build(self, input_shape):
        self.scale = self.add_weight(name="scale", shape=(), initializer="ones")
        super().build(input_shape)

    def call(self, inputs):
        return inputs + self.initial_scale * self.scale * inputs

    def get_config(self):
        return {**super().get_config(), "initial_scale": self.initial_scale}
```

هذا المثال يوضح دورة الطبقة، لا يضيف قدرة جديدة إلى تحويل الهوية. عند تحميل نموذج من ملف غير موثوق، لا تسمح باستيراد شيفرة مخصصة عشوائيًا؛ سجل الفئات وراجع مصدرها.

17.3 خسائر ومقاييس وسلوك تدريب خاص

تجمع الخسارة الكلية أهداف المخرجات بوزن معلوم:

$$L_{total} = \sum_{k=1}^{K} \lambda_k L_k$$

`L_k` خسارة المخرج `k` و `λ_k` أهميته النسبية. اختلاف مقياس الخسائر قد يجعل الوزن العددي غير ممثل للأهمية دون تطبيع.

اختر خسارة مخصصة حين لا تمثل الخسارة القياسية الهدف بدقة، واحتفظ بحساباتها قابلة للاختبار على مصفوفات صغيرة. يحتاج `train_step` إلى حساب التوقع والخسارة، واستخراج التدرج وتحديث المحسن، ثم تحديث المقاييس وإرجاع قيمها. عند وجود `sample_weight` أو مخرجات متعددة يجب تمرير الأوزان لكل خسارة ومقياس كما هو متوقع؛ إغفالها يغيّر معنى التدريب بصمت.

الامتداد	الواجهة الأساسية	مسؤولية الكاتب
Layer	`build` ، `call` ، `get_config`	أشكال وأوزان قابلة للحفظ
Model	`call`	تدفق النموذج
Custom train step	`train_step`	الخسارة والتدرج والمقاييس
Custom metric	`update_state` ، `result`	تجميع صحيح عبر الدفعات

17.4 حالة واقعية: خسارة موزونة لعدة مخرجات

يتنبأ نموذج مصنع بنوع العيب وموقعه. مخرج التصنيف يستخدم خسارة فئوية، ومخرج القناع يستخدم Dice أو خسارة بكسلية. يختار الفريق أوزان الخسائر على التحقق، ويتابع كل مقياس على حدة. إذا طغى مخرج على الآخر، يفحص مقاييس الوحدات وتوزيع الأهداف قبل تغيير البنية.

17.5 أمثلة تنفيذية إضافية

طبقة قابلة للتسلسل

```python
@keras.saving.register_keras_serializable(package="Book")
class Scale(keras.layers.Layer):
    def build(self, input_shape):
        self.weight = self.add_weight(shape=(), initializer="ones")
    def call(self, inputs):
        return self.weight * inputs
```

تحميل النموذج

```python
model.save("custom.keras")
restored = keras.models.load_model("custom.keras")
```

17.6 تعديل `train_step` دون كسر Keras

عند تجاوز `train_step`، حافظ على صيغة Keras للبيانات: الزوج `(x,y)` أو الثلاثي `(x,y,sample_weight)`. استخدم `keras.utils.unpack_x_y_sample_weight`، وحسب إصدار Keras الحالي استخدم `compute_loss` لمعالجة خسائر compile وregularizationg. حدّث كل مقياس على بيانات الدفعة وأعد قاموسًا بأسماء ثابتة لتظهر القيم في السجل.

```python
class Trainer(keras.Model):
    def train_step(self, data):
        x, y, sample_weight = keras.utils.unpack_x_y_sample_weight(data)
        with tf.GradientTape() as tape:
            y_pred = self(x, training=True)
            loss = self.compute_loss(x=x, y=y, y_pred=y_pred,
                                     sample_weight=sample_weight)
        gradients = tape.gradient(loss, self.trainable_variables)
        self.optimizer.apply_gradients(zip(gradients, self.trainable_variables))
        for metric in self.metrics:
            if metric.name == "loss":
                metric.update_state(loss)
            else:
                metric.update_state(y, y_pred, sample_weight=sample_weight)
        return {metric.name: metric.result() for metric in self.metrics}
```

اعتبر المثال قالبًا يراجع وفق إصدار Keras، وعرّف `test_step` إن كان evaluate يجب أن يعكس السلوك نفسه. اختبر sample weights يدويًا على أمثلة قليلة، لأن المتوسط الموزون يختلف عن متوسط المتوسطات عند تفاوت حجم الدفعات. في المخرجات المتعددة افصل الخسائر والمقاييس بأسماء واضحة.

17.7 الأقنعة والحالة في call

قد يستقبل `call(inputs, training=None, mask=None)` وضع التدريب والقناع. مرر `training` إلى Dropout وBatchNorm عند بناء طبقة ملتفة، ومرر mask إلى طبقة الانتباه. تجاهل القناع لا يسبب دائمًا استثناءً؛ قد يكون الخلل صامتًا ويجعل النموذج يتعلم من padding. اختبر مدخلًا بطول قصير مع حشو إضافي وتأكد من ثبات النتيجة.

17.8 التهيئة والتسلسل والتحميل

أنشئ الأوزان مرة واحدة في `build` عبر `add_weight`، ثم نفذ `super().build(input_shape)`. أعد كل إعدادات الطبقة من `get_config`، وسجل الكلاس باستخدام `register_keras_serializable`. صيغة `.keras` تحفظ التهيئة والأوزان لكنها لا تضمن وجود تعريف Python المخصص عند التحميل. افتح الملف في عملية جديدة بعد استيراد الحزمة المسجلة وقارن التنبؤات؛ هذا يختبر العقد الحقيقي للنشر.

اترك `safe_mode=True` عند التحميل الطبيعي. لا تحمل ملفات نموذج غير موثوقة تعطّل فحوص التسلسل؛ استخدام كائنات مخصصة يعني أن الشيفرة المرتبطة بها جزء من سطح الثقة. دوّن مصدر الملف وبصمته واعتمادات التشغيل.

17.9 عزل أخطاء التدرج

إذا لم تتغير الأوزان، تحقق أن `trainable_variables` غير فارغة وأن التدرجات ليست `None`. افحص هل عملية الحساب داخل `GradientTape` ، وهل تحولت Tensor إلى NumPy، وهل الطبقة أنشأت وزنًا جديدًا عند كل استدعاء. جرّب أن تفرط على دفعة واحدة أولًا؛ نجاح هذا الاختبار يثبت أن مسار النموذج والخسارة يمكنهما حفظ مثال صغير.

17.10 مقاييس مخصصة ذات حالة

المقياس الذي يحسب المتوسط يجب أن يحتفظ بمجموع القيم وعددها بدل متوسط كل batch ثم متوسط الدفعات. عند وجود sample weights، عرّف هل المقام مجموع الأوزان أم عدد الأمثلة. Keras Metric يتيح `result` , `update_state` , و `reset_state` ; اسم المقياس يصبح مفتاحًا في logs. اختبر حالتين دفعة غير متساويتين يدويًا، ثم تأكد من reset بين التدريب والتقييم.

17.11 `add_loss` وقيود الطبقة

يمكن لطبقة مخصصة إضافة عقوبة تعتمد تفعيلاتها باستخدام `self.add_loss` , فتدخل في خسارة النموذج. هذا مناسب لنشاط latent أو قيود تمثيل، لكن يجب أن تعرف هل تتجمع مرة لكل استدعاء أو لكل batch. سجل قيمة العقوبة منفصلة أثناء التطوير. في Keras الحديثة، لا تعتمد على واجهة `add_metric` القديمة في call بلا مراجعة الوثائق؛ مقاييس Model Metricg أكثر صراحة.

17.12 دعم أكثر من مخرج sample weightsg

نموذج multi-output قد يرجع قاموسًا للمخرجات. اجعل أسماء outputs ثابتة، واربط metricsg losses بالأسماء بدل ترتيب قائمة يصعب تذكره. `sample_weight` قد يكون لكل مثال أو لكل خطوة زمنية أو لكل بكسل؛ شكل الوزن يجب أن يبث فقط على المحاور المقصودة. أضف اختبارًا يغير وزن مثال واحد ويتأكد من أثره في الخسارة المتوقعة.

17.13 `tf.function` والتوافق مع التصدير

الطبقة التي تستخدم Python side effects أو شكلًا يتغير اعتمادًا على قيمة Tensor قد لا تتصرف كما تتوقع عند التتبع. اختبر graphg eager. العمليات المخصصة باستخدام `keras.ops` قد تكون أكثر قابلية للعمل عبر backends من استخدام TensorFlow مباشرة، لكن إذا اخترت TensorFlow-specific API فاكتب ذلك في عقد الطبقة. ليعمل export، يجب أن تكون المدخلات والموارد والتوقيع قابلة للتتبع ضمن runtime المستهدف.

17.14 خطة اختبار طبقة احترافية

- تحقق من `compute_output_shape` أو مواصفات الخرج تلقائيًا.
- اختبر mixed precisiong dtype float32 إن كانت مدعومة.
- افحص gradient على كل وزن قابل للتعلم.
- احفظ النموذج ثم أعد تحميله وقارن المخرجات.
- جرب batch size متغيرًا ومدخلًا مع mask.
- شغل `train_step` على batch واحدة ثم قارن loss مع الحساب اليدوي.

اختبارات الطبقة لا تضمن صحة علمية للهدف، لكنها تمنع أن يصبح خطأ plumbing جزءًا من ورقة أو خدمة.

17.15 طبقة Attention مصغرة

طبقة الانتباه المخصصة تحتاج Q/K/V projections وقناعًا وسلوك dropout اختياريًا. قبل إعادة تنفيذها يدويًا، جرّب `keras.layers.MultiHeadAttention`؛ إذا كان الهدف تعليميًا أو متطلبًا بحثيًا خاصًا، قارن مخرجات الطبقة المخصصة بالمرجع على أبعاد صغيرة، ثم قارن gradients. احتفظ بمعادلة لكل محور وحجم Tensor لتجنب تبديل sequence headg بالخطأ.

17.16 حساب الخسارة لعدة أهداف

عند multi-output، قيمة `self.compute_loss` تشمل خسائر compile وإضافات add_loss. لا تضف هذه الخسارة مرتين في loop. إذا استعملت `sample_weight` لكل output، مرر dict مطابقًا لمفاتيح المخرجات. اختبر الحالات: بدون أوزان، وزن موحد يعادل baseline، ووزن يضاعف مساهمة مثال واحد. افحص أن custom metric يأخذ الحقيقة والتوقع من المخرج الصحيح.

17.17 توزيع الطبقة المخصصة

تأكد أن `get_config` يعيد إعدادات primitive serializable، وأن `from_config` لا يحتاج معلومات خارجية مؤقتة. إذا كانت الطبقة تعتمد lookup table أو asset ملف، سجله بآلية assets. استخدم symbolic shape بدل قراءة قيمة Tensor في Python وقت بناء الرسم. جرّب save/load ثم `model.export`؛ بعض الوظائف تدعم `.keras` لكنها لا تحول تلقائيًا إلى LiteRT.

17.18 التوافق بين backends

Keras 3 متعدد backend. طبقة مكتوبة حصريًا بـ `tf.*` قد تعمل على TensorFlow فقط. إذا أردت دعم JAX/Torch، استخدم `keras.ops` وواجهات Keras abstract. وثق backend المطلوب واكتب CI لكل backend تدعمه. لا تدع ادعاء «Keras layer» يعني تلقائيًا توافقًا عامًا.

17.19 فشل مختبر التدريب المخصص

اختبر أولًا layer forward، ثم gradient، ثم optimizer update، ثم metrics، ثم checkpoint. إذا اختلفت loss عن `fit` القياسي، افحص distributed reductiong sample weightg regularization. في تعدد الأجهزة، يجب أن تقلل losses بالمقياس العالمي المقصود. هذا التدرج يعزل العطل بدل تصحيح train_step كامل دفعة واحدة.

17.20 دورة تدريب مخصصة عند الضرورة

استخدم `train_step` إذا احتجت حسابات لا يعبر عنها compile، مثل تحديثات متعددة أو adversarial objective. custom loop الكامل يجعلك مسؤولًا عن callbacks والتوزيع checkpointingg resumeg metricsg. ابدأ بمقارنة نتيجته مع `fit` القياسي على نفس batch، ثم أضف سلوكًا واحدًا. فشل loop لا يعني أن Keras `fit` محدود بالضرورة.

17.21 اختبار الطبقة عبر الأشكال والأنواع

اختبر batch dimension متغيرًا dtypeg معلومًا، gradientg لكل وزن، maskg على تسلسل يحوي padding. احفظ نموذجًا يحوي الطبقة وأعد تحميله في process نظيفة بعد استيراد package. اختبر `.export` إذا كان الاستدلال يحتاج LiteRT أو SavedModel؛ نجاح `.keras` لا يضمن أن العملية قابلة للتحويل.

17.22 دعم backend ومراجعة الشفرة

استخدام `keras.ops` يحافظ على توافق أوسع في Keras متعددة backend، أما `tf.*` فيربط الطبقة بـ TensorFlow. صرح بذلك في التوثيق والـ dependency. راجع API الحالية بدل كود قديم يستخدم loss/metric methods تمت إعادة تشكيلها. ثبّت إصدار Keras في CI وشغل tests صغيرة على save/load وgradient.

17.23 واجهة عامة وصيانة طويلة الأجل

استخدم أسماء متغيرات configg مستقرة، واكتب migration إذا تغير شكل الوزن. الطبقة المنشورة تدخل في checkpoints ومكتبات أخرى؛ لا تغير ترتيب القنوات أو serialization بصمت. سجل نسخة الـ package التي تعرّف الكائن وقت الحفظ والتحميل.

17.24 قراءة سجل التدرجات

أثناء التطوير، سجل norm للتدرجات وعدد المتغيرات التي وصلها gradient. إذا كانت `None`، افحص tape وقطع graph أو متغيرًا غير trainable. أزل التسجيل المكلف قبل benchmark النهائي.

17.25 دورة مراجعة على دفعة واحدة

اختبر المدخل والخرج والخسارة والتدرج ووزن العينة على batch صغيرة ذات أرقام يمكن حسابها. لا تبدأ بتشغيل مجموعة كاملة حين تكون الطبقة جديدة؛ فالفحص العددي يكشف أخطاء reduction والمحاور قبل أن تصبح مكلفة.

17.26 أخطاء شائعة

> ⚠ **تحذير**
>
> إنشاء وزن داخل `call` يعيد إنشاء الحالة أو يفشل تتبعها؛ أنشئه في `build` عبر `add_weight`.
>
> [!DANGER] كائن مخصص بلا `get_config` أو تسجيل قد يمنع التحميل في بيئة جديدة.

17.27 تمارين

1. اكتب طبقة تضيف انحيازًا قابلًا للتعلم. (المتوقع: وزن في `build`.)

2. أعد بناء طبقتك من config. (المتوقع: إعدادات متطابقة.)
3. أضف metric لمتوسط خطأ مطلق. (المتوقع: تجميع عبر الدفعات لا متوسط المتوسطات.)
4. اشرح موضع تطبيق `sample_weight`. (المتوقع: قبل تجميع الخسارة.)
5. صمم `train_step` لمخرجين. (المتوقع: خسارتان وتدرجات ومقاييس.)
6. أعد تحميل `.keras` في عملية جديدة. (المتوقع: تسجيل أو توفير الطبقة.)

17.28 ملخص

- افصل تهيئة الأوزان عن الحساب الأمامي.
- قابلية التسلسل جزء من تصميم الطبقة، لا خطوة نشر لاحقة.
- التدريب المخصص يجب أن يحافظ على الأوزان والمقاييس وعقد البيانات.

الجزء الرابع · الاحتراف والإنتاج

الفصل الثامن عشر

18

الأداء والتدريب الموزّع

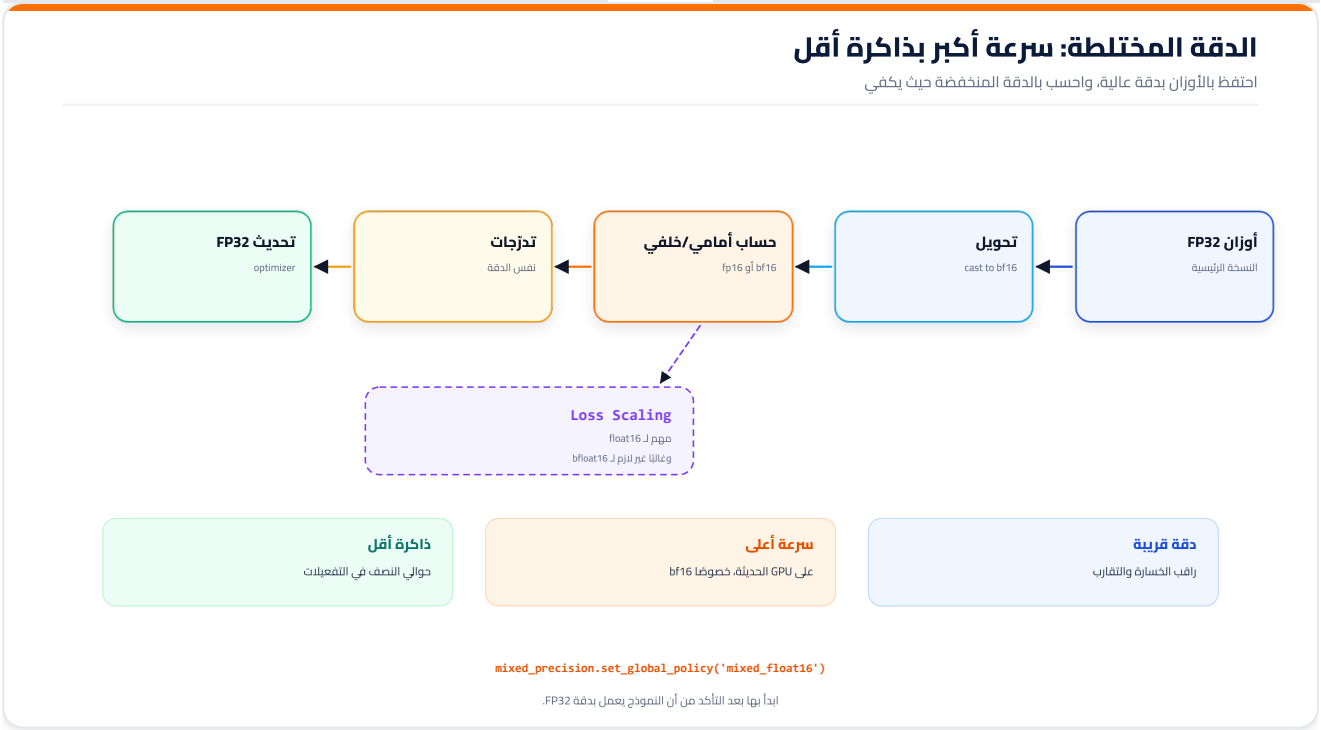
> **حكاية الفصل**
>
> نقل فريق نموذجه إلى GPU أغلى، لكن زمن الحقبة لم يتحسن. كشف القياس أن المعالجة المسبقة على CPU هي الاختناق، وأن الدفعة صغيرة فلا تستفيد البطاقة من التوازي. كيف نقيس قبل توسيع العتاد؟

18.1 أهداف الفصل

- [ ] تحديد الاختناق عبر القياس لا التخمين.
- [ ] استخدام graph وXLA والدقة المختلطة عند ملاءمتها.
- [ ] تقدير أثر تراكم التدرجات.
- [ ] فهم استراتيجيات التوزيع ومقياس الإنتاجية.

18.2 قِس خط أساس

سجل زمن الخطوة والحقبة، وعدد الأمثلة في الثانية، واستخدام المسرّع والذاكرة. تجاهل خطوات الإحماء الأولى عند التوقيت لأن التتبع وتهيئة المكتبات يضيفان كلفة. استخدم TensorBoard Profiler لتحديد فجوات إدخال البيانات أو عمليات CPU أو نقل متكرر بين الجهازين. حسّن أبطأ جزء ثم أعد القياس؛ لا تجمع تغييرات كثيرة في تجربة واحدة.



شكل 18-1 نقل من الدقة الكاملة إلى المختلطة



شكل 18-2 نسخ النموذج وتوزيع الدفعة على الأجهزة

```python
import time
import tensorflow as tf

@tf.function(jit_compile=False)
def step(x):
    return tf.linalg.matmul(x, x, transpose_b=True)

x = tf.random.normal((512, 128))
for _ in range(5):
    step(x)  # إحماء وتتبع
start = time.perf_counter()
for _ in range(50):
    step(x)
tf.experimental.async_wait()
elapsed = time.perf_counter() - start
print(f"{50 / elapsed:.1f} خطوة/ثانية")
```

قياس GPU غير المتزامن يحتاج مزامنة حتى لا نقيس إرسال العمل فقط. المثال مصغر لشرح المنهج؛ benchmark حقيقي يضم تحميل البيانات والنموذج والتحديث، ويكرر التجربة على الجهاز نفسه.

18.3 XLA والدقة المختلطة

$$Throughput = \frac{N_{examples}}{T_{elapsed}}$$

يحسب المعدل عدد الأمثلة المكتملة على الزمن المقاس بعد الإحماء. قارنه مع جودة النموذج والذاكرة؛ لا يكفي throughput منفردًا.

`tf.function` يقلل كلفة التنفيذ التفسيري عبر الرسم البياني، وXLA قد يدمج العمليات لكنه قد يضيف زمن ترجمة أو لا يدعم كل عملية. الدقة المختلطة تستخدم تمثيلات أقل بتًّا لتوفير الذاكرة وتسريع الحساب على عتاد مناسب؛ احتفظ بعمليات حساسة عدديًا بدقة أعلى، واستخدم loss scaling عند الحاجة. تحقق من التقارب والذاكرة لا من الزمن فقط.

18.4 التوزيع وتراكم التدرجات

MirroredStrategy تزامن نسخًا على عدة مسرّعات في جهاز واحد، وMultiWorkerMirroredStrategy توزعها بين عمال، وTPUStrategy تستهدف TPU. يجب ضبط حجم الدفعة العالمي، وتهيئة البيانات لكل عامل، ومزامنة المقاييس. تراكم التدرجات يجمع تحديثات عدة دفعات صغيرة قبل خطوة واحدة لتقريب دفعة أكبر، لكنه لا يخفض ذاكرة التفعيلات المطلوبة لكل خطوة بنفس الطريقة التي يفعلها تقليل النموذج.

طريقة التوسيع	فائدة	كلفة تشغيلية
جهاز واحد	خط أساس أبسط	سقف ذاكرة وحساب الجهاز
عدة GPU في جهاز	توازي البيانات	تزامن ودفعة أكبر
عدة عمال	قدرة أوسع	شبكة وفشل جزئي وإدارة
TPU	مصفوفات ضخمة مدعومة	تهيئة ونقل بيانات ملائم

18.5 حالة واقعية: قياس خدمة تدريب

يتتبع الفريق صورًا في الثانية وزمن الخطوة واستخدام GPU ومعدل إدخال البيانات. بعد `prefetch` متوازٍ أزيلت فجوات الانتظار، ثم رفعوا الدفعة حتى حد الذاكرة. قارنوا الدقة النهائية والجودة والزمن الكلي وكلفة الساعة؛ تحسن throughput بلا تحسن زمن الوصول إلى جودة الهدف ليس مكسبًا كافيًا.

18.6 أمثلة تنفيذية إضافية

استراتيجية التوزيع

```python
strategy = tf.distribute.MirroredStrategy()
with strategy.scope():
    model = make_model()
    model.compile(optimizer="adam", loss="mse")
```

قياس متزامن

```python
start = time.perf_counter()
for _ in range(20):
    step(batch)
tf.experimental.async_wait()
print(time.perf_counter() - start)
```

18.7 استراتيجيات التوزيع والتكلفة

في data parallelism تحمل كل replica نسخة من النموذج وتعمل على جزء من الدفعة، ثم تزامن التدرجات. أنشئ النموذج والمحسن داخل `strategy.scope()` حتى توزع المتغيرات. إذا كان لديك أربع وحدات globalg batch قدره 256، تعالج كل وحدة 64 مثالًا عند التقسيم المتساوي. قياس throughput على دفعة أكبر قد يتحسن لكنه قد يغير ديناميكا التعلم، لذا افصل تجربة حجم الدفعة عن تجربة عدد الأجهزة.

في MultiWorker يجب أن يرى كل عامل shard مختلفًا، وأن يكون سلوك الخطأ عند توقف عامل معلومًا. شبكة الاتصال قد تصبح أغلى من حساب النموذج. TPU يتطلب مسار تجهيز بيانات وأشكالًا ملائمة، وتدريبه لا يعني أن inference سينتقل إلى هاتف تلقائيًا. ابدأ بجهاز واحد كمرجع، ثم وسّع مع قياس scaling efficiency.

18.8 تراكم التدرج

إذا كانت الذاكرة لا تحتمل دفعة كبيرة، احسب التدرج على microbatches واجمعه ثم طبق تحديثًا واحدًا كل `K` أجزاء. عند خسارة متوسطية اقسم المجموع على `K`، وإلا تكبر خطوة المحسن. هذا يقرب أثر الدفعة الأكبر فقط عندما لا تعتمد الطبقات على إحصاءات العينة داخل الدفعة؛ BatchNorm مثال يختلف سلوكه لأن إحصاءاته ما زالت محلية لكل microbatch. كذلك يمكن أن يتغير أثر dropout والعشوائية.

18.9 اقرأ trace لا رقم واحد

ابدأ TensorBoard Profiler لخطوات مستقرة قليلة. هل GPU تنتظر `IteratorGetNext`؟ هل تتكرر عمليات نسخ host-to-device؟ هل kernel واحد بطيء؟ قارن تشغيلًا بمدخلات من الذاكرة مع تشغيل خط البيانات الكامل. اختبر prefetchg cache والمعالجة المتوازية كلًا على حدة، واحتفظ trace قبل وبعد كدليل.

18.10 الدقة المختلطة وحدود الأعداد

float16 يخفض التخزين ويستفيد من عتاد متخصص، لكن مجاله العددي أضيق؛ bfloat16 يوسع المجال مع دقة mantissa أقل. loss scaling يضخم الخسارة قبل الاشتقاق كي لا تضيع التدرجات الصغيرة ثم يعيد المقياس. اختبر NaN وInf ودقة النموذج على جهاز الهدف. أبقِ التجميعات أو المخرج الحساس بدقة أعلى إذا أظهر القياس حاجة إلى ذلك.

18.11 تقرير أداء قابل للتكرار

حدد الجهاز والتعريف والحزم، حجم الدفعة، شكل الإدخال، عدد warm-up steps، عدد الخطوات الموقوتة، وطريقة مزامنة الحساب غير المتزامن. أبلغ median وp95 أو نطاق التشتت، لا أسرع قراءة منفردة. سجل GPU utilization والذاكرة والصور في الثانية ومقياس الجودة. التغيير الصحيح هو الذي يصل إلى جودة الهدف بأقل زمن أو كلفة، لا الذي يرفع رقمًا واحدًا.

18.12 مختبر profiler خطوة بخطوة

ابدأ بخمس خطوات warm-up كي لا تخلط tracing أو XLA compilation بالقياس. سجل profiler في نطاق قصير من التدريب ثم افتح trace، وتحقق من فجوات GPU idle، زمن input pipeline، وعمليات H2D/D2H. إذا ظهر `IteratorGetNext` متكررًا، اختبر dataset اصطناعيًا في الذاكرة. إذا ظل GPU مشغولًا مع utilization مرتفع، اختبر حجم النموذج والدقة المختلطة أو XLA. احفظ trace قبل التغيير وبعده.

18.13 تقدير الذاكرة

وزن `float32` يستهلك أربعة بايتات، لكن التدريب يحتفظ أيضًا بأوزان المحسن وتفعيلات الطبقات والتدرجات والـ workspace المؤقت. زيادة batch تضرب ذاكرة التفعيلات تقريبًا خطيًا في حالات كثيرة، بينما attention قد يضيف مصفوفة مربعة في طول التسلسل. استخدم profiler لذروة الذاكرة، لا عدد أوزان النموذج وحده. checkpointing يبادل وقت إعادة الحساب بذاكرة أقل.

18.14 استراتيجية اختيار الدقة

جرب float32 baseline أولًا حتى تعرف صحة النموذج، ثم mixed precision. على GPU Tensor Core قد يكون float16 سريعًا، وعلى TPU قد يكون bfloat16 مناسبًا؛ العتاد يحدد الفائدة. راقب output في آخر طبقات، وتجنب أن تتحول قيم صغيرة إلى صفر. إذا اختلفت الجودة، افحص scaling والطبقات الحساسة قبل الرجوع لكل

النموذج إلى float32.

18.15 الاتصالات في التدريب الموزع

تبادل gradient قد يهيمن على العمل عندما يكون النموذج كبيرًا والدفعة صغيرة، أو الشبكة بطيئة. قياس scaling efficiency يقارن الزيادة الفعلية في throughput بالزيادة النظرية لعدد الأجهزة. لا ترفع عدد العمال لمجرد توفرها؛ قس مقدار الحساب لكل byte من الاتصال. Gradient compression أو communication overlap تقنيات متقدمة، لكنها تضيف تعقيدًا ويجب أن تثبت فائدتها تجريبيًا.

18.16 حوكمة القياس

وثق من أجرى benchmark، ومكان الجهاز، وإصدار التعريف، ودرجات حرارة GPU عند التدريب الطويل إن كانت تؤثر. أبلغ median وpercentiles من عدة runs. لا تقارن نتائج cloud مستأجرة على وحدات مختلفة تحت اسم GPU نفسه. إذا كان المقياس لصالح قرار شراء، احسب تكلفة الطاقة والزمن والحوسبة المتاحة وسرعة الوصول إلى الهدف.

18.17 Mixed precision حسب backend

للتفاصيل التشغيلية راجع مرجع Keras mixed precision ودليل التوزيع متعدد GPU على TensorFlow. اختبر إصدار backend والعتاد لديك؛ التوصية لا تعتمد على اسم dtype وحده.

اضبط `keras.mixed_precision.set_global_policy` قبل بناء النموذج. اقرأ `compute_dtype` و`variable_dtype`: غالبًا تحسب الطبقات بتنسيق منخفض وتخزن متغيراتها بدقة أعلى. آخر layer قد تحتاج خرج float32 لحساب probability أو loss بحساسية. هذا ليس قاعدة عمياء؛ افحص نوع الخرج وعدد القيم المنتهية وvalidation scores.

18.18 توازي البيانات والنموذج

data parallelism يكرر النموذج ويقسم batch. model parallelism يقسم الطبقات أو tensor عبر عدة أجهزة، ويناسب نموذجًا لا يتسع على جهاز واحد لكنه يحتاج اتصالًا بين الأجزاء. pipeline parallelism يمرر microbatches عبر المراحل لتقليل خمول الأجهزة. اختر وفق حجم النموذج وبنية الاتصالات. استخدام strategy جاهز قد يكفي حتى عدة GPUs؛ لا تبنِ نظام توزيع يدويًا قبل قياس حدود الجاهز.

18.19 مراجعة الأشكال الديناميكية

أشكال `None` قد تسبب retracing في `tf.function` عند تغير الأبعاد. ثبت batch/time shapes متى كان ذلك مناسبًا، أو استخدم `input_signature` متعدد الأشكال مدعومًا. لا تقفل طول تسلسل لمجرد الأداء إذا كانت الخدمة تقبل أطوالًا مختلفة؛ pad/truncate وفق عقد مدروس.

18.20 جمع الدرجات والتحقق من scale

في gradient accumulation، إذا كان آخر microbatch أصغر، المتوسط يجب أن يوزن بعدد أمثلته لا متوسط متوسطات الدفعات. في multi-replica، انتبه إلى reduction العالمي الذي تستخدمه Keras strategy. اكتب مقارنة على batch اصطناعية: خطوة واحدة بحجم 8 مقابل 4 microbatches بحجم 2، وقارن gradients قبل optimizer، مع تعطيل مصادر العشوائية والطبقات المعتمدة على batch.

18.21 لوحة أداء للفريق

احتفظ بتقرير يحتوي throughput، زمن epoch، GPU memory، استخدام CPU، وقت البيانات، checkpoint throughput، وأثر الجودة. قس inference latency منفصلًا من batch size 1 حتى throughput batch. عند رفع نموذج، أبلغ steady state، warm start، لأن cold start قد يهم المستخدم النهائي.

18.22 XLA وكلفة التجميع

`jit_compile=True` قد يدمج kernels، لكنه يضيف compile time وقد يفشل مع عمليات أو أشكال غير مدعومة. قس زمن أول invocation منفصلًا عن steady state. في خدمة قصيرة العمر قد يمحو startup cost المكسب؛ في تدريب طويل قد يتوزع على خطوات كثيرة. قارن المخرجات ضمن tolerance.

18.23 الذاكرة وحجم الدفعة

الأوزان ليست كل الذاكرة؛ التدريب يحتفظ بالتفعيلات والتدرجات وحالة المحسن وworkspace. تكبير batch يرفع التفعيلات غالبًا خطيًا، والانتباه قد يضيف مصفوفة تربيعية بطول التسلسل. قس ذروة الذاكرة عبر profiler. gradient checkpointing يبادل إعادة الحساب بذاكرة أقل.

18.24 اختبار تراكم التدرج

في microbatch accumulation، آخر جزء قد يكون أصغر؛ اجمع gradients موزونة بعدد أمثلته لتكافئ المتوسط العام. قارن مع دفعة كاملة على بيانات صغيرة، مع تعطيل dropout وتثبيت batch-dependent layers قدر الإمكان. اختلاف BatchNorm المتوقع يجب أن يذكر، لا أن يخفى.

18.25 استراتيجية نمو

data parallelism يكرر النموذج ويقسم الدفعة؛ model parallelism يقسم أجزاء النموذج؛ pipeline parallelism يمرر microbatches على مراحل. اختر بالنظر إلى حجم النموذج والاتصالات. أبلغ scaling efficiency مقابل عدد الأجهزة، وزمن reaching target quality، وكلفة الساعة؛ زيادة throughput دون تحسن الزمن الكلي ليست كافية.

18.26 أخطاء شائعة

> ⚠ **تحذير**
>
> لا تقارن تنفيذًا eager أول مرة بتنفيذ graph بعد الإحماء؛ زمن التتبع ليس زمن الخطوات المستقرة. [DANGER!] التوزيع قد يضاعف حجم الدفعة ويغير ديناميكا التدريب. اضبط معدل التعلم وقارن جودة الهدف.

18.27 تمارين

1. صمم جدول قياس لزمن التدريب. (المتوقع: إحماء، throughput، ذاكرة، مقياس جودة.)
2. فعّل `tf.function` على خطوة حسابية. (المتوقع: إعادة استخدام الرسم للشكل نفسه.)
3. فسّر متى لا يفيد XLA. (المتوقع: كلفة ترجمة أو عمليات غير ملائمة.)
4. احسب دفعة عالمية لأربع بطاقات. (المتوقع: دفعة لكل بطاقة × 4.)
5. اشرح تراكم التدرجات. (المتوقع: عدة microbatches قبل التحديث.)
6. شخّص GPU منخفضة الاستخدام. (المتوقع: فحص الإدخال ونقل البيانات وحجم الدفعة.)

18.28 ملخص

- الملف الشخصي يحدد الاختناق الحقيقي قبل الإنفاق على العتاد.
- graph وXLA والدقة المختلطة فرص قياس، وليست تسريعًا مضمونًا.
- التوزيع يغير الدفعة والتزامن والإدارة، فقارن الجودة والإنتاجية معًا.

الجزء الرابع · الاحتراف والإنتاج

الفصل التاسع عشر

19

النشر: TF Servingg TF.jsg LiteRT

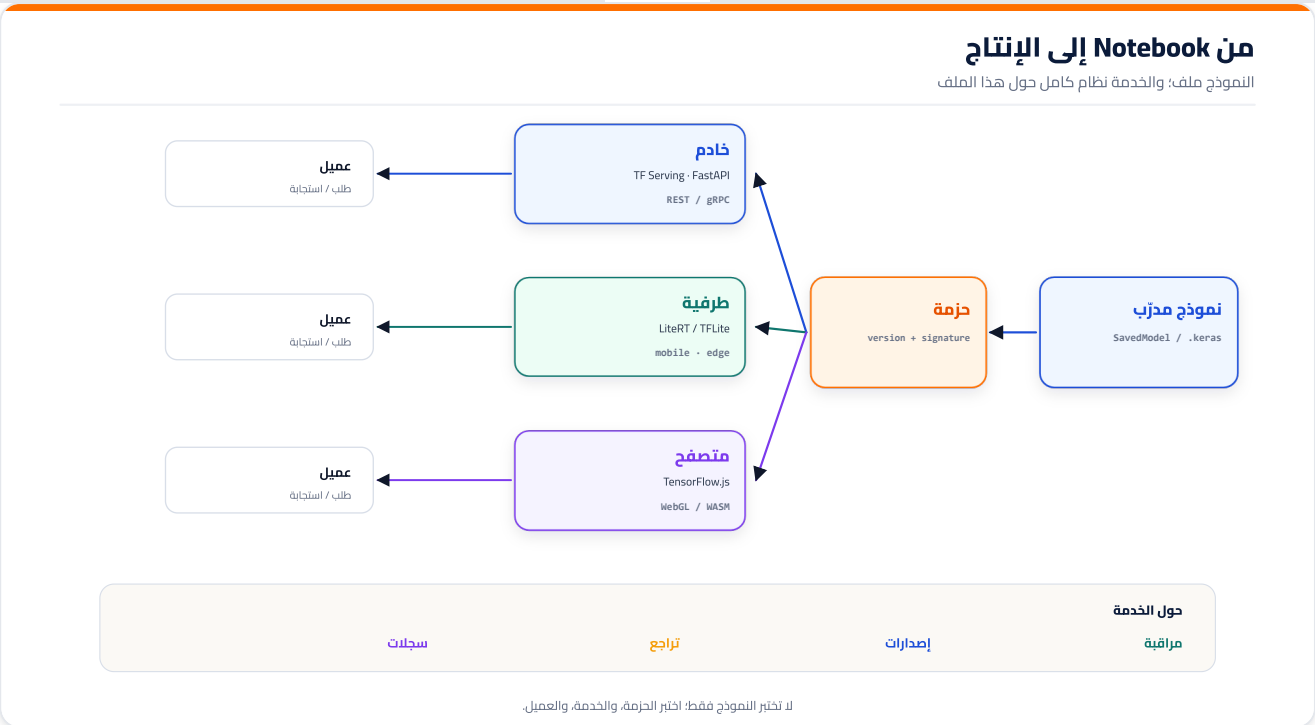
> **حكاية الفصل**
>
> نجح نموذج الجودة على محطة التدريب، لكن مصنعًا يريد تشغيله قرب الكاميرا دون اتصال دائم. لم يعد السؤال عن الدقة وحدها؛ صار عن حجم الملف والكمون والذاكرة وتوافق الإدخال. كيف نعبر من نموذج بحث إلى أثر قابل للتشغيل؟

19.1 أهداف الفصل

- ☐ التمييز بين `.keras` وSavedModelg.
- ☐ تحويل النموذج إلى LiteRT وشرح التكميم.
- ☐ مقارنة النشر الطرفي والمتصفح والخدمة.
- ☐ وضع فحوصات قبل الإصدار.

19.2 صيغة الاستدلال وعقده

في `model.save("model.keras")`، Keras 3 يحفظ النموذج لاستعادته، و`model.export("saved_model")` يصدر مسار الاستدلال بصيغة SavedModel. يحدد العقد شكل المدخل ونوعه ومداه ومعالجته، وشكل المخرج ومعناه. افصل المعالجة المسبقة وعتبة القرار عن افتراضات غير موثقة، أو ضمّنها في رسم يمكن تصديره.



شكل 19-1 مسار نشر نموذج التعلم الآلي

19.3 LiteRT على الأجهزة الطرفية

$$Model_quality_after_conversion \geq Quality_threshold$$

يمثل `Quality_threshold` الحد الأدنى المتفق عليه للمقياس الحرج على بيانات التحقق الواقعية. حجم الملف الأصغر لا يبرر خرق هذا الشرط.

تحوّل TFLite Converter نموذجًا إلى LiteRT (TFLite سابقًا). التكميم الديناميكي أو float16 قد يقلل الحجم؛ التكميم الصحيح الكامل يتطلب مجموعة تمثيلية لمعايرة النطاقات. اختبر مجموعة قيم حقيقية بعد التحويل، وقارن التنبؤات والكمون والذاكرة. Pruning يقلل أوزانًا صغيرة لكنه يحتاج بنية مدعومة أو تحويلًا يستفيد من التخلخل.

```python
import tensorflow as tf

model = tf.keras.models.load_model("model.keras")
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
lite_model = converter.convert()
with open("model.tflite", "wb") as f:
    f.write(lite_model)
print(len(lite_model))
# المتوقع: حجم ملف بالبايت؛ افحص الدقة على مجموعة تحقق بعد التحويل.
```

19.4 المتصفح والخدمات

TF.js يشغل نماذج متوافقة في المتصفح أو Node.js بعد التحويل، مع اختلاف دعم العمليات. TF Serving يقدم نماذج ذات إصدارات عبر REST أو FastAPI. gRPC يمنح تحكمًا في التحقق من المدخلات، والـ batching، والمصادقة، والسجلات، لكنه لا يعوض إدارة الموارد أو الحماية. ONNX خيار تبادل يحتاج إلى اختبار توافق العمليات والمخرجات.

الهدف	مسار محتمل	قيد رئيسي
هاتف/متحكم طرفي	LiteRT	عمليات مدعومة وحجم وذاكرة
متصفح	TF.js	دعم التحويل وموارد الجهاز
خادم عالي الحمل	TF Serving	تشغيل وإصدارات ومراقبة
واجهة أعمال مخصصة	FastAPI	تحقق وأمن وتوسع تشغيلي

19.5 حالة واقعية: فحص عيب دون إنترنت

يصدر الفريق نسخة LiteRT كمية إلى كاميرا المصنع. يختبر عينات من كل فئة وإضاءة، ويقيس زمن p95 لا المتوسط وحده. يحتفظ بإصدار رجوع، ويطابق تطبيع الألوان في الجهاز مع التدريب. إذا هبط استدعاء الفئة الحرجة بعد التكميم، تُرفض النسخة حتى لو صغر الملف.

19.6 أمثلة تنفيذية إضافية

فحص نموذج LiteRT

```python
interpreter = tf.lite.Interpreter(model_path="model.tflite")
interpreter.allocate_tensors()
print(interpreter.get_input_details())
```

مقارنة المخرجات

```python
np.testing.assert_allclose(reference, converted, rtol=0.05, atol=0.05)
```

19.7 عقد الإدخال وإصدارات النموذج

توضح واجهة التصدير في Keras الصيغ المدعومة وتوقيعات الاستدلال، ويوضح دليل LiteRT الحديث مسارات runtime الجديدة. واجهة `tf.lite.Interpreter` تظهر كـ legacy/deprecated في مسار LiteRT الموثق حديثًا؛ اختر runtime وفق المنصة والنسخة المستهدفة، ولا تنسخ مثالًا قديمًا دون اختبار.

اكتب لكل endpoint أسماء المدخلات وأشكالها وأنواعها ونطاق القيم ووحداتها ومعالجة الغياب، ثم وثق إخراج النموذج وما إذا كان logits أو احتمالات أو قيمة بوحدة أعمال. افحص مدخل الخدمة وارفض NaN والأحجام المفرطة والحقول غير المعروفة. احفظ preprocessing أو كوده بإصدار النموذج؛ الوزن وحده لا يحدد التنبؤ إذا كانت طريقة التطبيع أو قائمة الفئات اختلفت.

المسارات في Keras 3 واضحة: `model.save("classifier.keras")` لملف Keras كامل قابل لإعادة التحميل، و`model.export("saved_model", format="tf_saved_model")` لتصدير استدلال SavedModel، و`model.export("classifier.tflite", format="litert")` لتصدير LiteRT في الإصدارات التي تدعم هذه الواجهة. توثيق التصدير في Keras يشرح التنسيقات وواجهات signatures. طبّق التوقيع المتوافق مع الإدخال الواقعي، خصوصًا عند أبعاد ديناميكية أو نموذج متعدد المدخلات.

19.8 معايرة التكميم وقرار القبول

في integer quantization، خذ representative dataset من مجموعة تدريب أو معايرة منفصلة تمثل مجالات البيانات، ولا تستخدم مجموعة الاختبار لضبط التحويل. نفذ النموذج الأصلي والمحول على العينات نفسها، وقارن أكبر فروق المخرجات، ثم احسب precision/recall أو IoU وغيرها على مجموعة التقييم المحجوزة. اختيار tolerance يعتمد على حساسية القرار: مقارنة tensors وحدها لا تكفي إذا تغير ترتيب الحالات قرب عتبة القرار.

احفظ نتائج التحويل: الحجم، زمن latency p50/p95، warm-up، الذاكرة، الطاقة عند الإمكان، والدقة لكل شريحة. بعض العمليات قد تحتاج Select TF Ops أو custom operator، وهو ما يزيد حجم التطبيق واعتماداته. لا تدع converter يحول النموذج إلى عملية لا يدعمها runtime على جهاز العميل.

19.9 تشغيل الخدمة وطرح تدريجي

TF Serving مناسب لخدمة SavedModel بإصدارات وواجهات REST أو FastAPI. gRPC يعطيك سيطرة على schema والتوثيق والتكامل لكنه يحتاج إدارة تحميل النموذج والموارد والعمليات العاملة. ضمّن health endpoint وtimeouts وحجم طلب أقصى، وقس زمن queue لا زمن الاستدعاء وحده. راقب أخطاء parsing ومعدل المدخلات خارج المجال.

قبل نقل كل الحركة، شغل نسخة shadow أو canary على نسبة صغيرة. قارن نتائج النسخة القديمة والجديدة دون اتخاذ قرار مزدوج، وارصد الشرائح التي تبدل فيها التصنيف. احتفظ بإصدار سابق معلوم وجرّب rollback قبل الحاجة إليه. استخدم version id في السجلات مع تقليل ما قد يكشف بيانات شخصية.

19.10 أمان سلسلة النشر

تحقق من مصدر artifact وبصمته. لا تقبل مسار نموذج يتحكم به المستخدم، ولا تعطّل safe deserialization لملف مجهول. اعزل تبعيات التحويل عن runtime الإنتاج، وحدثها وفق سياسة مراجعة. في الجهاز الطرفي، احمِ الملفات المحلية إذا كانت الأوزان حساسة، وفكر في أن النموذج المنشور يمكن تحليله حتى مع إخفاء الكود.

19.11 تشغيل TensorFlow Serving

احفظ export في مسار إصدار مثل `models/classifier/17`، وقدم مجلد النموذج إلى TF Serving. استخدم توقيع إدخال ثابتًا باسم معروف، وافحص request/response على نسخة محلية قبل ربطه بالعميل. REST سهل التكامل، وgRPC يقلل كلفة serialization في خدمات عالية المرور؛ اختبر البروتوكول الفعلي وقياسه. إدارة نسخة النموذج تشمل تحميل الجديد والتحقق منه وإزالة القديم وفق سياسة rollback.

19.12 تطبيق طرفي: مسار اختبار كامل

على Android أو كاميرا طرفية، قس حجم الملف على القرص وذاكرة arena ووقت أول استدعاء والكمون المستقر. جرّب representative inputs تحوي أطول وأقصر الحالات وحدود القيم. تأكد من ترتيب الأبعاد وتوقيع input عند تخصيص tensor. إذا غيرت input shape، نفذ resize tensor قبل الاستدلال ضمن API runtime. لا تختبر فقط على جهاز المطور؛ فرق المعالج والحرارة يغيران latency.

LiteRT يواصل تطوير runtime جديد للأجهزة الطرفية، وبعض واجهات `tf.lite.Interpreter` القديمة تظهر بوصفها legacy أو deprecated في توثيق التصدير الحديث. صمم طبقة adapter بين تطبيقك runtimeg كي تبدل implementation دون تغيير عقد النموذج. راجع وثائق LiteRT الخاصة بالمنصة عند بدء منتج، وحدد هل تستخدم API جديدة مثل `ai_edge_litert` أم compatibility path المطلوب للإصدار المختار. دليل Keras للتصدير إلى LiteRT.

19.13 المراقبة بعد النشر

الخدمة لا ترى label دائمًا، لذلك يمكنها تسجيل failures، OOV rate، confidence distribution، input summary، latencyg مع تطبيق إخفاء البيانات. عندما تصل الحقيقة لاحقًا، اربطها بـ model version timestampg. راقب توزيع التوقعات لكن لا تعتبر تغيره وحده دليل تراجع. ضع حدًا يوقف الطلبات غير الصالحة، وآلية تبديل تلقائي أو يدوي إلى نسخة مستقرة.

19.14 خطة rollback جاهزة

احتفظ بمؤشر إصدار نشط يمكن إعادته، واحتفظ بمخرجات المعالجة متوافقة معه. اختبر deploy ثم rollback في staging. إذا كان التكميم قد غير mapping الفئات أو preprocessing، فإن الرجوع للوزن القديم مع معالج جديد قد يبقي الخطأ. نسخة rollback الكاملة تشمل model + tokenizer أو preprocessing + config + threshold + runtime version.

19.15 مثال استدلال يطابق العقد

```python
import numpy as np
from ai_edge_litert.interpreter import Interpreter

interpreter = Interpreter(model_path="classifier.tflite")
interpreter.allocate_tensors()
inp = interpreter.get_input_details()[0]
out = interpreter.get_output_details()[0]
batch = np.zeros((1, 224, 224, 3), dtype=inp["dtype"])
interpreter.set_tensor(inp["index"], batch)
interpreter.invoke()
scores = interpreter.get_tensor(out["index"])
print(scores.shape)
```

هذه الواجهة مرتبطة بحزمة runtime المختارة؛ تحقق من API وتوقيع النموذج في الإصدار المستهدف. إذا كان التطبيق يعتمد compatibility interpreter الأقدم، وثق سبب ذلك وخطة الانتقال. لا تنس أن تطبق preprocessing ذاتها قبل `set_tensor`، وإلا المثال يختبر شكلًا فقط لا دقة المنتج.

19.16 TFLite/LiteRT signatures

عند تصدير نموذج متعدد المدخلات، استخدم signatures وأسماء ذات معنى، واختبر signature runner إن كان runtime يوفره. استخدم `input_details` و `output_details` لفحص shapes/dtypes بدل افتراض index ثابت بعد التحويل. إذا كان batch ديناميكيًا، resize input tensor قبل allocate/invoke وفق قيود runtime. بعض المترجمات تثبت أبعادًا؛ اختبر boundary shapes.

19.17 المتصفح وخصوصية المستخدم

تشغيل النموذج على العميل يقلل إرسال البيانات لكنه يجعل تنزيل الأوزان متاحًا وقد يفرض حملًا على جهاز ضعيف. احسب حجم bundle، وقت التحميل، الذاكرة، وسياسات المتصفح. اختبر WebGPU/WebGL/CPU fallback. لا ترسل telemetry محتوى المستخدم افتراضيًا؛ سجل أقل قدر من معلومات الأداء بعد موافقة وسياسة واضحة.

19.18 اختبار الطرف إلى الطرف

ابدأ بصورة خام → postprocess → scores → runtime → tensor → preprocessing → decode. احفظ golden fixtures ببيانات غير حساسة وقارن كل خطوة. قارن النموذج الأصلي بالنسخة المحولة ضمن tolerance، واختبر المدخل المشوه والمنقطع ونفاد ذاكرة. في API، تحقق من المصادقة وrate limits وtimeouts ورسائل الخطأ. هذا الاختبار يكشف التباين بين نظامين الذي لا يظهر بتجربة `converter.convert()` .

19.19 خدمة API قابلة للتوسع

افصل HTTP handler عن model runner وعن preprocessing. حمّل النموذج عند بدء العملية، لا لكل request. احسب عدد workers بحسب الذاكرة، لأن كل عامل قد يحمل نسخة كاملة. batching الديناميكي يزيد throughput لكنه يضيف queue latency. أضف حدودًا للحجم والتزامن والمهلة، ومصادقة health checks ومقاييس p50/p95 والأخطاء.

19.20 اختبار الحمل والفشل

زد الحمل تدريجيًا وسجل throughput والكمون والذاكرة ونسبة 4xx/5xx. اختبر طلبًا ناقصًا، صورة تالفة، حجمًا زائدًا، وإيقاف runtime. أطلق candidate على نسبة صغيرة أو shadow، وقارن النتائج دون تغيير القرار حتى تجتاز البوابة. جرب rollback تحت حركة اختبارية؛ احتفظ بوزن النموذج والمعالجة والعتبة والإصدار معًا.

19.21 تقييم الطرف إلى الطرف

benchmark داخل notebook لا يقيس الشبكة وserialization والصف والتحميل الأولي. قس من العميل إلى العميل في بيئة staging. للهاتف، اختبر جهازًا من الفئة الأضعف والحرارة والتخزين والبطارية بعد تشغيل متكرر. أبلغ النموذج وruntime وإعدادات المسرع؛ اختلاف runtime قد يغير سرعة العمليات ودقتها.

19.22 مصفوفة توافق الإصدار

سجل Keras وTensorFlow وLiteRT runtime ونظام التشغيل ونوع المسرع في تقرير النشر. اختبر مجموعة العمليات المدعومة على الجهاز، لأن تصدير artifact لا يثبت أن كل operator متاح. احتفظ بصورة من فحص التوقيع والكمون ضمن release record.

19.23 التوثيق للمستخدم النهائي

اعرض confidence ومعناها وحدودها، وسياسة رفض الطلب، ومتطلبات الصورة، ووقت آخر تحديث. لا تقدم confidence كاحتمال معاير ما لم يختبر calibration. اذكر قناة الرجوع للدعم إذا كانت الخدمة تؤثر في قرار مالي أو طبي أو تشغيلي.

19.24 أخطاء شائعة

> ⚠ **تحذير**
>
> تحويل النموذج لا ينقل تلقائيًا قواعد تجهيز الصور أو عتبة القرار إلى الخدمة. [!DANGER] لا تقدم خدمة تنبؤ عامة بلا التحقق من الحجم والنوع ومعدل الطلبات والمصادقة.

19.25 تمارين

1. صدر نموذجًا إلى SavedModel. (المتوقع: توقيع استدلال محفوظ.)
2. حوله إلى LiteRT وقارن الحجم. (المتوقع: ملف أصغر أو تفسير سبب عدم ذلك.)
3. قارن الدقة قبل وبعد التكميم. (المتوقع: تقرير فرق على بيانات حقيقية.)
4. اختر مسارًا لخدمة داخلية. (المتوقع: مبررات الكمون والحمل والتشغيل.)
5. اكتب عقد مدخلات API. (المتوقع: أشكال وأنواع وحدود تحقق.)
6. خطط rollback لإصدار معيب. (المتوقع: رجوع سريع لإصدار معلوم.)

19.26 ملخص

- الحفظ للتدريب وإعادة التحميل يختلف عن تصدير الاستدلال.
- التكميم يضغط النموذج ويحتاج تحققًا من الجودة على الهدف.
- النشر عقد بيانات وتشغيل ومراقبة، وليس مجرد تحويل ملف.

الجزء الرابع · الاحتراف والإنتاج

الفصل العشرون

20

MLOps: TFX والمراقبة

> **حكاية الفصل**
>
> أطلق فريق نموذجًا جيدًا، وبعد أشهر تغيرت أنماط الشراء. لم يتغير البرنامج، لكن التوقعات تدهورت. لم يعرف أحد أي بيانات بنت الإصدار الحالي أو كيف يعاد بناؤه. كيف نجعل دورة النموذج قابلة للتتبع والرجوع؟

20.1 أهداف الفصل

- ☐ وصف مكونات خط TFX ودورة البيانات.
- ☐ تمييز انجراف البيانات عن انجراف العلاقة.
- ☐ تصميم تتبع التجربة وإعادة التدريب والرجوع.
- ☐ تحديد عناصر حوكمة النموذج.

20.2 خط إنتاج له نسب بيانات وإصدارات

يبدأ TFX بمصادر أمثلة، ثم يحسب إحصاءات ومخططًا، ويتحقق من البيانات، ويحول السمات، ويدرّب النموذج، ويقيّمه قبل دفعه. كل مكوّن يستهلك أثرًا وينتج أثرًا قابلًا للتتبع. لا يلزم استخدام TFX لكل مشروع؛ الفكرة الأساسية هي أن تكون التحويلات وبيانات التدريب وإصدار النموذج والقرار محفوظة.



شكل 20-1 حلقة MLOps من تربية النموذج إلى مراقبته

المكون	السؤال الذي يجيب عنه
ExampleGen	ما مصدر الأمثلة؟
Statistics/Schema	ما توزيع السمات وعقدها؟
ExampleValidator	هل خالفت البيانات المخطط أو ظهرت شذوذات؟
Transform	ما التحويلات الثابتة المستخدمة؟
Trainer/Evaluator	هل تحسن النموذج على معيار القبول؟
Pusher	هل يسمح بنشر الإصدار؟

MLflow أو TensorBoard يساعد في تسجيل المعلمات والمقاييس والآثار. Vertex AI أو Kubeflow قد يديران خطوطًا أكبر، لكن الخدمة المدارة لا تعوض تعريف معيار قبول أو الاحتفاظ بأصل البيانات.

20.3 المراقبة ترى التغير لا الحقيقة وحدها

$$Release = Quality_gate \wedge Data_validation \wedge Operational_readiness$$

الإصدار يمر فقط إذا اجتاز بوابة الجودة وفحص البيانات والاستعداد التشغيلي؛ كل شرط يغطي نوع فشل مختلفًا.

انجراف البيانات هو تغير توزيع المدخلات. انجراف المفهوم يعني تغير العلاقة بين المدخل والهدف. راقب كذلك الكمون والأخطاء ونسبة القيم المفقودة ومعدلات الفئات. إذا تأخرت الوسوم، يمكن مراقبة مدخلات مؤقتًا ثم إجراء تقييم مؤكد بعد وصول الحقيقة. لا تعتبر اختلاف توزيع السمة دليلًا كافيًا على فشل النموذج دون اختبار أثره.

20.4 دورة إصدار قابلة للرجوع

عرّف مجموعة قبول ثابتة، وسجل نسخة البيانات والشيفرة والاعتمادات والبذرة والمعالجة. لا ينشر المرشح قبل اجتياز الاختبارات التقنية ومقاييس الشرائح الحرجة. أطلق تدريجيًا، وراقب الكمون والفشل والتوزيع. احتفظ بإصدار مستقر وخطة rollback، وحدد من يوافق على إعادة التدريب وتواترها.

```python
# سجل الحد الأدنى للتجربة في منصة تتبع مثل MLflow:
run_metadata = {
    "data_version": "orders-2026-04",
    "code_revision": "git-sha-recorded-by-ci",
    "seed": 42,
    "metric_name": "macro_f1",
    "model_version": "candidate-17",
}
print(run_metadata)
# المتوقع: سجل يربط القياس بمصدر البيانات والشيفرة والنموذج.
```

20.5 حالة واقعية: تغير سلوك العملاء

بعد حملة ترويج، تتغير أعداد الطلبات ومصادر الزيارات. يعرض النظام الانجراف حسب القناة والمنطقة، ويؤجل قرار إعادة التدريب حتى تتوفر نتائج فعلية. يُعاد التقييم على فترة مستقبلية، وتفحص شرائح صغيرة قبل تعميم النموذج. بذلك لا تتحول كل حملة مؤقتة إلى إعادة تدريب آلية غير مضبوطة.

20.6 أمثلة تنفيذية إضافية

سجل التجربة

```python
record = {"data_version": "v3", "seed": 42, "macro_f1": 0.81}
# Link the record to code, preprocessing, and model versions.
```

إشارة الانجراف

```python
baseline_mean = train_feature.mean()
current_mean = production_feature.mean()
print(current_mean - baseline_mean)
```

20.7 خطوط التشغيل ليست مجرد رسم مربعات

يُحدد كل component مدخلاته ومخرجاته وإصدار التنفيذ. ExampleGen يقرأ مصادر البيانات ويقسمها، StatisticsGen يصف التوزيعات، SchemaGen يساعد على استنتاج أنواع وقيود، ExampleValidatorg يفحص الشذوذات. Transform يضمن تطبيق تجهيز متعلم من التدريب وقت الاستدلال، Trainerg يدرب، Evaluatorg يقارن ببوابة قبول، Pusherg ينشر الإصدار الذي اجتازها. في الواقع، قد تحتاج اختبارات خصائص وجودة بيانات وحوكمة قبل هذه المكونات وبعدها.

لا تفترض أن schema مستنتج من البيانات هو عقد صحيح: إذا لم تظهر فئة نادرة في عينة، قد يضع النظام قيدًا ناقصًا. راجع المخطط بشريًا وحدد نطاقات وقيم OOV وقيودًا زمنية. احتفظ بمصدر البيانات وتاريخها ومعرف الاستعلام، لأن نفس جدول المستودع يمكن أن يتغير في الخلفية.

20.8 مراقبة متعددة الطبقات

سلامة الخدمة

راقب الكمون، throughput، نسبة الأخطاء، إعادة المحاولة، ذاكرة العملية، ووقت تحميل النموذج. ارتفاع أخطاء 5xx قد يكون مشكلة اعتماد لا انجراف تعلم. اربط سجل الخطأ برقم إصدار دون تسجيل مدخل حساس كاملًا.

جودة البيانات

راقب تغيرات توزيع السمات، القيم المفقودة، فئات OOV، وحدود القيم. قد يفيد PSI أو KS أو إحصاء آخر بحسب نوع السمة، لكن عتبة الانذار قرار تطبيقي. لا تقارن نافذتين مختلفتي الحجم بلا اختبار دلالة أو مجال ثقة، ولا تعمم عتبة واحدة على كل ميزات البيانات.

جودة القرار

عندما تصل labels متأخرة، اربطها بالنسخة والتنبؤ السابق الصحيحين، ثم احسب المقياس على cohort زمني. لا تعيد تسمية سجل بتوقع حديث. قارن الأداء حسب اللغة والموقع والفئة ذات الأثر العالي، واعرض عدم اليقين وحجم العينة. انجراف المدخلات علامة تستحق التحقيق، لا إثبات أن النموذج تدهور.

20.9 دورة إصدار تحمي من التحسن الزائف

ثبت مرشحًا baseline وسجل معيار النجاح قبل التدريب الجديد. تحقق من عدم تداخل بيانات التدريب مع مجموعة القبول، ومن عدم تغير تعريف label. شغل اختبارات تكامل للتحويلات وتوقيع النموذج والكمون، ثم جرب canary أو shadow. قارن مؤشرات الإنتاج ومخرجات النسختين وخطط rollback. كل خطوة لها صاحب قرار وسجل تدقيق، خاصة في التطبيقات المنظمة.

20.10 إعادة التدريب والاستدامة

إعادة التدريب قد تكون مجدولة، أو عند تجاوز signal، أو عند توفر labels جديدة. قبل تشغيله، تحقق من اكتمال المصدر وجودة labels؛ بيانات حديثة رديئة قد تفسد نموذجًا جيدًا. احتفظ بخط زمني للنسخ، وقارن مرشحًا جديدًا بأفضل نسخة منشورة على نطاق زمني حديث وثابت. سجّل تكاليف التخزين والحوسبة والتشغيل، ونظف artifacts وفق سياسة الاحتفاظ.

20.11 نموذج أولي وتدرج الاحتراف

في مشروع طالب، قد يكفي ملف بيانات بإصدار scriptg تدريب وسجل JSON للنتائج. عند تعدد الباحثين، أضف تتبع التجارب ومخزن artifacts. عند نشر خدمة مستمرة، أضف lineage validationg monitoringg rollbackg. الأداة تتبع التعقيد المطلوب؛ لا يضيف استخدام TFX قيمة إذا لم توجد مشكلة pipeline أو تدقيق تحتاجه.

20.12 قائمة قبول لنموذج جديد

قبل النشر، اسأل: هل مصدر البيانات وترخيصها معروفان؟ هل المخطط اجتاز فحوص الجودة؟ هل preprocessing واحدة للتدريب والاستدلال؟ هل تفوق المرشح على baseline في فترة لم تستخدم للضبط؟ هل بقيت المقاييس ضمن الحدود على الشرائح ذات الأثر؟ هل latency والذاكرة مقبولان؟ هل بطاقة النموذج وخطة الرجوع جاهزتان؟ المجهول يعني تجربة أو مراجعة، لا إطلاقًا.

قارن المرشح عبر seeds أو folds عندما تكون البيانات قليلة، وأبلغ فرق المقياس وعدم اليقين، لا رقمًا منفردًا. في التجارب online، حدد metric أساسيًا وguardrails، واحسب مدة تكفي لرصد الموسمية. لا توقف الاختبار مبكرًا عند ظهور فرق مرغوب دون تصميم إحصائي مناسب.

20.13 تتبع التجارب والآثار

كل run يحتاج معرفًا يرتبط بتغييرات الشيفرة، config، إصدار البيانات، بيئة التنفيذ، artifacts، metrics، والبذرة. اجعل أسماء المقاييس ووحداتها ثابتة. منصة التتبع ليست المصدر الوحيد إن لم تحفظ artifacts بطريقة آمنة. افصل بيانات الوصول السرية عن config المسجل.

Lineage يجيب: أي ملفات وإصدارات ومكونات أنتجت هذا النموذج؟ لإعادة البناء استخدم snapshots أو مدخلات immutable، لا اسم جدول متحرك أو `latest`. خزّن hash لمجموعات المصدر وقواعد التحويل. احتفظ بمخرجات evaluator والشرائح التي اجتازت البوابة.

20.14 تصميم مراقبة الانجراف

اختر نافذة baseline تمثل تشغيلًا مستقرًا، وحدد نافذة الإنتاج وطريقة التعامل مع اختلاف حجم العينات والموسمية. للميزات العددية استخدم quantiles أو إحصاء توزيع مناسب، وللفئوية راقب OOV والتوزيع، وللنص راقب طول tokens دون كشف محتوى. اختر عتبات عملية، واربط التنبيه بإجراء واضح: التحقيق، fallback، أو تقييم مرشح.

تغيير التوزيع إشارة للتحقيق وليس برهانًا على تراجع. عندما تتوفر labels، اربطها بالتنبؤ والإصدار والوقت المناسب ثم احسب الأداء حسب cohort زمني. لا تخلط توقع النسخة الحالية مع labels قديمة دون توثيق. اعرض فواصل الثقة وحجم الشريحة.

20.15 مسؤوليات الفريق والاستدامة

العالم يحدد الهدف والوسم والمقياس، ومهندس البيانات يضمن المصدر والتدفق، ومهندس ML يبني ويقيّم، وSRE والأمن يراجعان الموثوقية والصلاحيات. في مشروع صغير قد ينفذ شخص واحد هذه الأدوار، لكن الفصل الذهني لها يقلل الثغرات. عرف من يوافق على إعادة التدريب ومن يوقف الخدمة.

احتفظ بسجل تغيير فيه سبب الإطلاق والمقاييس والمراجع والنسخة السابقة. راجع صلاحيات البيانات ومدة الاحتفاظ. سجل التكلفة والطاقة وزمن إعادة التدريب؛ الاحتراف يقاس بموثوقية القرار وكلفة الحفاظ عليه، لا بعدد النماذج المنشورة.

20.16 مؤشرات تشغيلية وبوابات قرار

اجعل التنبيه يملك runbook: ما الذي يتحقق منه المناوب؟ من يقرر fallback؟ ما الحد الذي يوقف الطرح؟ دون إجراء مرتبط، يتراكم dashboard بلا استجابة. قس عدد alerts الكاذبة وراجع العتبات لتقليل الإنذار الإرهاقي.

20.17 فشل pipeline والتعافي

عرّف retry آمنًا للمكونات القابلة للتكرار، وحدد checkpoint لنقطة الاستئناف. لا تعِد خطوة side-effect مثل Pusher دون idempotency؛ قد تنشر نسختين أو تعيد تفعيل candidate قديم. أرسل تنبيهًا يتضمن component وrun id وخطوة الفشل، وحدد مالكًا. بعد عودة الخدمة، قارن lineage للمرشح قبل استكمال النشر.

20.18 اتفاقية تشغيل بين الفرق

اكتب SLO للكمون والتوفر، وSLA للجهات المستهلكة عند الحاجة، ومسؤولية جودة label ومصدر البيانات. حدد ما يحدث إذا تجاوز drift حدًا، ومن يوافق على rollback، والمدة المقبولة لغياب labels. هذه الاتفاقيات تمنع أن تصبح ML pipeline مملوكة للجميع ولا يملكها أحد.

20.19 أمان البيانات والنماذج

طبّق أقل صلاحية على حسابات pipeline، وافصل دور قراءة البيانات عن دور دفع النموذج إلى الإنتاج. تحقق من ملفات container imageg dependencies، وامسح الأسرار من logs. ضع artifacts الحساسة في مخزن مشفر ومحدود الوصول. امنع تعديل model registry بعد الموافقة دون سجل تغيير؛ سلامة artifact جزء من موثوقية القرار.

20.20 مستوى نضج تدريجي

ابدأ بإعادة تشغيل تدريب موثقة، ثم سجل datasetsg runs، ثم أضف evaluator وبوابة نشر، ثم راقب التشغيل والـ rollback. لا تبنِ منصة كاملة قبل أن تعرف ما الذي يتكرر أو يؤلم الفريق. راجع كل مرحلة مع مستخدميها حتى تقل كلفة التشغيل بدل نقل التعقيد إلى فريق آخر.

20.21 حادث انجراف: سيناريو استجابة

لنفترض ارتفاع نسبة OOV بعد إطلاق سوق جديد. لا يعيد الفريق التدريب مباشرة. يبدأ بتحديد إصدار feature transformer والوقت الذي بدأ فيه الارتفاع، ثم يفحص هل أضيفت الفئات إلى المصدر أو تغيرت طريقة استخراجها. يقارن الأداء عندما تصل labels حسب السوق، ويراجع قائمة الأخطاء مع مالك المنتج. إن كان العطب preprocessing، يصلح العقد ويعيد إصدار المعالجة؛ إن كان تحولًا حقيقيًا، يبني candidate على بيانات حديثة ويختبره عبر بوابة القبول. التمييز يمنع إعادة التدريب على عيب تشغيلي.

20.22 تخزين النسب والاحتفاظ

اربط أي artifact بمعرف البيانات والشفرة والتبعيات وخط التحويل. لا تحتفظ بكل صورة raw في منصة التتبع إذا كانت تحتوي بيانات شخصية؛ احفظ معرفًا مشفرًا أو إحصاءً وفق سياسة الخصوصية. حدد retention لكل logs وmodel checkpointsg، واحتفظ بما يلزم لإعادة البناء أو التحقيق القانوني فقط. التوازن بين auditability والحد الأدنى للبيانات يجب أن يكون قرارًا واضحًا.

20.23 بوابة قبول مرشحة

يمكن أن تشترط البوابة أن يتفوق `macro_f1` على baseline، وألا يتراجع recall في شريحة حرجة أكثر من حد متفق، وأن يفي p95 inference بميزانية الخدمة. اجعل مقارنة المرشح والنسخة الحالية على cohort واحدة، واستخدم فاصل ثقة عند قلة الأمثلة. لا تجمع مقاييس لا تتصل بهدف العمل في score اعتباطي واحد. احتفظ

بتقرير evaluator مع قرار الموافقة واسم المراجع.

20.24 أخطاء شائعة

⚠ تحذير

إعادة التدريب على جدول زمني ثابت بلا مراقبة جودة قد تنشر تراجعًا جديدًا. [!DANGER] لا تحتفظ بسجل metric دون نسخة بيانات وشيفرة؛ لن تعرف ما الذي أنتج النتيجة.

20.25 تمارين

1. ارسم خط بيانات إلى نموذج منشور. (المتوقع: آثار ومكونات موثقة.)
2. ميّز انجراف البيانات عن انجراف المفهوم. (المتوقع: تغير المدخلات مقابل تغير العلاقة.)
3. اختر إشارات تشغيلية لمراقبتها. (المتوقع: جودة وكمون وأخطاء ومدخلات.)
4. عرف بوابة قبول إصدار. (المتوقع: مقاييس ثابتة وشرائح حرجة.)
5. صمم خطة rollback. (المتوقع: نسخة سابقة وقرار استعادة.)
6. ناقش مخاطر التدريب الآلي. (المتوقع: تحقق قبل النشر وتتبّع.)

20.26 ملخص

- MLOps سلسلة آثار قابلة لإعادة الإنتاج والقبول والتشغيل.
- المراقبة تشمل الجودة والبيانات والخدمة، والانجراف ليس حكمًا منفردًا.
- كل إصدار يحتاج مصدرًا معلومًا وقرار نشر وخطة رجوع.

الجزء الرابع · الاحتراف والإنتاج

الفصل الحادي والعشرون

21

التعلم المعزز

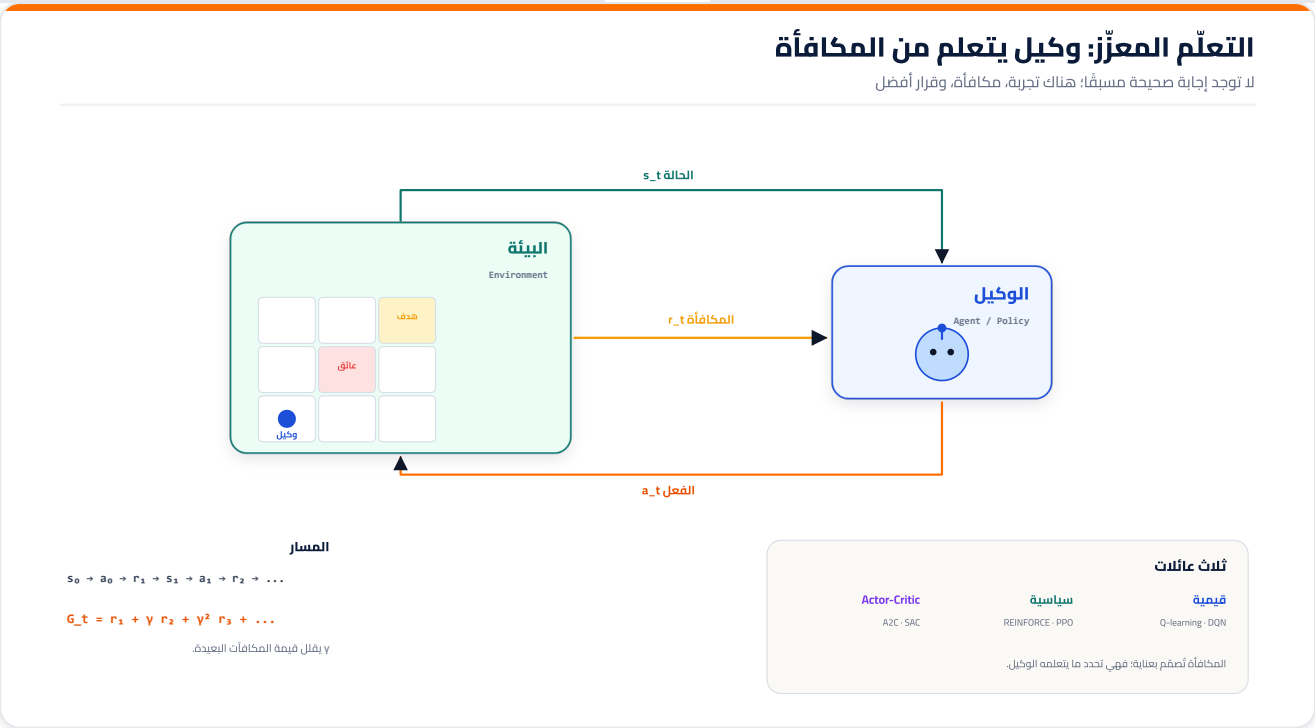
> **حكاية الفصل**
>
> تحرك روبوت مستودع نحو أقصر طريق، لكنه اصطدم بعربة لأن المكافأة عاقبته على طول المسار فقط. لم يكن الخطأ في الاستكشاف وحده؛ الهدف صاغ السلوك غير المرغوب. كيف نمثل قرارات متتابعة تتأثر نتائجها بما سيحدث لاحقًا؟

21.1 أهداف الفصل

- [ ] تعريف MDP والحالة والفعل والمكافأة والسياسة.
- [ ] قراءة معادلة Bellman وفهم Q-learning.
- [ ] شرح مكونات DQN ومزالق الاستقرار.
- [ ] تحديد متى لا يكون RL الاختيار المناسب.

21.2 عملية قرار ماركوف

يصف MDP الحالات s والأفعال a والمكافآت والانتقالات واحتمال الانتقال ومعامل الخصم γ. السياسة تختار فعلًا من الحالة، وتسعى لتعظيم المكافأة المتراكمة لا مكافأة الخطوة وحدها. صياغة المكافأة تحدد الحوافز؛ قد يستغل الوكيل الاختصار الذي كتبه المطور ويحقق الرقم مع خرق المقصد.



شكل 21-1 تفاعل الوكيل مع بيئة مرحلة

$$Q(s,a) = \mathbb{E}[r + \gamma \max_{a'} Q(s',a')]$$

`Q(s,a)` قيمة تنفيذ الفعل، و `r` المكافأة الحالية، و `γ` وزن المستقبل، و `s'` الحالة التالية. التوقع يأخذ انتقالات البيئة في الحسبان.

21.3 من Q-learning إلى DQN

يحدّث Q-learning القيمة بعد كل انتقال بفرق TD. عندما تصبح الحالات كثيرة، تقرب شبكة عصبية دالة Q. يستخدم DQN مخزن إعادة لتقليل ترابط العينات، وشبكة هدف أبطأ لتقليل الحركة المستمرة لهدف التعلم. الاستكشاف مثل ε-greedy يمنع الاكتفاء المبكر بأفعال تبدو جيدة في عينات قليلة.

العنصر	دوره	ما الذي يحدث عند غيابه؟
Replay Buffer	خلط خبرات سابقة	عينات مترابطة بشدة
Target Network	هدف أكثر ثباتًا	تذبذب بين المتنبئ وهدفه
Exploration	تجربة أفعال بديلة	استغلال مبكر لمسار ضعيف
Reward design	تحديد المقصد	سلوك يطابق المقياس لا النية

Policy Gradient يتعلم توزيع أفعال مباشرة. PPO يقيد مقدار تحديث السياسة للمحافظة على استقرار نسبي. هذه الخوارزميات تستهلك تفاعلات وتجارب؛ المحاكاة لا تكفي إن كانت مختلفة عن العالم الحقيقي.

21.4 تجربة آمنة في CartPole

ابدأ ببيئة معيارية وبذور محددة، وسجل المكافأة وطول الحلقة وتباين عدة تشغيلات. في TensorFlow يمكن بناء DQN، أو استخدام مكتبة متخصصة إن كانت متوافقة ومدعومة. افصل البيئة والتقييم عن حلقة التعلم، ولا تعرض وكيلًا غير مدرب على جهاز حقيقي دون حدود أمان.

21.5 حالة واقعية: روبوت المستودع

تُحدد المكافأة لسلامة الأشخاص أولًا، ثم زمن الحركة واستهلاك الطاقة. تختبر السياسة في محاكاة تتنوع فيها مواقع العوائق والاحتكاك، ثم تعمل في وضع مراقبة بلا تحكم. أي انتقال للتشغيل الفعلي يحتاج حد توقف آمن ومشرفًا بشريًا. في كثير من حالات التخطيط، قد يكون البحث أو التحسين التقليدي أبسط وأوثق من RL.

21.6 أمثلة تنفيذية إضافية

تحديث Q-learning

```python
target = reward + gamma * max_next_q
new_q = old_q + learning_rate * (target - old_q)
```

اختيار عمل بمنهج ε-greedy

```python
if rng.random() < epsilon:
    action = env.action_space.sample()
else:
    action = int(q_values.argmax())
```

21.7 صياغة Bellmang MDP

في MDP، تمثل `s_t` حالة البيئة، و `a_t` الفعل، و `r_{t+1}` المكافأة بعد الانتقال، و `P(s'|s,a)` احتمال الحالة التالية. إذا كانت الحالة لا تكفي لتوقع المستقبل (مثلاً ملاحظة جزئية من كاميرا)، فقد لا تحقق خاصية ماركوف؛ قد يحتاج الوكيل سجلًا أو belief state. اختيار الحالة جزء من تصميم المسألة.

معامل الخصم `γ` يقرب المكافآت المستقبلية؛ `γ` صغير يفضل مكافأة قريبة، وكبير يولي المستقبل وزنًا أعلى. إذا كانت المهمة تنتهي، عالج terminal state بحيث لا تضيف قيمة مستقبلية بعد النهاية. خلط `done` مع نهاية حلقة بسبب حد زمني يغير الهدف؛ Gymnasium يميز terminated عن truncated في واجهات حديثة.

21.8 DQNg Q-learning عمليًا

تحديث tabular Q يستخدم هدفًا مؤقتًا `r + γ max_a' Q(s',a')`. مقدار خطأ TD يحدد حركة القيمة. في DQN، تأخذ الشبكة الحالة وتخرج قيمة كل فعل، وتختار `argmax` عند الاستغلال. تتكون replay sample من `(s,a,r,s',done)`. احسب قيمة الفعل المنفذ، وقارنها بالهدف من target network، ثم أوقف التدرج عن الهدف حتى لا يطارد الشبكة نفسها.

```python
import tensorflow as tf

q_values = online_network(states)
chosen_q = tf.reduce_sum(q_values * tf.one_hot(actions, num_actions), axis=1)
next_q = tf.reduce_max(target_network(next_states), axis=1)
target = rewards + gamma * (1.0 - terminals) * next_q
loss = tf.reduce_mean(tf.square(tf.stop_gradient(target) - chosen_q))
```

التحديث الكامل يحتاج replay buffer، وتزامن target كل عدد خطوات، والاستكشاف، وتطبيع observations، وقياس عدة بذور. Double DQN يفصل اختيار الفعل التالي عن تقييمه لتخفيف overestimation. لا تضف تقنيات متقدمة قبل أن يستقر baseline صغير على بيئة بسيطة.

21.9 REINFORCE وPPO

Policy gradient يتعلم توزيع السياسة مباشرة ويستخدم مكافآت الحلقات لتحديثها. REINFORCE عالي التباين؛ baseline متعلم يمكنه خفض التباين دون تغيير التوقع في شروط معينة. PPO يقيد تغير النسبة بين السياسة الجديدة والقديمة ويحسب objective clipped، ويشيع معه critic لمزايا الأفعال. اختيار خوارزمية يعتمد على فضاء الأفعال، عينة البيئة، واستقرار التحديث.

21.10 تقييم قبل الواقع

شغل عدة seeds، وقس متوسط العائد والتشتت وعدد الخطوات إلى مستوى النجاح. اختبر تغييرات عشوائية في البيئة مثل احتكاك أو إضاءة أو مواضع العوائق، لا الحالة الثابتة فقط. reward hacking يظهر حين يجد الوكيل مسارًا يزيد المكافأة دون تحقيق الغرض؛ افحص trajectory فيديو أو log للحالات والأفعال. ضع محددات آمنة في البيئة ولا تعتمد على أن السياسة ستتجنب الحالات الخطرة من تلقاء نفسها.

21.11 متى لا تستخدم RL؟

إذا كانت قواعد القرار معروفة أو يمكن تحسينها بخوارزمية تخطيط، فقد يكون ذلك أوضح وأسهل في التحقق. RL مكلف في جمع التفاعل، حساس لصياغة المكافأة، وقد يكون صعب التفسير. يصلح عندما توجد قرارات متسلسلة وتغذية راجعة وتأثير زمني لا يختزل إلى supervised label، مع بيئة تدريب آمنة وقياس موثوق.

21.12 درس CartPole كتجربة علمية

تلاحظ العربة موضعها وسرعتها وزاوية العمود وسرعته، وتختار دفعًا إلى اليمين أو اليسار. تكافئ البيئة بقاء العمود قائمًا وتنهي الحلقة عند تجاوز حد أو بلوغ الأفق. هذا مختبر للتفاعل، لا دليلًا أن DQN صالح لعربة صناعية. ثبت إصدار البيئة وseed max episode length، لأن wrappers أو قواعد الإنهاء قد تتغير.

ابدأ بسياسة عشوائية كخط أساس، ثم heuristic بسيط، ثم DQN. خفّض epsilon وفق عدد خطوات البيئة، لا epochs البيانات؛ RL يجمع خبرة من التفاعل. اختبر سياسة greedy بلا استكشاف على عدة حلقات، واحفظ checkpoint دوريًا. اعرض العائد المتوسط والتشتت عبر بذور مستقلة، لا أفضل تشغيل فقط.

21.13 ضبط المكافأة دون خلق تحايل

المكافأة الكثيفة تسهل التعلم لكنها قد تغير الهدف. مكافأة كل حركة باتجاه هدف قد تُستغل بالاهتزاز إذا كانت الإحداثيات أو الإشارة خاطئة. المكافأة النهائية أمينة لكنها sparse وقد تتطلب استكشافًا طويلًا. Potential-based shaping له شروط نظرية تحفظ السياسة المثلى في صياغة معينة؛ يجب تطبيقها بدقة عند الحالات النهائية.

في الروبوت، ضع حدود سرعة ومسافة أمان صلبة خارج المكافأة، واختبر السياسة على محاكاة ذات خصائص عشوائية. Domain randomization يقلل التعلق بنموذج فيزيائي واحد لكنه لا يعالج فجوة بين المحاكاة والعالم. قبل التشغيل، استخدم shadow policy ومشرف إيقاف واختبارًا تدريجيًا.

21.14 Replay وحالات نهاية الحلقة

Replay buffer يخزن `(s,a,r,s',done)`. عند النهاية، لا تستبدل الحالة التالية بالحالة الابتدائية للحلقة التالية. ميز بين انتهاء المهمة فعليًا وبين انتهاء حد زمني؛ Gymnasium الحديث يفصل `terminated` عن `truncated`، ولهما أثر مختلف على bootstrap. حجم buffer الكبير يزيد تنوع الخبرة لكنه يحتفظ بتوزيع قديم، والصغير قد ينسى تجارب نادرة.

Prioritized replay يعطي أولوية لانتقالات ذات TD error عالٍ، لكنه يسبب انحيازًا في العينة ويحتاج importance weights. Double DQN يفصل اختيار الفعل من تقييمه لتقليل overestimation. أضف هذه التحسينات بالتدريج بعد استقرار DQN الأساسي، وسجل hyperparameters لكل واحدة.

21.15 قياس المتانة وقرار الانتقال للواقع

اختبر تغييرات لم تستخدم في التدريب: احتكاك، كتلة، تأخير استشعار، مواضع عوائق. أبلغ متوسط العائد، معدل النجاح، زمن الحلقة، وأسوأ الحالات. reward hacking قد يظهر فقط في trajectory؛ راجع فيديو أو سجل حالات وأفعال. لا تستنتج عائد إنتاجي من simulation دون تجربة مراقبة مضبوطة وحدود سلامة.

21.16 بيانات بشرية كخط أساس

في بعض المسائل، سياسة خبير أو demonstration أفضل نقطة بدء من الاستكشاف العشوائي. وثق من أنتج demonstrations، وهل تغطي الحالات النادرة، وكيف تتعامل مع الاختلاف بينهم. قارن policy المتعلمة بأداء الخبير ومعدل الأخطاء الحرجة، لا المكافأة الاصطناعية فقط.

21.17 انحياز وتغطية replay

إذا كانت البيئة تنتج تكرارات كثيرة لحالة شائعة، قد يغرق replay buffer في خبرة معتادة ويهمل حالات طرفية نادرة. راقب توزيع الحالات والأفعال، ولا توازن عينات replay اعتباطًا دون إدراك أنه يغير التوزيع. افحص الحلقات الفاشلة وتأكد من أن terminal events الحرجة ممثلة بما يكفي للتعلم والتقييم.

21.18 وثائق بيئة قابلة للتكرار

احفظ نسخة البيئة ونسخة wrappers والـ observation/action spaces. seedg عند تغيير dynamics، أعد baseline من الصفر حتى لا تنسب تحسن سياسة إلى نسخة بيئة أسهل. شارك رسمًا لمسار الحلقة وعائدها لا checkpoint وحده.

21.19 فحص سياسة قبل نشرها

استخدم deterministic evaluation للسياسة greedy، ثم stochastic evaluation إذا كانت السياسة احتمالية. سجل trajectories للحالات الحدية، ومعدل النجاح والفشل، ومقدار constraint violations. لا تخفِ failure episodes عند حساب متوسط return. اختبر سياسة emergency fallback مستقلة عن نموذج RL.

21.20 Offline RL والحذر من البيانات

إذا تعلم الوكيل من dataset ثابتة دون تفاعل، فالسياسة قد تقترح أفعالًا خارج دعم البيانات. extrapolation في Q-values خطر؛ اذكر توزيع البيانات وسياسة الجمع. قارن action coverage للسياسة الجديدة بما ورد في سجلات جمع الخبرة، واستخدم قيودًا أو مراجعة بشرية قبل التنفيذ.

21.21 خريطة تنقل للمبتدئ والباحث

يبدأ الطالب بـ bandit ثم GridWorld، ليفصل اختيار الفعل عن حالة معقدة. ينتقل إلى tabular Q-learning قبل DQN. الباحث يسجل عدة بذور وبيئة ونسخة wrappers ويقارن ablationg baseline. المهندس يضيف safety constraints وshadow modeg وfallback قبل ربط أي policy بمعدة. هذا التدرج يكشف المفاهيم دون القفز إلى نظام لا يمكن اختباره.

21.22 مؤشرات فشل التدريب

العائد الثابت عند مستوى منخفض قد يعني أن الاستكشاف توقف مبكرًا أو أن reward لا يحمل إشارة، والانفجار قد يشير إلى bootstrap غير مستقر أو مقياس observation. اعرض TD error وتوزيع Q-values وepsilong ومدة الحلقات. قارن target network بالـ online network. قيم reward على trajectory بشرية المعنى، لا على المنحنى وحده.

21.23 التعلم من بيانات ثابتة

Offline RL يتعلم من سجل سياسة سابقة دون الاستكشاف في البيئة. التحدي أن policy الجديدة قد تختار actions نادرة خارج dataset، ويبالغ value estimator في تقييمها. اذكر سياسة جمع البيانات والتغطية والفئات غير المرصودة. ابدأ بمقارنة سلوك policy على الأفعال المدعومة بالبيانات، واستخدم تقييمًا محافظًا بدل تقديم performance غير مستند إلى تفاعل.

21.24 أخطاء شائعة

> ⚠ **تحذير**
>
> إعادة استخدام البيئة نفسها بلا بذور أو قياس عدة حلقات تجعل مقارنة السياسات مضللة. [DANGER!] لا تختبر سياسة استكشافية غير مقيدة على معدات أو أشخاص في العالم الحقيقي.

تكرار الحلقة ببذرة معلومة

سجّل عدة حلقات، لأن نتيجة تشغيل واحد لا تكفي لمقارنة سياسات متقلبة:

```python
returns = []
for episode in range(5):
    state, _ = env.reset(seed=100 + episode)
    returns.append(run_episode(policy, env, state))
print(sum(returns) / len(returns))
```

21.25 تمارين

1. عرف حالات وأفعال ومكافآت لمسألة بسيطة. (المتوقع: MDP محدد.)
2. احسب تحديث Q لقيم صغيرة. (المتوقع: مكافأة + قيمة مستقبلية مخفضة.)
3. فسّر Replay Buffer. (المتوقع: تقليل الترابط وإعادة استخدام الخبرة.)
4. صمم مكافأة لا تشجع اصطدامًا. (المتوقع: حد سلامة مستقل عن مكافأة السرعة.)
5. قارن DQN وPPO. (المتوقع: قيمة أفعال مقابل سياسة مباشرة.)
6. قرر إن كانت مسألة ما تحتاج RL. (المتوقع: مقارنة ببحث أو تحكم أو تحسين أبسط.)

21.26 ملخص

- RL يوازن مكافأة فورية وعوائد مستقبلية عبر سياسة تتفاعل مع البيئة.
- DQN يضيف إعادة خبرة وشبكة هدف إلى تقريب Q.
- صياغة المكافأة والسلامة والمحاكاة تحدد صلاحية الانتقال إلى العالم الحقيقي.

الجزء الرابع · الاحتراف والإنتاج

الفصل الثاني والعشرون

22

البيانات الجدولية والتوصيات والرسوم البيانية

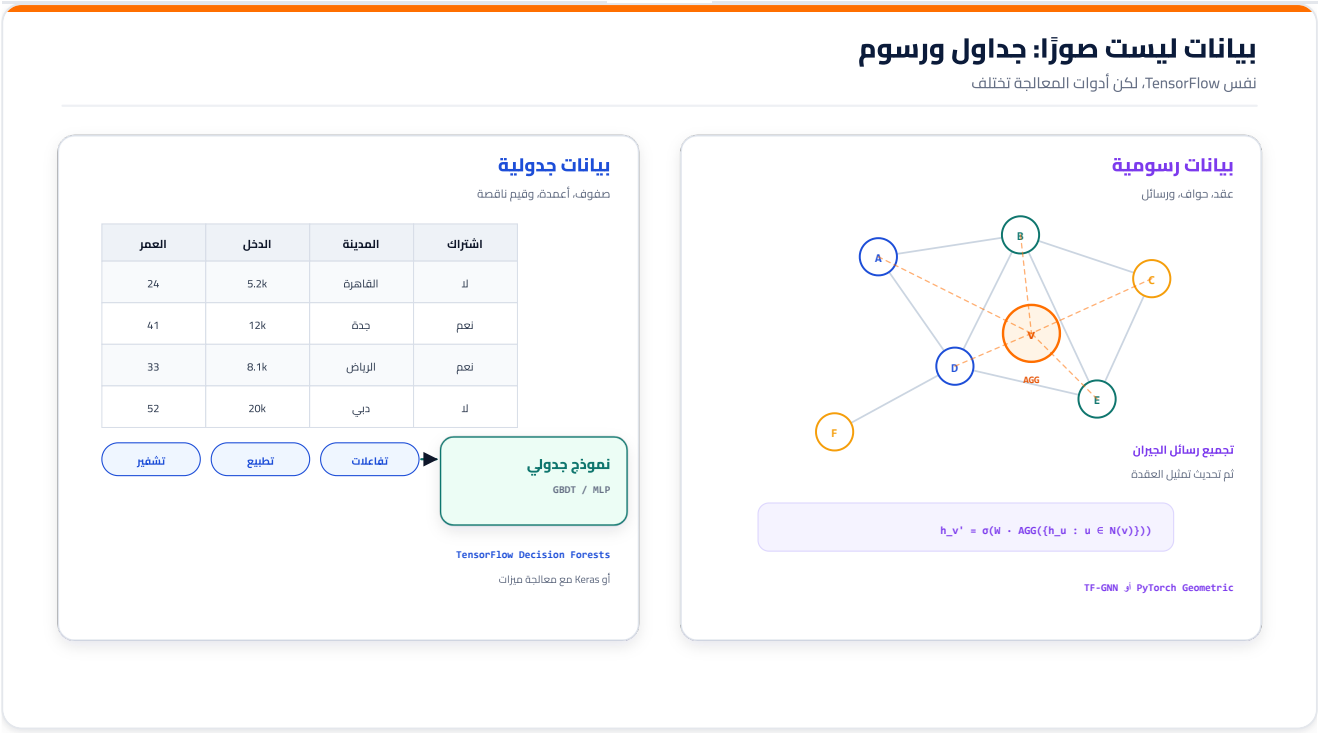
> **حكاية الفصل**
>
> توقّع نموذج متجر احتمالات شراء جيدة، لكنه أخطأ عند ظهور فئة منتج جديدة، وارتبك مع قيم مفقودة. لم تكن البيانات صورة أو نصًا؛ كانت أعمدة ذات دلالات مختلفة. متى تفيد الشبكة العميقة على جدول، وكيف نمثل العلاقات بين المستخدمين والمنتجات؟

22.1 أهداف الفصل

- [ ] تجهيز السمات العددية والفئوية بطبقات Keras.
- [ ] شرح Wide & Deep وembeddings وأنظمة التوصية.
- [ ] تمييز retrieval عن ranking.
- [ ] وصف message passing ومخاطر الرسوم البيانية.

22.2 اعرف العمود قبل ترميزه

تتعامل `Normalization` مع القيم العددية باستخدام إحصاءات التدريب، و`StringLookup` يحول الفئات إلى معرفات مع رمز OOV، و`CategoryEncoding` ينتج تمثيلًا متناثرًا أو كثيفًا حسب الحالة. ضمّن المعالجة في النموذج أو احفظ حالتها كي لا تتعلم الخدمة إحصاءات مختلفة. عالج المفقود والجديد صراحة؛ لا تستبدل القيمة بمعلومة محسوبة من المستقبل.



شكل 22-1 تجميع تمثيلات الجيران في GNN

Wide & Deep يجمع ميزات خطية وتقاطعات صريحة مع تمثيل عميق يتعلم تفاعلات. قد تتفوق أشجار القرار على شبكات عميقة في جداول صغيرة أو متوسطة؛ قارن بخط أساس مثل gradient-boosted trees قبل اختيار البنية.

22.3 توصية على مرحلتين

$$Score(u, i) = f(e_u, e_i, context)$$

يمثل `e_u` تضمين المستخدم و`e_i` تضمين العنصر، ويجمع `context` معلومات الجلسة. تقدر مرحلة الاسترجاع التوافق بكفاءة، ثم يحسن المرتّب القرار بسياق أغنى.

تسترجع مرحلة retrieval عددًا صغيرًا من العناصر المرشحة من ملايين، ثم ترتب مرحلة ranking المرشحين وفق المستخدم والسياق. Two-tower يتعلم تمثيل المستخدم وتمثيل العنصر ويقارن بينهما، ما يجعل فهرسة العناصر ممكنة. تقييم offline يجب أن يحترم الزمن؛ تفاعلات المستقبل لا تدخل تدريب المرشحات.

المهمة	حجم المرشحين	مقياس نموذجي
Retrieval	كبير إلى صغير	Recall@K
Ranking	قائمة قصيرة	NDCG أو AUC/مقياس أعمال
تصنيف جدول	ثابت	F1 أو AUC حسب القرار
تنبؤ كثافة رسم	عقد/حواف	مقياس المهمة مع فصل العقد

22.4 الرسوم ومرور الرسائل

في GNN تجمع العقدة معلومات جيرانها ثم تحدث تمثيلها. تكرر العملية لالتقاط جوار أوسع، لكن العمق الكبير قد يجعل تمثيلات العقد متشابهة (over-smoothing). درجات الجيران والحواف واتجاهها جزء من المعنى. TF-GNN يوفر مكونات للرسوم، بينما يمكن أن تكون حلول أخرى أنسب بحسب نوع الرسم والمنصة.

22.5 حالة واقعية: اقتراح المنتج التالي

يُبنى تقسيم زمني بحيث لا تتسرب مشتريات المستقبل. تقاس Recall@K في retrieval ثم NDCG في ranking، وتفحص النتائج حسب المستخدم الجديد والمنتج قليل التفاعل. للمستخدم البارد قد يلزم محتوى المنتج أو قواعد عمل؛ embeddings لا تخلق تاريخًا غير موجود.

22.6 أمثلة تنفيذية إضافية

تطبيع السمات

```python
normalizer = keras.layers.Normalization()
normalizer.adapt(train_numeric)  # بيانات التدريب فقط
normalized = normalizer(train_numeric)
```

استرجاع ثم ترتيب

```python
candidate_ids = retriever(user_vector, k=100)
scores = ranker.score(user_id, candidate_ids)
recommendations = candidate_ids[np.argsort(scores)[::-1][:10]]
```

22.7 معالجة الجدول ضمن النموذج

قسّم البيانات أولًا، ثم `adapt` لطبقات التطبيع والبحث على التدريب فقط. استخدم رمز OOV للفئات الجديدة، واحتفظ بقائمة فئات ثابتة للإصدار. لا تستخدم معرفًا رقميًا متسلسلًا لفئة اسمية ثم تطلب من Dense افتراض أن الفئة 8 أكبر من الفئة 2. one-hot يفيد مع معجم صغير، وembedding يضغط معجمًا كبيرًا إلى فضاء كثيف لكنه يحتاج تاريخًا كافيًا لكل فئة.

البيانات المفقودة لها معنى محتمل؛ أضف indicator إن كان غياب القيمة يحمل معلومة، وعالج القيم الشاذة وفق معرفة المجال. Tree ensembles مثل TF-DF أو أنظمة boosting غالبًا baseline قوي للجدول، وأسهل ضبطًا من شبكة عميقة. قارن زمن الإعداد وجودة الشرائح وصيانة preprocessing، لا metric وحيدة.

22.8 ترتيب التوصية من الطرف إلى الطرف

في retrieval، يتعلم two-tower تمثيل المستخدم والعنصر ثم يسترجع أقرب العناصر بواسطة فهرس متجهي. هذا يسمح بتحديث المستخدم سريعًا، لكنه يعني أن تمثيل عنصر جديد يحتاج استراتيجية cold-start. Ranking يستقبل مجموعة مرشحين ويستخدم ميزات أغنى مثل السياق والحداثة والتوفر. قيّم recall@K للاسترجاع وNDCG أو مقياس أعمال للترتيب، مع تقسيم زمني يحاكي لحظة التوصية.

راقب كذلك تنوع القائمة، تكرار عرض المنتج، والآثار التجارية أو العدالة بين البائعين. تحسين النقر قد يزيد توصيات مثيرة لكن منخفضة الجودة. التجربة online تتطلب توزيعًا عشوائيًا سليماً ومقاييس guardrail، لا مجرد مقارنة offline.

22.9 الرسم البياني وتمرير الرسائل

في message passing، تجمع كل عقدة سمات الجيران ثم تحدث تمثيلها بدالة تتعلم. بعد طبقتين يصل التأثير إلى الجيران من درجتين. Neighbor sampling يحد انفجار عدد الجيران في الرسوم الكثيفة، لكنه يضيف عشوائية ويحتاج تثبيت seed للتقييم. heterogenous graph يتضمن أنواع عقد أو حواف مختلفة، فينبغي أن يحافظ

التحويل على نوع العلاقة واتجاهها.

مع زيادة العمق قد تتشابه embeddings العقد (over-smoothing)، ويؤدي تسرب الحواف أو تسميات الجيران إلى تقييم متفائل. حدد هدفك: node classification أم link prediction أم توصية. قسم الحواف زمنيًا إن كان التنبؤ بعلاقات مستقبلية، واحجب الهدف من message passing أثناء التقييم. تحقق من أن feature preprocessing لا تستخدم معلومات لم تكن متاحة عند القرار.

22.10 التعميم على فئات وعناصر جديدة

خطط لاختبار user/item cold-start صراحة. المحتوى الوصفي للعنصر أو قواعد fallback قد تكون أفضل من embedding عشوائي. في جدول مستمر، راقب فئات OOV والانحراف بعد إطلاق منتج جديد. عند label imbalance، افحص recall لكل فئة وتكلفة false positive؛ لا ترفع threshold دون حساب أثره على قائمة الترتيب.

22.11 شجرة قرار مقابل شبكة عميقة

جدول الأعمال غالبًا يضم عددًا متوسطًا من الصفوف وأنواع ميزات مختلطة. في هذا النطاق قد تفوز gradient-boosted trees بجهد أقل. الشبكات مناسبة لدمج تمثيلات متعددة أو scale كبير. ابدأ بـ majority/median ثم نموذج خطي أو شجري، واحفظ split ثابتًا. لا تستخدم embedding لمجرد أن عمودًا فئوي؛ قارن one-hot والشجرة والتضمين على الجودة وسرعة التحديث وcold-start.

22.12 منع تسرب السمات

نفذ `adapt` لطبقة التطبيع والترميز على التدريب فقط، ثم أعد استخدامها دون إعادة تعلم الإحصاءات على test. أي ميزة محسوبة من بعد وقت القرار تسرب المستقبل، حتى لو لم يدخل label نفسه. أضف data dictionary يذكر وقت توفر كل عمود ومصدره ووحدته، واختبر as-of joins. فحص التسرب يجب أن يكون دلاليًا لا تقنيًا فقط؛ Tensor قد يمر بالشكل الصحيح ويحمل معلومة غير متاحة وقت التنبؤ.

22.13 تجربة توصية من التفاعل إلى العرض

حدد impression clickg purchaseg. غياب click بعد عدم عرض المنتج ليس negative label. أنشئ negatives من مجموعة العناصر التي كانت متاحة فعلًا للمستخدم في ذلك الوقت. افصل البيانات زمنيًا واحجز مستخدمين أو عناصر جددًا إذا كان cold-start حالة منتج. sampled evaluation قد ترفع جودة الترتيب؛ اشرح قائمة

المرشحين وطريقة أخذ العينات.

في online A/B، حدد وحدة randomization وتجنب نقل المستخدم بين المجموعتين. قس guardrails مثل التنوع والكمون والمخزون إلى جانب click-through. نظام يوصي بالعناصر الأكثر رواجًا قد يخلق popularity bias ويقلل فرص عناصر جديدة؛ سجل سياسة الترتيب التي ولّدت البيانات.

22.14 بناء GNN قابلة للتوسع

حدد اتجاه الحواف وأنواع العقد والعلاقات ووقت حدوثها. في message passing اجمع جيرانًا ثم حدث تمثيل العقدة، مع normalization أو residual وفق العمارة. الرسم الكبير قد لا يناسب full-batch؛ neighbor sampling يقلل الذاكرة لكنه يغير توزيع التدريب. اختبر link predictiong node classification ببروتوكولات فصل مختلفة.

في تقييم link prediction، احجب الحافة الهدف عن message passing. إذا كانت المهمة تتنبأ بعلاقات مستقبلية، اقسم زمنيًا حتى لا تكشف حواف المستقبل. راقب over-smoothing مع العمق، وfan-outg الذي قد ينمو أسيًا. لا تقارن نموذجين على رسمين أو فهرسين مختلفين دون توثيق.

22.15 تغطية العناصر الجديدة والنتائج

اختبر فئات OOV ومستخدمًا جديدًا وعنصرًا قليل التفاعل. المحتوى الوصفي أو fallback rule قد يكون أفضل من embedding غير مدرب. أبلغ missingness وعدد عقد وحواف ونسبة OOV، ومقاييس الشرائح. الخطة الإنتاجية تتضمن مسارًا واضحًا عندما يتعذر استرجاع embedding أو ينقطع مخزن التوصية.

22.16 خدمة الميزات والفهارس

أعد بناء فهرس العناصر بنفس طريقة تطبيع embedding المستخدمة وقت التدريب. عند استبدال تمثيلات العنصر، حدّث الفهرس كنسخة متوافقة أو قد تقارن user vectors بإصدارات قديمة. خطط لتحديث incremental أو rebuild كامل، وراقب عمر الفهرس ومعدل العناصر المفقودة.

22.17 مراقبة عرض التوصية

قِس نسبة العناصر الجديدة في القائمة، وتكرار عرض العنصر، وتغطية البائعين، ومعدل البيانات الوصفية المفقودة. إذا انخفض التنوع، افحص popularity feedback loop قبل زيادة وزن diversity. شغل تجارب controlled، لأن تغيير ترتيب المنتجات يؤثر في البيانات التي ستسجل لاحقًا.

22.18 واجهة fallback واضحة

حدد ما تعرضه الخدمة عند عدم وجود embedding أو فهرس قديم أو user بلا تاريخ. يمكن تقديم popular items ضمن المنطقة أو اقتراح عام، مع وسم المصدر حتى تراقب fallback rate. لا تخلط إخفاق retrieval الصامت بتوصية طبيعية.

22.19 Feature store ومزامنة الزمن

اجعل feature pipeline واحدًا للتدريب والخدمة أو اختبر التكافؤ بينهما. سجل وقت feature materialization ووقت توفر القيمة؛ join غير as-of قد يجعل التدريب يرى تحديثًا لم يكن موجودًا عند التنبؤ. عند تحديث catalog، اكتب timestamp أو version للفهرس، وأعد حساب embedding للعناصر التي تغير وصفها.

22.20 حذف المستخدم والخصوصية

أنظمة التوصية تجمع تفاعلات سلوكية. عرف سياسة retention وحذف حساب يطلب إزالة بياناته، وحدد كيف ينعكس ذلك على embeddings والفهرس والنسخ الاحتياطية. طبّق وصولًا محدودًا للسجلات، ولا تسجل تاريخًا دقيقًا إن لم يكن ضروريًا. تحليل cold-start يجب ألا يكشف بيانات مستخدمين آخرين.

22.21 سجلات الدلالة وجودة الوسم

العمود نفسه قد يعني «قيمة حالية» في نظام و«قيمة شهر سابق» في نظام آخر. احتفظ data dictionary ووحدة القياس والمصدر وتوقيت توفر القيمة. افحص label generation ومعدلات missingness بحسب المصدر. إذا كان target مشتقًا من قاعدة أعمال، طبق النسخة نفسها عند التدريب والتقييم ولا تستنتج الحقيقة من تنبؤ قديم.

22.22 تحديث نظام التوصية

حدد cadence لتحديث embeddings والفهرس وتحقق من اتساق المعرفات. إذا أضيف عنصر أثناء rebuild، احتفظ queue للتحديثات أو أوقف النشر حتى يكتمل الفهرس. استخدم index version مع model version، واختبر retrieval recall بعد التحديث. راقب زمن البحث والتحديث، لأن فهرسًا عالي الجودة لكنه stale قد يعرض عناصر منتهية.

22.23 الاختبار بحسب السيناريو

اعرض النتائج للمستخدم العائد والجديد، والعنصر الشائع والنادر، والمنطقة واسعة البيانات وقليلتها. اختبر كتالوجًا صغيرًا حيث يمكن التحقق من كل المرشحين، ثم اختبر scale حقيقيًا. قس أثر قواعد السلامة مثل عدم التوصية بمخزون غير متاح. هذه الشروط التشغيلية جزء من ranking system وليست تعديلًا جانبيًا.

22.24 أخطاء شائعة

> ⚠ **تحذير**
>
> طابق معرفات الفئات وقت التدريب والاستدلال، وحدد رمزًا للفئات الجديدة. [DANGER!] لا تقسّم رسمًا عشوائيًا إذا كانت الحواف تنقل تسميات الاختبار إلى جيران التدريب.

رمز للفئات الجديدة

حدد مسار OOV قبل تحويل الفئات إلى معرفات حتى لا ينهار الاستدلال عند إدخال قيمة لم تظهر في التدريب:

```python
lookup = keras.layers.StringLookup(mask_token=None, num_oov_indices=1)
lookup.adapt(train_categories)
ids = lookup(["known-category", "new-category"])
print(ids.numpy())
```

22.25 تمارين

1. صمم تطبيعًا لعمود عددّي. (المتوقع: ملاءمة على التدريب فقط.)
2. عالج فئة لم تظهر سابقًا. (المتوقع: OOV أو مسار احتياطي.)
3. اشرح retrieval مقابل ranking. (المتوقع: تقليل مرشحين ثم ترتيبهم.)
4. اختر مقياسًا لتوصية top-k. (المتوقع: Recall@K أو NDCG بحسب المرحلة.)
5. ارسم message passing لجولة واحدة. (المتوقع: تجميع الجيران ثم تحديث العقدة.)
6. افحص تسرب الحواف. (المتوقع: تقسيم يمنع تسريب الهدف أو المستقبل.)

22.26 ملخص

- للجدول معالجة متخصصة، وخط أساس شجري ضروري للمقارنة.
- أنظمة التوصية تفصل الاسترجاع عن الترتيب.
- GNN تتعلم من الجوار، ويجب تصميم تقسيم يمنع تسرب الروابط والوسوم.

الجزء الرابع · الاحتراف والإنتاج

الفصل الثالث والعشرون

23

الأخلاق والخصوصية والتفسير والأمان

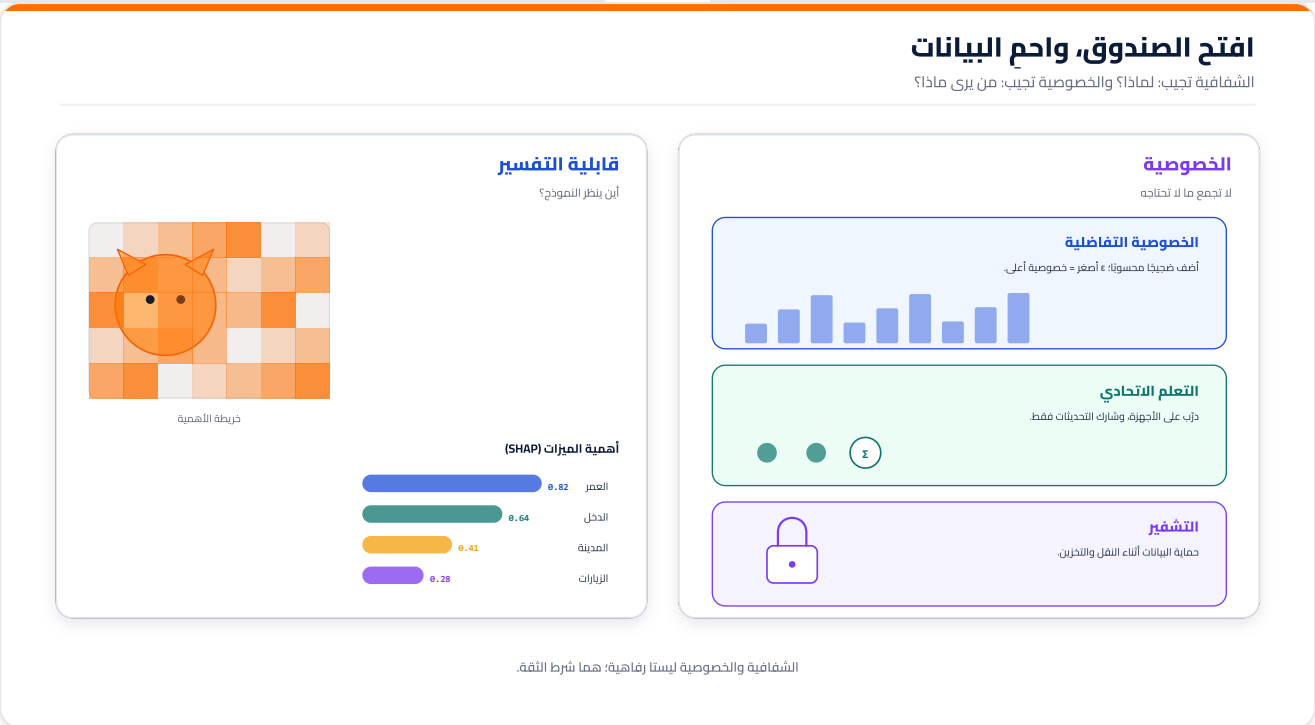
> **حكاية الفصل**
>
> رتّب نموذج توظيف المرشحين، ثم اكتشف الفريق أن نتائج مجموعة صغيرة أقل بكثير رغم تشابه المؤهلات. لم تكن الدقة الكلية كافية لإظهار الضرر. كيف نراجع عدالة القرار وخصوصيته وأمانه قبل منح النموذج سلطة؟

23.1 أهداف الفصل

- [ ] اختيار شرائح ومقاييس عدالة مناسبة للقرار.
- [ ] شرح الخصوصية التفاضلية والتعلم الاتحادي وحدودهما.
- [ ] استخدام Integrated Gradients وSHAP وGrad-CAM بحذر.
- [ ] وصف هجمات الخصومة والحوكمة.

23.2 العدالة سؤال عن القرار والسياق

ابدأ بتحديد المتأثرين، والقرار، والضرر الذي نحاول منعه. قارن معدلات القبول والخطأ والمعايرة بين الشرائح ذات الصلة، مع الإفصاح عن حجم العينة وعدم اليقين. قد تتعارض معايير العدالة رياضيًا عند اختلاف معدلات الأساس؛ لذلك يحتاج اختيار المقياس إلى سياسة معلنة ومراجعة أصحاب المصلحة، لا تعديل عتبة آليًا.



شكل 23-1 التفسير والخصوصية والأمان

الخصوصية التفاضلية تضيف ضوضاء محسوبة إلى التعلم أو النتائج لتحد من أثر سجل واحد، وتوازن بين الخصوصية والمنفعة عبر ميزانية مثل `ε`. التعلم الاتحادي يبقي البيانات محليًا ويجمع تحديثات، لكنه لا يضمن الخصوصية تلقائيًا؛ يمكن استنتاج معلومات من التدرجات، لذا قد يلزم secure aggregation وDP.

23.3 التفسير أداة فحص

$$Risk = Impact \times Likelihood$$

هذا تقدير أولي نوعي: الأثر يصف حجم الضرر، والاحتمال يصف فرصة وقوعه. لا تختزل الحقوق أو العدالة إلى حاصل ضرب واحد؛ استخدمه لترتيب مراجعة المخاطر فقط.

Integrated Gradients يقارن مدخلًا بخط أساس ويدمج التدرجات على المسار. SHAP يوزع الإسهام وفق قيمة Shapley تقريبية، وGrad-CAM يبرز مناطق صورة مرتبطة بدرجة فئة. كل تفسير يعتمد على اختيار خط الأساس والتمثيل والنموذج، ولا يثبت السببية. افحص ثبات الشرح عند تغييرات صغيرة واطلب تفسيرًا من خبراء المجال.

الخطر	تجربة فحص	استجابة محتملة
تحيز شريحة	معدل خطأ ومعايرة حسب المجموعة	جمع بيانات ومراجعة قرار
استدلال عضوية	اختبار تمييز بيانات التدريب	تقليل الإفراط وخصوصية مدروسة
FGSM	اضطراب متدرج صغير	تقييم متانة ودفاع ملائم
تسميم بيانات	مراجعة المصدر والشذوذ	تتبع المنشأ وفصل الصلاحيات

23.4 أمان الخصومة والحوكمة

FGSM يضيف اضطرابًا باتجاه تدرج الخسارة، ويكشف حساسية محلية لا يغطي جميع الهجمات الواقعية. التسميم يغير بيانات التدريب، والتلاعب بالاستدلال يستهدف مدخل الخدمة. وثق بطاقة النموذج: الاستخدام المقصود، البيانات، القيود، المقاييس، والشرائح غير المختبرة. ضمّن الإنسان في القرار عند الأثر الكبير وحدد آلية اعتراض ومراجعة.

23.5 حالة واقعية: نموذج توظيف

يفحص الفريق معدلات الدعوة للمقابلة والرفض الخاطئ لكل مجموعة، ويختبر تأثير حذف السمات الحساسة مع الانتباه إلى المتغيرات البديلة. يراجع أصحاب القرار عينة من الحالات ويقارنون الترتيب بمقيّمين بشريين. لا يصدر القرار وحده؛ يوضح النظام سبب التوصية وحدودها ويحفظ سجلًا محدودًا الوصول.

23.6 أمثلة تنفيذية إضافية

الأخطاء بحسب المجموعة

```python
for group in groups:
    mask = sensitive_group == group
    print(group, (y_pred[mask] ≠ y_true[mask]).mean(), mask.sum())
```

التنبؤion stability

```python
score_a = model(x, training=False)
score_b = model(x_perturbed, training=False)
delta = tf.reduce_max(tf.abs(score_a - score_b))
print(float(delta))
```

23.7 تقييم عدالة القرار خطوةً خطوة

ابدأ بتحديد تعريف كل مجموعة ومن له حق الوصول إليها، ثم عرّف قرار النموذج ومن يتأثر به. اجمع confusion matrix لكل شريحة ومعدلات القبول والرفض، واحسب فواصل ثقة أو أظهر أحجام العينات. اختلاف recall بين شريحتين لا يفسر وحده سبب الفرق، لكنه يوجه التحقيق إلى جودة البيانات والوسم والظروف التشغيلية. تجنب إظهار استنتاجات قوية من مجموعات صغيرة جدًا.

لا توجد قاعدة رياضية واحدة تحقق كل مفاهيم العدالة عند اختلاف معدلات الأساس. اختيار parity أو equalized odds أو calibration مفاضلة سياسة وسياق وقانون. افصل الاختبار عن بيانات اتخاذ القرار، واشرك أصحاب المصلحة المتأثرين، وحدد مسار اعتراض بشري. لا تستخدم السمات الحساسة أو تحذفها تلقائيًا دون فحص قانوني وأخلاقي للسياق.

23.8 الخصوصية التفاضلية والتعلم الاتحادي

آلية DP تقيد أثر مساهمة فرد واحد ضمن مقياس خصوصية، غالبًا مع epsilon وdeltag. الخصوصية تنخفض عادة حين تتكرر الاستعلامات أو التدريب دون حساب ميزانية مجمعة. في تدريب DP-SGD تقص التدرجات لكل مثال ثم تضيف ضوضاء؛ clipping norm وحجم الضوضاء والدفعة تؤثر في الخصوصية والمنفعة. استخدم محاسبًا موثوقًا بدل وصف إعداد بأنه «خاص» لمجرد إضافة ضوضاء.

التعلم الاتحادي يبقي السجلات على الأجهزة ويجمع تحديثات محلية، لكنه لا يخفي التحديثات وحده. secure aggregation يقلل رؤية تحديث فرد، وDP قد يضيف ضمانًا إضافيًا. عدم تساوي العملاء أو انقطاعهم وتغير توزيع الأجهزة تحديات جودة وتشغيل. قس النموذج على الجهات الأقل بيانات، لا متوسطًا فقط.

23.9 التفسير والتحقق من متانته

Integrated Gradients يختار خط أساس ويكامل التدرج من baseline إلى المدخل. SHAP يحاول توزيع إسهام السمات ويعتمد على تعريف مجموعة الخلفية. Grad-CAM يربط درجات التصنيف بمناطق مكانية في CNN. كل واحدة إجابة لسؤال مختلف، وقد يتغير التفسير باختيار baseline أو background. اختبر sanity checks: هل تتغير الإشارات عندما تبدل أوزان النموذج عشوائيًا؟ وهل تظل الخريطة نفسها عند إزالة السمة؟ لا تعرض heatmap كدليل سببي.

23.10 التهديدات: FGSM والتسميم وإساءة الاستعمال

FGSM اضطراب خطوة واحدة باتجاه sign gradient، ويفيد اختبار حساسية مبدئيًا لكنه لا يمثل كل خصم. هجمات أقوى قد تكون متعددة الخطوات أو تستهدف واجهة الإدخال. تسميم البيانات يغير أمثلة التدريب أو وسومها؛ راقب مصدر كل ملف، وصلاحيات تعديل البيانات، وشذوذات labels. threat model يحدد خصمًا وقدراته وأهدافه، ثم اختبارات تناسبها.

23.11 بطاقة نموذج ومراجعة إطلاق

سجل الغرض المتوقع والاستخدامات المحظورة، مصادر البيانات وتراخيصها، تقسيم التقييم، الأداء الكلي وحسب الشرائح، القيود المعروفة، متطلبات الإشراف البشري، وقنوات الإبلاغ. اربط البطاقة بإصدار النموذج لا باسم مشروع عام. بعد الإطلاق، راقب الانحراف والشكاوى وطلبات الاعتراض، وحدد من يوقف النظام ومتى.

23.12 دراسة تدقيق: نموذج توظيف

حدد أن النموذج يساعد لجنة على ترتيب الطلبات ولا يتخذ رفضًا نهائيًا بمفرده. راجع مصدر labels؛ قد تعكس قرارات موظفين سابقين لا جودة المرشح. قارن معدلات الخطأ والمعايرة بين الشرائح، واعرض حجم كل شريحة وفاصل عدم اليقين. افحص مجموعات تقاطعية بعناية؛ المتوسط العام قد يخفي ضررًا في فئة صغيرة.

افحص السمات الوكيلة مثل الرمز البريدي أو نمط الانقطاع، وقارن السيناريو مع السمات وحذفها، لكن لا تعتبر حذف عمود حساس علاجًا كافيًا. اطلب مراجعة أصحاب مصلحة، وأوضح مسار الاعتراض والمراجعة البشرية، واحفظ سجلًا محدود الوصول. لا تعلن أن النموذج عادل لأن قيمة metric واحدة متقاربة.

23.13 الخصوصية في جمع البيانات والتسجيل

اجمع الحد الأدنى، وحدد الغرض ومدة الحذف والصلاحيات. لا تسجل النصوص الشخصية كاملة لمجرد تصحيح خطأ إن كان نوع الحقل أو checksum كافيًا. DP-SGD يقص التدرجات لكل مثال ويضيف ضوضاء: clipping norm وحجم noise والدفعة تدخل في حساب الخصوصية والمنفعة. استخدم محاسبًا صحيحًا واعرض epsilon/delta مع الفرضيات، ولا تصف التدريب بأنه private بلا ضمان قابل للمراجعة.

التعلم الاتحادي يبقي السجلات محلية لكنه لا يخفي تحديثات الأوزان تلقائيًا. secure aggregation يحد من رؤية تحديث جهاز واحد، وDP يمكن أن يضيف حدًا رياضيًا. تفاوت العملاء وانقطاعهم والتحول بين الأجهزة تحديات يجب أن تظهر في evaluation.

23.14 شرح النموذج وتدخل المشغل

Integrated Gradients يعتمد baseline، وSHAP يعتمد background وتعريف التعاون بين السمات، وGrad-CAM ينتج منطقة مكانية. هذه طرق مختلفة للإجابة عن أسئلة مختلفة وليست أدلة سببية. افحص ثبات الشرح بتغيير baseline، وقارن تفسيرًا بأمثلة حذف سمة. في واجهة الإنسان، لا تجعل explanation يرسخ قرارًا خاطئًا؛ اختبر ما إذا كان المشغل يراجع الدليل المستقل.

23.15 نموذج تهديد مختصر

حدد الأصول (البيانات والأوزان وAPI)، والخصم وقدراته، وحدود النظام، والنتيجة التي يخشاها الفريق. اختبر poisoning عند ingestion، model extraction عبر API، adversarial perturbation للصورة، وعضوية التدريب إذا كانت حساسية البيانات عالية. احمِ مصدر البيانات والصلاحيات والمفاتيح، وحدد rate limits وسجلات لرصد السلوك. الدفاع يتبع threat model، وليس إضافة قائمة كلمات أمنية.

23.16 دورة الحوكمة بعد الإطلاق

قبل التجربة، حدد المالك والمراجع المستقل والفئات المتأثرة. قبل النشر، أرفق بطاقة نموذج ومخاطر وحدود وقناة اعتراض. بعد النشر، راقب الشكاوى والشرائح والانجراف وحوادث الأمن. حدد من يستطيع إيقاف النظام، وكيف تحافظ على مراجعة بشرية أثناء الإيقاف. عقب حادث، احفظ أدلة كافية للتدقيق دون جمع زائد، وراجع الجذر والتعويض والتواصل.

23.17 مراجعة إطلاق موثقة

احتفظ بسجل القرار: المخاطر التي نوقشت، الشرائح المختبرة، الفحوصات غير المنفذة، صاحب الموافقة، وتاريخ المراجعة التالية. عند تغير البيانات أو الغرض، أعد تقييم البطاقة. الحوكمة عملية متجددة لا صفحة disclaimer تثبت لمرة واحدة.

23.18 إدارة حادث أو اعتراض

عند اعتراض قرار، اربطه بإصدار النموذج والبيانات المتاحة وقت التنبؤ، وامنح مراجعًا صلاحية إعادة النظر. أوقف استخدام السمة أو المسار إن ظهر ضرر جسيم، واحفظ الحد الأدنى اللازم للتحقيق. حدّث التقييم وبطاقة النموذج بعد الإصلاح، وأبلغ أصحاب المصلحة بالتغيير.

23.19 مراجعة مستقلة للتغييرات الكبيرة

عند تغيير مصدر البيانات أو الغرض أو threshold، أعد مراجعة الخصوصية والشرائح والأمن. احتفظ بسجل سبب التغيير والجهة المخولة. إشراك مراجع مستقل قد يكشف تضارب حوافز الفريق الذي بنى النموذج.

23.20 مقاييس متانة وخصوصية مفيدة

اختبر حساسية الأداء لتغيير سمة أو تشويش صغير في صورة أو ترتيب tokens. في تجربة privacy، اختبر membership inference على مجموعة مضبوطة، وفسر حدود الاختبار: نجاح هجوم واحد لا يثبت كل اختراق، وفشله لا يضمن خصوصية كاملة. استخدم هذه النتائج لتقليل حفظ التدريب أو ضبط DP حسب مستوى المخاطر.

23.21 التوثيق بلغة المستخدم

أخبر الشخص أن قرارًا آليًا ساهم، وما البيانات العامة التي أثرت، وكيف يمكن الاعتراض أو طلب مراجعة. تجنب شرحًا يوحي بوجود سبب حتمي إذا كان التفسير تقريبيًا محليًا. اعرض حدود الثقة بلغة بسيطة واستخدم رقمًا فقط عندما يكون معايرًا ومفهومًا.

23.22 مصفوفة مخاطر قبل الجمع والنشر

قيّم الاحتمال والأثر والتعرض: هل يمكن أن ينكشف سجل؟ هل يغير القرار أهلية الشخص؟ هل يستطيع الفرد الاعتراض؟ صنف إجراءات تخفيف قابلة للتحقق، مثل فصل الهوية عن السجل، وتقييد access، ومراجعة شرائح محددة، ثم عين مالكًا وموعدًا لإعادة النظر. احتفظ بالمخاطر المتبقية بدل وصف المشروع بأنه خالٍ من المخاطر.

23.23 تصميم تفسير يتجنب الإفراط في الثقة

اعرض تفسيرًا بوصفه إشارة مساعدة، وأظهر ميزات أو مناطق مؤثرة مع لغة احتمالية. اختبر المتانة بتغيير baseline والميزات، وقارن بما يراه الخبير. إذا تغير التفسير جذريًا مع perturbation صغيرة، لا تقدمه كسبب مستقر. اشرك مستخدمين فعليين في اختبار الواجهة وتأكد أن الرسم لا يجعلهم يتجاهلون معلومات مخالفة.

23.24 موازنة الخصوصية والفائدة

قبل تفعيل DP، حدد adjacency notion: هل الفرق سجل واحد أم مستخدم كامل؟ التجميع على عدة أحداث للمستخدم يتطلب privacy accounting على مستوى المستخدم إن كان الخطر متعلقًا به. قس utility عند budget مختارة على شرائح، وسجل آلية المحاسب وإصدار المكتبة. لا تكتب epsilon دون delta وعدد خطوات وحجم sample rate.

23.25 أخطاء شائعة

> ⚠ **تحذير**
>
> تفسير محلي مقنع لا يثبت أن النموذج عادل أو سببي؛ افحص النتائج على شرائح وسياقات مختلفة.
>
> [DANGER!] حذف عمود حساس لا يمنع الوكلاء (proxies) من حمل معلوماته.

حساب فجوة معدلات الخطأ

ابدأ بعرض حجم كل مجموعة مع المقياس؛ الفجوة بلا حجم عينة قد توحي بيقين غير موجود:

```python
group_rates = {
    group: ((y_pred[group_mask] ≠ y_true[group_mask]).mean(), int(group_mask.sum()))
    for group, group_mask in masks.items()
}
print(group_rates)
```

23.26 تمارين

1. اختر مقياسين لقرار توظيف وعلّل. (المتوقع: ربط بالمخاطر وحجم الشريحة.)
2. وضح ميزانية الخصوصية. (المتوقع: مفاضلة منفعة/خصوصية لا وعد مطلق.)
3. فسّر حدود التعلم الاتحادي. (المتوقع: البيانات محلية لكن التحديثات قد تسرب.)
4. قارن Grad-CAM وSHAP. (المتوقع: تفسير بصري مكاني مقابل إسهام ميزات.)
5. صمم اختبار FGSM. (المتوقع: مقياس متانة عبر اضطراب معلن.)
6. اكتب عناصر بطاقة نموذج. (المتوقع: غرض وبيانات وحدود ومقاييس.)

23.27 ملخص

- العدالة تتعلق بتوزيع الأثر والقرار، ولا تختزل بمقياس واحد.
- تقنيات الخصوصية والتفسير لها شروط وحدود.
- الأمان والحوكمة مستمران من مصدر البيانات حتى تشغيل الخدمة والاعتراض.

الجزء الرابع · الاحتراف والإنتاج

الفصل الرابع والعشرون

24

مشاريع ختامية وخارطة الطريق

> **حكاية الفصل**
>
> في مقابلة عمل، عرض مهندس شاب أربعة مشاريع: مصنّف صور بدقة 98%، ونموذج تنبؤ بالطلب، ومحلّل مشاعر عربي، وكاشف احتيال. أعجبت اللجنة بالأرقام، ثم سأله رئيس الفريق: «ما خط الأساس الذي تغلبت عليه؟ وكيف تعرف أن النموذج ما زال صالحًا بعد ستة أشهر؟». صمت الشاب؛ كان قد درّب نماذج، لكنه لم يبنِ أنظمة. تذكّر أنه لم يكتب يومًا ما يحدث حين تصل صورة من كاميرا جديدة، أو يتغيّر سلوك المشترين بعد عرض ترويجي. فما الفرق بين مشروع يعمل على دفتر، ومشروع يعيش في الإنتاج؟

24.1 أهداف الفصل

- [ ] تحويل مشكلة عمل غامضة إلى تعريف واضح للقرار والبيانات والمقياس.
- [ ] بناء أربعة مشاريع ختامية متكاملة: تصنيف صور، تنبؤ طلب، مشاعر عربي، كشف احتيال.
- [ ] اختيار خط أساس وتقسيم بيانات وتقييم يعكس تكلفة الخطأ لا سهولة الرقم.
- [ ] نشر النموذج ومراقبته وتوثيق حدوده ومخاطره.
- [ ] اتباع خارطة تعلّم عملية وبناء معرض أعمال يقنع المهندسين لا الخوارزميات.

24.2 ما الذي يفرّق المشروع الختامي عن دفتر تجارب؟

المشروع الختامي ليس دفترًا أطول، ولا شبكة أعمق. إنه أصغر نظام كامل يمكن الدفاع عنه أمام مهندس متشكّك: مشكلة محددة، وبيانات تعرف مصدرها وحدودها، وخط أساس، وتدريب قابل للتكرار، وتقييم يقيس ما يهم، ونشر، ومراقبة. إن غاب عنصر واحد، فما لديك تجربة، لا مشروع.

قبل أول سطر كود، أجب عن أربعة أسئلة: ما القرار الذي سيغيّره التنبؤ؟ ما المعلومات المتاحة فعلًا لحظة التنبؤ؟ ما تكلفة الخطأ من كل نوع؟ وكيف سنكتشف التدهور قبل المستخدم؟ هذه الأسئلة تحدد المقياس، والمقياس يحدد كل شيء بعده.

ولأن القرار هو الغاية، فلا تبدأ بالدقة، بل بالتكلفة:

$$\text{Cost}(\tau) = C_{FN}\, N_{FN}(\tau) + C_{FP}\, N_{FP}(\tau)$$

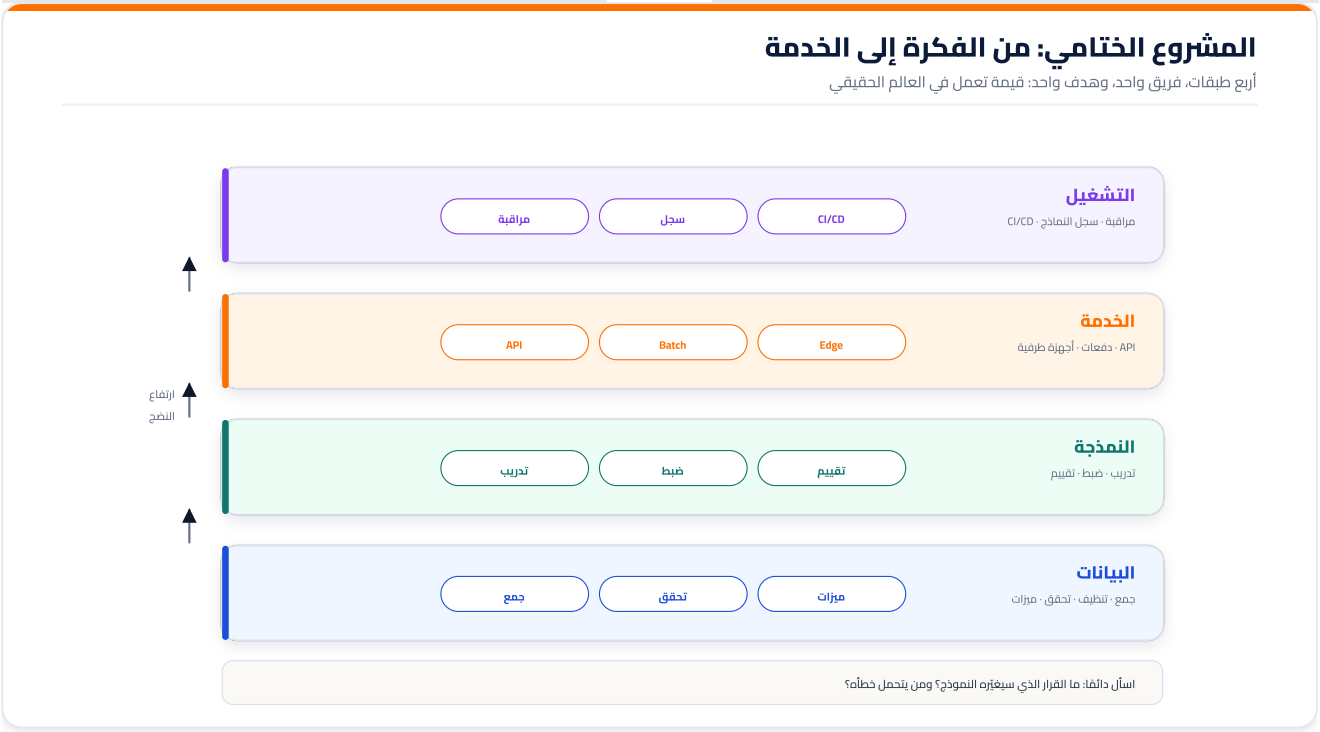
حيث τ عتبة القرار التي نحوّل عندها الاحتمال إلى فعل، و$N_{FN}(\tau)$ عدد الحالات الموجبة التي فاتت عند تلك العتبة، و$N_{FP}(\tau)$ عدد الإنذارات الكاذبة، وC_{FN} تكلفة تفويت حالة موجبة، وC_{FP} تكلفة الإنذار الكاذب. يجب أن تكون التكاليف بالوحدة نفسها: ريال، أو ساعة مراجعة، أو حالة متضررة.

وحين لا تكون التكاليف معروفة، استخدم مقياسًا يرجّح أحد النوعين:

$$F_\beta = (1+\beta^2)\frac{P \cdot R}{\beta^2 P + R}$$

حيث P الدقة (Precision)، وR الاستدعاء (Recall)، وβ وزن يحدد ميل المقياس. عندما يكون $\beta > 1$ يُعاقب المقياسُ تفويتَ الحالات الموجبة أكثر، وعندما يكون $\beta < 1$ يُعاقب الإنذارات الكاذبة أكثر. في كشف الاحتيال، قد يكون $\beta = 2$ أو أعلى؛ لأن تفويت عملية احتيال أغلى من مراجعة عملية سليمة.

المشروع	القرار	البيانات	البنية المرجعية	المقياس الأساسي	الخطر الأول
تصنيف صور	هل القطعة معيبة؟	صور خط الإنتاج، ثلاث فئات	MobileNetV2 + نقل تعلّم	استدعاء العيوب عند دقة 0.90	انحراف الكاميرا والإضاءة
تنبؤ طلب	كم نطلب الأسبوع القادم؟	مبيعات يومية، تقويم، عروض	خط أساس موسمي ثم GRU	WAPE على الأفق كاملًا	طلب مقيّد بنفاد المخزون
مشاعر عربي	هل التقييم سلبي؟	مراجعات متعددة اللهجات	TextVectorization + BiLSTM أو BERT	F1 الماكرو	انحراف اللهجة والسخرية
كشف احتيال	هل نوقف العملية الآن؟	معاملات وتصنيف متأخر	MLP أو GBDT + معايرة	استدعاء عند دقة ثابتة	تسريب ميزات وتغيّر عدائي



شكل 24-1 المشروع الختامي: من الفكرة إلى الخدمة

الفكرة الجامعة أن كل مشروع يختبر قدرة مختلفة: الأول يختبر التعميم البصري، والثاني يختبر احترام الزمن، والثالث يختبر هشاشة اللغة، والرابع يختبر اتخاذ قرار مكلف تحت عدم التوازن. لكن دورة العمل واحدة: بيانات، خط أساس، نموذج، تقييم، نشر، مراقبة.

24.3 تصنيف الصور: حين تكون الدقة أسهل ما في المشروع

المشكلة والبيانات. في خط إنتاج للمعادن، يفحص المفتّشون كل قطعة بحثًا عن خدش أو انبعاج. نريد مصنّفًا لثلاث فئات: سليم، خدش، انبعاج. لدينا اثنا عشر ألف صورة من أربع كاميرات، والفئة المعيبة نحو 6% فقط. الخطأ في البيانات ليس عشوائيًا: بعض الصور الصعبة وُسمت على عجل، وبعض الخدوش الدقيقة لا تظهر إلا تحت إضاءة معينة. لذلك نكتب بطاقة بيانات تذكر الكاميرات والإضاءة والدقة وزمن التصوير، ونحجز مجموعة اختبار من خط إنتاج لم يظهر في التدريب: التقسيم العشوائي هنا يجمّل الرقم ويكذب.

البنية والتدريب. نبدأ بنقل تعلّم من MobileNetV2: نموذج صغير نسبيًا، سريع على المعالجات الطرفية، ومدرب على ImageNet. نجمّد القاعدة، وندرب رأسًا جديدًا، ثم نفكّ تجميد الكتل الأخيرة بمعدّل تعلّم منخفض. التكبير (Augmentation) يكون خفيفًا: دوران طفيف وتقريب، ولا نطبّق قلبًا أفقيًا إن كان اتجاه الخدش يحمل معنى. نستخدم أوزان فئات لأن العيوب نادرة، ونراقب استدعاء الفئة المعيبة لا الدقة الكلية.

التقييم والنشر. الأرقام المهمة: F1 الماكرو، واستدعاء العيوب عند دقة لا تقل عن 0.90، ومصفوفة الالتباس لكل كاميرا، وفحص Grad-CAM للتأكد من أن النموذج ينظر إلى العيب لا إلى خلفية المصنع. نضيف خيار الرفض (Reject): إذا كان أقصى احتمال أقل من عتبة، تُحوَّل الصورة إلى مفتّش بشري. بعد التصدير إلى SavedModel، نحوّل النموذج إلى LiteRT بضغط float16 أو int8 ونقيس الزمن على الجهاز الفعلي، لا على حاسوب التطوير.

المخاطر. الخطر الأكبر هو انحراف المجال: كاميرا جديدة أو إضاءة مختلفة تُسقط الدقة بصمت. يليه الاعتماد على إشارات مختصرة (خلفية أو لون حامل القطعة)، ثم ضجيج الوسم. في النسخة الطبية من المشروع، أضف موافقة مؤسسية وإخفاء هوية ومراجعة طبيب؛ لا تجعل النموذج يقرر وحده.

```python
import tensorflow as tf
import keras

keras.utils.set_random_seed(42)          # تكرار قدر الإمكان

IMG_SIZE = (224, 224)
BATCH = 32

# واختبر الأنابيب cache/prefetch خط بيانات بسيط: في الواقع أضف
train_ds = keras.utils.image_dataset_from_directory(
    "data/train", image_size=IMG_SIZE, batch_size=BATCH,
    label_mode="int", shuffle=True, seed=42)
val_ds = keras.utils.image_dataset_from_directory(
    "data/val", image_size=IMG_SIZE, batch_size=BATCH,
    label_mode="int", shuffle=False).prefetch(tf.data.AUTOTUNE)

# MobileNetV2 [1 ,1-] يتوقع قيمًا في المجال
preprocess = keras.layers.Rescaling(1 / 127.5, offset=-1)
augment = keras.Sequential([
    keras.layers.RandomFlip("horizontal"),
    keras.layers.RandomRotation(0.02),
    keras.layers.RandomZoom(0.1),
])

# المرحلة الأولى: قاعدة مجمّدة ورأس جديد
base = keras.applications.MobileNetV2(
    input_shape=(*IMG_SIZE, 3), include_top=False, weights="imagenet")
base.trainable = False

inputs = keras.Input(shape=(*IMG_SIZE, 3))
x = augment(inputs)
x = preprocess(x)
x = base(x, training=False)              # BatchNorm يبقى في وضع الاستدلال
x = keras.layers.GlobalAveragePooling2D()(x)
```

```
x = keras.layers.Dropout(0.2)(x)
outputs = keras.layers.Dense(3, activation="softmax")(x)
model = keras.Model(inputs, outputs)

model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
callbacks = [
    keras.callbacks.EarlyStopping(monitor="val_loss", patience=4,
                                  restore_best_weights=True),
    keras.callbacks.ModelCheckpoint("best.keras", save_best_only=True),
]
model.fit(train_ds, validation_data=val_ds, epochs=15,
          callbacks=callbacks)

# المرحلة الثانية: فكّ تجميد آخر الكتل بمعدّل تعلّم صغير
base.trainable = True
for layer in base.layers[:-30]:
    layer.trainable = False
model.compile(optimizer=keras.optimizers.Adam(1e-5),
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
model.fit(train_ds, validation_data=val_ds, epochs=8,
          callbacks=callbacks)

# على الجهاز الطرفي LiteRT للنشر؛ ثم يُحوَّل إلى SavedModel
model.export("artifacts/defect_classifier")
```

24.4 تنبؤ الطلب: الزمن لا يُخلط

المشكلة والبيانات. تريد سلسلة تجزئة معرفة الطلب على كل منتج في كل متجر خلال أربعة عشر يومًا، لتخطط الشراء والتوظيف. البيانات يومية على مدى سنتين: مبيعات، سعر، عروض، أعياد، ومخزون. هنا فخّان: الأول أن المبيعات ليست الطلب: حين ينفد المنتج، تتوقف المبيعات ويبدو الطلب منخفضًا، وهذا **رقابة على البيانات (Censoring)**. الثاني أن العروض المستقبلية قد تكون معروفة، أما نتائجها فليست.

الميزات والنوافذ. نبني نافذة إدخال من 60 يومًا وأفقًا من 14 يومًا. الميزات: قيم متأخرة (1، 7، 14، 28 يومًا)، متوسطات وانحرافات متحركة، يوم الأسبوع، الشهر، العيد، وجود عرض، السعر، وحدود فوريه لالتقاط الموسمية. كل ميزة متحركة تُزاح (Shift) قبل الاستخدام، وكل تجميع يُحسب من الماضي فقط. التقسيم زمني، مع فجوة تساوي طول الأفق بين التدريب والاختبار، حتى لا يتسرّب المستقبل إلى الماضي.

البنية والتقييم. لا تبدأ بـGRU. ابدأ بالخط الأساس الساذج الموسمي، ثم المتوسط المتحرك، ثم انحدار خطي بسيط. قارن الجميع بمقياس واحد: WAPE، أي $\mathrm{WAPE} = \sum_i |y_i - \hat{y}_i| / \sum_i |y_i|$، لأنه يجمع الأحجام المختلفة ويحتمل القيم القريبة من الصفر. بعد ذلك فقط جرّب نموذجًا عالميًا بـGRU أو TCN، مع تضمين للمتجر والمنتج. في البيانات الجدولية، غالبًا يتفوق GBDT؛ لا تستبدله بشبكة لمجرد أنها «أحدث».

النشر والمخاطر. التنبؤ يعمل كدفعة ليلية، ويُكتب في جدول تتساءل عنه أنظمة الشراء. راقب WAPE لكل شريحة، لا رقمًا واحدًا: منتج جديد، متجر صغير، موسم عيد. أعد التدريب شهريًا أو عند انحراف واضح. المخاطر: طلب مقيّد، عرض لم يظهر في التاريخ، تغيّر هيكلي، وتكلفة غير متناظرة بين النقص والزيادة. لا تُخفِ تدخّل البشر: اجعل التوقّع مدخلًا لقرار، لا قرارًا.

```python
import numpy as np
import keras

keras.utils.set_random_seed(42)

# بيانات اصطناعية: ثلاث سنوات يومية لمتجر واحد
rng = np.random.default_rng(42)
t = np.arange(3 * 365)
sales = (50 + 10 * np.sin(2 * np.pi * t / 7) + 0.01 * t
         + rng.normal(0, 3, size=t.shape)).astype("float32")

def make_windows(series, input_len=60, horizon=14):
    """يحوّل سلسلة إلى أزواج (ماضٍ، مستقبل) دون خلط الزمن."""
    X, y = [], []
    for i in range(len(series) - input_len - horizon + 1):
        X.append(series[i:i + input_len])
        y.append(series[i + input_len:i + input_len + horizon])
    return np.expand_dims(np.array(X), -1), np.array(y)

X, y = make_windows(sales)

# تقسيم زمني: آخر 20% للاختبار النهائي فقط
cut = int(len(X) * 0.8)
X_train, y_train = X[:cut], y[:cut]
X_test, y_test = X[cut:], y[cut:]

# خط الأساس: الأسبوع نفسه من الأسبوع الماضي
naive = X_test[:, -7:, 0]
mae_naive = np.mean(np.abs(naive - y_test[:, :7]))
print(f"MAE (أسبوع واحد) للخط الأساسي: {mae_naive:.2f}")

# التطبيع من التدريب فقط، لا من الاختبار
mu, sigma = X_train.mean(), X_train.std()
```

```
X_train_n = (X_train - mu) / sigma
X_test_n = (X_test - mu) / sigma

model = keras.Sequential([
    keras.Input(shape=(60, 1)),
    keras.layers.GRU(64),
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dense(14),              # أفق أربعة عشر يومًا
])
model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss="mae", metrics=["mae"])
model.fit(X_train_n, y_train, validation_split=0.1,
          epochs=20, batch_size=64, verbose=0)

mae_model = model.evaluate(X_test_n, y_test, verbose=0)[0]
print(f"MAE للنموذج: {mae_model:.2f}")
# قارن دائمًا بالخط الأساسي؛ التفوق ليس مضمونًا
```

◆ **ملاحظة**

حالة واقعية: في سلسلة تجزئة متوسطة الحجم، أضاف الفريق ميزة «الطلبات الملغاة» إلى نموذج التنبؤ، لأنها بدت مفيدة. على الورق انخفض WAPE إلى 12%، لكن النموذج في الإنتاج ساء إلى 31%. السبب أن الإلغاء لا يُعرف إلا بعد التسليم، فكانت الميزة تصف المستقبل لا الحاضر. الحل لم يكن نموذجًا أفضل، بل **عقد ميزات** يحدد لكل عمود لحظة توفره، واختبار آلي يحذف كل ميزة لا تتوفر عند التنبؤ. الدرس: التسريب لا يظهر كخطأ برمجي، بل كرقم جميل.

24.5 تحليل المشاعر العربي: اللغة أصعب من النموذج

المشكلة والبيانات. تطبيق تجارة إلكترونية يريد تصنيف مراجعات العملاء إلى سلبية أو محايدة أو إيجابية، لتوجيه الشكاوى بسرعة. لدينا خمسة وعشرون ألف مراجعة من متاجر التطبيقات ومنصات التواصل، تمزج الفصحى بالخليجية والمصرية والشامية، وبعضها عربي مكتوب بحروف لاتينية (Arabizi)، وبعضها سخرية خالصة. الوسم ليس حقيقة مقدسة: ثلاثة معلّقين، واتفاق متوسط، وحالات خلاف تُحسم بمراجعة رابعة. احسب اتفاق المعلّقين قبل أن تحلم بدقة 95%.

التنظيف والتمثيل. وحّد الألف (أ، إ، آ إلى ا) والياء (ى إلى ي)، وأزل التشكيل والتطويل، وحوّل الأرقام العربية الهندية إلى لاتينية، واحتفظ بعلامات الترقيم المؤثرة وبالرموز التعبيرية لأنها تحمل مشاعر. لا تحذف «لا» و«ما» و«مش» بحجة أنها كلمات وقوف؛ فهي تقلب المعنى. لا تجذّع النص العربي بعنف؛ الجذر يفقد السوابق واللواحق التي تحدد الزمن والنفي. أزل التكرار القريب قبل التقسيم، وإلا وجدت المراجعة نفسها في التدريب والاختبار.

البنية والتقييم. للنموذج الصغير: طبقة TextVectorization، ثم Embedding، ثم BiLSTM، ثم تصنيف. هذا نموذج يُدرَّب على CPU في دقائق، ومناسب كخط أساس قوي. للأداء الأعلى، استخدم BERT عربيًا عبر KerasHub: `keras_hub.models.BertClassifier.from_preset` مع preset عربي من قائمة النماذج، ثم اضبطه بمعدّل تعلّم صغير. قيّم بـF1 الماكرو، لأن الفئة المحايدة صعبة وقليلة، واعرض النتائج حسب اللهجة. أضف مراجعة بشرية لمئتي عينة قبل الثقة.

النشر والمخاطر. خدمة نصية تُطبّق التنظيف نفسه المستخدم في التدريب، ثم تعيد الفئة والاحتمال. خزّن النسخة والوقت، ولا تحتفظ بالنص الخام أطول من اللازم، واحجب المعلومات الشخصية. راقب انحراف المفردات: لهجة جديدة أو حملة سخرية تُغيّر التوزيع. المخاطر: انحراف اللهجة، ضجيج الوسم، السخرية، والتعميم على لهجة لم تُرَ. واحذر تحيّز النموذج: قد يعاقب لهجة بعينها بسبب توزيع بياناتها لا بسبب مشاعرها.

```python
import re
import numpy as np
import keras
from keras import layers

keras.utils.set_random_seed(42)

def normalize_arabic(text):
    """تنظيف محافظ: يوحّد الحروف ولا يمسّ النفي."""
    text = re.sub(r"[\u064B-\u0652]", "", text)   # التشكيل
    text = re.sub(r"ـ+", "", text)                # التطويل
    text = re.sub(r"[أإآ]", "ا", text)
    text = re.sub(r"ي$", "ى", text)
    text = re.sub(r"[٠-٩]", lambda m: str(ord(m.group()) - 0x0660), text)
    text = re.sub(r"http\S+|@\w+", " ", text)
    return re.sub(r"\s+", " ", text).strip()

texts = np.array([
    "الخدمة ممتازة والموظفون لطفاء جدًا",
    "لم يعجبني المنتج، التأخير غير مقبول",
    "جيد لكن السعر مرتفع قليلًا",
    "لا أنصح به أبدًا",
], dtype=object)
labels = np.array([2, 0, 1, 0], dtype="int32")

# في الواقع: adapt على مجموعة التدريب فقط، لا على الاختبار
vectorizer = layers.TextVectorization(
    max_tokens=30000, output_mode="int", output_sequence_length=64,
    standardize=normalize_arabic)
vectorizer.adapt(texts)
```

```
model = keras.Sequential([
    keras.Input(shape=(1,), dtype="string"),
    vectorizer,
    layers.Embedding(30000, 128, mask_zero=True),
    layers.Bidirectional(layers.LSTM(64)),
    layers.Dropout(0.3),
    layers.Dense(64, activation="relu"),
    layers.Dense(3, activation="softmax"),    # سلبي، محايد، إيجابي
])
model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
model.summary()    # المخرجات تعتمد على المفردات الفعلية
```

24.6 كشف الاحتيال: حين يكون الخطأ مكلفًا

المشكلة والبيانات. نظام دفع يريد تقييم كل عملية شراء عبر الإنترنت خلال أقل من مئة ميلي ثانية، قبل الموافقة. نسبة الاحتيال نحو 0.3%، والوسم لا يصل إلا بعد ثلاثين إلى تسعين يومًا حين تصل الشكاوى. هذا يعني أن أحدث البيانات ناقصة التسمية، وأن التدريب يجب أن يستخدم نافذة قديمة مكتملة. الميزات: المبلغ، فئة التاجر، الجهاز، البلد، سرعة تكرار العمليات، وسلوك العميل السابق. كل ميزة يجب أن تكون متاحة لحظة التفويض، لا بعده.

البنية والمعايرة. الدقة هنا عديمة الفائدة: نموذج يقول «سليم» دائمًا يصيب 99.7%. استخدم PR-AUC، والاستدعاء عند دقة تشغيلية ثابتة، والتكلفة المتوقعة. عالج عدم التوازن بأوزان الفئات أو خسارة بؤرية، لكن افعل ذلك داخل مجموعة التدريب فقط، ولا تكرر العينات قبل التقسيم. شبكة صغيرة أو GBDT تكفي؛ المهم المعايرة. أوزان الفئات تشوّه الاحتمالات، لذا أعد المعايرة على مجموعة تحقق غير موزونة، وارسم منحنى الموثوقية واحسب Brier وECE.

منع التسريب والحوكمة. اكتب عقد ميزات يحدد لكل حقل مصدره وزمن توفره ودرجة حداثته. استخدم انضمامًا زمنيًا صحيحًا (Point-in-time Join)؛ لا تحسب متوسط إنفاق العميل من بيانات تشمل العملية الحالية. افصل طبقة القرار عن النموذج: النموذج يعطي احتمالًا، والسياسة تحدد العتبة والإجراء والمراجعة. ابدأ بوضع الظل (Shadow Mode) حيث يُسجَّل القرار دون تنفيذه، ثم انتقل تدريجيًا. سجّل كل قرار لأغراض التدقيق، وراقب معدلات الرفض الخاطئ حسب المجموعات، ووثّق النموذج في بطاقة واضحة.

المخاطر. الاحتيال عدائي: يتغير المحتالون عندما يتغير النموذج. وهناك حلقة تغذية راجعة: العملية المرفوضة لا تُوسم لاحقًا، فتختفي من بيانات التدريب. أضف مراجعة عشوائية لعينات مرفوضة، وراقب الانحراف في الميزات والنتائج. الخطأ الإيجابي الكاذب ليس رقمًا: إنه عميل مُحرَج أو عملية مشروعة رُفضت. لذلك الحوكمة والعدالة جزء من المتطلب، لا إضافة تجميلية.

```python
import numpy as np
import keras

keras.utils.set_random_seed(42)

# بيانات اصطناعية شديدة الاختلال (توضيحية فقط)
rng = np.random.default_rng(42)
n = 20000
X = rng.normal(size=(n, 12)).astype("float32")
p_true = 1 / (1 + np.exp(-(2.0 * X[:, 0] + X[:, 1] - 3.0)))
y = (rng.random(n) < 0.005 * p_true / p_true.mean()).astype("int32")

# في الواقع: تقسيم زمني بفجوة تغطي تأخر الوسم
cut = int(n * 0.7)
X_train, y_train = X[:cut], y[:cut]
X_val, y_val = X[cut:], y[cut:]

pos = int(y_train.sum())
neg = len(y_train) - pos
class_weight = {0: 1.0, 1: float(neg / max(pos, 1))}

model = keras.Sequential([
    keras.Input(shape=(12,)),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dense(1),                # logit واحد بلا sigmoid
])
model.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss=keras.losses.BinaryCrossentropy(from_logits=True),
    metrics=[keras.metrics.AUC(curve="PR", from_logits=True,
                               name="pr_auc")],
)
model.fit(X_train, y_train, validation_data=(X_val, y_val),
          epochs=20, batch_size=256, class_weight=class_weight,
          verbose=0)

# العتبة تُختار على التحقق وفق التكلفة: 5 لتفويت الاحتيال، 1 لإنذار كاذب
logits = model.predict(X_val, verbose=0).ravel()
probs = 1 / (1 + np.exp(-logits))
for tau in (0.2, 0.5, 0.8):
    pred = (probs >= tau).astype(int)
    fn = int(((y_val == 1) & (pred == 0)).sum())
    fp = int(((y_val == 0) & (pred == 1)).sum())
    print(f"tau={tau:.1f}  FN={fn}  FP={fp}  cost={5 * fn + fp}")
```

```
brier = np.mean((probs - y_val) ** 2)
print(f"Brier score: {brier:.4f}")      # أقل أفضل، لكنه يتأثر بالاختلال
```

24.7 ست حالات واقعية تكمل الأمثلة الأربعة

الأمثلة السابقة تعلّمك الصورة والسلسلة الزمنية واللغة والاحتيال. لكن هل تتغير المشكلة حين ينتقل النموذج إلى عيادة أو مزرعة أو قاعة دراسة؟ نعم: المدخلات قد تتشابه، أما طريقة جمع الوسم، ومَن يتأثر بالقرار، وكلفة الخطأ، وشروط التشغيل فتختلف. يختصر الشكل ستة مجالات إضافية؛ ثم نفكك كل حالة إلى قرار وبيانات وتقسيم ومقياس وطريقة نشر.



شكل 24-2 ستة مجالات تطبيقية، ولكل مجال مقياس وضابط يحمي من الاستخدام الخاطئ

الصحة: ترتيب صور تحتاج إلى مراجعة اختصاصي

لنفترض أن مستشفى يريد ترتيب صور شبكية العين بحسب أولوية مراجعتها، كي يراجع اختصاصي العيون الحالات الأشد أولًا. السؤال ليس «هل يشخّص النموذج المرض؟» بل «هل يلتقط الحالات التي تتطلب مراجعة سريعة، دون أن يحوّل نتيجة آلية إلى تشخيص؟». هذا التحديد يغير المنتج والتقييم معًا.

يجب أن يكون تقسيم البيانات على مستوى **المريض** لا الصورة؛ فقد يملك المريض صورًا متكررة، وإذا ظهرت صور منه في التدريب والاختبار حفظ النموذج إشارات خاصة به. احتفظ أيضًا بموقع أو جهاز تصوير كامل للاختبار الخارجي. اجمع الوسوم من اختصاصيين وفق بروتوكول معلن، وسجّل الخلاف وحالات عدم كفاية الصورة بدل إجبارها على فئة صحيحة/مريضة.

اعرض الحساسية عند حدّ محدد للنوعية، ومعايرة الاحتمالات، ومصفوفة الأداء حسب الجهاز والعمر والموقع؛ ولا تختزل النتيجة في الدقة. ابدأ تشغيلًا صامتًا لا يؤثر في الرعاية، ثم راجع النتائج مع المختصين، وبعد تحقق مستقبلي مستقل يمكن دراسة نظام مساعدة يتيح للطبيب رؤية الصورة والدرجة وسبب الإحالة. لا يُستخدم المثال لإصدار تشخيص أو علاج، وتخضع البيانات الصحية لضوابط الخصوصية والموافقات واللوائح المحلية.

الصناعة: فحص العيوب من صور خط الإنتاج

في مصنع إلكترونيات، تلتقط كاميرا لوحة الدارة بعد اللحام. الهدف كشف عيب ظاهر وتحويل الصورة إلى مفتّش قبل أن تتوقف المحطة التالية. لا تقسّم الصور عشوائيًا إذا كانت الصور المتقاربة من اللوحة نفسها أو الوردية نفسها؛ ضع دفعات إنتاج لاحقة، أو كاميرا أخرى، في الاختبار كي تقيس التعميم الحقيقي. واحفظ صور «لا يوجد عيب» من ظروف إضاءة متعددة، لا من يوم واحد.

هذا خط تدريب بنقل التعلّم. جهّز المجلدات بعد تقسيمها حسب الدفعة/الزمن، لا عبر تقسيم عشوائي داخل الدفعة. رتّب الفئات بحيث تكون `defect` هي الفئة الموجبة. يعرض المثال متوسط الدقة تحت منحنى PR، لكن قرار الفحص النهائي يختار عتبة تحقق حدًا مقبولًا للدقة أو الاستدعاء بحسب كلفة إعادة الفحص وفوات العيب.

```python
from pathlib import Path
import keras
from keras import layers
import tensorflow as tf

AUTOTUNE = tf.data.AUTOTUNE
CLASS_NAMES = ["normal", "defect"]  # الفئة الموجبة: defect

def read_split(path, shuffle):
    ds = keras.utils.image_dataset_from_directory(
        path,
        class_names=CLASS_NAMES,
        label_mode="binary",
        image_size=(224, 224),
        batch_size=32,
        shuffle=shuffle,
        seed=42,
    )
```

```
    def prepare(images, labels):
        images = keras.applications.mobilenet_v2.preprocess_input(images)
        return images, labels

    return ds.map(prepare, num_parallel_calls=AUTOTUNE).prefetch(AUTOTUNE)

train_ds = read_split("data/split/train", shuffle=True)
val_ds = read_split("data/split/validation", shuffle=False)

backbone = keras.applications.MobileNetV2(
    include_top=False, weights="imagenet", input_shape=(224, 224, 3)
)
backbone.trainable = False
inputs = keras.Input(shape=(224, 224, 3), name="image")
x = backbone(inputs, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.2)(x)
probability = layers.Dense(1, activation="sigmoid", name="defect_probability")(x)
model = keras.Model(inputs, probability)
model.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss=keras.losses.BinaryCrossentropy(),
    metrics=[keras.metrics.AUC(curve="PR", name="pr_auc")],
)
callbacks = [keras.callbacks.EarlyStopping(
    monitor="val_pr_auc", mode="max", patience=4, restore_best_weights=True
)]
model.fit(train_ds, validation_data=val_ds, epochs=20, callbacks=callbacks)
```

لا تجعل `0.5` عتبة تشغيل افتراضية لمجرد أنها شائعة. استخرج احتمالات مجموعة التحقق، ثم اختر عتبة وفق تكلفة العيب الفائت وزمن المراجعة، وثبّتها قبل قراءة مجموعة الاختبار. افحص الأداء لكل خط وكاميرا، وقِس p95 لزمن الاستدلال بعد التصدير إلى الجهاز المقصود. إذا تغيّر موضع الكاميرا أو الإضاءة، أرسل الحالات الملتبسة إلى الإنسان وراقب نسبة الرفض.

الزراعة: رصد المحصول من صورة متعددة الأطياف

قد يستخدم فريق زراعي صورًا جوية لتحديد غطاء محصول أو مناطق إجهاد مائي. الصورة هنا قد تحتوي قنوات مرئية وقنوات تحت الحمراء القريبة؛ لذلك لا تنطبق عليها معالجة RGB تلقائيًا. ابدأ بمعايرة قيم المستشعر، وقناع السحب والظلال، وسجل يوم التصوير ونوع المستشعر والطقس. إذا كان الهدف تحديد المناطق داخل الصورة فالمسألة **تجزئة**، وتناسبها بنية مثل U-Net أكثر من مصنف يعطي فئة واحدة للصورة كلها.

قسّم بحسب الحقل والموسم، لا بحسب رقع متجاورة عشوائيًا؛ فالرقع من الحقل نفسه تشترك في التربة والطقس. قِس IoU لكل فئة وMacro-F1، وافحص الخريطة على محصول أو مستشعر غير ممثل. ينبغي أن يرى المهندس الزراعي خريطة الثقة والبقع غير المؤكدة، وأن يبقى قرار الري أو المعالجة بيد المختص. راقب تغير الموسم والنمو؛ فالنموذج المدرب على شهر لا يَعِد بأداء مماثل بعد تغير مرحلة النبات.

التوصية: ترتيب ما قد يهم مستخدمًا

تطبيق متجر أو مكتبة يريد ترتيب المنتجات لكل مستخدم. ابدأ بخط أساس شائع مثل الأكثر مشاهدة في الفئة، ثم افصل **استرجاع المرشحين** عن **ترتيبهم** إذا كبر الكتالوج. لا تعدّ كل عنصر لم ينقره المستخدم مثالًا سلبيًا؛ فقد لا يكون قد رآه أصلًا. ابنِ أمثلة التدريب من أحداث معروفة، واسحب السلبيات من العناصر التي كانت متاحة عند وقت الحدث، لا من الكتالوج المستقبلي.

هذا مثال مبسط لعاملية مصفوفة ضمن Keras: يتعلم لكل مستخدم ولكل عنصر متجهًا، ويحسب توافقهما بالضرب النقطي. جهّز المعرّفات لتكون أعدادًا صحيحة ضمن النطاق، واجعل `train_labels` واحدًا للتفاعل الموجب وصفرًا لسلبية جرى أخذها من عناصر مكشوفة للمستخدم.

```python
import numpy as np
import keras
from keras import layers

# هذه المصفوفات تأتي من سجل تفاعلات مقسوم زمنيًا ومن سلبية مأخوذة بعناية.
# شكل كل من المعرّفات والوسوم: (عدد التفاعلات، 1).
num_users, num_items, factors = 5000, 8000, 64
user_input = keras.Input(shape=(), dtype="int32", name="user_id")
item_input = keras.Input(shape=(), dtype="int32", name="item_id")
user_vector = layers.Embedding(num_users, factors)(user_input)
item_vector = layers.Embedding(num_items, factors)(item_input)
score = layers.Dot(axes=1, name="compatibility")([user_vector, item_vector])
ranker = keras.Model({"user_id": user_input, "item_id": item_input}, score)
ranker.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss=keras.losses.BinaryCrossentropy(from_logits=True),
    metrics=[keras.metrics.AUC(curve="PR", from_logits=True, name="pr_auc")],
)
ranker.fit(
    {"user_id": train_users, "item_id": train_items}, train_labels,
    validation_data=({"user_id": val_users, "item_id": val_items}, val_labels),
    batch_size=1024, epochs=8,
)
```

التحقق يجب أن يأتي من فترة تالية للتدريب. أضف Recall@K وNDCG@K وقِس التغطية والتنوع، ثم اختبر على شرائح المستخدمين الجدد والعناصر الجديدة. العوامل الكامنة لا تحل مشكلة البداية الباردة وحدها؛ يحتاج المستخدم أو العنصر الجديد إلى مسار بديل قائم على المحتوى أو الشعبية أو سؤال اختياري عن التفضيلات. وعند الإطلاق، راقب عدم توازن الظهور وتأثير التوصية على البائعين، واختبر التغييرات تدريجيًا بمقياس حارس لتجربة المستخدم.

التعليم: اقتراح دعم مبكر مع إبقاء القرار إنسانيًا

قد يريد فريق تعليمي معرفة من قد يستفيد من جلسة دعم إضافية، باستخدام تسليم الواجبات ونتائج الاختبارات القصيرة وطلب المساعدة. لا تسمِّ النموذج «متنبئًا بالفشل»؛ النتيجة تتأثر بالفرصة والاتصال بالإنترنت والظروف المحيطة، وقد يتغير الوسم نفسه لأن المعلم تدخّل.

قسّم بحسب الفصل الدراسي أو المدرسة، وراقب المعايرة ومعدلات الاستدعاء والإنذارات لكل مجموعة، لا الأداء الكلي فقط. اجعل استخدام الدرجة دعوةً إلى مساعدة اختيارية، ولا تمنع تسجيل مادة أو فرصة بناء عليها. اجمع أقل قدر من البيانات، وضع مدة حذف معلومة وصلاحيات وصول، وأتح للطالب أو المعلم تصحيح بيانات خاطئة. ولقياس الأثر، قارِن تدخلًا تعليميًا مصممًا أخلاقيًا بممارسة الرعاية المعتادة؛ تحسن التنبؤ لا يثبت أن التدخل نافع.

الطاقة والنقل: توقع الذروة قبل أن تصبح مشكلة

تريد جهة تشغيل توقع حمل الكهرباء في الساعات المقبلة أو زمن الازدحام على طريق. خط الأساس قد يكون «قيمة الساعة نفسها الأسبوع الماضي» أو متوسط الأيام المماثلة. أضف التقويم والعطل والطقس المتوقع فقط إذا كان متاحًا وقت التنبؤ، واحفظ إصدار توقع الطقس حتى لا تتسرب القيم المرصودة لاحقًا. اختبر على أسابيع أو أشهر مستقبلية متتالية، مع فترة تحقق لاختيار المعلمات والعتبة.

لا يكفي MAE إذا كانت أخطاء الذروة أخطر من أخطاء الليل. أبلغ الخطأ حسب الساعة والمنطقة والموسم، وأضف خطأ الذروة أو فواصل كمية تبين عدم اليقين. في النقل، قِس أيضًا زمن الرحلة عند p90 لا المتوسط فقط. شغّل النموذج إلى جانب النظام القديم، وحدد من يتلقى الإنذار وما الإجراء المسموح، ثم راقب تغير الأنماط عند العطل أو إغلاق الطرق أو تغير الطقس.

المجال	القرار الذي يدعمه النموذج	مجموعة اختبار ذات معنى	المقاييس والضابط التشغيلي
الصحة	ترتيب الحالات لمراجعة اختصاصي	مرضى أو مستشفى/جهاز غير مرئي	الحساسية والمعايرة؛ مراجعة بشرية، لا تشخيص آلي
الصناعة	فحص عيب على خط الإنتاج	دفعة أو كاميرا أو وردية لاحقة	الاستدعاء عند دقة محددة؛ مسار للمراجعة اليدوية
الزراعة	تحديد محصول أو إجهاد على الخريطة	حقل وموسم ومستشعر جديد	Macro-F1 وIoU؛ اعتماد المختص الزراعي
التوصية	ترتيب عناصر متاحة للمستخدم	تفاعلات مستقبلية وتغطية عناصر جديدة	Recall@K وNDCG؛ مراقبة التعرض والتنوع
التعليم	عرض دعم إضافي على من قد يحتاجه	مدرسة أو فصل دراسي لاحق	معايرة حسب المجموعة؛ لا حرمان من فرصة
الطاقة/النقل	توقع الحمل أو الازدحام	فترة زمنية مستقبلية	خطأ الذروة وp90؛ إنذار يراجعه المشغل

هذه الحالات ليست وصفات جاهزة لتدريب نموذج؛ إنها قوالب تفكير. غيّر تعريف الهدف والوسم وخط الأساس والتقسيم والمقياس حتى تعكس عملية العمل الفعلية. إذا لم تستطع شرح من يستخدم الناتج، ومتى، وما الذي يحدث عند خطئه، فالمشروع لم يُعرَّف بعد.

24.8 خارطة الطريق ومعرض الأعمال

الخارطة ليست وعدًا بالتخصص في اثني عشر شهرًا، بل عقد مع نفسك: مرحلة، مخرج ملموس، وبوابة لا تُعبر بالقراءة وحدها. لا تنتقل إلى مرحلة قبل أن تستطيع إعادة بناء مخرجها من الذاكرة، وشرح قراراته لشخص آخر.

المرحلة	الأشهر	المحور	المخرج الملموس
الأساس	3-1	بايثون، الجبر والاحتمال، التنسورات، Keras، أساسيات التقييم	MLP من الصفر، مشروع جدولي صغير، تقرير تقييم
التطبيق	6-4	CNN، نقل التعلّم، التسلسلات، NLP، أساسيات الانتباه	مشروعان من الأربعة، مع خط أساس وتحليل أخطاء
الإنتاج	9-7	tf.data والأداء، التصدير، LiteRT، الخدمة، الاختبارات، المراقبة	خدمة كاملة مع سجل ومراقبة وخطة تراجع
التخصص	12-10	مجال واحد: رؤية، لغة، سلاسل، أو جدولية	مشروع رائد، إعادة إنتاج ورقة، مقال أو حديث

اثنا عشر شهرًا بلا حشو

في الأشهر الثلاثة الأولى، ابنِ الأساس مرة واحدة: بايثون، NumPy، الجبر الخطي، الاحتمال، ثم TensorFlow وKeras. لا تجمع الدورات؛ اكتب MLP من الصفر، ثم درّبه على بيانات صغيرة، ثم قيّمه بمصفوفة التباس. المخرج ليس شهادة، بل دفتر تستطيع شرح كل سطر فيه.

في المرحلة الثانية، اختر مشروعين لا أربعة. مشروع رؤية ومشروع نصوص، مثلًا، لأن كلًّا منهما يعلّمك أنبوب بيانات مختلفًا. طبّق نقل التعلّم، وقارن خط الأساس، واكتب تحليل أخطاء من عشرين عينة خاطئة. هنا تتشكل العادة الأهم: لا تنشر رقمًا بلا سياق.

في مرحلة الإنتاج، توقّف عن تدريب نماذج جديدة وأصلح النظام: tf.data، الدفعات، التصدير، خدمة، حاوية، اختبارات، سجلات، تنبيهات، وتراجع عن الإصدار. جرّب أن تُسقط الخدمة عمدًا ثم تستعيدها. هذه المرحلة هي التي تفصل من «يعرف التدريب» عن «يعرف الإنتاج».

في المرحلة الأخيرة، تخصّص. اختر مجالًا واحدًا واقرأ فيه خمس أوراق، وأعد إنتاج واحدة على بيانات صغيرة، واكتب ما فشل. المساهمة في مشروع مفتوح أو نشر تحليل تجربة يعلّمك أكثر من مشروع آخر نسخَه من درس. العمق في مجال واحد أقوى من سطحية في عشرة.

معرض الأعمال والسيرة الذاتية

المُوظِّف يقرأ أول صفحتين، لا كل ما كتبت. اجعل المستودع المثبّت (Pinned) ثلاثة فقط، واكتب لكل واحد ملف README يبدأ بالمشكلة والقرار، ثم بطاقة البيانات والقيود، ثم خط الأساس والمقياس، ثم مخطط البنية، ثم جدول نتائج بشرائح، ثم خطوات التشغيل، وأخيرًا الحدود وما ستفعله لاحقًا. أرفق بطاقة نموذج، وبيئة قابلة للتكرار، واختبارًا واحدًا على الأقل.

في السيرة الذاتية، لا تكتب «أتقن TensorFlow». اكتب فعلًا وطريقة ورقمًا وقيدًا: «بنيت مصنّف عيوب بنقل تعلّم من MobileNetV2، ورفعت استدعاء العيوب من 0.71 إلى 0.89 عند دقة 0.90، بزمن استدلال 45ms على جهاز طرفي». هكذا يرى القارئ قرارك، لا قائمة أدواتك. ونظّف المستودعات من دفاتر التجارب الفوضوية؛ ثلاثة مشاريع مكتملة أفضل من عشرين بداية.

في المقابلة، ستُسأل عن خط الأساس، والتسريب، وفشل النموذج، والمراقبة، والتكلفة. جهّز قصة مشروع واحد من البداية إلى النهاية، واحكِ قرارًا خاطئًا صحّحته. الصدق في الحدود يبني ثقة أكثر من ادعاء دقة كاملة.

24.9 أخطاء شائعة

⚠ تحذير

العرض: تبدأ بالمعمارية قبل أن تحسب خط أساس. **السبب:** الإعجاب بالنموذج الجديد يسبق فهم المشكلة. **الحل:** ابنِ أولًا تنبؤًا ساذجًا أو انحدارًا بسيطًا، وسجّل رقمه؛ إن لم يتفوق نموذجك عليه، فالمشكلة في الميزات أو التقييم.

⛔ خطر

العرض: تقسيم عشوائي لبيانات زمنية أو لمجموعات مترابطة. **السبب:** العينات القريبة تتشابه، فيتسرّب المستقبل أو نفس المريض إلى الاختبار. **الحل:** تقسيم زمني بفجوة، أو تقسيم حسب المجموعة/المتجر/المريض، ثم اختبار أن التوزيعات لا تتقاطع.

⚠ تحذير

العرض: قياس الدقة على بيانات غير متوازنة والاكتفاء بها. **السبب:** الفئة الكبرى تسيطر، فيبدو النموذج ممتازًا وهو يتجاهل الأقلية. **الحل:** استخدم PR-AUC، والاستدعاء عند دقة ثابتة، والتكلفة المتوقعة، واعرض النتائج حسب الشريحة.

⛔ خطر

العرض: تدريب النموذج على ميزة لا تتوفر وقت التنبؤ. **السبب:** سهولة إضافتها في الدفتر، وغفلة عن زمن توفرها الحقيقي. **الحل:** عقد ميزات يحدد لكل حقل لحظة توفره، واختبار آلي يحذف الميزات غير المتاحة، ومراجعة الانضمام الزمني.

> ⚠ **تحذير**
>
> **العرض:** نشر النموذج دون سجلات أو تنبيهات أو خطة تراجع. **السبب:** التركيز على الدقة ونسيان أن التوزيع يتغير. **الحل:** سجّل المدخلات والقرارات، وراقب الانحراف والأداء، واحتفظ بإصدار سابق جاهز، ووثّق حدود النموذج في بطاقة.

24.10 تمارين

1. **حساب التكلفة:** مصفوفة التباس تعطي 40 حالة موجبة فائتة و120 إنذارًا كاذبًا. إذا كانت تكلفة التفويت خمسة أضعاف الإنذار الكاذب، احسب التكلفة الكلية. (المتوقع: 40×5 + 120 = 320 وحدة تكلفة.)
2. **اختيار العتبة:** من بيانات التحقق، لديك عتبتان: الأولى تعطي استدعاء 0.90 ودقة 0.60، والثانية استدعاء 0.70 ودقة 0.85. أيّهما تختار لنظام احتيال يكلّف التفويت خمسة أضعاف الإنذار؟ برّر. (المتوقع: اختيار يعتمد على عدد الحالات والتكلفة، مع ميل للاستدعاء الأعلى إن كان الفارق كبيرًا.)
3. **تقسيم زمني:** صمّم تقسيمًا لبيانات يومية على سنتين مع أفق 14 يومًا وفجوة 14 يومًا. (المتوقع: تدريب حتى حد معين، فجوة، تحقق، فجوة، اختبار في النهاية.)
4. **بطاقة بيانات:** اكتب ستة حقول لبطاقة بيانات مشروع صور: المصدر، الفئات، التوزيع، الكاميرات، طريقة الوسم، القيود. (المتوقع: بطاقة تذكر عدم تمثيل خط إنتاج جديد وتفاوت الإضاءة.)
5. **F-beta:** إذا كانت الدقة 0.8 والاستدعاء 0.5، احسب F_1 وF_2. (المتوقع: $F_1 \approx 0.615$ و$F_2 \approx 0.556$؛ انخفاض F_2 يعكس أن الاستدعاء ضعيف.)
6. **تنظيف عربي:** خذ خمس جمل فيها نفي وسخرية ورموز تعبيرية، وطبّق التنظيف المقترح. أي قرار قد يضرّ المشاعر؟ (المتوقع: حذف الرموز أو «لا» يغيّر المعنى، وتوحيد الحروف قد يخلط كلمات متشابهة.)
7. **الخط الأساس:** شغّل كود التنبؤ وقارن MAE للنموذج بالخط الأساسي. إن لم يتفوق، اقترح ميزتين. (المتوقع: مقارنة صريحة، ثم إضافة ميزات موسمية أو إزالة الاتجاه قبل التدريب.)
8. **معرض الأعمال:** حوّل أحد مشاريعك إلى README من سبعة أقسام وسطرين للسيرة الذاتية. (المتوقع: صفحة تبدأ بالمشكلة والمقياس، وتذكر القيود وخطة التحسين.)

24.11 ملخص

- المشروع الختامي نظام كامل: مشكلة، بيانات، خط أساس، تقييم، نشر، مراقبة، وتوثيق.
- المقياس يتبع القرار والتكلفة، لا العكس؛ والدقة وحدها تخفي الفشل في البيانات غير المتوازنة.
- الزمن والمجموعات مصدران أساسيان للتسريب؛ التقسيم العشوائي ليس بريئًا.
- لكل مجال فخّه: انحراف الكاميرا في الصور، رقابة الطلب في السلاسل، اللهجات في اللغة، والتغيّر العدائي في الاحتيال.
- الخارطة أربع مراحل، ولكل مرحلة مخرج ملموس وبوابة فهم، لا شهادة.
- معرض الأعمال يبيع القرار والقياس والحدود، لا قائمة المكتبات؛ والمقابلة نقاش هندسي لا حفظ.

الملاحق · مراجع وأدوات

ملحق A

الملحق A: الرياضيات العملية

هذا الملحق مرجع سريع للرموز المتكررة. الهدف أن تصبح المعادلة وصفًا لعملية يمكن فحصها.

المتجهات والمصفوفات

المتجه قائمة مرتبة من القيم، والمصفوفة ترتيب ثنائي الأبعاد. إذا كانت `X` دفعة أمثلة بشكل `(n, d)` و`w` أوزانًا بشكل `(d, 1)`، فإن `Xw` ينتج درجة لكل مثال. يجب أن يتطابق البعد الداخلي. الضرب العنصري يختلف عن ضرب المصفوفات.

$$y = Xw + b$$

في الانحدار الخطي تمثل `X` السمات، و`w` الأوزان، و`b` انحيازًا، و`y` التنبؤات.

التفاضل وقاعدة السلسلة

المشتقة تقيس تغير المخرج مع تغير المدخل محليًا. إذا كان `z=g(x)` و`y=f(z)`، فإن مشتقة `y` بالنسبة إلى `x` هي حاصل مشتقتي المرحلتين. الشبكة تركيب دوال، لذا يعيد الانتشار الخلفي استخدام قاعدة السلسلة عبر طبقاتها.

$$\nabla_w L = \left[\frac{\partial L}{\partial w_1}, \ldots, \frac{\partial L}{\partial w_d}\right]$$

اتجاه التدرج يزيد الخسارة محليًا؛ يتجه تحديث الانحدار المتدرج عكسه. معدل التعلم يضبط طول الخطوة.

الاحتمال والتوقع

الاحتمال قيمة بين صفر وواحد، والتوزيع يحدد احتمالات نتائج ممكنة. التوقع متوسط موزون بالاحتمال. Cross-Entropy تعاقب التوزيع المتوقع حين يضع احتمالًا منخفضًا للحقيقة. في التصنيف متعدد الفئات، logits درجات غير مطبّعة؛ softmax يحولها إلى توزيع.

$$softmax(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

`z_i` درجة الفئة، والمقام مجموع الدرجات الأسية، فتكون المخرجات موجبة ومجموعها واحد.

حساب يدوي صغير

للنقطة `x=2` والنموذج `ŷ=wx+b` حيث `w=3` و `b=1`، يكون التنبؤ 7. إذا كان الهدف 5 وخسارة المربع، فالخسارة 4. مشتقة الخسارة بالنسبة إلى `w` هي `2(ŷ-5)x=8` وبالنسبة إلى `b` هي 4. خطوة بمعدل 0.1 تحدث الوزن إلى 2.2 والانحياز إلى 0.6. استخدم الحساب اليدوي لفحص إشارة التدرج.

أحجام الأشكال الشائعة

الكيان	شكل نموذجي	المعنى
دفعة متجهات	`(N, D)`	أمثلة × سمات
صور NHWC	`(N, H, W, C)`	دفعة × ارتفاع × عرض × قنوات
رموز	`(N, T)`	دفعة × طول تسلسل
تمثيل تسلسلي	`(N, T, D)`	دفعة × زمن × ميزات

حين يظهر خطأ أبعاد، اكتب معنى كل محور قبل تعديل الشفرة. لا تحل مشكلة شكل بإعادة تشكيل عمياء؛ قد يمر الحساب وتفسد دلالته.

قائمة مراجعة

- اكتب شكل كل طرف قبل الضرب.
- عرّف ما تمثله المحاور.
- ميّز بين الاحتمال وlogit.
- افحص إشارة التدرج بمثال صغير.
- قارن المعادلة بحساب يدوي أو اختبار عددي.

الملاحق · مراجع وأدوات

ملحق B

الملحق B: دليل البيئة والأعطال

إعداد نظيف

1. اختر إصدار Python تدعمه عجلة TensorFlow التي تريدها.
2. أنشئ بيئة افتراضية داخل المشروع.
3. ثبّت الإصدارات المعتمدة ثم احفظ ملف القفل.
4. شغّل استيرادًا صغيرًا وفحص الأجهزة.
5. سجّل النظام والمعالج وتعريف المسرع.

```bash
python -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install tensorflow==2.21.0 keras==3.15.1
python -m pip freeze > requirements-lock.txt
```

في PowerShell يكون التنشيط `\.venv\Scripts\Activate.ps1`. استخدم `python -m pip` لتفادي تثبيت الحزمة إلى مفسّر مختلف.

التحقق

```python
import sys
import keras
import tensorflow as tf

print(sys.executable)
print(tf.__version__, keras.__version__)
print(tf.config.list_physical_devices())
```

لا تقارن رقم GPU المعلن برقم GPU المستخدمة فعليًا دون تشغيل عملية وقياسها.

جدول الأعطال

العرض	ما تفحصه أولًا	إجراء آمن
لا توجد توزيعة مناسبة	نسخة Python، المعمارية	اختر مفسرًا وعجلة متوافقين
DLL مفقودة	مصدر التثبيت runtimeg النظام	أعد بناء venv وافحص التبعيات الرسمية
تعارض NumPy	توافق ABI وإصدارات الحزم	ثبّت نسخة متوافقة من ملف القفل
GPU غير مكتشفة	تعريف NVIDIA وWSL/الحاوية	افحص الجهاز ثم TensorFlow
نفاد VRAM	حجم الدفعة والتفعيلات	خفّض الدفعة أو دقة الصورة
تدريب بطيء	خط البيانات والإحماء	قِس profiler بدل تغيير CUDA عشوائيًا
نتائج غير متكررة	بذور وتقسيم وإصدارات	ثبّت مصادر العشوائية وسجل البيانات

GPU وWSL2 والحاويات

تحقق من دليل إصدار TensorFlow الرسمي لمصفوفة CUDA وcuDNN. على Windows استخدم مسارًا موثقًا مثل WSL2 أو صورة رسمية متوافقة. الحاوية تعزل مساحة المستخدم لكنها لا تستبدل تعريف المضيف. Colab وKaggle مناسبان للتجربة، وقد تتغير الموارد والجلسة؛ احفظ الملفات في تخزين دائم.

تقرير مشكلة مفيد

ضمّن نسخة Python وTensorFlow وKeras، نظام التشغيل، نوع الجهاز، طريقة التثبيت، أقل شفرة تعيد الخطأ، والنص الكامل للرسالة. أزل الرموز السرية ومسارات البيانات الخاصة. لا ترسل بيانات شخصية علنًا.

الملاحق · مراجع وأدوات

ملحق C

الملحق C: المصطلحات — عربي ↔ إنجليزي

المصطلح العربي	English	تعريف موجز
التعلّم العميق	Deep Learning	نماذج متعددة الطبقات تتعلم تمثيلات
التنسور	Tensor	مصفوفة متعددة الأبعاد
الرتبة	Rank	عدد محاور التنسور
الشكل	Shape	طول كل محور
نوع البيانات	Data type	دقة وتمثيل القيم
البث	Broadcasting	محاذاة أبعاد متوافقة في عملية عنصرية
الاشتقاق التلقائي	Automatic differentiation	حساب مشتقات من سجل العمليات
التدرج	Gradient	مشتقات الخسارة بالنسبة للمعاملات
الانتشار الخلفي	Backpropagation	تطبيق قاعدة السلسلة عبر الشبكة
دالة الخسارة	Loss function	هدف عددي يحسنه التدريب
المحسّن	Optimizer	قاعدة تحديث المعاملات
معدل التعلّم	Learning rate	طول خطوة التحديث
دفعة	Batch	مجموعة أمثلة في خطوة تدريب
حقبة	Epoch	مرور على مجموعة التدريب
فرط التعلّم	Overfitting	تحسن التدريب مع ضعف التعميم
التنظيم	Regularization	تقييد التعلم لتقليل التخصيص للعينة
التسرب	Data leakage	دخول معلومات لا تتاح وقت التنبؤ

المصطلح العربي	English	تعريف موجز
استدعاء	Recall	نسبة الإيجابيات الحقيقية المكتشفة
إحكام	Precision	نسبة التنبؤات الإيجابية الصحيحة
معايرة	Calibration	توافق الثقة المقدرة مع التكرار الحقيقي
مصفوفة الالتباس	Confusion matrix	جدول التنبؤات والحقيقة
تحقق متقاطع	Cross-validation	إعادة التقسيم للتقييم المتعدد
خط بيانات	Data pipeline	خطوات تجهيز وتدفق البيانات
تكميم	Quantization	تمثيل الأوزان أو التفعيلات بدقة أقل
استدلال	Inference	تشغيل نموذج لإنتاج توقع
تضمين	Embedding	تمثيل متعلم كثيف لرمز أو عنصر
انتباه	Attention	ترجيح معلومات سياقية بحسب الاستعلام
محوّل	Transformer	معمارية تعتمد الانتباه والطبقات الأمامية
شبكة تلافيفية	CNN	شبكة تستفيد من البنية المكانية
شبكة متكررة	RNN	شبكة تحمل حالة عبر خطوات تسلسل
تعلم بالنقل	Transfer learning	إعادة استخدام أوزان مدربة لمهمة جديدة
ضبط دقيق	Fine-tuning	تحديث جزء من نموذج مدرب مسبقًا
تجزئة دلالية	Semantic segmentation	تصنيف كل بكسل في صورة
تقاطع على اتحاد	IoU	نسبة تداخل منطقتين إلى اتحادهما
انتشار	Diffusion	توليد بعكس عملية إضافة الضوضاء
شبكة خصومية توليدية	GAN	مولد ومميز يتنافسان بالتدريب
أثر التجربة	Experiment tracking	تسجيل إعداد ونتيجة ونسخة بيانات

المصطلح العربي	English	تعريف موجز
انجراف البيانات	Data drift	تغير توزيع المدخلات عبر الزمن
انجراف المفهوم	Concept drift	تغير علاقة السمات بالهدف
تعلم معزز	Reinforcement learning	تعلم سياسة عبر تفاعل ومكافأة
سياسة	Policy	قاعدة اختيار الفعل من الحالة
قيمة الفعل	Q-value	العائد المتوقع من فعل في حالة
خصوصية تفاضلية	Differential privacy	حد رياضي لأثر سجل فردي
تعلم اتحادي	Federated learning	تدريب منسق مع إبقاء البيانات محلية
قابلية التفسير	Interpretability	طرق فحص سلوك النموذج ومخرجاته
هجوم خصومي	Adversarial attack	مدخل مصمم لإرباك النموذج
بطاقة نموذج	Model card	وثيقة الاستخدام والبيانات والقيود
الانحدار الخطي	Linear regression	نموذج يتنبأ بقيمة مستمرة من مجموع موزون
الانحدار اللوجستي	Logistic regression	نموذج تصنيف يحول الدرجة إلى احتمال
الإدراك	Perceptron	وحدة خطية تتبعها دالة تنشيط
شبكة متعددة الطبقات	Multilayer perceptron	شبكة كثيفة من طبقات متتابعة
الانتشار الأمامي	Forward pass	حساب التفعيلات من المدخل حتى المخرج
تهيئة الأوزان	Weight initialization	تعيين قيم البدء للمعاملات
Glorot	Glorot/Xavier initialization	تهيئة توازن التباين بين الطبقات
He initialization	He initialization	تهيئة شائعة مع ReLU
الزخم	Momentum	متوسط متحرك ينعّم اتجاه التدرج
اضمحلال الوزن	Weight decay	تقليص المعاملات ضمن قاعدة المحسن

المصطلح العربي	English	تعريف موجز
قص التدرج	Gradient clipping	تحديد معيار التدرج لمنع تحديث منفجر
تطبيع الدفعة	Batch normalization	تطبيع وفق إحصاءات الدفعة أثناء التدريب
تطبيع الطبقة	Layer normalization	تطبيع ضمن تمثيل المثال
إسقاط عشوائي	Dropout	تعطيل عشوائي لوحدات أثناء التدريب
إيقاف مبكر	Early stopping	إيقاف التدريب عند توقف مقياس التحقق
جدول معدل التعلم	Learning-rate schedule	تغيير معدل التعلم مع الخطوات
معدل التعلم	Learning rate	حجم خطوة تحديث المعاملات
دفعة مصغرة	Mini-batch	مجموعة صغيرة لتقدير التدرج
خطوة تحسين	Optimization step	تطبيق تحديث واحد للمعاملات
الدقة المختلطة	Mixed precision	استخدام أنواع عائمة مختلفة خلال الحساب
تحجيم الخسارة	Loss scaling	تكبير مؤقت للخسارة لحفظ دقة التدرجات
دقة عائمة	Floating-point precision	عدد البتات المخصصة لتمثيل القيمة
إعادة التتبع	Retracing	إنشاء رسم جديد لدالة بسبب تغير توقيعها
الرسم البياني	Computation graph	تمثيل عمليات الحساب وروابطها
التنفيذ الفوري	Eager execution	تنفيذ العمليات فور استدعائها
التوقيع	Signature	عقد أشكال وأنواع مدخلات الدالة
محور	Axis	اتجاه أو بعد في مصفوفة متعددة الأبعاد
فهرسة	Indexing	اختيار عنصر أو جزء وفق موضعه
تقطيع	Slicing	استخراج نطاق من محور أو أكثر
تبديل المحاور	Transpose	إعادة ترتيب محاور التنسور

المصطلح العربي	English	تعريف موجز
إعادة تشكيل	Reshape	تغيير تقسيم العناصر على الأبعاد
تكديس	Stack	إضافة محور وجمع موترات عليه
ربط المحاور	Concatenate	وصل موترات على محور موجود
متغير قابل للتعلم	Trainable variable	حالة عددية يحدثها التدريب
مسار التدرج	Gradient path	سلسلة العمليات التي تربط المعامل بالخسارة
تفاضل عددي	Numerical differentiation	تقريب مشتقة عبر فروق صغيرة
هسيان	Hessian	مصفوفة مشتقات الخسارة من الدرجة الثانية
جاكوبي	Jacobian	مصفوفة مشتقات المخرجات بالنسبة للمدخلات
خسارة	Loss	قيمة هدف يحاول التدريب خفضها
إنتروبيا متقاطعة	Cross-entropy	خسارة لتوزيع التنبؤ مقابل الفئة الحقيقية
خسارة Huber	Huber loss	خسارة انحدار تجمع سلوكًا تربيعيًا وخطيًا
فجوة التعميم	Generalization gap	فرق الأداء بين التدريب والتحقق/الاختبار
نقص التعلّم	Underfitting	عجز النموذج عن تمثيل النمط حتى التدريب
ثبات النموذج	Calibration	توافق الاحتمالات مع تكرار الصحة الفعلي
عتبة قرار	Decision threshold	حد يحول الدرجة إلى فئة أو إجراء
المنحنى الدقيق	Precision-recall curve	مقايضة الإحكام والاستدعاء عبر العتبات
منحنى ROC	ROC curve	مقايضة الحساسية ومعدل الإيجابيات الكاذبة
الإيجابي الحقيقي	True positive	حالة موجبة تنبأ النموذج بها موجبة
الإيجابي الكاذب	False positive	حالة سالبة تنبأ النموذج بها موجبة
السلبي الحقيقي	True negative	حالة سالبة تنبأ النموذج بها سالبة

المصطلح العربي	English	تعريف موجز
السلبي الكاذب	False negative	حالة موجبة تنبأ النموذج بها سالبة
التحقق الطبقي	Stratification	حفظ نسب الفئات عند التقسيم
تقسيم مجموعاتي	Group split	فصل مجموعات المصدر لمنع تسرب العينات المرتبطة
تحقق زمني	Temporal validation	تقييم على فترة تلي بيانات التدريب
تتبع التجربة	Experiment tracking	حفظ إعدادات ونتائج run قابل لإعادة التعرف
بصمة البيانات	Data fingerprint	ملخص أو hash يميز نسخة المصدر
إصدارة بيانات	Dataset version	هوية ثابتة لمجموعة البيانات وتقسيمها
تدفق البيانات	Data streaming	معالجة عينات تدريجيًا دون تحميل الكل
مخزن خلط	Shuffle buffer	عناصر تحتفظ بها عملية الخلط مؤقتًا
تحميل مسبق	Prefetching	تجهيز دفعات مستقبلية أثناء الحساب
تداخل القراءة	Interleave	قراءة عناصر من مصادر متعددة بالتوازي
سجل تسلسلي	TFRecord	ملف تدفق لسجلات TensorFlow المتسلسلة
مخطط بيانات	Schema	تعريف أنواع الحقول وحدودها المتوقعة
زيادة بيانات	Data augmentation	تحويل يحافظ على وسم العينة لتوسيع التدريب
تسرب الهدف	Target leakage	سمة تحمل معلومة لا تتاح وقت القرار
تحوّل المجال	Domain shift	اختلاف توزيع التدريب عن مجال الاستخدام
التعلم بالنقل	Transfer learning	إعادة استخدام تمثيلات مدربة مسبقًا
رأس التصنيف	Classification head	طبقات إخراج مضافة لمهمة تصنيف جديدة
تجميد طبقة	Layer freezing	منع تحديث أوزان طبقة أثناء التدريب
فك التجميد	Unfreezing	إعادة جعل أوزان طبقات قابلة للتحديث

المصطلح العربي	English	تعريف موجز
كشف كائنات	Object detection	تحديد فئات وأماكن الأجسام داخل الصورة
صندوق إحاطة	Bounding box	مستطيل يحدد موضع جسم في الصورة
قمع غير الأعظم	Non-maximum suppression	حذف صناديق متكررة بحسب التداخل والثقة
متوسط الدقة	Average precision	مساحة/متوسط منحنى الدقة والاستدعاء لفئة
تجزئة	Segmentation	إسناد فئة أو قناع لمواضع الصورة
قناع	Mask	Tensor يحدد المواضع الفعالة أو فئات البكسل
توسيع حقل الرؤية	Receptive field	منطقة الإدخال المؤثرة في وحدة داخلية
وصلة متبقية	Residual connection	مسار يضيف تمثيلًا سابقًا إلى خرج طبقة
شبكة هرمية	Feature pyramid	دمج خرائط سمات بمقاييس مكانية متعددة
زمن رجوعي	Backpropagation through time	انتشار التدرج عبر خطوات تسلسل
حالة مخفية	Hidden state	ذاكرة متعلمة تمثل ملخص الخطوات السابقة
بوابة النسيان	Forget gate	تحكم LSTM في بقاء الحالة السابقة
تنبؤ متعدد الخطوات	Multi-horizon forecast	إخراج توقع لعدة أزمنة مستقبلية
نافذة منزلقة	Sliding window	مقطع زمني متحرك لتدريب أو استدلال
خط أساس موسمي	Seasonal naive baseline	استخدام قيمة الموسم السابق كتوقع مرجعي
تكييف النموذج	Model conditioning	إدخال معلومة إضافية توجه المخرج
استعلام	Query	تمثيل يطلب معلومات من مجموعة مفاتيح
مفتاح	Key	تمثيل يقارن بالاستعلام لتوليد وزن الانتباه
قيمة	Value	تمثيل تدمجه الأوزان لإنتاج السياق
ترميز موضعي	Positional encoding	إضافة معلومة ترتيب الرموز للمحول

المصطلح العربي	English	تعريف موجز
قناع سببي	Causal mask	منع رمز من الاطلاع على مواضع مستقبلية
قناع حشو	Padding mask	إخفاء مواضع أضيفت لمعادلة أطوال التسلسل
رأس انتباه	Attention head	إسقاط فرعي يتعلم علاقات سياقية
attention flash	Flash Attention	تنفيذ مجزأ يقلل حركة الذاكرة للانتباه
نموذج مشفر فقط	Encoder-only model	Transformer لفهم سياق كامل غالبًا
نموذج مولد	Autoregressive model	تنبؤ متتابع بالرمز التالي
رمز خارج المفردات	OOV token	تمثيل قيمة غير معروفة للمعجم
ترميز أجزاء الكلمة	Subword tokenization	تقسيم الكلمة إلى وحدات متكررة فرعية
توليد مشروط	Conditional generation	توليد عينات وفق شرط أو فئة
انهيار الأنماط	Mode collapse	فقدان المولد تنوع التوزيع في GAN
نموذج مشفر تغايري	Variational autoencoder	Autoencoder يتعلم توزيعًا كامنًا منظمًا
مفكك	Decoder	شبكة تحول تمثيلًا كامنًا إلى بيانات
فضاء كامن	Latent space	تمثيل داخلي مضغوط للعينات
إعادة المعلمة	Reparameterization trick	فصل عشوائية العينة عن معاملات التوزيع
استدلال DDPM	DDPM sampling	عكس إضافة الضوضاء بخطوات احتمالية
توجيه بلا مصنف	Classifier-free guidance	دمج توقع مشروط وغير مشروط لتوجيه التوليد
تقدير التوزيع	Density estimation	نمذجة احتمالية البيانات أو العينات
شبكة هدف	Target network	نسخة أبطأ لتثبيت أهداف Q-learning
مخزن إعادة	Replay buffer	ذاكرة انتقالات يعاد أخذ عينات منها في RL
فرق زمني	Temporal-difference error	فرق القيمة المتوقعة والهدف المؤقت

المصطلح العربي	English	تعريف موجز
استكشاف	Exploration	تجربة أفعال لاكتساب معلومات عن البيئة
استغلال	Exploitation	اختيار الفعل الأفضل وفق المعرفة الحالية
دالة قيمة	Value function	تقدير العائد المتوقع من حالة أو فعل
استرجاع مرشحين	Candidate retrieval	اختيار مجموعة صغيرة قبل مرحلة الترتيب
ترتيب	Ranking	فرز مرشحين وفق منفعة أو احتمال تفاعل
نظام توصية ثنائي البرج	Two-tower recommender	تمثيل المستخدم والعنصر ببرجين منفصلين
مرور رسائل	Message passing	تحديث عقدة عبر تجميع معلومات الجيران
تنعيم مفرط	Over-smoothing	تشابه تمثيلات العقد مع زيادة عمق GNN
مقياس خصوصية ε	Privacy budget epsilon	حد خصوصية في صياغة DP المختارة
تجميع آمن	Secure aggregation	جمع تحديثات دون كشف مساهمة كل جهاز
قص التدرج لكل مثال	Per-example clipping	الحد من مساهمة سجل قبل إضافة الضوضاء
إسناد متكامل	Integrated gradients	إسناد مخرجات إلى مدخلات عبر مسار baseline
تفسير شابلي	SHAP values	توزيع إسهام السمات وفق قيمة Shapley
مثال مضاد	Counterfactual	تغيير مدخل محدود يبدل قرار النموذج
متانة خصومية	Adversarial robustness	ثبات التنبؤ تحت اضطرابات مصممة
تسميم البيانات	Data poisoning	إدخال بيانات تدريب خبيثة أو مضللة
بطاقة بيانات	Dataset card	وثيقة مصدر البيانات وبنائها وحدودها
إصدار مرشح	Candidate release	نسخة اجتازت التقييم قبل الطرح العام
طرح تدريجي	Canary deployment	إرسال نسبة محدودة للنسخة الجديدة أولًا
رجوع إصدار	Rollback	استعادة نسخة سابقة موثوقة من الخدمة

المصطلح العربي	English	تعريف موجز
كمون المئة الخامسة والتسعين	p95 latency	زمن لا يتجاوزه 95% من الطلبات
زمن أول استدعاء	Cold-start latency	زمن تهيئة الخدمة أو النموذج لأول طلب
معايرة عتبة	Threshold tuning	اختيار حد القرار على مجموعة تحقق
تشغيل ظلي	Shadow deployment	مقارنة نسخة جديدة دون تغيير القرار الفعلي
مجموعة تحدٍ	Challenge set	أمثلة صعبة مصممة لاختبار حدود النموذج
اختبار رجعي	Backtesting	تقييم تنبؤ زمني على نقاط قطع تاريخية
تجميع تدريجي	Gradient accumulation	جمع تدرجات microbatches قبل تحديث واحد
قياس الإنتاجية	Throughput benchmark	عدد الأمثلة المنفذة لكل وحدة زمن
تقسيم حسب المجموعة	Group-based split	إسناد الكيانات المترابطة إلى قسم واحد لمنع التسرب
الانحراف بين المجالات	Domain shift	اختلاف توزيع أو طريقة جمع بيانات التشغيل عن التدريب
البداية الباردة	Cold start	غياب تفاعلات كافية لمستخدم أو عنصر جديد
الاستدعاء عند K	Recall@K	نسبة العناصر ذات الصلة التي تظهر ضمن أول K نتائج
الكسب التراكمي المخصوم المطبع	NDCG	مقياس لترتيب النتائج المهمة قرب أعلى القائمة
إنسان ضمن حلقة القرار	Human-in-the-loop	نقطة مراجعة يشارك فيها إنسان قبل تنفيذ إجراء حساس

عند أول ورود لمصطلح في المتن، يذكر العربي ثم الإنجليزي بين قوسين. الترجمة الاصطلاحية قد تختلف حسب المجال.

الملاحق · مراجع وأدوات

ملحق D

الملحق D: مرجع TensorFlow/Keras السريع

التنسورات والتدرج

```python
import tensorflow as tf

x = tf.constant([[1., 2.], [3., 4.]])
v = tf.Variable(2.)
with tf.GradientTape() as tape:
    loss = tf.reduce_sum(x * v)
grad = tape.gradient(loss, v)
```

- `tf.shape(x)` شكل ديناميكي؛ `x.shape` شكل معروف أثناء البناء.
- `tf.stack` ، `tf.concat` ، `tf.transpose` ، `tf.reshape` لتحويل الأبعاد.
- `tf.data.AUTOTUNE` يختار توازيًا تلقائيًا لعمليات خط البيانات.
- `@tf.function` يحول دالة إلى رسم عند أول استدعاء متوافق.

نموذج Keras

```python
import keras
from keras import layers

model = keras.Sequential([
    layers.Input((20,)),
    layers.Dense(32, activation="relu"),
    layers.Dense(1),
])
model.compile(optimizer="adam", loss="mse", metrics=["mae"])
```

المهمة	واجهة شائعة
تدريب	`model.fit(x, y, validation_data=...)`
تقييم	`model.evaluate(x_test, y_test)`
توقع	`model.predict(x_new)`
حفظ/تحميل	`model.save("m.keras")` ، `keras.models.load_model(...)`
تصدير استدلال	`model.export("saved_model")`
إيقاف مبكر	`keras.callbacks.EarlyStopping(...)`
ضبط البذرة	`keras.utils.set_random_seed(42)`

خسائر ومقاييس

- تصنيف ثنائي بمخرج sigmoid: `BinaryCrossentropy`.
- تصنيف ثنائي بـ logits: `BinaryCrossentropy(from_logits=True)`.
- متعدد الفئات بأعداد فئات صحيحة: `SparseCategoricalCrossentropy` .
- انحدار: `MeanSquaredError` أو `MeanAbsoluteError` .
- عدم التوازن: افحص `AUC` ، `Recall` ، `Precision` ومقاييس كل شريحة.

خطوط البيانات والنشر

`from_tensor_slices → shuffle → map → batch → prefetch` نمط شائع للتدريب، ويتغير ترتيبه بحسب المصدر والمعالجة. لا تستخدم الخلط على مجموعة اختبار زمنية. استخدم TFLite Converter لإنشاء نموذج LiteRT، ثم افحص المخرجات على بيانات تحقق بعد التحويل.

تذكير

- سجّل نسخة البيانات والمعالجة مع النموذج.
- افصل التدريب والتحقق والاختبار قبل ضبط الإعدادات.
- لا تخلط `.keras` مع SavedModel.
- تحقق من توافق إصدارات TensorFlow Pythong والمسرع.

الملاحق · مراجع وأدوات

ملحق E

الملحق E: حلول التمارين المختارة

هذه حلول إرشادية لعينة من الأفكار. تقبل تصميمات أخرى إذا حافظت على الفرضيات وفسرت الاختيار.

حل تمرين الأبعاد

لـ Tensor شكله `(8, 28, 28, 1)` الرتبة 4 وعدد العناصر `8×28×28×1=6272` . البعد الأول دفعة، وأبعاد المثال `(28,28,1)` عند تعريف `Input` .

حل تمرين الانحدار

للنموذج `ŷ=wx+b` والخسارة `L=(ŷ-y)^2` ، تعطي قاعدة السلسلة `∂L/∂w=2(ŷ-y)x` و `∂L/∂b=2(ŷ-y)` . إذا كان الخطأ والمدخل موجبين، ينقص تحديث الانحدار المتدرج الوزن. افحص الإشارة بمثال عددي قبل كتابة حلقة التدريب.

حل تمرين مصفوفة الالتباس

إذا كانت `FN=2` , `FP=6` , `TP=18` ، فإن precision يساوي `18/(18+6)=0.75` ، وrecall يساوي `18/(18+2)=0.90` . ارتفاع الاستدعاء مع إحكام أقل قد يناسب الفحص الأولي إذا راجع الإنسان الإنذارات.

حل تمرين عدم توازن الفئات

ابدأ بمصفوفة الالتباس والاستدعاء للفئة النادرة، لا بالدقة الخام. اختر العتبة على التحقق وفق كلفة الخطأ وسعة المراجعة، وجرب أوزان الفئات. احتفظ بالاختبار حتى القرار النهائي.

حل تمرين تقسيم السلسلة الزمنية

رتب الأحداث زمنيًا، وحدد نقطة قطع، ودرّب على ما قبلها واختبر على ما بعدها. احسب معاملات التطبيع من التدريب فقط. إذا تداخلت نوافذ التدريب والاختبار، فحدد هل ذلك يطابق سيناريو الاستخدام؛ وإلا اترك فجوة مناسبة.

حل تمرين نقل التعلم

جمّد قاعدة النموذج، درب الرأس، ثم أعد `compile` وافتح كتلًا عليا بمعدل أصغر. راقب بيانات تحقق مفصولة بحسب المصدر. إذا تراجع الأداء بعد فك التجميد، ارجع للرأس المجمد وافحص معدل التعلم وحجم البيانات.

حل تمرين النشر الطرفي

أنشئ نسخة LiteRT، وقارن الحجم والكمون والذاكرة، ثم قارن التنبؤات مع النموذج المرجعي على عينات تغطي الفئات والإضاءة. اقبل التحويل إذا بقي المقياس الحرج ضمن حد متفق عليه، واحتفظ بإصدار رجوع.

حل تمرين المراجعة الأخلاقية

حدد الفئات المتأثرة وآثار القرار، ثم اعرض معدلات القبول والأخطاء والمعايرة مع حجم كل شريحة. راجع حالات ممثلة، وحدد حق الاعتراض ومراجعة بشرية. لا تدّعِ العدالة من غياب عمود حساس؛ افحص السمات البديلة والنتائج.

الملاحق · مراجع وأدوات

ملحق F

الملحق F: المراجع والمصادر

تاريخ التحقق من صفحات الإصدارات والواجهات: 24 أكتوبر 2026. TensorFlow 2.21.0 هو خط الإصدار المستقر المرجعي، مع ظهور rc0-2.22.0 مرشحًا للإصدار؛ لذلك لا تُعامل المرشحات بوصفها إصدارات إنتاج مستقرة. راجع المصادر الحية عند إعادة الطباعة؛ جداول الدعم والواجهات تتغير.

الوثائق الرسمية

- TensorFlow: https://www.tensorflow.org/
- TensorFlow API: https://www.tensorflow.org/api_docs
- Keras Guides: https://keras.io/guides/
- Keras API: https://keras.io/api/
- تثبيت TensorFlow: https://www.tensorflow.org/install
- تثبيت TensorFlow ومصفوفة Python المدعومة: https://www.tensorflow.org/install/pip
- TensorFlow tutorials: https://www.tensorflow.org/tutorials
- تحسين خطوط `tf.data`: https://www.tensorflow.org/guide/data_performance
- LiteRT/TensorFlow Lite: https://www.tensorflow.org/lite
- TensorFlow.js: https://www.tensorflow.org/js
- TensorFlow Serving: https://www.tensorflow.org/tfx/guide/serving
- TFX: https://www.tensorflow.org/tfx
- KerasHub: https://keras.io/keras_hub/
- مصفوفة بناء TensorFlow وإصدارات Python:
  https://github.com/tensorflow/docs/blob/master/site/en/install/source.md
- ملاحظات TensorFlow 2.21: https://github.com/tensorflow/tensorflow/releases/tag/v2.21.0

- إصدار TensorFlow 2.22.0-rc0 (مرشح، وليس خط الإنتاج المرجعي):
  https://github.com/tensorflow/tensorflow/releases/tag/v2.22.0-rc0
- ملاحظات Keras 3.15.1: https://github.com/keras-team/keras/releases/tag/v3.15.1
- واجهات التدريب في Keras (`compile` و `fit` و `evaluate`):
  https://keras.io/api/models/model_training_apis/
- حفظ نموذج Keras وصيغته: https://keras.io/guides/serialization_and_saving/ (`.keras`)
- تصدير Keras للاستدلال والصيغ المدعومة: https://keras.io/api/models/model_saving_apis/export/
- تصدير Keras إلى LiteRT: https://keras.io/guides/litert_export/
- مرجع Keras mixed precision: https://keras.io/api/mixed_precision/
- Keras متعدد GPU مع TensorFlow: https://keras.io/guides/distributed_training_with_tensorflow/
- ترحيل Keras 3 وفروق الحفظ: https://keras.io/guides/migrating_to_keras_3/

تحقق من مصفوفة دعم Python وCUDA ومتطلبات النظام عند التنفيذ؛ تتغير الإصدارات وواجهات التثبيت بمرور الوقت.

كتب تأسيسية

- Goodfellow, Bengio, Courville. **Deep Learning**. MIT Press, 2016.
- Géron. **Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow**. O'Reilly.
- Murphy. **Probabilistic Machine Learning: An Introduction**. MIT Press, 2022.
- Sutton and Barto. **Reinforcement Learning: An Introduction**, second edition, 2018.

أوراق محورية

- Rumelhart, Hinton, Williams. "Learning representations by back-propagating errors." Nature, 1986.
- Hochreiter and Schmidhuber. "Long Short-Term Memory." Neural Computation, 1997.
- Vaswani et al. "Attention Is All You Need." NeurIPS, 2017.
- Kingma and Welling. "Auto-Encoding Variational Bayes." ICLR, 2014.
- Ho, Jain, Abbeel. "Denoising Diffusion Probabilistic Models." NeurIPS, 2020.
- Abadi et al. "Deep Learning with Differential Privacy." CCS, 2016.

- Ho and Salimans. “Classifier-Free Diffusion Guidance.” NeurIPS Workshop, 2021.
- Ouyang et al. “Training language models to follow instructions with human feedback.” NeurIPS, 2022.
- Dao et al. “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.” NeurIPS, 2022.
- Kairouz et al. “Advances and Open Problems in Federated Learning.” Foundations and Trends in Machine Learning, 2021.

استخدام مجموعات البيانات

استخدم مجموعات Keras المضمنة أو مصادر بحثية موثقة مع مراجعة الترخيص وشروط الاستخدام. سجّل المصدر والإصدار وتاريخ الحصول والتقسيم. لا ترفع بيانات شخصية أو سريرية إلى تخزين عام لمجرد أن المثال يعمل عليها.

الملاحق · مراجع وأدوات

ملحق G

الملحق G: فهرس الموضوعات

الفهرس مرتّب بحسب الموضوع. أرقام الفصول مراجع ثابتة؛ أرقام صفحات النسخة المطبوعة يضيفها مولّد PDF.

الموضوع	موضعه
Adam وAdamW	الفصل 7
Autoencoder وVAE وGAN	الفصل 15
BatchNorm وLayerNorm	الفصول 7 و10
BERT ومعالجة العربية	الفصل 14
CNN وGrad-CAM	الفصول 10 و23
Cross-Validation	الفصل 8
Dataset وTFRecord	الفصل 9
DQN وBellman	الفصل 21
EarlyStopping	الفصول 5 و7
Embedding	الفصول 12 و14 و22
FGSM	الفصل 23
Fine-tuning	الفصل 11
GradientTape	الفصول 4 و6 و17
IoU وmAP	الفصل 11
IoU في خرائط المحاصيل	الفصل 24
Keras Functional API	الفصل 5
LiteRT / TFLite	الفصل 19
LSTM وGRU	الفصل 12
MLOps وTFX	الفصل 20
MultiHeadAttention	الفصل 13

الموضوع	موضعه
Python وتوافق البيئة	الفصل 2 والملحق B
SparseTensorg RaggedTensor	الفصل 3
ROC وAUCg PR	الفصل 8
SavedModel و `model.export`	الفصل 19
Integrated Gradientsg SHAP	الفصل 23
TensorBoard Profiler	الفصول 8 و18
TensorFlow	الفصول 1-24
Transformer	الفصل 13
Transfer Learning	الفصل 11
XLA والدقة المختلطة	الفصل 18
التحيز والخصوصية	الفصل 23
التدرج المتلاشي	الفصلان 6 و12
التكميم	الفصل 19
التنسورات والأشكال	الفصل 3 والملحق A
الانتشار والتوليد	الفصلان 15 و16
الانتشار الخلفي	الفصلان 4 و6
الانجراف	الفصل 20
التوصية والاسترجاع	الفصل 22
التوصية والترتيب والبداية الباردة	الفصل 24
التدريب الموزع	الفصل 18

الموضوع	موضعه
التقسيم والتسرب	الفصل 8
الرسم البياني والتتبع	الفصلان 4 و18
السلاسل الزمنية	الفصل 12
الشبكات العصبية	الفصول 6-7
العدالة وبطاقة النموذج	الفصل 23
خط البيانات	الفصل 9
المشتقة التلقائية	الفصل 4
النشر	الفصل 19 والجزء الرابع
المعايرة	الفصل 8
الصحة ومراجعة الاختصاصي	الفصل 24
الصناعة وفحص العيوب	الفصلان 24 و11
الزراعة والصور متعددة الأطياف	الفصل 24
التعليم والدعم المبكر	الفصل 24
الطاقة وتوقع الذروة	الفصلان 24 و12
NDCG وRecall@K	الفصل 24
المحولات والانتباه	الفصل 13
مصفوفة الالتباس	الفصل 8
نموذج مخصص	الفصل 17